Práctica 1:

Introducción

En esta práctica se presenta una introducción a *Clean Architecture* y la propuesta de *Robert C. Martin*.

Objetivos

Al finalizar esta práctica, los estudiantes podrán:

- Describir qué es *Clean Architecture*.
- Describir las ventajas de implementar *Clean Architecture* para el desarrollo de aplicaciones.
- Describir los elementos principales del diagrama *Clean Architecture* de *Robert C. Martin*.

¿Qué es Clean Architecture?

Clean Architecture es un concepto popularizado por ***Robert Cecil Martin*** conocido como "***Uncle Bob***". *Clean Architecture*, se compone de un conjunto de patrones, prácticas y principios para crear una arquitectura de software que sea simple, comprensible, flexible, comprobable y mantenible.

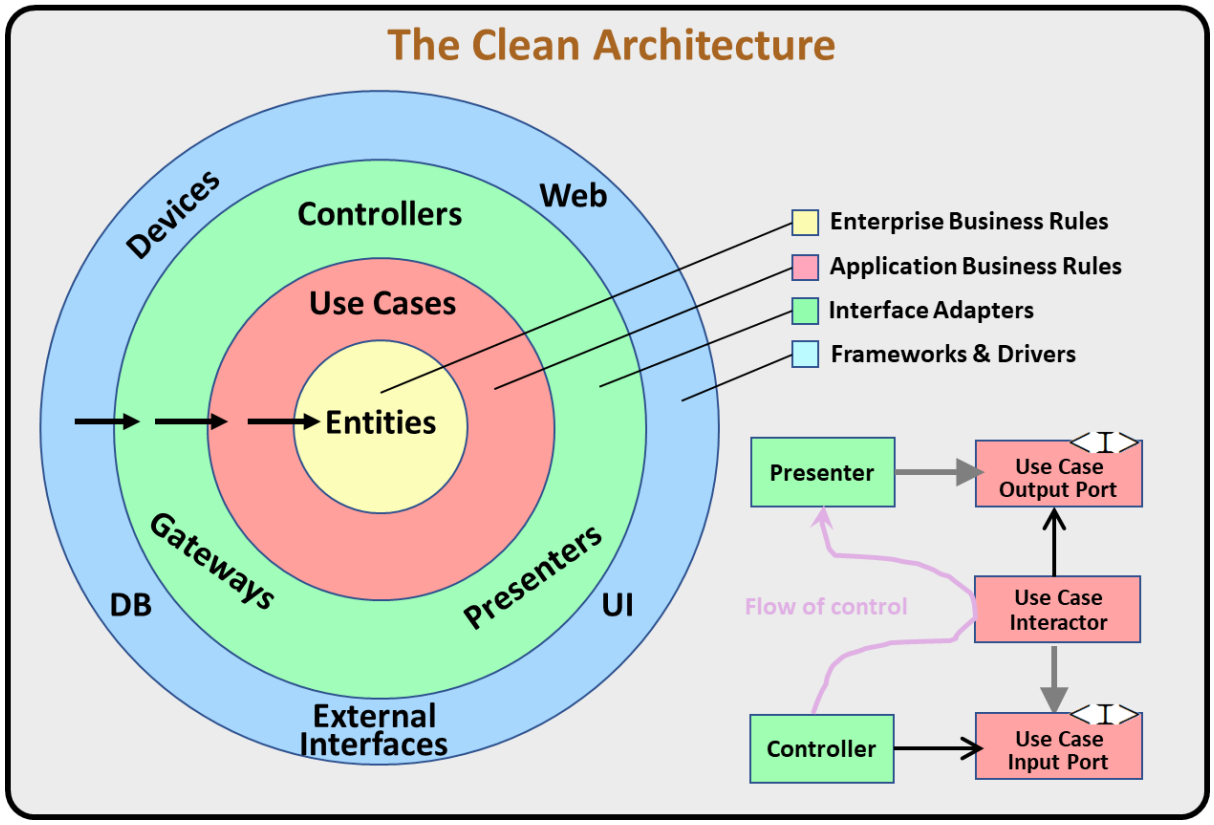
El propósito principal de la arquitectura es respaldar el ciclo de vida del sistema. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de implementar. El objetivo final es minimizar el costo de vida útil del sistema y maximizar la productividad del programador.

Clean Architecture propone la estructuración del código en capas donde cada capa debe realizar su propia tarea, cada capa debe tener su propia responsabilidad. Tomando como base esta idea, podemos encontrar artículos que tratan sobre *Clean Architecture*, *Onion Architecture*, *Hexagonal Architecture*, *Screaming Architecture*, etc., todas ellas tienen diferentes enfoques, pero comparten la idea de que cada capa debe realizar su propia tarea y que cada capa debe tener su propia responsabilidad.

La fuente de la propuesta de *Clean Architecture* puede encontrarse en el siguiente enlace:

https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

La siguiente imagen muestra el diagrama de *Clean Architecture* propuesto por *Robert C. Martin*.



El objetivo principal de una arquitectura limpia es la separación de responsabilidades y esto se logra dividiendo el software en capas. Al menos se tiene una capa para la regla de negocios y otra para las interfaces.

Los sistemas con una arquitectura limpia:

a) ***Son independientes de Frameworks***. La arquitectura no depende de la existencia de alguna biblioteca de software conteniendo diversas funciones. Esto nos permite utilizar esos frameworks como herramientas, en lugar de tener que obstaculizar nuestro sistema con sus limitadas funcionalidades.

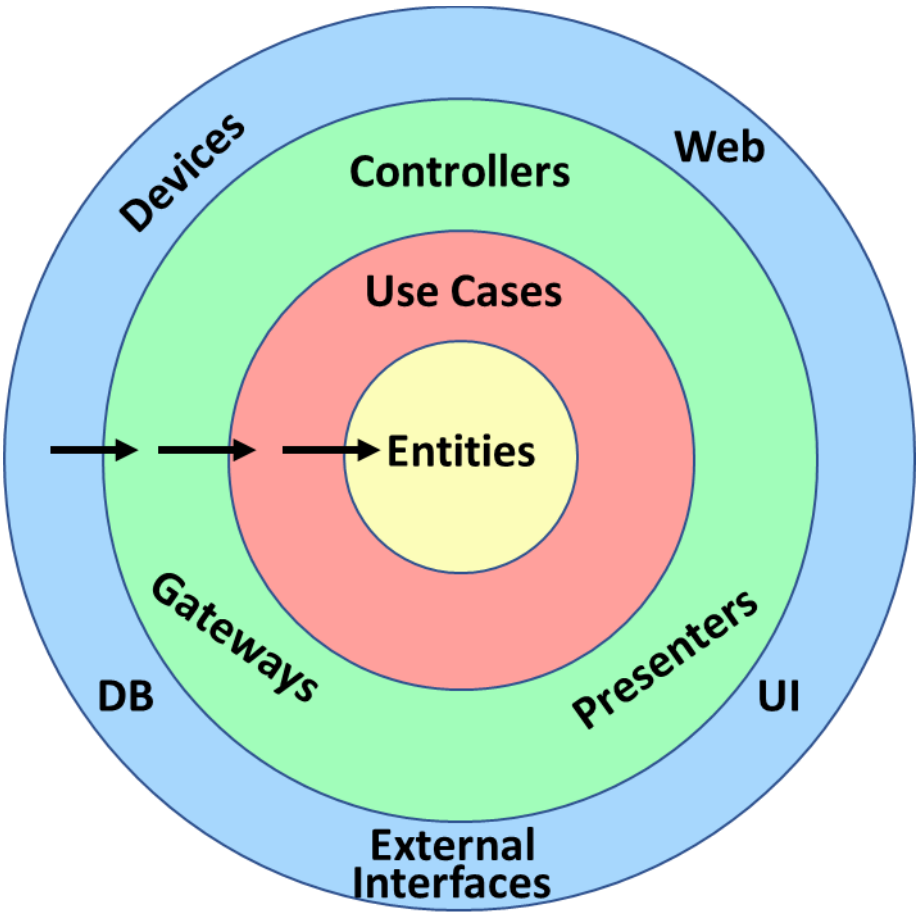
b) ***Facilitan la realización de pruebas (Testables)***. Las reglas de negocio pueden ser probadas sin la interfaz de usuario, la base de datos, el servidor web o cualquier otro elemento externo.

c) ***Independientes de la Interfaz de Usuario***. La interfaz de usuario puede cambiar fácilmente, sin cambiar el resto del sistema. Por ejemplo, una interfaz de usuario web podría reemplazarse por una interfaz de usuario de consola sin cambiar las reglas de negocio.

d) ***Independientes de la base de datos.*** Podemos cambiar *Oracle* o *SQL Server*, por *Mongo*, *BigTable*, *CouchDB* u otro manejador de base de datos. Las reglas de negocio no están vinculadas a la base de datos.

e) ***Independientes de cualquier agente externo***. Las reglas de negocio simplemente no saben nada sobre el mundo exterior.

La Regla de la Dependencia



Los círculos concéntricos del diagrama representan diferentes áreas del software. Los círculos exteriores son implementaciones. Los círculos internos son políticas o definiciones.

La regla primordial que hace que la arquitectura limpia funcione es ***La Regla De Dependencia***. Esta regla dice que las dependencias del código fuente solo pueden apuntar hacia adentro. Nada en un círculo interno puede saber nada sobre algo en un círculo externo. En particular, el nombre de algo declarado en un círculo exterior no debe ser mencionado por el código en un círculo interior. Esto incluye funciones, clases, variables, o cualquier otra entidad de software con nombre. Nada en un círculo externo debe afectar a los círculos internos.

Reglas de negocio empresariales (Enterprise Business Rules)

Las entidades encapsulan las reglas de negocio de toda la empresa. Una entidad puede ser un objeto con métodos o puede ser un conjunto de estructuras de datos y funciones. No importa lo que sea siempre que las entidades puedan ser utilizadas por otras aplicaciones diferentes en la empresa.

En caso de que no estemos desarrollando una aplicación de un sistema empresarial y simplemente estemos desarrollando una aplicación independiente, las entidades serán simplemente los objetos de negocio de la aplicación.

Las entidades encapsulan las reglas más generales y de alto nivel. Son las menos propensas a cambiar cuando algo externo cambia. Por ejemplo, las entidades no se tienen que ver afectadas por un cambio en la navegación de la página o la seguridad. Ningún cambio operativo en una aplicación en particular debería afectar la capa de *Entidades*.

En resumen, podemos decir que la capa *Enterprise Business Rules* contiene el código que puede ser compartido por diferentes aplicaciones con el fin de la reutilización del código. Algunos de los elementos que pueden ubicarse en esta capa son los siguientes:

- Definición de interfaces.
- Entidades POCO que representan elementos únicos (por ejemplo, un *Producto* de la tabla *Products*).
- Excepciones personalizadas.
- *Objetos de Valor* (*Value Object)* que, a diferencia de las entidades, los *Objectos de Valor* no tienen una identificación específica, solo describen características como un *Color* o una *Dirección de envío*.
- Eventos para comunicar los sucesos que suceden en el dominio.
- Especificaciones de lo que se puede hacer con una entidad dejando a otros componentes la realización de esta.
- Validaciones sobre las entidades.
- DTOs.
- Etc.

Reglas de negocio de la aplicación (Application Business Rules)

El software de esta capa contiene reglas de negocio específicas de la aplicación. Encapsula e implementa todos los casos de uso del sistema. Los casos de uso orquestan el flujo de datos hacia y desde las entidades para que estas apliquen sus reglas de negocio empresariales y permitan lograr los objetivos del caso de uso.

Los cambios en esta capa no deben afectar a las entidades. Esta capa tampoco debe ser afectada por cambios externos como la base de datos, la interfaz de usuario o cualquiera de los frameworks utilizados.

Sin embargo, si hay algún cambio en el funcionamiento de la aplicación, esto podría afectar a los casos de uso y, por lo tanto, al software en esta capa. Si los detalles de un caso de uso cambian, es seguro que algún código de esta capa se verá afectado.

En esta capa podemos localizar los siguientes elementos:

- ***Use Case Interactor***. Es el elemento que contiene el código con la lógica de negocios que resuelve un caso de uso. Este elemento implementa la abstracción representada por el elemento *Use Case Input Port*. En términos de programación orientada a objetos, el ***Interactor*** puede ser una clase que implementa una *Interface* o *clase abstracta* (***Input Port***).
- ***Use Case Input Port***. Es una abstracción que permite al *Interactor* recibir los datos necesarios para resolver el caso de uso proporcionados por un elemento de la capa externa (***Controller***). En términos de programación orientada a objetos, el *Input Port* puede ser definido como una *Interface* o *clase abstracta* que el *Interactor* debe implementar y el *Controller* debe utilizar.
- ***Use Case Output Port***. Es una abstracción que permite al *Interactor* devolver el resultado del caso de uso a un elemento de la capa externa (***Presenter***). En términos de programación orientada a objetos, el *Output Port* puede ser definido como una *Interface* o *clase abstracta* que el *Presenter* debe implementar y el *Interactor* debe utilizar.

En resumen, podemos decir que en esta capa se encuentra el código encargado de dar solución a una aplicación particular. El código en esta capa normalmente no puede ser reutilizado por otras aplicaciones ya que dan solución a problemas distintos. Por ejemplo, el código de una aplicación de facturación no nos serviría en una aplicación de seguimiento de clientes.

Algunos de los elementos que pueden ubicarse en esta capa son similares a los que se pueden definir en la capa de *Reglas de Negocio Empresariales,* pero con enfoque en las necesidades de la aplicación que se esté desarrollando:

- Definición de interfaces.
- Servicios para implementar la lógica de negocios.
- Excepciones personalizadas.

- Eventos.
- Especificaciones.
- Validaciones.
- DTOs,
- Etc.

Adaptadores de Interfaz (Interface Adapters)

El software en esta capa es un conjunto de adaptadores que convierten los datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para algún agente externo como la Base de Datos o la Web.

Es esta capa, por ejemplo, la que contendrá por completo la arquitectura MVC de una aplicación con Interfaz de Usuario Gráfica (GUI). *Presenters*, *Views* y *Controllers* pertenecen a esta capa. Es probable que los modelos sean solo estructuras de datos (*DTOs*) que se pasan de los controladores a los casos de uso y luego de los casos de uso a los presentadores y vistas.

De manera similar, los datos se convierten en esta capa de la forma más conveniente para las entidades y los casos de uso a la forma más conveniente para cualquier framework de persistencia que se esté utilizando, es decir, la base de datos. Ningún código interno de este círculo debería saber algo sobre la base de datos. Aquí podemos encontrar a los repositorios que funcionan como puerta de enlace (*Gateways*) hacia la fuente de datos.

También en esta capa puede existir cualquier otro adaptador necesario para convertir datos de algún formato externo, tal como el de un servicio externo, al formato interno utilizado por los casos de uso y las entidades.

Frameworks y Drivers

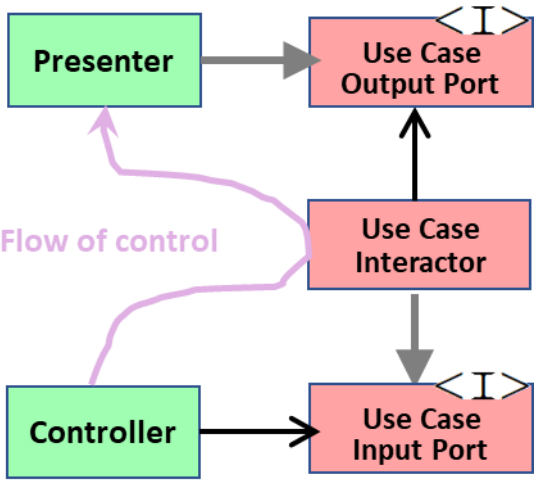
La capa más externa generalmente se compone de frameworks y herramientas como la base de datos, el framework web, etc. Generalmente, no se escribe mucho código en esta capa que no sea el código necesario para comunicarse con el siguiente círculo hacia adentro.

Esta capa es donde van todos los detalles. La Web es considerada como un detalle, esto es, un *dispositivo de entrada y salida* de información, así como cualquier otra *Interfaz de Usuario* como la Consola. La base de datos también es considerada como un detalle. Guardamos estas cosas en el exterior, donde pueden hacer poco daño.

¿Solo 4 círculos?

No, los círculos son esquemáticos. No hay ninguna regla que diga que siempre debemos tener solo estas cuatro capas. Sin embargo, siempre se aplica *la regla de dependencia*. Las dependencias del código fuente siempre apuntan hacia adentro. A medida que avanzamos hacia adentro, aumenta el nivel de abstracción y de políticas. El círculo más externo consiste en detalles concretos de bajo nivel. A medida que avanzamos hacia adentro, el software se vuelve más abstracto y encapsula políticas de alto nivel. El círculo más interno es el más general y de más alto nivel.

Cruzando los limites



En la parte inferior derecha del diagrama hay un ejemplo de cómo cruzamos los límites de los círculos. El diagrama muestra a los *Controladores* y *Presentadores* comunicándose con los *Casos de Uso* en la siguiente capa. El flujo de control comienza en el controlador, se mueve a través del caso de uso y luego termina ejecutándose en el presentador. Debemos tener en cuenta también las dependencias del código fuente. Cada una de ellas apunta hacia adentro hacia los casos de uso. Esta aparente contradicción se resuelve utilizando el *Principio de Inversión de Dependencia*. En C#, por ejemplo, podemos utilizar interfaces o clases abstractas para que en lugar de que los controladores y presentadores accedan directamente a las implementaciones, accedan a abstracciones.

Por ejemplo, consideremos que el *Interactor* del caso de uso necesita invocar al presentador. Esta llamada no debe ser directa porque violaría la *Regla de dependencia*: ningún nombre en un círculo externo puede ser mencionado por un círculo interno. Por lo tanto, podemos hacer que el caso de uso utilice una interface (que se muestra en el diagrama como *Puerto de salida del caso de uso – Use Case Output Port*) en el círculo interno, y hacer que el presentador en el círculo externo lo implemente.

La misma técnica es utilizada para cruzar todos los límites en la arquitectura. Aprovechamos el polimorfismo para crear dependencias de código fuente que se oponen al flujo de control para que podamos cumplir con la *Regla de dependencia* sin importar en qué dirección vaya el flujo de control.

En el diagrama podemos ver que *Use Case Input Port* es una Interface (abstracción) que es utilizada por el *Controlador* y es implementada por el *Interactor*. De la misma forma podemos ver que el *Use Case Output Port* es una Interface que es utilizada por el *Interactor* e implementada por el *Presentador*. Ambas interfaces se encuentran definidas en la capa de *Reglas de negocio de la aplicación*.

¿Qué datos cruzan los límites?

Normalmente, los datos que cruzan los límites son estructuras de datos simples. Podemos utilizar estructuras básicas u objetos simples de transferencia de datos (*DTOs*) si lo deseamos. O los datos pueden ser simplemente argumentos en llamadas a funciones. O podemos empaquetarlos en un objeto. El punto importante es que estructuras de datos simples y aisladas son pasadas a través de los límites. No debemos pasar, por ejemplo, registros o filas de bases de datos o entidades que tengan algún tipo de dependencia que viole la *Regla de dependencia*.

Por ejemplo, muchos frameworks de base de datos devuelven un formato de datos conveniente en respuesta a una consulta (por ejemplo, un objeto proxy en una entidad devuelta por *Entity Framework*). Podríamos llamar a esto una *estructura de fila*. No debemos pasar esa *estructura de fila* hacia adentro a través de un límite. Eso violaría la *Regla de dependencia* porque obligaría a un círculo interno a saber algo sobre un círculo externo.

Por lo tanto, cuando pasemos datos a través de un límite, siempre deben estar en la forma que sea más conveniente para el círculo interno.

Conclusión

Cumplir con estas simples reglas no es difícil y nos ahorrará muchos dolores de cabeza en el futuro. Al separar el software en capas y cumplir con la *Regla de dependencia*, crearemos un sistema que sea intrínsecamente comprobable, con todos los beneficios que ello implica. Cuando alguna de las partes externas del sistema se vuelva obsoleta, como la base de datos o el framework web, podremos reemplazar esos elementos obsoletos con un mínimo de esfuerzo.

Práctica 2:

Definición del caso de uso

En esta práctica describiremos el caso de uso que tomaremos como base para desarrollar una aplicación .NET implementando *Clean Architecture*.

Después de definir el caso de uso, crearemos la estructura inicial de la solución utilizando Visual Studio.

Objetivos

En esta práctica los estudiantes:

- Conocerán el caso de uso a implementar.
- Crearán la estructura inicial de la solución con Visual Studio.

El caso de uso: Crear orden de compra

Para ejemplificar el desarrollo de aplicaciones .NET implementando *Clean Architecture*, empezaremos con la implementación de un caso de uso simple para la empresa ficticia *NorthWind*. En lecciones posteriores agregaremos más funcionalidad al caso de uso.

Historia de Usuario: Crear orden de compra

"Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema registra la orden de compra.
3. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.

- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Podemos notar que el caso de uso solo describe lo que se necesita en la interacción entre el usuario y el sistema. No especifica la forma en que se debe guardar la información o la forma de introducir la información, por ejemplo, a través de un sitio Web, una aplicación de consola o vía telefónica. Esos son detalles, o cómo lo indica *Robert C. Martin* "*las reglas de negocio deben ser agnósticas de los mecanismos de entrega*".

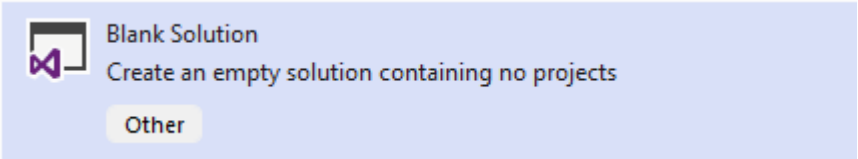
Crear la estructura inicial de la solución

Empecemos por crear la estructura de la solución utilizando Visual Studio.

Las indicaciones de cada ejercicio de este entrenamiento están basadas en *Visual Studio 2022* y el framework ***.NET 8***. Algo muy importante de mencionar es que las imágenes presentadas en las instrucciones podrían no coincidir con la interfaz actual de Visual Studio debido a los cambios constantes del mismo.

La estructura de la solución que crearemos está planteada con fines didácticos para poder ejemplificar los elementos de la propuesta *Clean Architecture* de *Robert C. Martin*, sin embargo, la propuesta no especifica que debamos tener esta misma estructura de directorios, tampoco especifica si debemos tener una o más soluciones o uno o más proyectos. De hecho, *Clean Architecture* es independiente del *IDE*, del lenguaje o framework que utilicemos para desarrollar la aplicación.

1. Abre Visual Studio bajo el contexto de *Administrador*.
2. Crea una solución vacía llamada ***NorthWind*** utilizando la plantilla ***Blank Solution***.

   La solución que crearemos incluirá el desarrollo de 2 aplicaciones, una *API Web* y una aplicación *Blazor WebAssembly*.

3. Agrega un nuevo directorio a la solución llamado ***1. Enterprise Business Rules***.

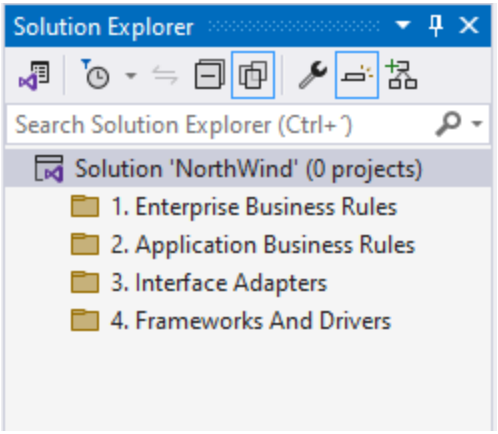
   En este directorio colocaremos las entidades que encapsulen las reglas de negocio que pueden ser compartidas por las aplicaciones de nuestra solución.

   Las entidades encapsulan las reglas más generales y de alto nivel. Son las menos propensas a cambiar cuando algo externo cambia. Por ejemplo, las entidades no se tienen que ver afectadas por un cambio en la navegación de la página o la seguridad. Ningún cambio operativo en una aplicación en particular debería afectar la capa de *Entidades*.

4. Agrega un nuevo directorio a la solución llamado ***2. Application Business Rules***. En este directorio colocaremos el código propio de una aplicación, por ejemplo, el código que resuelve el caso de uso "*Crear orden de compra*".
5. Agrega un nuevo directorio a la solución llamado ***3. Interface Adapters***. Aquí colocaremos los *Controladores*, *Presentadores*, *Vistas* y *ViewModels* de la aplicación. También colocaremos aquí los enlaces (*Gateways*) para consumir servicios externos como una API Web o los repositorios para acceso a los datos.

6. Agrega un nuevo directorio llamado ***4. Frameworks And Drivers*** a la solución. Aquí colocaremos las aplicaciones que interactuarán directamente con el usuario o con otras aplicaciones. Por ejemplo, aquí colocaremos una *Web API* y una aplicación *Blazor WebAssembly*.

   La estructura de la solución será similar a la siguiente.



Es importante volver a mencionar que hemos creado la solución con esta estructura únicamente con fines didácticos y para poder ejemplificar cada uno de los elementos de la propuesta *Clean Architecture* de *Robert C. Martin*. En aplicaciones reales, no es necesario tener la misma estructura. La propuesta de *Clean Architecture* no especifica que debamos tener esta misma estructura de directorios, tampoco especifica si debemos tener una o más soluciones o uno o más proyectos. De hecho, *Clean Architecture* es independiente del *IDE*, del lenguaje o framework que utilicemos para desarrollar la aplicación.

No hay ninguna regla que diga que siempre debemos tener esta estructura. Sin embargo, lo que si debemos tener siempre presente es cumplir con ***la regla de dependencia***. Las dependencias del código fuente siempre deben apuntar hacia dentro.

Cada proyecto que agreguemos en una capa solo podrá hacer referencia a proyectos de la misma capa o de capas inferiores. Por ejemplo, los proyectos que agreguemos en la capa *Enterprise Business Rules* no podrán hacer referencia a ninguno de los proyectos de las otras capas. Sin embargo, los proyectos de la capa *Frameworks And Drivers* podrán agregar referencias de cualquier proyecto de la misma capa o de las otras capas. De igual forma, los proyectos de la capa *Application Business Rules* podrán agregar referencias de proyectos de la misma capa y de la capa *Enterprise Business Rules,* pero no podrán agregar referencias de proyectos de las otras capas superiores.

Práctica 3:

Implementando la capa de Reglas de negocio Empresariales

Las entidades encapsulan las reglas de negocio de toda la empresa. Una entidad puede ser un objeto con métodos o puede ser un conjunto de estructuras de datos y funciones. No importa lo que sea siempre que las entidades puedan ser utilizadas por diferentes aplicaciones en la empresa.

Las entidades encapsulan las reglas más generales y de alto nivel. Son las menos propensas a cambiar cuando algo externo cambia. Por ejemplo, las entidades no se tienen que ver afectadas por un cambio en la navegación de la página o la seguridad. Ningún cambio operativo en una aplicación en particular debería afectar la capa de *Entidades*.

En esta capa podemos definir *Interfaces*, *entidades POCO*, *Excepciones personalizadas*, *Objetos de Valor* (*Value Object)*, *Eventos*, *Especificaciones*, *DTOs*, etc.

En esta práctica definiremos las reglas de negocio empresariales que serán requeridas para resolver el caso de uso *Crear Orden de compra* y que podrán ser utilizadas por la aplicación *API Web* y por la aplicación *Blazor WebAssembly*.

Objetivos

En esta práctica los estudiantes:

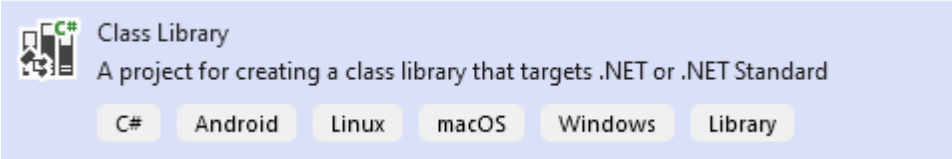
- Definirán las reglas de negocio empresariales de la empresa *NorthWind*.

Definir la capa de entidades

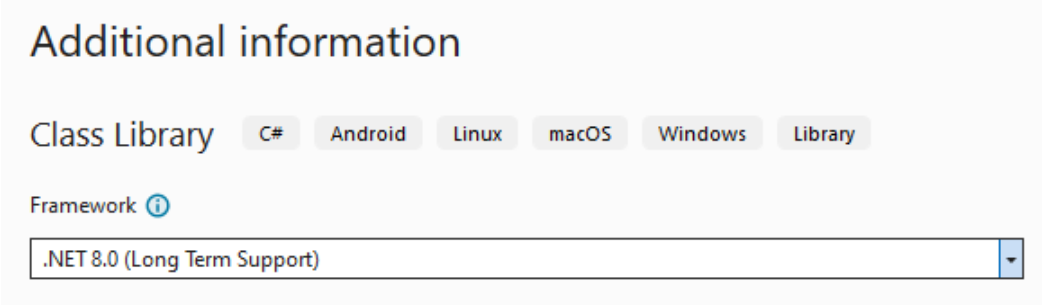
Para dar solución al caso de uso "*Crear orden de compra*", desarrollaremos 2 aplicaciones, una *API Web* que implemente el código *backend* y una aplicación *Blazor WebAssembly* que implemente el código *frontend*. El código backend se encontrará en el espacio de nombres ***NorthWind.Sales.Backend***, el código frontend se encontrará en el espacio de nombres ***NorthWind.Sales.Frontend*** y el código compartido por las aplicaciones se encontrará en el espacio de nombres ***NorthWind.Sales***.

Empecemos por agregar el primer proyecto que nos permitirá definir las reglas de negocio empresariales, este proyecto podrá ser compartido por la aplicación backend y la aplicación frontend.

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.Entities*** dentro del directorio ***1. Enterprise Business Rules***.



   Selecciona la versión más reciente de .NET. Al momento de crear el contenido de este entrenamiento, la versión más reciente es .NET 8 y es la que utilizaremos en todos los proyectos que crearemos en este entrenamiento.



2. Elimina el archivo ***Class1.cs*** creado.

Agregar DTOs

Normalmente, los datos que cruzan los límites de las capas en *Clean Architecture* son estructuras de datos simples. Podemos utilizar estructuras básicas u objetos simples de transferencia de datos (*DTOs*) si lo deseamos. Los *DTOs* nos permiten también ocultar información que no sea necesaria para las capas exteriores y normalmente son inmutables.

Agreguemos los *DTOs* necesarios para recibir los datos de entrada del caso de uso que serán pasados desde la aplicación frontend hacia la aplicación backend.

1. Crea un directorio llamado ***Dtos*** en la raíz del proyecto.
2. Crea un nuevo directorio llamado ***CreateOrder*** en el directorio *Dtos*. En este directorio colocaremos los *Dtos* necesarios del caso de uso *Crear Orden de compra*. Cuando agreguemos más casos de uso, agregaremos más directorios, uno por cada caso de uso.
3. Agrega un nuevo archivo de clase llamado ***CreateOrderDetailDto.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
public class CreateOrderDetailDto(int productId, decimal unitPrice,
        short quantity)
{
    public int ProductId => productId;
    public decimal UnitPrice => unitPrice;
    public short Quantity => quantity;
}
```

   Estamos utilizando la característica ***Primary Constructors*** de *C# 12* para definir el constructor de la clase e inicializar las propiedades de solo lectura. Normalmente los *DTOs* son objetos de solo lectura que son utilizados para transportar información.

4. Agrega un nuevo archivo de clase llamado ***CreateOrderDto.cs*** en el directorio *CreateOrder* con el siguiente código.

```
public class CreateOrderDto(string customerId, string shipAddress,
        string shipCity, string shipCountry, string shipPostalCode,
        IEnumerable<CreateOrderDetailDto> orderDetails)
{
    public string CustomerId => customerId;
    public string ShipAddress => shipAddress;
    public string ShipCity => shipCity;
    public string ShipCountry => shipCountry;
    public string ShipPostalCode => shipPostalCode;
    public IEnumerable<CreateOrderDetailDto> OrderDetails => orderDetails;
}
```

Puedes notar que ambos *DTOs* incluyen únicamente las propiedades necesarias para recibir los datos de entrada del caso de uso. Datos como la fecha de la orden, transporte o descuento no son incluidos.

Un DTO solo debe contener datos, no lógica empresarial. Es algo simple y pequeño que debe hacer una sola tarea: transportar datos.

Práctica 4:

Implementando la capa de Reglas de negocio de la aplicación

El software de esta capa contiene reglas de negocio específicas de la aplicación. Encapsula e implementa todos los casos de uso de la aplicación. Los casos de uso orquestan el flujo de datos hacia y desde las entidades para que estas apliquen sus reglas de negocio empresariales y permitan lograr los objetivos del caso de uso.

Los cambios en esta capa no deben afectar a las entidades. Esta capa tampoco debe ser afectada por cambios externos como la base de datos, la interfaz de usuario o cualquiera de los framework utilizados.

Si hay algún cambio en el funcionamiento de la aplicación, esto podría afectar a los casos de uso y, por lo tanto, al software en esta capa. Si los detalles de un caso de uso cambian, es seguro que algún código de esta capa se verá afectado.

Los elementos exclusivos de esta capa son los *Use Case Interactors*, *Use Case Input Ports* y *Use Case Output Ports*.

Algunos otros elementos que pueden encontrarse en esta capa son similares a los que se pueden definir en la capa de *Reglas de Negocio Empresariales,* pero con enfoque en las necesidades de la aplicación que se esté desarrollando.

En esta práctica definiremos las reglas de negocio de la aplicación que serán requeridas para resolver el caso de uso *Crear Orden de compra*.

Objetivos

En esta práctica los estudiantes:

- Definirán los objetos de negocio de la aplicación incluyendo enumeraciones, entidades POCO, *Value Objects*, *Agregados*, *DTOs*, abstracciones de *Puertos*, *Presentadores*, *Controladores* y *Repositorios*.
- Implementarán el caso de uso *Crear orden de compra*.

Definir los objetos de negocio de la aplicación

Los objetos de negocio de la aplicación son similares a las entidades definidas en la capa de reglas de negocio empresariales, la diferencia radica en que los objetos de negocio son entidades exclusivas de la aplicación que no son útiles a otras aplicaciones ya que ayudan a resolver los casos de uso propios de una aplicación específica. Cada aplicación tendrá sus propios objetos de negocio.

Empecemos por definir los objetos de negocio del código backend.

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.BusinessObjects*** dentro del directorio ***2. Application Business Rules***.
2. Elimina el archivo ***Class1.cs***.
3. Agrega una referencia del proyecto ***NorthWind.Sales.Entities***.

Agregar enumeraciones

Para implementar las consideraciones del caso de uso relacionadas con los medios de transporte y tipos de descuento, definiremos un par de enumeraciones.

1. Crea un nuevo directorio llamado ***Enums*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ShippingType.cs*** en el directorio *Enums* remplazando el código de la clase por el siguiente.

```
public enum ShippingType
{
    Sea, Air, Rail, Road
}
```

3. Agrega un nuevo archivo de clase llamado ***DiscountType.cs*** en el directorio *Enums* remplazando el código de la clase por el siguiente.

```
public enum DiscountType
{
    Flat, Percentage
}
```

Agregar entidades POCO

Las entidades *POCO* representan elementos con identidad que los hace únicos (por ejemplo, una *Orden* específica del conjunto de *Ordenes*).

Agreguemos ahora una clase que nos permita identificar una orden de compra.

1. Crea un nuevo directorio llamado ***POCOEntities*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***Order.cs*** en el directorio *POCOEntities* y remplaza el código de la clase con el siguiente código.

```
public class Order
{
    public int Id { get; set; }
    public string CustomerId { get; set; }
    public string ShipAddress { get; set; }
    public string ShipCity { get; set; }
    public string ShipCountry { get; set; }
    public string ShipPostalCode { get; set; }

    public ShippingType ShippingType { get; set; } = ShippingType.Road;
    public DiscountType DiscountType { get; set; } = DiscountType.Percentage;
    public double Discount { get; set; } = 10;
    public DateTime OrderDate { get; set; } = DateTime.Now;
}
```

Puedes notar que en el código estamos implementado 3 requerimientos:

- El tipo de transporte predeterminado es *Terrestre*.
- El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponde a la fecha y hora en la que es creada.

El código anterior requerirá importar el espacio de nombres de las enumeraciones. En la mayoría de las ocasiones, Visual Studio agregará al inicio del archivo el código para importar los espacios de nombres requeridos conforme vayamos escribiendo el código. Para evitar repetir el mismo código en los diversos archivos que lo requieran, haremos uso de la característica ***Global using directives*** de *C# 10* y moveremos el código que importa los espacios de nombres hacia un archivo independiente, el archivo será un archivo de clase que puede tener cualquier nombre. En nuestros ejemplos utilizaremos el nombre de archivo ***GlobalUsings.cs***.

3. Agrega un nuevo archivo de clase llamado ***GlobalUsings.cs*** en la raíz del proyecto.
4. Elimina el contenido actual del archivo.
5. Agrega el siguiente código en el archivo *GlobalUsings.cs* para importar de forma global el espacio de nombres de las enumeraciones.

```
global using NorthWind.Sales.Backend.BusinessObjects.Enums;
```

6. Elimina la línea de código con la directiva *using* en el archivo *Order.cs*.

En los siguientes ejercicios de este entrenamiento, siempre que se indique importar los espacios de nombres requeridos hacia el archivo *GlobalUsings.cs*, se asume que el archivo existe en la raíz del proyecto. En caso de no existir, deberás crearlo manualmente.

Puedes instalar la extensión de Visual Studio "***Make usings as global usings***" para realizar la tarea de mover las directivas *using* del archivo actual hacia un archivo *GlobalUsings.cs*. La extensión puede crear el archivo *GlobalUsings.cs* en la raíz del proyecto en caso de que este no exista.

Make usings as global usings
https://marketplace.visualstudio.com/items?itemName=MSMDotNet.MakeUsingsAsGlobalUsings

Agregar Value Objects

Los *Objetos de Valor* (*Value Object)* a diferencia de las entidades, no tienen una identificación específica, solo describen características como un *Color,* una *Dirección de envío* o el detalle de una orden.

Agreguemos una clase que nos permita agregar los valores para el detalle de la orden.

1. Crea un nuevo directorio llamado ***ValueObjects*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***OrderDetail.cs*** en el directorio ***ValueObjects*** y remplaza la clase con el siguiente código.

```
public class OrderDetail(int productId, decimal unitPrice, short quantity)
{
    public int ProductId => productId;
    public decimal UnitPrice => unitPrice;
    public short Quantity => quantity;
}
```

Los *Value Objects* tienen 2 características principales: no tienen identidad y son inmutables.

Agregar Agregados (Aggregates)

Un *Agregado* (*Aggregate*) es un grupo de objetos de dominio que pueden ser tratados como una unidad. Un ejemplo puede ser una orden con su detalle.

Un *Agregado* tendrá uno de sus objetos componentes como raíz del *Agregado*. Cualquier referencia desde fuera del *Agregado* solo debe ir a la raíz del *Agregado*. De este modo, la raíz puede garantizar la integridad del *Agregado* en su conjunto.

Agreguemos ahora un *Agregado* que nos permita tratar una orden con su detalle como una unidad.

1. Crea un directorio llamado ***Aggregates*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***OrderAggregate.cs*** en el directorio *Aggregates* y remplaza el código de la clase con el siguiente código.

```
public class OrderAggregate : Order
{
    readonly List<OrderDetail> OrderDetailsField = [];
    public IReadOnlyCollection<OrderDetail> OrderDetails =>
        OrderDetailsField;

    public void AddDetail(int productId, decimal unitPrice, short quantity)
    {
        var ExistingOrderDetail =
            OrderDetailsField
            .FirstOrDefault(o => o.ProductId == productId);

        if (ExistingOrderDetail != default)
        {
            quantity += ExistingOrderDetail.Quantity;
            OrderDetailsField.Remove(ExistingOrderDetail);
        }

        OrderDetailsField.Add(
            new OrderDetail(productId, unitPrice, quantity));
    }

    public static OrderAggregate From(CreateOrderDto orderDto)
    {
        OrderAggregate OrderAggregate = new OrderAggregate
        {
            CustomerId = orderDto.CustomerId,
            ShipAddress = orderDto.ShipAddress,
            ShipCity = orderDto.ShipCity,
            ShipCountry = orderDto.ShipCountry,
            ShipPostalCode = orderDto.ShipPostalCode
        };

        foreach (var Item in orderDto.OrderDetails)
        {
            OrderAggregate.AddDetail(Item.ProductId,
                Item.UnitPrice, Item.Quantity);
        }

        return OrderAggregate;
    }
}
```

Del código anterior podemos ver que la única forma de agregar un detalle a la orden es a través del método *AddDetail*. El método ***AddDetail*** implementa uno de los requerimientos:

"*Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.*"

El código anterior también incluye un método que permite crear un objeto *OrderAggregate* a partir de los datos de un objeto *CreateOrderDto*. La aplicación frontend nos enviará los datos de una orden en un objeto *CreateOrderDto* que necesitaremos convertir en un objeto *OrderAggregate*.

Puedes notar también que utilizamos la característica ***Collection expressions*** de *C# 12* para inicializar la colección *OrderDetailsField* que se manejará de forma interna en el agregado y se expondrá al exterior a través de la propiedad *OrderDetails*.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar la abstracción Unit Of Work

Unit of Work es un patrón de diseño que nos permite mantener una lista de objetos afectados por una transacción de negocio y coordinar el guardado de los cambios, así como la resolución de conflictos de concurrencia.

Unit of Work es responsable de confirmar los cambios en la fuente de datos, lo que garantiza una transacción completa, sin pérdida de datos, de forma atómica.

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***Common*** en el directorio *Interfaces*. En este directorio agregaremos las abstracciones que sean comunes a todos los casos de uso.
3. Agrega un nuevo archivo de *Interface* llamado ***IUnitOfWork.cs*** en el directorio *Common* y remplaza el código de la interface con el siguiente código.

```
public interface IUnitOfWork
{
    Task SaveChanges();
}
```

El método *SaveChanges* especifica un valor devuelto de tipo *Task* para permitir que la implementación pueda ser síncrona o asíncrona.

Agregar las abstracciones de los Puertos

Los *Puertos* son abstracciones que permiten al *Interactor del caso de uso* recibir los datos de entrada y proporcionar el resultado del caso de uso.

Agreguemos los puertos del caso de uso.

1. Crea un directorio llamado ***CreateOrder*** en el directorio *Interfaces*.
2. Agrega un nuevo archivo de tipo *Interface* llamado ***ICreateOrderInputPort.cs*** dentro del directorio *CreateOrder* y remplaza el código de la interface con el siguiente código.

```
public interface ICreateOrderInputPort
{
    Task Handle(CreateOrderDto orderDto);
}
```

   El *InputPort* del caso de uso es una abstracción que permite al *Interactor* recibir los datos necesarios para resolver el caso de uso proporcionados por un elemento de la capa externa (***Controller***). En términos de programación orientada a objetos, el *InputPort* puede ser definido como una *Interface* o *clase abstracta* que el *Interactor* debe implementar y el *Controller* debe utilizar.

3. Agrega un nuevo archivo de tipo *Interface* llamado ***ICreateOrderOutputPort.cs*** dentro del directorio *CreateOrder* y remplaza el código de la interface con el siguiente código.

```
public interface ICreateOrderOutputPort
{
    int OrderId { get; }
    Task Handle(OrderAggregate addedOrder);
}
```

   El *OutputPort* del caso de uso es una abstracción que permite al *Interactor* devolver el resultado del caso de uso a un elemento de la capa externa (***Presenter***). En términos de programación orientada a objetos, el *OutputPort* puede ser definido como una *Interface* o *clase abstracta* que el *Presenter* debe implementar y el *Interactor* debe utilizar. El *OutputPort* que hemos definido recibe una instancia de la orden que se haya agregado.

   La función del *Presenter* es la de convertir los datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para algún agente externo como la Base de Datos o la Web. En nuestro ejemplo, el *Presenter* simplemente devolverá el *Id* de la orden creada.

Es importante mencionar que no siempre necesitamos un *Presenter*, sobre todo cuando no hay un resultado que devolver o cuando no necesitamos realizar una transformación del resultado devuelto por el *Interactor*. En nuestro caso, agregaremos el *Presenter* para ejemplificar cada uno de los elementos del diagrama de *Clean Architecture* de *Robert C. Martin*.

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar la abstracción del Repositorio

El patrón ***Repositorio*** (***Repository***) es un patrón de diseño que consiste en separar la lógica de negocios de la lógica que recupera los datos y los asigna a un modelo de entidad (como *Entity Framework Core* o *Dapper*). Un *Repositorio* media entre el dominio y las capas de mapeo de datos actuando como una colección de objetos de dominio en memoria.

Un *Repositorio* es una clase que oculta la lógica necesaria para almacenar o recuperar datos. Por lo tanto, a una aplicación no le importará qué tipo de *ORM* estemos usando ya que todo lo relacionado con el *ORM* se maneja dentro de una capa de *Repositorio*. Esto nos permite tener una separación más limpia de responsabilidades. El patrón *Repository* es uno de los patrones de diseño más utilizados para crear soluciones más limpias y se utiliza a menudo junto con el patrón *Unit Of Work*.

Para realizar la persistencia de datos implementaremos un repositorio, el repositorio que definiremos deberá implementar el patrón *Unit Of Work*.

1. Crea un directorio llamado ***Repositories*** en el directorio *Interfaces*.
2. Agrega un nuevo archivo de interface llamado ***ICommandsRepository.cs*** dentro del directorio *Repositories* y remplaza el código de la interface con el siguiente código.

```
public interface ICommandsRepository : IUnitOfWork
{
    Task CreateOrder(OrderAggregate order);
}
```

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

   La abstracción del repositorio que hemos creado nos permitirá realizar operaciones de modificación de datos para los casos de uso de la aplicación *NorthWind.Sales*. Esto nos facilitará una posible implementación del patrón ***CQRS*** que propone la separación de comandos y consultas.

Por el momento nuestro repositorio solo incluye una operación, la de crear la orden. Puedes notar que no estamos especificando cómo se debe implementar el repositorio, tampoco estamos especificando el framework a utilizar para realizar la persistencia de datos.

Implementar el caso de uso

Después de haber definido las abstracciones de la aplicación, podemos ahora empezar a implementar el caso de uso.

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.UseCases*** dentro del directorio ***2. Application Business Rules***.
2. Elimina el archivo ***Class1.cs***.
3. Agrega una referencia del proyecto ***NorthWind.Sales.Backend.BusinessObjects***.

Agregar el Interactor

1. Agrega un nuevo directorio llamado ***CreateOrder*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***CreateOrderInteractor.cs*** en el directorio *CreateOrder* y remplaza el código de la clase con el siguiente código.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository) : ICreateOrderInputPort
{
    public async Task Handle(CreateOrderDto orderDto)
    {
        OrderAggregate Order = OrderAggregate.From(orderDto);

        await repository.CreateOrder(Order);
        await repository.SaveChanges();
        await outputPort.Handle(Order);
    }
}
```

El código de la clase recibe los servicios requeridos mediante inyección de dependencias a través del constructor.

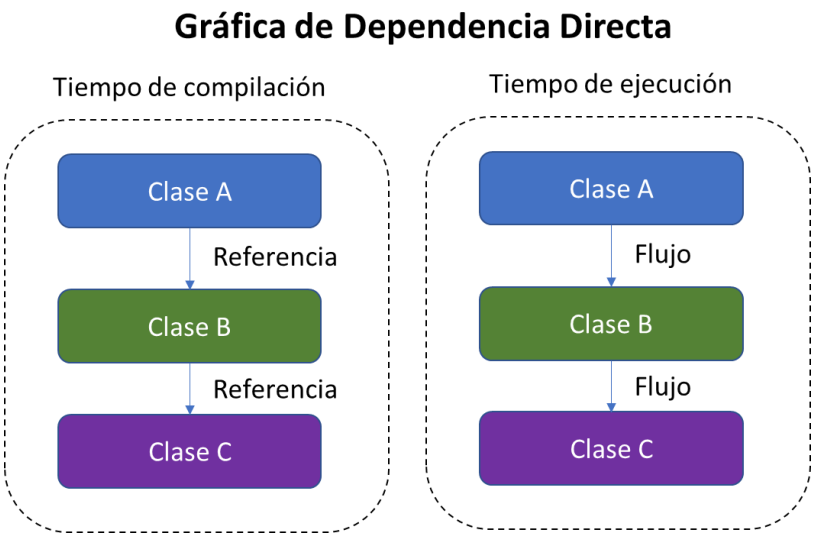
El método *Handle* simplemente crea un objeto *OrderAggregate* con los datos de entrada que recibe como parámetro. Posteriormente, realiza la persistencia y envía los datos de la orden creada al *OutputPort* invocando su método *Handle*.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Realizar la inversión de Control

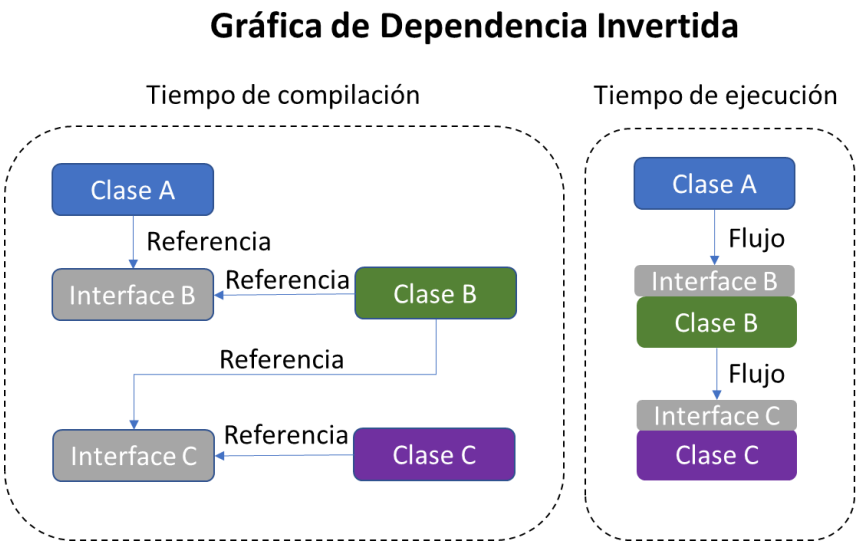
La dirección de dependencia dentro de la aplicación debe estar en la dirección de la abstracción, no de los detalles de implementación.

La mayoría de las aplicaciones se escriben de manera que la dependencia del tiempo de compilación fluya en la dirección del tiempo de ejecución, lo que produce un gráfico de dependencias directa. Es decir, si la clase ***A*** invoca a un método de la clase ***B*** y la clase ***B*** invoca a un método de la clase ***C***, en tiempo de compilación la clase ***A*** dependerá de la clase ***B*** y la clase ***B*** de la clase ***C***, como se muestra en la siguiente figura.



Inversión de control (*Inversion of Control en inglés* o simplemente *IoC*) es un principio de diseño de software en el que el flujo de ejecución de un programa se invierte respecto a los métodos de programación tradicionales.

Aplicando el principio de *inversión de control* permite que la clase ***A*** invoque a métodos de una abstracción que ***B*** implementa, lo que hace posible que ***A*** invoque métodos de ***B*** en tiempo de ejecución, pero que ***B*** dependa de una *Interface* controlada por ***A*** en tiempo de compilación (por tanto, se invierte la típica dependencia de tiempo de compilación). En tiempo de ejecución, el flujo de ejecución del programa no cambia, pero la introducción de interfaces hace posible que se puedan conectar fácilmente otras implementaciones de estas interfaces.



La ***Inversión de control*** es una parte fundamental de la creación de aplicaciones de acoplamiento flexible, ya que se pueden escribir detalles de implementación de los que depender e implementar abstracciones de nivel superior, en lugar de hacerlo al revés. Como resultado, las aplicaciones son modulares y más fáciles de probar y mantener.

La *Inversión de control* puede implementarse de diferentes maneras, por ejemplo, se puede implementar mediante eventos o mediante *Inyección de Dependencias*.

En este entrenamiento realizaremos la *Inversión de Control* mediante la técnica de *Inyección de Dependencias*. En cada proyecto que implementemos abstracciones agregaremos un archivo con código que facilite el registro de dependencias utilizando el servicio de inyección de dependencias de .NET.

Agregar el contenedor de dependencias

Debido a que hemos implementado un servicio (*ICreateOrderInputPort*), debemos crear el código que permita el registro de la implementación en el contenedor de dependencias.

Una técnica para facilitar el registro de dependencias es crear una clase estática que exponga un método de extensión de ***IServiceCollection*** donde se registren los servicios implementados. Posteriormente, el cliente podrá invocar a ese método para registrar los servicios en el contenedor de dependencias.

1. Agrega el paquete NuGet `Microsoft.Extensions.DependencyInjection.Abstractions` al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el código del archivo con el siguiente código.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddUseCasesServices(
        this IServiceCollection services)
    {
        services.AddScoped<ICreateOrderInputPort,
            CreateOrderInteractor> ();

        return services;
    }
}
```

Puedes notar que el espacio de nombres de la clase *DependencyContainer* se encuentra en el espacio de nombres *Microsoft.Extensions.DependencyInjection*. Esto evitará el uso de sentencias *using* adicionales en el código consumidor del método *AddUseCasesServices*.

El nombre del archivo que utilizamos fue ***DependencyContainer.cs*** pero puede ser cualquier otro nombre.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Práctica 5:

Implementando la capa de Adaptadores de Interfaz

El software en esta capa está formado por un conjunto de adaptadores que convierten los datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para algún agente externo como la Base de Datos o la Web.

En esta capa también puede existir cualquier otro adaptador necesario para convertir datos de algún formato externo, tal como el de un servicio externo, al formato interno utilizado por los casos de uso y las entidades.

Objetivos

En esta práctica los estudiantes:

- Implementarán el *Repositorio*.
- Implementarán el *Presenter*.
- Implementarán el *Controller*.
- Agregarán un proyecto para centralizar el registro de dependencias.

El Repositorio

El patrón ***Repository*** es un patrón de diseño del enfoque *DDD* cuyo objetivo es mantener los detalles de persistencia fuera del modelo de dominio del sistema. Una o varias abstracciones de persistencia (interfaces) se definen en el modelo de dominio, y estas abstracciones tienen implementaciones en forma de adaptadores específicos de persistencia definidos en otra parte de la aplicación.

Hemos definido las abstracciones en el proyecto ***NorthWind.Sales.Backend.BusinessObjects*** y ahora implementaremos esas abstracciones en un nuevo proyecto. La forma en que implementaremos los repositorios será considerando el uso de alguna herramienta *ORM*, por ejemplo, *Entity Framework Core*, sin embargo, si lo deseáramos, podríamos realizar implementaciones que transformen los objetos de negocio en instrucciones SQL directamente para comunicarnos con el medio de persistencia.

1. Agrega un nuevo directorio llamado ***Gateways*** dentro del directorio ***3. Interface Adapters***.
2. Agrega un nuevo directorio llamado ***Repositories*** dentro del directorio *Gateways*.
3. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.Repositories*** en el directorio *Repositories*.
4. Elimina el archivo ***Class1.cs***.
5. Agrega la referencia del proyecto ***NorthWind.Sales.Backend.BusinessObjects***.

Crear las entidades del modelo de datos

1. Agrega un nuevo directorio llamado ***Entities*** en la raíz del proyecto. En este directorio definiremos las entidades con las cuales nos comunicaremos con la herramienta *ORM*.
2. Agrega un nuevo archivo de clase llamado ***OrderDetail.cs*** en el directorio *Entities* y remplaza el código de la clase por el siguiente código.

```
public class OrderDetail
{
    public int OrderId { get; set; }
    public Order Order { get; set; }
    public int ProductId { get; set; }
    public decimal UnitPrice { get; set; }
    public short Quantity { get; set; }
}
```

   La propiedad ***Order*** de esta clase nos permitirá establecer la relación entre una orden y su detalle. Para este caso de uso, la entidad *OrderDetail* junto con la entidad *Order* serán suficientes.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear la interfaces para los contextos de datos

Definamos ahora las abstracciones que utilizaremos para comunicarnos con la herramienta *ORM*.

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo llamado ***INorthWindSalesCommandsDataContext.cs*** de tipo *Interface* en el directorio *Interfaces* y remplaza el contenido de la interface por el siguiente.

```
public interface INorthWindSalesCommandsDataContext
{
    Task AddOrderAsync(Order order);
    Task AddOrderDetailsAsync(IEnumerable<Entities.OrderDetail> orderDetails);
    Task SaveChangesAsync();
}
```

El contexto de datos nos permitirá agregar al repositorio una orden, el detalle de la orden y nos permitirá guardar la información invocando al método *SaveChangesAsync*.

Por el momento, esta es la única abstracción de contexto de datos que necesitaremos.

Implementar el Repositorio

1. Agrega un nuevo directorio llamado ***Repositories*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***CommandsRepository.cs*** en el directorio *Repositories* y remplaza el contenido de la clase por el siguiente.

```
internal class CommandsRepository(INorthWindSalesCommandsDataContext context) :
    ICommandsRepository
{
    public async Task CreateOrder(OrderAggregate order)
    {
        await context.AddOrderAsync(order);
        await context.AddOrderDetailsAsync(
            order.OrderDetails
            .Select(d => new Entities.OrderDetail
            {
                Order = order,
                ProductId = d.ProductId,
                Quantity = d.Quantity,
                UnitPrice = d.UnitPrice

            }).ToArray());
    }

    public async Task SaveChanges() =>
        await context.SaveChangesAsync();
}
```

La implementación incluye el método para agregar la orden al repositorio y el método para realizar la persistencia. Todo ello a través del contexto de datos.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega el paquete NuGet `Microsoft.Extensions.DependencyInjection.Abstractions` al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el código del archivo con el siguiente código.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddRepositories(
        this IServiceCollection services)
    {
        services.AddScoped<ICommandsRepository, CommandsRepository>();

        return services;
    }
}
```

3. Importa los espacios de nombres en el archivo *GlobalUsings.cs*.

El Presenter

La función del *Presenter* es la de convertir los datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para algún agente externo como la Web.

Agreguemos la clase *Presenter* que se encargará de implementar el *Output Port* del caso de uso.

1. Agrega un nuevo directorio llamado ***Presenters*** en el directorio ***3. Interface Adapters***.
2. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.Presenters*** en el directorio *Presenters*.
3. Elimina el archivo ***Class1.cs***.
4. Agrega la referencia del proyecto ***NorthWind.Sales.Backend.BusinessObjects***.

Implementar el Presenter

1. Agrega un nuevo directorio llamado ***CreateOrder*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***CreateOrderPresenter.cs*** en el directorio *CreateOrder* y remplaza el código de la clase con el siguiente.

```
internal class CreateOrderPresenter : ICreateOrderOutputPort
{
    public int OrderId { get; private set; }

    public Task Handle(OrderAggregate addedOrder)
    {
        OrderId = addedOrder.Id;
        return Task.CompletedTask;
    }
}
```

   El *Interactor del caso de uso* invocará al método *Handle* definido por *ICreateOrderOutputPort* para enviar el resultado del caso de uso. El *Presentador* en este caso únicamente almacenará el valor recibido para que después sea obtenido por el *Controlador*. Este caso de uso específico no requiere de una transformación del resultado, por ejemplo, convertirlo a un formato JSON, XML o algún otro formato.
3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega el paquete NuGet `Microsoft.Extensions.DependencyInjection.Abstractions` al proyecto.

2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido con el siguiente código.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddPresenters(
        this IServiceCollection services)
    {
        services.AddScoped<ICreateOrderOutputPort,
            CreateOrderPresenter>();

        return services;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

El Controller

La función del *Controller* será la de proporcionar los datos de entrada al *Interactor* a través del *InputPort* para posteriormente obtener el resultado a través del *Presenter*.

Implementemos ahora el *Controlador* del caso de uso.

1. Crea un directorio llamado ***Controllers*** en el directorio ***3. Interface Adapters***.
2. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.Controllers*** en el directorio *Controllers*.
3. Elimina el archivo ***Class1.cs***.
4. Agrega la referencia del proyecto ***NorthWind.Sales.Backend.BusinessObjects***.
5. Agrega el siguiente código al archivo del proyecto para agregar una referencia del framework ***Microsoft.AspNetCore.app*** que contiene ensamblados de soporte de aplicaciones ASP.NET Core.

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.app" />
</ItemGroup>
```

Definir los Endpoints del código Backend

El código *backend* expondrá puntos de conexión para los clientes remotos. Para facilitar la definición de los nombres de los *endpoints*, crearemos una clase que contenga los nombres de los endpoints expuestos. Los nombres de los *endpoints* serán accesibles tanto por el código *backend* como por el código *frontend*, por lo tanto, definiremos una clase en la capa de entidades.

1. Agrega un nuevo directorio llamado ***ValueObjects*** en la raíz del proyecto ***NorthWind.Sales.Entities***.
2. Agrega un nuevo archivo de clase llamado ***Endpoints.cs*** en el directorio ***ValueObjects*** y remplaza el contenido de la clase por el siguiente.

```
public class Endpoints
{
    public const string CreateOrder = $"/{nameof(CreateOrder)}";
}
```

Implementar el Controller

1. Agrega un nuevo directorio llamado ***CreateOrder*** en la raíz del proyecto ***NorthWind.Sales.Backend.Controllers.***

2. Agrega un nuevo archivo de clase llamado ***CreateOrderController.cs*** en el directorio *Createorder* y remplaza el contenido del archivo con el siguiente código.

```
namespace Microsoft.AspNetCore.Builder;
public static class CreateOrderController
{
    public static WebApplication UseCreateOrderController(
        this WebApplication app)
    {
        app.MapPost(Endpoints.CreateOrder, CreateOrder);

        return app;
    }

    public static async Task<int> CreateOrder(
        CreateOrderDto orderDto,
        ICreateOrderInputPort inputPort,
        ICreateOrderOutputPort presenter)
    {
        await inputPort.Handle(orderDto);
        return presenter.OrderId;
    }
}
```

En el código anterior, estamos extendiendo la clase ***WebApplication*** para registrar los endpoints de la aplicación. También puedes notar que estamos colocando a la clase en el espacio de nombres `Microsoft.AspNetCore.Builder` para evitar el uso de la sentencia *using* en el código consumidor.

El método *UseCreateOrderController* envuelve al método *CreateOrder* que simplemente invoca al método *Handle* del *InputPort* y devuelve el resultado desde el *Presenter*.

UseCreateOrderController podrá ser consumido por un cliente remoto a través de la API Web mientras que *CreateOrder* podrá ser consumido por alguna aplicación de escritorio o backend, por ejemplo, una aplicación MVC.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Práctica 6:

Implementando la capa Frameworks And Drivers

La capa más externa generalmente se compone de frameworks y herramientas como la base de datos, el framework web, etc. Generalmente, no se escribe mucho código en esta capa que no sea el código necesario para comunicarse con el siguiente círculo hacia adentro.

Esta capa es donde van todos los detalles. La Web es considerada como un detalle, esto es, un *dispositivo de entrada y salida* de información, así como cualquier otra *Interfaz de Usuario* como la de una aplicación de consola, web, escritorio o móvil. La base de datos también es considerada como un detalle. Guardamos estas cosas en el exterior, donde pueden hacer poco daño.

Objetivos

En esta práctica los estudiantes:

- Implementaran los contextos de persistencia de datos utilizando *Entity Framework Core*.
- Implementarán un proyecto para realizar la inversión de control de los servicios backend.
- Implementarán la interfaz de usuario *Swagger* de una aplicación *Web API.*
- Implementarán la interfaz de usuario de un cliente *Blazor WebAssembly*.

Implementar los contextos de datos

Los repositorios utilizan un contexto de datos para realizar la persistencia de la información. Para persistir la información utilizaremos ***Microsoft SQL Server*** y el enfoque ***Code-First*** de ***Entity Framework Core*** para crear la base de datos. Por tal motivo, agregaremos un nuevo proyecto que centralizará la creación de la base de datos y los contextos de datos que la aplicación pueda necesitar.

1. Agrega un nuevo directorio llamado ***Frameworks*** en la raíz del directorio ***4. Frameworks And Drivers***.
2. Agrega un nuevo directorio llamado ***DataContexts*** en el directorio *Frameworks*.
3. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.DataContexts.EFCore*** en el directorio *DataContexts*.
4. Elimina el archivo ***Class1.cs***.
5. Agrega la referencia del proyecto ***NorthWind.Sales.Backend.Repositories***.
6. Agrega el paquete NuGet ***Microsoft.EntityFrameworkCore.SqlServer***.
7. Agrega el paquete NuGet ***Microsoft.EntityFrameworkCore.Tools***.

Crear la configuración de las entidades

Agreguemos ahora la configuración de las entidades del modelo de datos que permitirán a *Entity Framework Core* construir el esquema de la base de datos.

1. Agrega un nuevo directorio llamado ***Configurations*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***OrderConfiguration.cs*** en el directorio *Configurations* y remplaza el código de la clase con el siguiente código.

```
internal class OrderConfiguration :
        IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.Property(o => o.CustomerId)
            .IsRequired()
            .HasMaxLength(5)
            .IsFixedLength();

        builder.Property(o => o.ShipAddress)
            .IsRequired()
            .HasMaxLength(60);

        builder.Property(o => o.ShipCity)
            .HasMaxLength(15);

        builder.Property(o => o.ShipCountry)
```

```
            .HasMaxLength(15);

        builder.Property(o => o.ShipPostalCode)
            .HasMaxLength(10);
    }
}
```

Puedes notar que estamos especificando las características de cada propiedad de la entidad *Order* para crear los campos apropiados en la base de datos.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.
4. Agrega un nuevo archivo de clase llamado ***OrderDetailConfiguration.cs*** en el directorio *Configurations* y remplaza el código de la clase con el siguiente código.

```
internal class OrderDetailConfiguration :
    IEntityTypeConfiguration<Repositories.Entities.OrderDetail>
{
    public void Configure(
        EntityTypeBuilder<Repositories.Entities.OrderDetail> builder)
    {
        builder.HasKey(d => new { d.OrderId, d.ProductId });
        builder.Property(d => d.UnitPrice)
            .HasPrecision(8, 2);
    }
}
```

Puedes notar que únicamente estamos especificando la llave primaria y la precisión del campo *UnitPrice*.

Crear el contexto de datos para creación de la base de datos

Agreguemos ahora un contexto de datos que utilizaremos exclusivamente en tiempo de diseño para crear y modificar el esquema de la base de datos a través de migraciones de *Entity Framework Core*.

1. Agrega un nuevo directorio llamado ***DataContexts*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***NorthWindContext.cs*** en el directorio *DataContexts* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindContext : DbContext
{
    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=NorthWindDB");
    }

    public DbSet<Order> Orders { get; set; }
```

```
    public DbSet<Repositories.Entities.OrderDetail> OrderDetails { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            Assembly.GetExecutingAssembly());
    }
}
```

El contexto que estamos creando tiene el único propósito de permitir la creación de las migraciones de *Entity Framework Core* en tiempo de diseño, este contexto no será utilizado en producción por lo que estamos estableciendo la cadena de conexión como código duro.

Cuando necesitemos agregar más tablas a la base de datos, lo haremos a través de este contexto de datos.

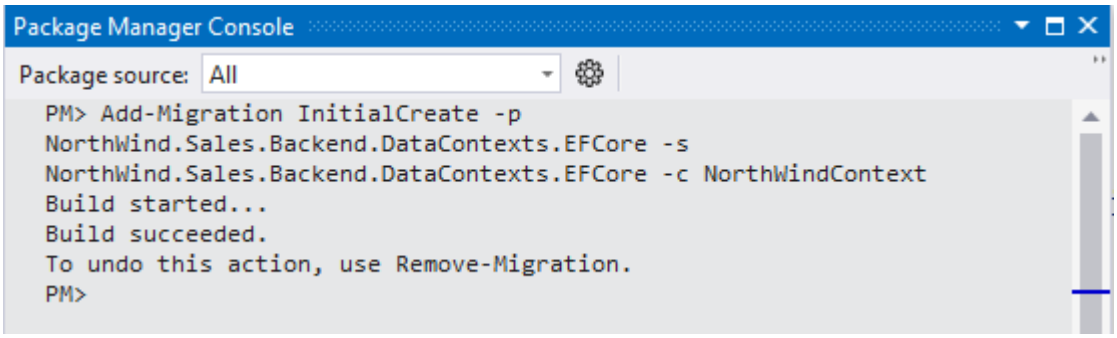
El código en el método *OnModelCreating* permitirá a las herramientas de *Entity Framework Core* aplicar la configuración de las entidades.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear la base de datos

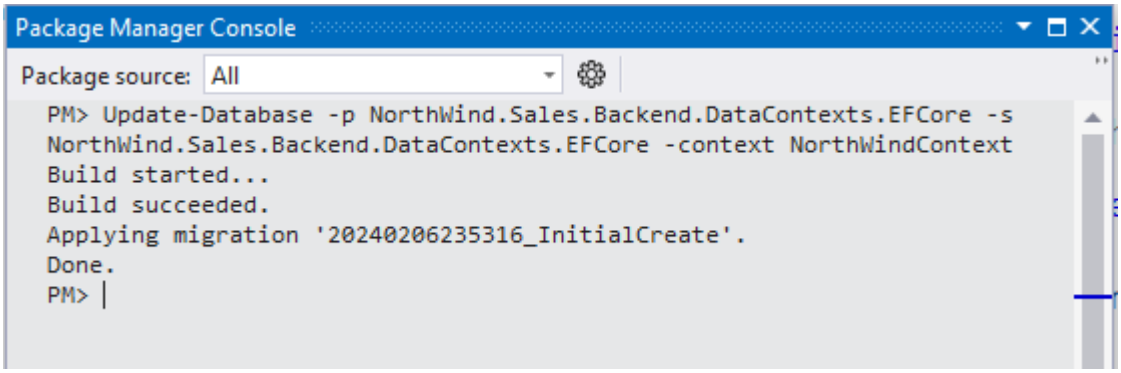
1. Ejecuta el siguiente comando en la *Consola de Administración de Paquetes NuGet* (*Package Manager Console*) para crear la migración inicial y verifica que no haya errores.

```
Add-Migration InitialCreate -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -c NorthWindContext
```



2. Ejecuta el siguiente comando para crear la base de datos y verifica que no haya errores.

```
Update-Database -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -context NorthWindContext
```



Agregar las opciones del contexto de base de datos

El contexto de base de datos requerirá de una cadena de conexión para poder conectarse al servidor de base de datos de la aplicación. Para poder obtener el valor de la cadena de conexión implementaremos el patrón *Options*.

1. Agrega un nuevo directorio llamado ***Options*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***DBOptions.cs*** en el directorio ***Options*** y remplaza el código de la clase con el siguiente.

```
public class DBOptions
{
    public const string SectionKey = nameof(DBOptions);
    public string ConnectionString { get; set; }
}
```

Por el momento únicamente necesitamos la cadena de conexión al servidor de la base de datos. El valor de la cadena de conexión podrá ser almacenada en la sección "*DBOptions*" de los valores de configuración de la aplicación.

Crear el contexto de datos para producción

Es momento de crear el contexto de datos que será utilizado en ambiente de producción.

1. Agrega un nuevo archivo de clase llamado ***NorthWindSalesContext.cs*** en el directorio *DataContexts* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindSalesContext(IOptions<DBOptions> dbOptions) : DbContext
{
    public DbSet<Order> Orders { get; set; }
    public DbSet<Repositories.Entities.OrderDetail> OrderDetails { get; set; }

    protected override void OnConfiguring(
```

```
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            dbOptions.Value.ConnectionString);
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(
            Assembly.GetExecutingAssembly());
    }
}
```

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar el contexto de datos para el repositorio

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***NorthWindSalesCommandsDataContext.cs*** en el directorio *Services* y remplaza el contenido de la clase por el siguiente.

```
internal class NorthWindSalesCommandsDataContext(
    IOptions<DBOptions> dbOptions) :
    NorthWindSalesContext(dbOptions),
    INorthWindSalesCommandsDataContext
{
    public async Task AddOrderAsync(Order order) =>
        await AddAsync(order);

    public async Task AddOrderDetailsAsync(
        IEnumerable<Repositories.Entities.OrderDetail> orderDetails) =>
        await AddRangeAsync(orderDetails);

    public async Task SaveChangesAsync() =>
        await base.SaveChangesAsync();
}
```

Puedes notar que la implementación del contexto de datos simplemente encapsula la funcionalidad del contexto de datos de *Entity Framework Core*. Esto nos permitiría remplazar *Entity Framework Core* por algún otro framework.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el código del archivo con el siguiente contenido.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddDataContexts(
        this IServiceCollection services,
        Action<DBOptions> configureDBOptions)
    {
        services.Configure(configureDBOptions);

        services.AddScoped<INorthWindSalesCommandsDataContext,
            NorthWindSalesCommandsDataContext>();

        return services;
    }
}
```

Puedes notar que estamos registrando las opciones de la conexión a la base de datos y la implementación del contexto de datos del dominio.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*

Agregar el proyecto para Inversión de Control global

Para facilitar el registro de dependencias en las aplicaciones cliente, por ejemplo, en una *Web API*, crearemos un proyecto que consolide los servicios que expone cada uno de los proyectos de nuestra solución *backend*.

1. Agrega un nuevo directorio llamado ***IoC*** en el directorio ***4. Frameworks And Drivers***.
2. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.IoC*** en el directorio *IoC*.
3. Elimina el archivo ***Class1.cs***.

Agregar la referencia de los proyectos

1. Agrega la referencia de los siguientes proyectos:
    a. NorthWind.Sales.Backend.UseCases
    b. NorthWind.Sales.Backend.Controllers
    c. NorthWind.Sales.Backend.Repositories
    d. NorthWind.Sales.Backend.Presenters
    e. NorthWind.Sales.Backend.DataContexts.EFCore

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido con el siguiente código.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddNorthWindSalesServices(
        this IServiceCollection services,
        Action<DBOptions> configureDBOptions)
    {
        services.AddUseCasesServices()
            .AddRepositories()
            .AddDataContexts(configureDBOptions)
            .AddPresenters();

        return services;
    }
}
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar el contenedor de Endpoints

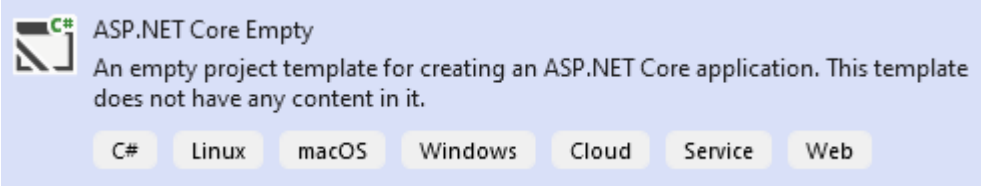
1. Agrega un nuevo archivo de clase llamado ***EndpointsContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo con el siguiente código.

```
namespace Microsoft.AspNetCore.Builder;
public static class EndpointsContainer
{
    public static WebApplication MapNorthWindSalesEndpoints(
        this WebApplication app)
    {
        app.UseCreateOrderController();
        return app;
    }
}
```

Crear la aplicación Web API

Para la creación de la *Web API* podríamos utilizar la plantilla ***ASP.NET Core Web API*** con la característica *Minimal API* o podríamos utilizar la plantilla ***ASP.NET Core Empty***. En este entrenamiento utilizaremos la plantilla ***ASP.NET Core Empty*** para ir describiendo cada elemento de la *Web API*.

1. Agrega un nuevo directorio llamado ***UI*** dentro del directorio ***4. Frameworks And Drivers***.
2. Utiliza la plantilla ***ASP.NET Core Empty*** con soporte *HTTPS* para agregar un nuevo proyecto llamado ***NorthWind.Sales.WebApi*** en el directorio *UI*.



3. Agrega la referencia del proyecto ***NorthWind.Sales.Backend.IoC***.
4. Agrega el paquete NuGet ***Swashbuckle.AspNetCore*** que contiene las herramientas de *Swagger* para documentar APIs en *ASP.NET Core*.

Agregar la clase auxiliar para configurar los servicios de la aplicación

1. Agrega un nuevo archivo de clase llamado ***Startup.cs*** en la raíz del proyecto y remplaza el contenido de la clase por el siguiente.

```
internal static class Startup
{
    public static WebApplication CreateWebApplication(
            this WebApplicationBuilder builder)
    {
        // Configurar APIExplorer para descubrir y exponer
        // los metadatos de los endpoints de la aplicación.
        builder.Services.AddEndpointsApiExplorer();

        // Agregar el generador que construye los objetos de
        // documentación de Swagger con la funcionalidad del APIExplorer.
        builder.Services.AddSwaggerGen();

        // Registrar los servicios de la aplicación
        builder.Services.AddNorthWindSalesServices(dbObtions =>
             builder.Configuration.GetSection(DBOptions.SectionKey)
             .Bind(dbObtions));

        // Agregar el servicio CORS para clientes que se ejecutan
        // en el navegador Web (como Blazor WebAssembly).
        builder.Services.AddCors(options =>
        {
```

```
            options.AddDefaultPolicy(config =>
            {
                config.AllowAnyMethod();
                config.AllowAnyHeader();
                config.AllowAnyOrigin();
            });
        });

        return builder.Build();
    }

    public static WebApplication ConfigureWebApplication(
        this WebApplication app)
    {
        // Habilitar el middleware para servir el documento
        // JSON generado y la interfaz UI de Swagger en el
        // ambiente de desarrollo.
        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        // Registrar los endpoints de la aplicación
        app.MapNorthWindSalesEndpoints();

        // Agregar el Middleware CORS
        app.UseCors();

        return app;
    }
}
```

El código anterior expone dos métodos de extensión para configurar los servicios y agregar los middlewares y endpoints de la Web API.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar la cadena de conexión

1. Agrega el siguiente código en el archivo ***appsettings.json*** para definir los valores de la sección *DBOptions* que contienen la cadena de conexión.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "DBOptions": {
    "ConnectionString": "Server=(localdb)\\mssqllocaldb;Database=NorthWindDB"
```

```
    }
}
```

Modificar el archivo Program.cs

1. Modifica el código del archivo ***Program.cs*** para que sea similar al siguiente.

```
WebApplication.CreateBuilder(args)
    .CreateWebApplication()
    .ConfigureWebApplication()
    .Run();
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

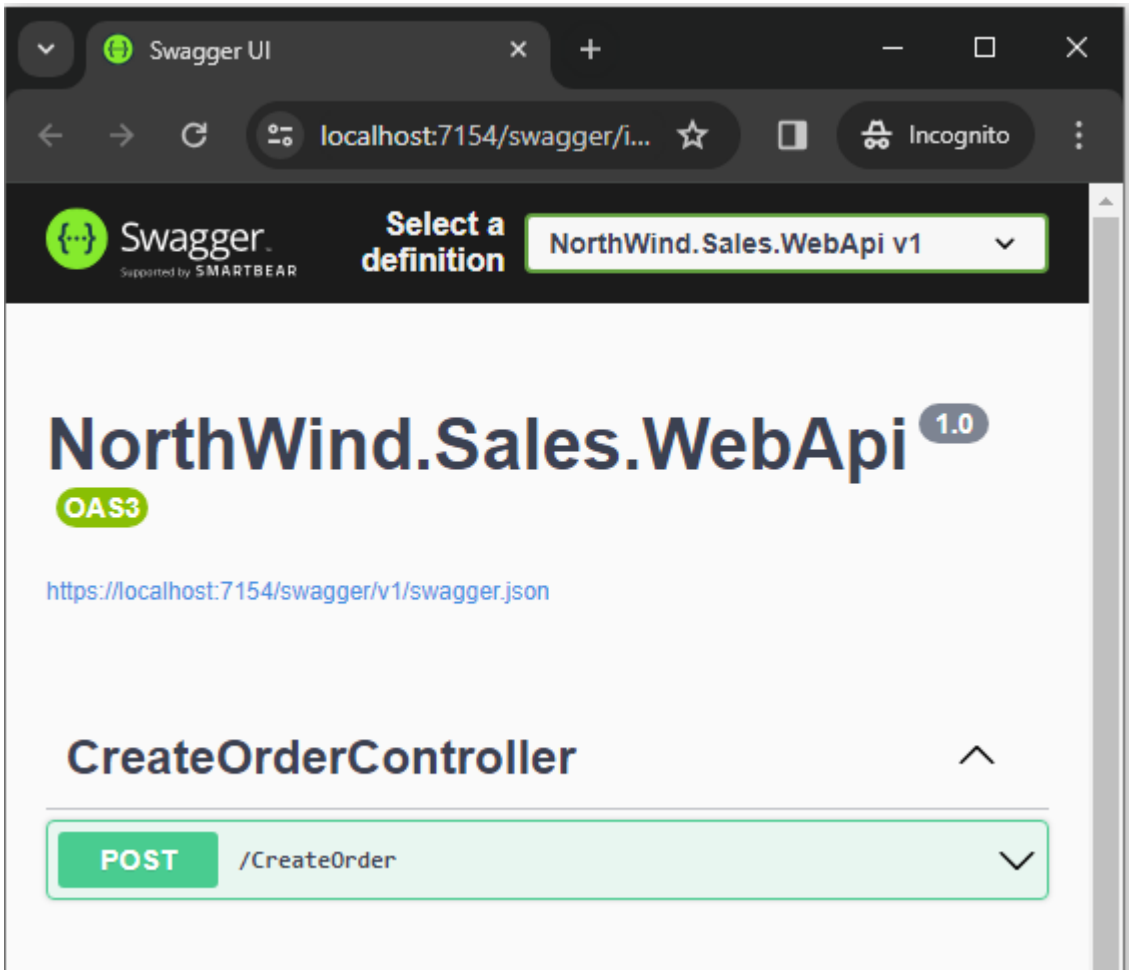
Probar la funcionalidad

Finalmente, podemos probar la funcionalidad de la Web API.

1. Establece el proyecto ***NorthWind.Sales.WebApi*** como proyecto de inicio.
2. Abre el archivo ***Properties\launchSettings.json***.
3. Agrega el siguiente código en el perfil "***https***" para establecer el endpoint ***swagger*** como Url de inicio.

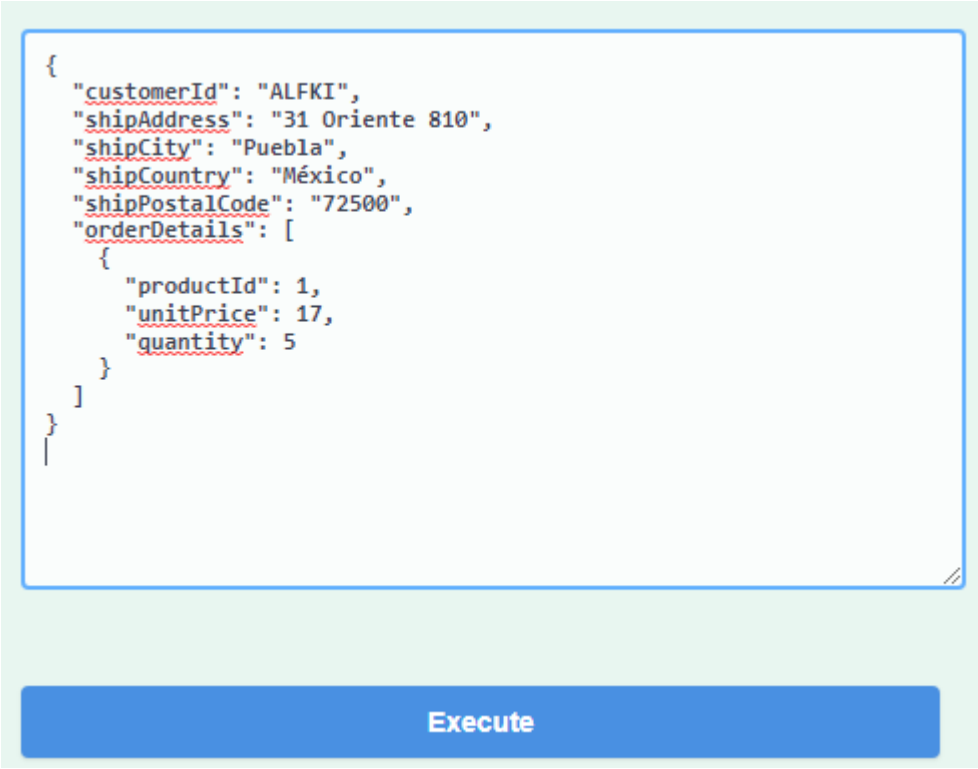
```
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7154;http://localhost:5254",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
```

4. Ejecuta la aplicación. Se mostrará la página ***index.html*** de ***swagger*** que mostrará el endpoint ***/CreateOrder***.



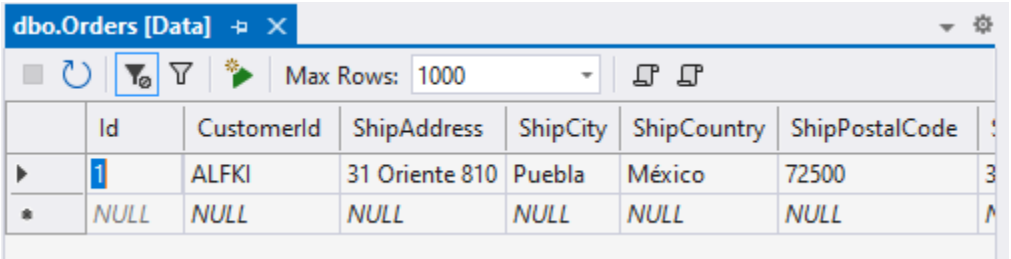
5. Haz clic en el botón ***POST /CreateOrder***.
6. Haz clic en el botón ***Try it out***.
7. Proporciona los siguientes datos para la orden.

```
{
  "customerId": "ALFKI",
  "shipAddress": "31 Oriente 810",
  "shipCity": "Puebla",
  "shipCountry": "México",
  "shipPostalCode": "72500",
  "orderDetails": [
    {
      "productId": 1,
      "unitPrice": 17,
      "quantity": 5
    }
  ]
}
```

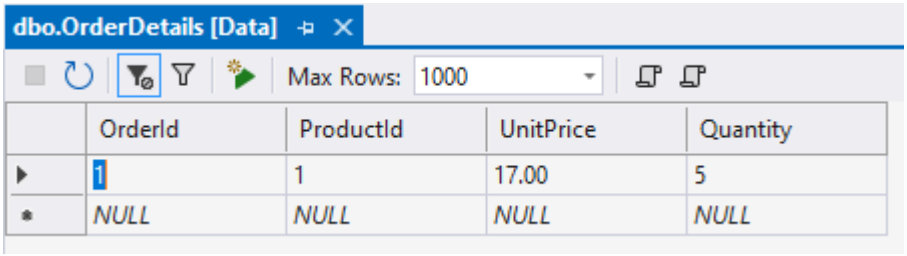


8. Haz clic en el botón ***Execute*** para enviar la petición. Podrás recibir la respuesta con el estado ***HTTP 200 OK*** y el número de la orden creada.

9. Verifica que la tabla ***Orders*** de la base de datos contenga los datos de la orden.



10. Verifica que la tabla ***OrderDetails*** de la base de datos contenga los datos de la orden.



11. Regresa a *Visual Studio* y detén la ejecución.

Crear la aplicación Blazor WebAssembly

Agreguemos ahora una aplicación *Blazor WebAssembly* para interactuar con el usuario.

1. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Frontend.BusinessObjects*** en el directorio ***2. Application Business Rules***.
2. Elimina el archivo ***Class1.cs***.
3. Agrega una referencia del proyecto ***NorthWind.Sales.Entities***.

Agregar los objetos de negocio

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo de *Interface* llamado ***ICreateOrderGateway.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface ICreateOrderGateway
{
    Task<int> CreateOrderAsync(CreateOrderDto order);
}
```

   Esta interface nos permitirá implementar una clase para encapsular el código cliente que consuma la Web API para crear la orden.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el Gateway

La capa *Interface Adapters* es donde colocaremos el código que consuma la Web API.

1. Agrega un nuevo directorio llamado ***WebApiGateways*** en el directorio ***3. Interface Adapters\Gateways***.
2. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Frontend.WebApiGateways*** en el directorio ***WebApiGateways***.
3. Elimina el archivo ***Class1.cs***.
4. Agrega la referencia del proyecto ***NorthWind.Sales.Frontend.BusinessObjects***.
5. Agrega un nuevo archivo de clase llamado ***CreateOrderGateway.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
internal class CreateOrderGateway(HttpClient client) : ICreateOrderGateway
{
    public async Task<int> CreateOrderAsync(CreateOrderDto order)
    {
        int OrderId = 0;
        var Response = await client.PostAsJsonAsync(
            Endpoints.CreateOrder, order);

        if (Response.IsSuccessStatusCode)
        {
            OrderId =
                await Response.Content.ReadFromJsonAsync<int>();
        }

        return OrderId;
    }
}
```

6. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias.

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.
2. Agrega el paquete NuGet ***Microsoft.Extensions.Http*** al proyecto. Este paquete contiene los tipos necesarios para implementar el patrón *HttpClient Factory*.
3. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

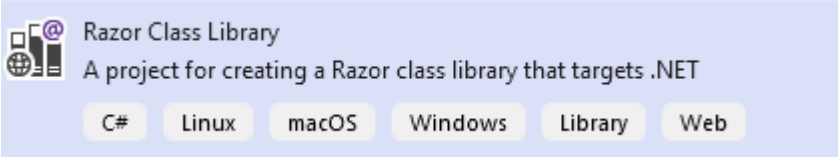
```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddWebApiGateways(
        this IServiceCollection services,
        Action<HttpClient> configureHttpClient)
    {
        services.AddHttpClient<ICreateOrderGateway,
            CreateOrderGateway>(configureHttpClient);

        return services;
    }
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

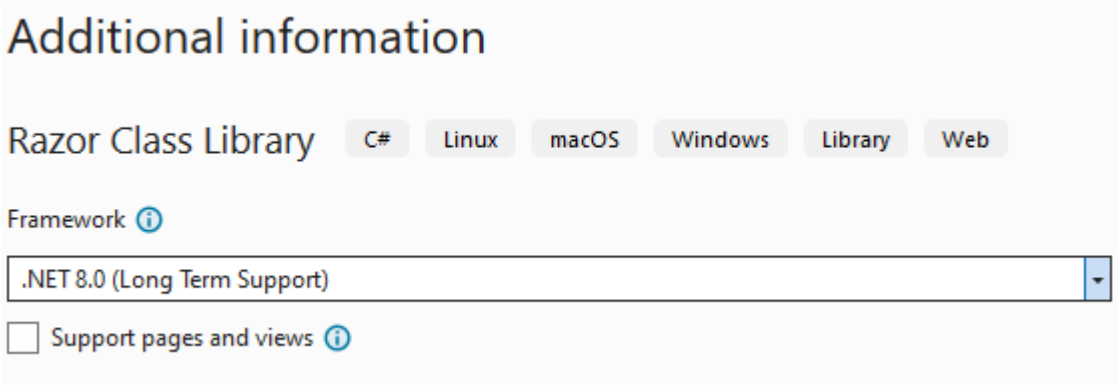
Agregar componentes Razor de proposito general

Agreguemos un proyecto independiente para implementar algunos componentes *Razor* de propósito general que podrán ser reutilizados en diferentes proyectos *Razor*.

1. Agrega un nuevo directorio llamado ***Views*** en el directorio ***3. Interface Adapters***.
2. Utiliza la plantilla ***Razor Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.RazorComponents*** en el directorio ***Views***.



   En la ventana ***Additional information*** asegúrate de no seleccionar la opción ***Support pages and views***.



3. Elimina los siguientes archivos:
   a. Component1.razor
   b. ExampleJsInterop.cs
   c. wwwroot\background.png
   d. wwwroot\exampleJsInterop.js

Agregar un componente para mostrar Excepciones

Agreguemos un componente *razor* que nos permita mostrar las posibles excepciones que puedan generarse durante la ejecución de la aplicación.

1. Agrega un nuevo directorio llamado ***Exceptions*** en la raíz del proyecto.

2. Utiliza la plantilla ***Razor Component*** para agregar un nuevo componente *razor* llamado ***ExceptionDisplay.razor*** en el directorio *Exceptions* y remplaza su contenido por el siguiente.

```
<div class="main alert alert-danger">
      <div>@Exception.Message</div>
</div>

@code {
      [Parameter]
      public Exception Exception { get; set; }
}
```

3. Agrega un nuevo archivo de tipo *Style Sheet* llamado ***ExceptionDisplay.razor.css*** en el directorio *Exceptions* con el siguiente contenido.

```
.main {
    overflow-wrap:anywhere;
}
```

Agregar componentes de Navegación

Agreguemos un componente que nos permita mostrar una barra de navegación con las distintas opciones de una aplicación.

Agregar un componente NavMenuItem

1. Agrega un nuevo directorio llamado ***NavBars*** en la raíz del proyecto.
2. Agrega un nuevo componente *razor* llamado ***NavMenuItem.razor*** en el directorio *NavBars* con el siguiente contenido.

```
<div class="nav-item px-3">
    <NavLink class="nav-link" href=@HRef Match=@Match>
        @ChildContent
    </NavLink>
</div>

@code {
    [Parameter]
    public RenderFragment ChildContent { get; set; }

    [Parameter]
    public string HRef { get; set; }

    [Parameter]
    public NavLinkMatch Match { get; set; }
}
```

Este componente nos permitirá especificar los elementos de la barra de navegación. Este componente simplemente envuelve al componente *NavLink* para aplicar un estilo CSS.

3. Agrega el siguiente código en el archivo ***_Imports.razor*** para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Routing
```

4. Agrega un nuevo archivo de tipo *Style Sheet* llamado ***NavMenuItem.razor.css*** en el directorio *NavBars* con el siguiente contenido.

```
.nav-item {
    font-size: 0.9rem;
    padding-top: 0.3rem;
}

    .nav-item ::deep a {
        color: #d7d7d7;
        border-radius: 4px;
        height: 3rem;
        display: flex;
        align-items: center;
        line-height: 3rem;
    }

        .nav-item ::deep a.active {
            background-color: rgba(255,255,255,0.37);
            color: white;
        }

        .nav-item ::deep a:hover {
            background-color: rgba(255,255,255,0.1);
            color: white;
        }
```

Agregar un componente NavMenu

1. Agrega un nuevo archivo de clase llamado ***NavMenu.razor.cs*** en el directorio *NavBars* y remplaza el contenido de la clase por el siguiente.

```
public partial class NavMenu
{
    [Parameter]
    public RenderFragment NavBarBrand { get; set; }

    [Parameter]
    public RenderFragment NavBarItems { get; set; }

    bool CollapseNavMenu = true;
```

```
    string NavMenuCssClass => CollapseNavMenu ? "collapse" : null;

    void ToggleNavMenu()
    {
        CollapseNavMenu = !CollapseNavMenu;
    }
}
```

Este componente renderizará una barra de navegación con un encabezado (*NavBarBrand*) y las opciones definidas por componentes *NavMenuItem* (*NavBarItems*).

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
3. Agrega un nuevo componente *razor* llamado ***NavMenu.razor*** en el directorio *NavBars* con el siguiente contenido.

```
<div class="top-row ps-3 navbar navbar-dark">
    <div class="container-fluid">
        @NavBarBrand
        <button title="Navigation menu" class="navbar-toggler"
                @onclick="ToggleNavMenu">
            <span class="navbar-toggler-icon"></span>
        </button>
    </div>
</div>

<div class="@NavMenuCssClass nav-scrollable" @onclick="ToggleNavMenu">
    <nav class="flex-column">
        @NavBarItems
    </nav>
</div>
```

4. Agrega un nuevo archivo de tipo *Style Sheet* llamado ***NavMenu.razor.css*** en el directorio *NavBars* con el siguiente contenido.

```
.navbar-toggler {
    background-color: rgba(255, 255, 255, 0.1);
}

@media (min-width: 641px) {
    .navbar-toggler {
        display: none;
    }

    .collapse {
        display: block;
    }

    .nav-scrollable {
        height: calc(100vh - 3.5rem);
        overflow-y: auto;
    }
```

```
}
```

Agregar un componente Layout

Agreguemos un componente que nos permita definir la estructura de las distintas páginas de la aplicación.

1. Agrega un nuevo directorio llamado ***Layouts*** en la raíz del proyecto.
2. Agrega un nuevo componente *Razor* llamado ***NavMenuLayout.razor*** el en directorio *Layouts* con el siguiente contenido.

```
@inherits LayoutComponentBase

<div class="page">
    <div class="sidebar">
        <NavMenu>
            <NavBarBrand>
                <SectionOutlet SectionId="NavBarBrandSection" />
            </NavBarBrand>
            <NavBarItems>
                <SectionOutlet SectionId="NavBarItemsSection" />
            </NavBarItems>
        </NavMenu>
    </div>

    <main>
        <div class="top-row px-4">
            <SectionOutlet SectionId="HeaderSection"/>
        </div>
        <article class="content px-4">
            @Body
        </article>
    </main>
</div>

@code{
    public static object NavBarBrandSection = new();
    public static object NavBarItemsSection = new();
    public static object HeaderSection = new();
}
```

El componente expone la sección para establecer el encabezado de la barra de navegación (*NavBarBrandSection*), la sección para establecer los elementos de navegación (*NavBarItemsSection*) y la sección para establecer un encabezado de página (*HeaderSection*).

3. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using Microsoft.AspNetCore.Components.Sections
```

```
@using NorthWind.RazorComponents.NavBars
```

4. Agrega un nuevo archivo de tipo *Style Sheet* llamado ***NavMenuLayout.razor.css*** en el directorio *Layouts* con el siguiente contenido.

```
.page {
    position: relative;
    display: flex;
    flex-direction: column;
}

main {
    flex: 1;
}

.content {
    padding-top: 1.1rem;
}

.sidebar {
    background-image: linear-gradient(180deg, rgb(5, 39, 103) 0%, #3a0647 70%);
}


.top-row {
    background-color: #f7f7f7;
    border-bottom: 1px solid #d6d5d5;
    justify-content: flex-end;
    height: 3.5rem;
    display: flex;
    align-items: center;
}
    .top-row ::deep a, .top-row ::deep .btn-link {
        white-space: nowrap;
        margin-left: 1.5rem;
        text-decoration: none;
    }
        .top-row ::deep a:hover, .top-row ::deep .btn-link:hover {
            text-decoration: underline;
        }

        .top-row ::deep a:first-child {
            overflow: hidden;
            text-overflow: ellipsis;
        }

@media (min-width: 641px) {
    .page {
        flex-direction: row;
    }

    .sidebar {
        width: 250px;
        height: 100vh;
        position: sticky;
        top: 0;
```

```
    }
}
```

Agregar las vistas de la aplicación

1. Utiliza la plantilla ***Razor Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.Frontend.Views*** en el directorio ***Views*** sin seleccionar la opción ***Support pages and views***.
2. Elimina los siguientes archivos:
    a. ExampleJsInterop.cs
    b. Component1.razor
    c. wwwroot\background.png
    d. wwwroot\exampleJsInterop.js
3. Agrega una referencia del proyecto ***NorthWind.Sales.Frontend.BusinessObjects***.
4. Agrega una referencia del proyecto ***NorthWind.RazorComponents***.

Agregar archivos de recursos

Agreguemos archivos de recursos que nos permitirán almacenar los mensajes que mostraremos en las vistas de la interfaz de usuario.

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo ***Resources File*** llamado ***CreateOrderMessages.resx*** en el directorio *Resources*.
3. Agrega los siguientes valores en el diseñador del archivo de recursos, estos mensajes serán utilizados en las vistas de creación de orden.

Nombre	Valor
CustomerIdLabel	Id del cliente
ShipAddressLabel	Dirección
ShipCityLabel	Ciudad
ShipCountryLabel	País
ShipPostalCodeLabel	Código Postal
ProductIdLabel	Id de Producto
UnitPriceLabel	Precio
QuantityLabel	Cantidad

AddProductButtonText	Agregar producto
SendButtonText	Enviar
CreatedOrderTemplate	Orden {0} creada.
CreateOrderMenuItemTitle	Crear orden
CreateOrderPageTitle	Crear orden de compra

4. Agrega un nuevo archivo de tipo ***Resources File*** llamado ***CommonMessages.resx*** en el directorio *Resources*.
5. Agrega los siguientes valores en el diseñador del archivo de recursos, los mensajes que podremos aquí serán utilizados por distintas vistas de la interfaz de usuario.

Nombre	Valor
OK	Aceptar
NavBarBrandTitle	NorthWind Sales
HomeMenuItemTitle	Inicio
AboutTitle	Acerca de

Para obtener información sobre la *localización* de contenido, pueden consultarse los siguientes enlaces:

Make an ASP.NET Core app's content localizable
https://learn.microsoft.com/aspnet/core/fundamentals/localization/make-content-localizable

Localization in .NET
https://learn.microsoft.com/dotnet/core/extensions/localization

Agregar los ViewModels

Agreguemos las clases *ViewModel* que utilizaremos para obtener la información desde la interfaz de usuario para enviarlos a la API Web utilizando los DTOs.

1. Agrega un nuevo directorio llamado ***ViewModels*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***CreateOrder*** en el directorio *ViewModels*.
3. Agrega un nuevo archivo de clase llamado ***CreateOrderDetailViewModel.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
public class CreateOrderDetailViewModel
```

```
{
    public int ProductId { get; set; }
    public decimal UnitPrice { get; set; }
    public short Quantity { get; set; }
}
```

Esta clase contendrá los datos que enviaremos a la API Web a través de *CreateOrderDetailDto*.

4. Agrega un nuevo archivo de clase llamado ***CreateOrderViewModel.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
public class CreateOrderViewModel(ICreateOrderGateway gateway)
{
    #region Propiedades relacionadas a CreateOrderDto
    public string CustomerId { get; set; }
    public string ShipAddress { get; set; }
    public string ShipCity { get; set; }
    public string ShipCountry { get; set; }
    public string ShipPostalCode { get; set; }
    public List<CreateOrderDetailViewModel> OrderDetails { get; set; } = [];
    #endregion

    public string InformationMessage { get; private set; }

    public void AddNewOrderDetailItem()
    {
        OrderDetails.Add(
            new CreateOrderDetailViewModel());
    }

    public async Task Send()
    {
        InformationMessage = "";
        var OrderId = await gateway.CreateOrderAsync(
            (CreateOrderDto)this);
        InformationMessage = string.Format(
            CreateOrderMessages.CreatedOrderTemplate, OrderId);
    }

    public static explicit operator CreateOrderDto(
        CreateOrderViewModel model) =>
        new CreateOrderDto(
            model.CustomerId, model.ShipAddress, model.ShipCity,
            model.ShipCountry, model.ShipPostalCode,
            model.OrderDetails.Select(d => new CreateOrderDetailDto(
                d.ProductId, d.UnitPrice, d.Quantity)
            ));
}
```

Esta clase contendrá los datos que enviaremos a la API Web a través de *CreateOrderDto.*

En el código anterior hemos definido el método `AddNewOrderDetailItem` que permitirá al usuario especificar los datos de un nuevo producto de la orden.

El método `Send` enviará los datos a la *API Web* a través del *Gateway* que creamos previamente y que recibimos en el constructor.

Puedes notar que hemos implementado una sobrecarga del operador `explicit` para poder convertir un tipo `CreateOrderViewModel` al tipo `CreateOrderDto`.

5. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el componente CreateOrder

Agreguemos un componente *razor* que permita construir la interfaz de usuario para poder proporcionar los datos de la orden que deseamos crear.

1. Agrega un nuevo directorio llamado ***Components*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***CreateOrderComponent.razor.cs*** en el directorio *Components* y remplaza el contenido de la clase por el siguiente.

```
public partial class CreateOrderComponent
{
    [Parameter]
     public CreateOrderViewModel Order { get; set; }
}
```

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.
4. Utiliza la plantilla *Razor Component* para agregar un nuevo componente *razor* llamado ***CreateOrderComponent.razor*** en el directorio *Components* y remplaza su contenido por el siguiente.

```
<EditForm Model=Order OnValidSubmit="Order.Send">
    <div class="form-group">
        <label for="customerId">
            @CreateOrderMessages.CustomerIdLabel
        </label>
        <InputText
            class="form-control" id="customerId"
            placeholder="@CreateOrderMessages.CustomerIdLabel"
            @bind-Value=Order.CustomerId/>
    </div>
    <div class="form-group">
        <label for="shipAddress">
            @CreateOrderMessages.ShipAddressLabel
        </label>
        <InputText
            class="form-control" id="shipAddress"
            placeholder="@CreateOrderMessages.ShipAddressLabel"
            @bind-Value=Order.ShipAddress />
```

```
    </div>
    <div class="form-group">
        <label for="shipCity">
            @CreateOrderMessages.ShipCityLabel
        </label>
        <InputText
            class="form-control" id="shipCity"
            placeholder="@CreateOrderMessages.ShipCityLabel"
            @bind-Value=Order.ShipCity />
    </div>
    <div class="form-group">
        <label for="shipCountry">
            @CreateOrderMessages.ShipCountryLabel
        </label>
        <InputText
            class="form-control" id="shipCountry"
            placeholder="@CreateOrderMessages.ShipCountryLabel"
            @bind-Value=Order.ShipCountry />
    </div>
    <div class="form-group">
        <label for="shipPostalCode">
            @CreateOrderMessages.ShipPostalCodeLabel
        </label>
        <InputText
            class="form-control" id="shipPostalCode"
            placeholder="@CreateOrderMessages.ShipPostalCodeLabel"
            @bind-Value=Order.ShipPostalCode />
    </div>
    <hr />
    <div class="row">
        <div class="col">
            @CreateOrderMessages.ProductIdLabel
        </div>
        <div class="col">
            @CreateOrderMessages.UnitPriceLabel
        </div>
        <div class="col">
            @CreateOrderMessages.QuantityLabel
        </div>
    </div>

    @foreach (var Item in Order.OrderDetails)
    {
        <div class="row">
            <div class="col">
                <InputNumber
                    class="form-control"
                    placeholder="@CreateOrderMessages.ProductIdLabel"
                    @bind-Value="Item.ProductId" />
            </div>
            <div class="col">
                <InputNumber
                    class="form-control"
                    placeholder="@CreateOrderMessages.UnitPriceLabel"
                    @bind-Value="Item.UnitPrice" />
            </div>
            <div class="col">
                <InputNumber
```

```
                    class="form-control"
                    placeholder="@CreateOrderMessages.QuantityLabel"
                @bind-Value="Item.Quantity" />
            </div>
        </div>
    }

    <div class="row p-3">
        <div class="col">
            <button type="button" class="btn btn-primary"
            @onclick=Order.AddNewOrderDetailItem>
                @CreateOrderMessages.AddProductButtonText
            </button>
        </div>
        <div class="col">
            <button class="btn btn-primary">
                @CreateOrderMessages.SendButtonText
            </button>
        </div>
    </div>
</EditForm>

<hr />
@if (!string.IsNullOrWhiteSpace(Order.InformationMessage))
{
    <div class="alert alert-info">
        @Order.InformationMessage
    </div>
}
```

5. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Forms
```

Agregar la página de creación de orden

Agreguemos ahora la página de creación de la orden.

1. Agrega un nuevo directorio llamado ***Pages*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***CreateOrder.razor.cs*** en el directorio *Pages* y remplaza el código de la clase por el siguiente código.

```
public partial class CreateOrder
{
      [Inject]
      CreateOrderViewModel ViewModel { get; set; }

      ErrorBoundary ErrorBoundaryRef;

      void Recover()
```

```
        {
                ErrorBoundaryRef?.Recover();
        }
}
```

En el código anterior estamos definiendo una variable para almacenar la referencia de un componente ***ErrorBoundary***. El componente *ErrorBoundary* nos permite atrapar excepciones no manejadas que pueden generarse durante la ejecución de la aplicación. El método ***Recover*** nos permitirá restablecer el estado de error indicando al componente que ya hemos manejado el error.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. Utiliza la plantilla *Razor Component* para agregar un nuevo componente *razor* llamado ***CreateOrder.razor*** en el directorio *Pages* y remplaza su contenido por el siguiente.

```
@page "/createorder"

<PageTitle>@CreateOrderMessages.CreateOrderPageTitle</PageTitle>

<ErrorBoundary @ref=ErrorBoundaryRef>
        <ChildContent>
                <CreateOrderComponent Order=ViewModel/>
        </ChildContent>
        <ErrorContent>
                <ExceptionDisplay Exception=context />
                <button class="btn btn-danger" @onclick=Recover>
                        @CommonMessages.OK
            </button>
        </ErrorContent>
</ErrorBoundary>
```

5. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using NorthWind.RazorComponents.Exceptions
@using NorthWind.Sales.Frontend.Views.Components
```

Agregar un componente Layout

1. Agrega un nuevo directorio llamado ***Layout*** en la raíz del proyecto.
2. Agrega un nuevo componente *razor* llamado ***MainLayout.razor*** el en directorio *Layout* con el siguiente contenido.

```
@inherits NavMenuLayout
```

Este componente hereda del componente *NavMenuLayout* que creamos previamente.

3. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using NorthWind.RazorComponents.Layouts
```

Agreguemos ahora las secciones definidas en el componente base *NavMenuLayout*.

4. Agrega el siguiente código en el archivo *MainLayout.razor* para establecer el valor de la sección *NavBarBrandSection*.

```
<SectionContent SectionId="NavBarBrandSection">
    <a class="navbar-brand" href="">
        @CommonMessages.NavBarBrandTitle
    </a>
</SectionContent>
```

5. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Sections
```

6. Agrega el siguiente código en el archivo *MainLayout.razor* para establecer el valor de la sección *NavBarItemsSection*.

```
<SectionContent SectionId="NavBarItemsSection">

</SectionContent>
```

7. Agrega el siguiente código dentro del elemento *SectionContent* anterior para definir el elemento de menú de la página de inicio.

```
<NavMenuItem HRef="" Match="NavLinkMatch.All">

</NavMenuItem>
```

8. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using NorthWind.RazorComponents.NavBars
```

```
@using Microsoft.AspNetCore.Components.Routing
```

9. Agrega el siguiente código dentro del elemento *NavMenuItem* anterior para establecer el ícono que será mostrado en el elemento de menú.

```
<svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
fill="currentColor" class="bi bi-house-door-fill" viewBox="0 0 16 16">
    <path d="M6.5 14.5v-3.505c0-.245.25-.495.5-.495h2c.25 0 .5.25.5.5v3.5a.5.5 0
0 0 .5.5h4a.5.5 0 0 0 .5-.5v-7a.5.5 0 0 0-.146-.354L13 5.793V2.5a.5.5 0 0 0-.5-
.5h-1a.5.5 0 0 0-.5.5v1.293L8.354 1.146a.5.5 0 0 0-.708 0l-6 6A.5.5 0 0 0 1.5
7.5v7a.5.5 0 0 0 .5.5h4a.5.5 0 0 0 .5-.5" />
</svg>
```

Este código corresponde al ícono `House door fill` que puede ser obtenido desde el enlace https://icons.getbootstrap.com/icons/house-door-fill/.

10. Agrega el siguiente código a continuación del código anterior para establecer el texto del elemento de menú.

```
@CommonMessages.HomeMenuItemTitle
```

11. Agrega el siguiente código después del elemento *NavMenuItem* anterior para definir el elemento de menú de creación de orden.

```
<NavMenuItem HRef="CreateOrder">

</NavMenuItem>
```

12. Agrega el siguiente código dentro del elemento *NavMenuItem* anterior para establecer el ícono que será mostrado en el elemento de menú.

```
<svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
fill="currentColor" class="bi bi-card-checklist" viewBox="0 0 16 16">
    <path d="M14.5 3a.5.5 0 0 1 .5.5v9a.5.5 0 0 1-.5.5h-13a.5.5 0 0 1-.5-.5v-
9a.5.5 0 0 1 .5-.5zm-13-1A1.5 1.5 0 0 0 0 3.5v9A1.5 1.5 0 0 0 1.5 14h13a1.5 1.5 0
0 0 1.5-1.5v-9A1.5 1.5 0 0 0 14.5 2z" />
    <path d="M7 5.5a.5.5 0 0 1 .5-.5h5a.5.5 0 0 1 0 1h-5a.5.5 0 0 1-.5-.5m-1.496-
.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-.708 0l-.5-.5a.5.5 0 1 1 .708-
.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0M7 9.5a.5.5 0 0 1 .5-.5h5a.5.5 0 0 1 0
1h-5a.5.5 0 0 1-.5-.5m-1.496-.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-.708 0l-
.5-.5a.5.5 0 0 1 .708-.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0" />
</svg>
```

Este código corresponde al ícono `Card checklist` que puede ser obtenido desde el enlace https://icons.getbootstrap.com/icons/card-checklist/.

13. Agrega el siguiente código a continuación del código anterior para establecer el texto del elemento de menú.

```
@CreateOrderMessages.CreateOrderMenuItemTitle
```

14. Agrega el siguiente código después del elemento ***SelectionContent*** anterior para establecer el valor de la sección *HeaderSection*.

```
<SectionContent SectionId="HeaderSection">
    <a href="https://learn.microsoft.com/aspnet/core/"
       target="_blank">
       @CommonMessages.AboutTitle
    </a>
</SectionContent>
```

15. Agrega el siguiente código después del elemento *SectionContent* anterior para forzar la renderización del contenido del componente base *NavMenuLayout*.

```
@((RenderFragment)(builder => base.BuildRenderTree(builder)))
```

Sin este código, el contenido del *layout* no sería renderizado y simplemente se mostraría una página en blanco al ejecutar la aplicación.

16. Agrega un nuevo archivo de tipo *Style Sheet* llamado ***MainLayout.razor.css*** en el directorio *Layout* con el siguiente contenido.

```
.bi {
    display: inline-block;
    position: relative;
    width: 1.25rem;
    height: 1.25rem;
    margin-right: 0.75rem;
    top: -1px;
    background-size: cover;
}
```

Agregar el contenedor de dependencias.

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
```

```
{
    public static IServiceCollection AddViewsServices(
        this IServiceCollection services)
    {
        services.AddScoped<CreateOrderViewModel>();

        return services;
    }
}
```

Agregar el proyecto para Inversión de Control global

Para facilitar el registro de dependencias en la aplicación cliente *Blazor*, crearemos un proyecto que consolide los servicios que expone cada uno de los proyectos de nuestra solución frontend.

1. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Frontend.IoC*** en el directorio ***4. Frameworks And Drivers\IoC***.
2. Elimina el archivo ***Class1.cs***.
3. Agrega la referencia de los siguientes proyectos:
    a. NorthWind.Sales.Frontend.WebApiGateways
    b. NorthWind.Sales.Frontend.Views

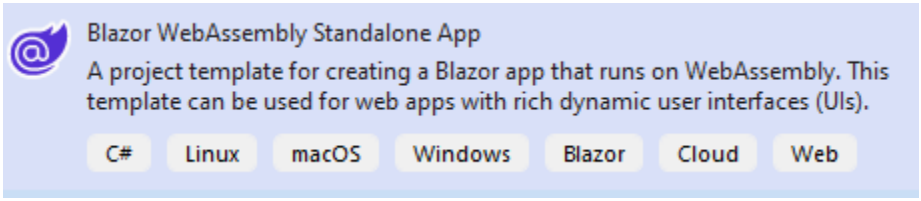
Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido con el siguiente código.

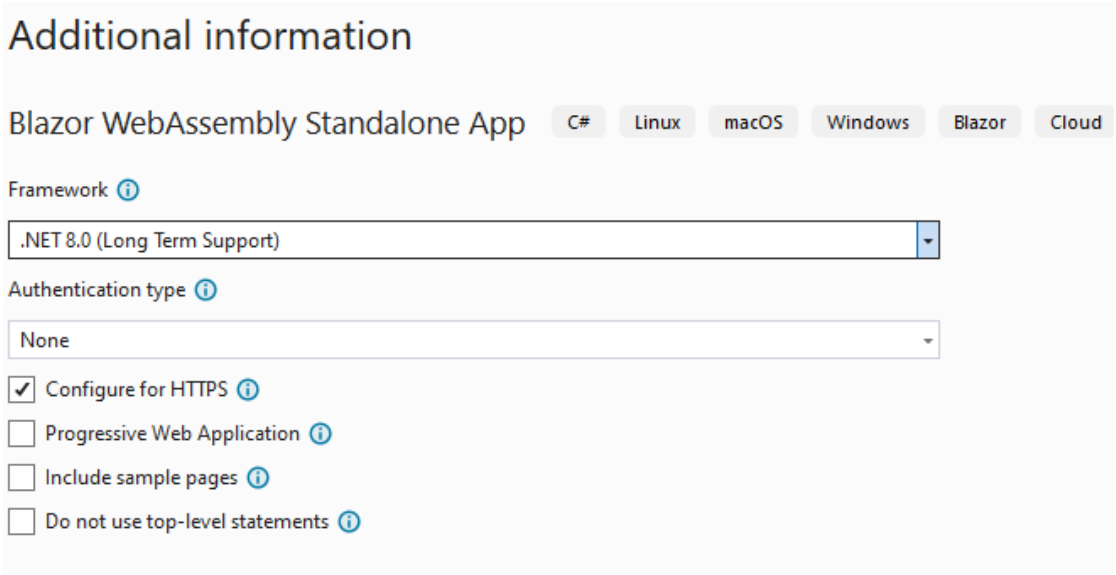
```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
      public static IServiceCollection AddNorthWindSalesServices(
             this IServiceCollection services,
             Action<HttpClient> configureHttpClient)
      {
             services.AddWebApiGateways(configureHttpClient)
                    .AddViewsServices();

             return services;
      }
}
```

Agregar el cliente Blazor WebAssembly

1. Utiliza la plantilla ***Blazor WebAssembly Standalone App*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.BlazorClient*** en el directorio ***4. Frameworks And Drivers\UIs***.



En la ventana ***Additional information*** verifica que solo esté seleccionada la opción ***Configure for HTTPS***.



2. Agrega la referencia del proyecto ***NorthWind.Sales.Frontend.IoC***.
3. Elimina el archivo ***MainLayout.razor*** del directorio *Layout*.
4. Elimina el siguiente código del archivo ***_Imports.razor***.

```
@using NorthWind.Sales.BlazorClient.Layout
```

Registrar los servicios

1. Agrega el siguiente código antes de la última línea de código del archivo ***Program.cs***.

```
builder.Services.AddNorthWindSalesServices(client =>
{
      client.BaseAddress =
      new Uri(builder.Configuration["WebApiAddress"]);
});
```

2. Elimina la siguiente línea de código.

```
builder.Services.AddScoped(sp => new HttpClient { BaseAddress = new
Uri(builder.HostEnvironment.BaseAddress) });
```

3. Importa los espacios de nombres requeridos en el archivo *GlobaUsings.cs*.

   El contenido del archivo ***Program.cs*** será similar al siguiente.

```
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

builder.Services.AddNorthWindSalesServices(client =>
{
      client.BaseAddress =
      new Uri(builder.Configuration["WebApiAddress"]);
});

await builder.Build().RunAsync();
```

Agregar el Uri de la Web Api

1. Agrega un nuevo archivo JSON llamado ***appsettings.json*** en el directorio ***wwwroot*** con el siguiente contenido utilizando la dirección de tu API Web.

```
{
  "WebApiAddress": "https://localhost:7154"
}
```

Registrar el ensamblado con las vistas

1. Abre el archivo ***App.razor***.
2. Agrega el siguiente código al final del archivo.

```
@code{
    Assembly[] AdditionalAssemblies =>
        new Assembly[]
        {
            typeof(NorthWind.Sales.Frontend.Views.Pages.CreateOrder)
            .Assembly
        };
}
```

   El código anterior devuelve un arreglo de ensamblados con un único elemento, el ensamblado que contiene la página *CreateOrder*.

3. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using System.Reflection
@using NorthWind.Sales.Frontend.Views.Layout
```

4. Modifica la etiqueta ***Router*** para agregar el ensamblado que contiene las vistas de la aplicación *Blazor*. El código será similar al siguiente.

```
<Router AppAssembly="@typeof(App).Assembly"
        AdditionalAssemblies=AdditionalAssemblies>
```

Agregar la referencia del framework Bootstrap

1. Abre el archivo ***index.html*** del directorio ***wwwroot***.
2. Agrega el siguiente código después del elemento *<base>* de la sección `<head>` del archivo *index.html*.

```
<link
 rel="stylesheet"
 href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.css">
```

3. Quita el comentario de la siguiente línea de código.

```
<link href="NorthWind.Sales.BlazorClient.styles.css" rel="stylesheet" />
```

El contenido del elemento `<head>` será similar al siguiente.

```
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>NorthWind.Sales.BlazorClient</title>
    <base href="/" />
    <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.css">

    <link rel="stylesheet" href="css/app.css" />
    <!-- If you add any scoped CSS files, uncomment the following to load them-->
    <link href="NorthWind.Sales.BlazorClient.styles.css" rel="stylesheet" />
</head>
```

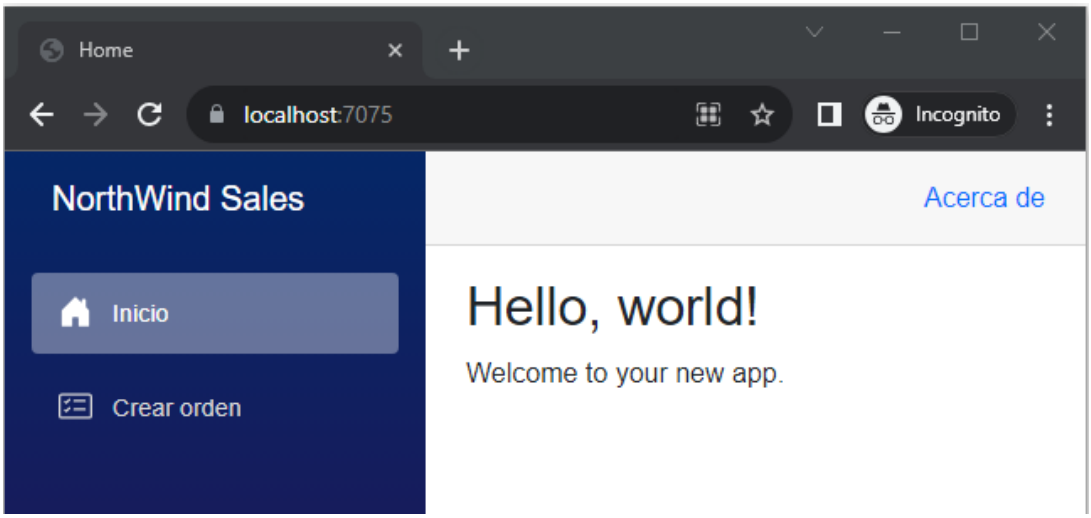
Agregar estilos CSS de la aplicación

1. Abre el archivo ***wwwroot\css\app.css***.
2. Agrega el siguiente contenido al inicio del archivo.

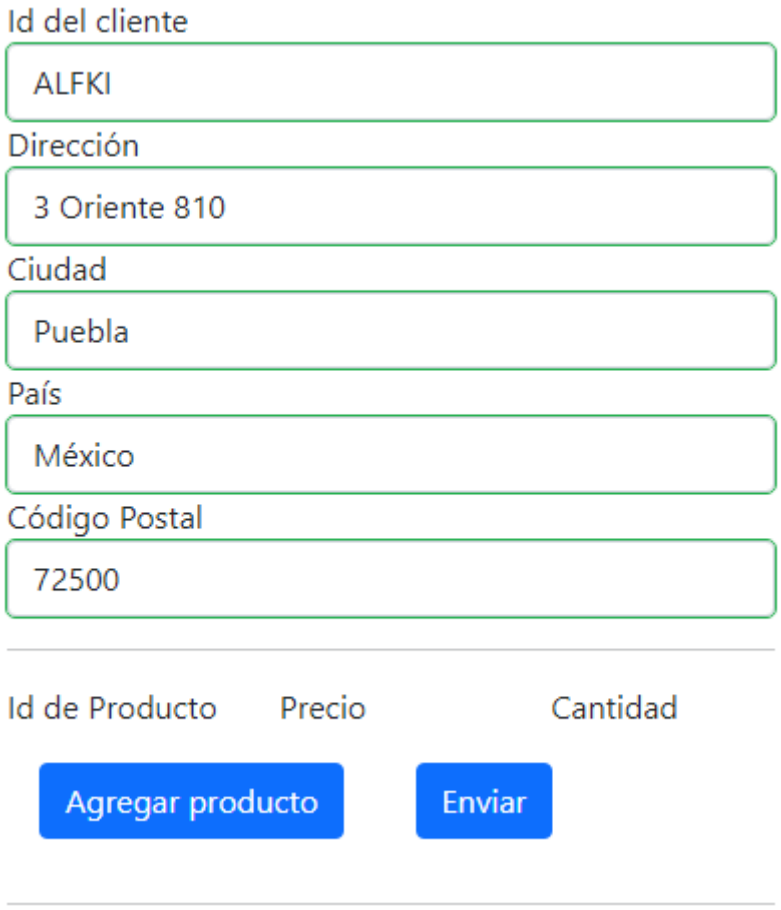
```
html, body {
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
}

h1:focus {
    outline: none;
}
```

Probar la funcionalidad

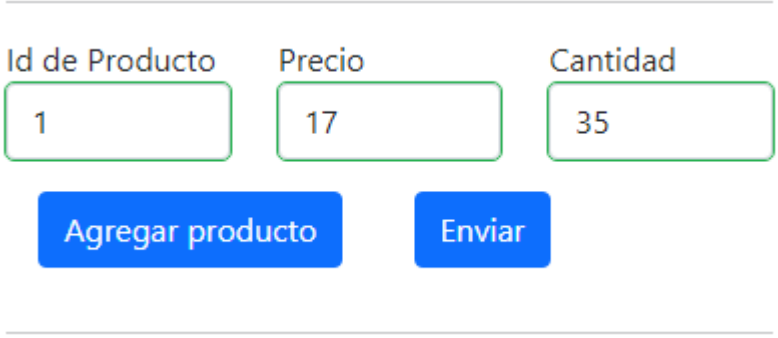
1. Establece los proyectos Web API y Blazor como proyectos de inicio.
2. Ejecuta la aplicación. Se mostrará una página similar a la siguiente.



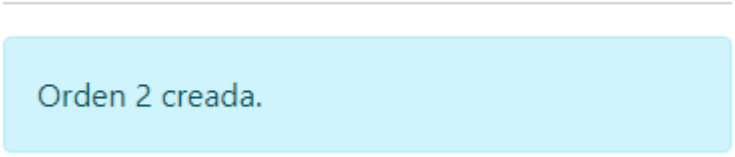
3. Haz clic en enlace *Crear Orden*.
4. Proporciona los datos de la orden a crear.



5. Haz clic en el botón ***Agregar producto*** para poder agregar el detalle de la orden.
6. Proporciona los datos del producto.



7. Haz clic en el botón ***Enviar*** para crear la orden. Podrás ver un resultado similar al siguiente.



8. Regresa a Visual Studio y detén la ejecución.

En este punto hemos creado 2 aplicaciones simples implementando *Clean Architecture*: una aplicación *Web API* y una aplicación *Blazor Web Assembly*.

Hemos dado los primeros pasos en el desarrollo de aplicaciones .NET implementando *Clean Architecture*. En lecciones posteriores agregaremos funcionalidad adicional al caso de uso, esto implicará agregar más abstracciones e implementaciones en las capas de nuestra solución.

¡Empecemos agregando *validación* de los datos de entrada en nuestra siguiente lección!

Práctica 7:

Implementando validación

La validación de los datos de entrada es algo común que debemos implementar en la mayoría de las aplicaciones. En esta lección modificaremos el caso de uso para agregar criterios de validación de los datos de entrada y mostraremos la forma de implementar esa validación en una aplicación implementando *Clean Architecture*.

Para implementar la validación en la aplicación, implementaremos un framework de validación que podrá ser consumido por diversas aplicaciones. Utilizaremos la biblioteca de validación para .NET ***FluentValidation*** como una herramienta de validación y demostraremos la forma de utilizar bibliotecas o frameworks de terceros como simples herramientas que pueden ser remplazadas fácilmente por alguna otra herramienta sin afectar a otros componentes de la aplicación.

Objetivos

En esta práctica los estudiantes:

- Implementarán validación de los datos de entrada.
- Utilizarán la biblioteca *FluentValidation* como herramienta de validación.
- Realizarán validación buscando valores existentes en la base de datos.

El Caso de uso: Crear orden de compra

Agreguemos algunos criterios de validación al caso de uso inicial para mostrar una forma en la que podemos implementar validación en *Clean Architecture*.

Historia de Usuario: Crear orden de compra

"*Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén*".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema valida los datos. (nuevo requerimiento)
3. El sistema registra la orden de compra.
4. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación (Nuevo requerimiento)

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Validaciones (Nuevo requerimiento)

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.
- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.

Implementar un framework de validación

Empecemos por implementar un framework de validación que contenga las abstracciones y servicios para que las aplicaciones que lo consuman puedan crear clases de validación que satisfagan reglas de negocio.

La validación puede ser realizada a través de una serie de instrucciones ***If*** o puede ser implementada utilizando alguna herramienta propia o de terceros. En esta lección utilizaremos la biblioteca ***FluentValidation*** como una herramienta de validación que quedará encapsulada en el framework sin exponerla fuera de este.

FluentValidation es una biblioteca de validación para .NET de uso gratuito y de código abierto para crear reglas de validación fuertemente tipadas. *FluentValidation* nos ayuda a implementar validaciones limpias, fáciles de crear y mantener. Incluso funciona en modelos externos de los que no tenemos acceso a su código fuente. Con esta biblioteca, podemos separar las clases modelo de la lógica de validación como sugiere una arquitectura limpia. No hace falta que las clases modelo se "ensucien" como lo hacen las anotaciones de datos. Además, un mejor control de la validación es algo que hace que los desarrolladores prefieran *FluentValidation*.

FluentValidation utiliza *expresiones lambda* para construir reglas de validación.

Es probable que, para sistemas pequeños, la estrategia de validación recomendada sea *Anotaciones de Datos* debido a su facilidad de implementación. Sin embargo, para sistemas más grandes y complejos, la recomendación es separar la responsabilidad de la validación utilizando objetos de validación con *Validación Fluida*.

Para obtener más información sobre la biblioteca ***FluentValidation*** puede consultarse el siguiente enlace:

FluentValidation
https://docs.fluentvalidation.net/en/latest/index.html

Definir las entidades de validación

Debido a que la validación puede ser requerida en diversas aplicaciones tal como la aplicación *Web API* o la aplicación *Blazor*, agregaremos un nuevo proyecto en la capa de *Reglas de negocio Empresariales* para definir las entidades de validación.

Antes de empezar a realizar cambios en la solución actual, si lo deseas, puedes realizar un respaldo de la solución actual para que puedas tener los distintos estados de la solución.

1. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Validation.Entities*** en el directorio ***1. Enterprise Business Rules***.
2. Elimina el archivo ***Class1.cs***.

Agregar una enumeración

1. Agrega un nuevo directorio llamado ***Enums*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ValidationConstraint.cs*** en el directorio *Enums* y remplaza el código de la clase por el siguiente.

```
public enum ValidationConstraint
{
    AlwaysValidate,
    ValidateIfThereAreNoPreviousErrors
}
```

Cuando se registren distintas clases de validación para un modelo, esta enumeración nos permitirá especificar si un validador se ejecutara siempre o solo cuando no haya errores previos de algún otro validador.

Agregar un ValueObject

1. Agrega un nuevo directorio llamado ***ValueObjects*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ValidationError.cs*** en el directorio *ValueObjects* y remplaza el código de la clase por el siguiente.

```
public class ValidationError(string propertyName, string message)
{
    public string PropertyName => propertyName;
    public string Message => message;
}
```

Esta clase nos permitirá identificar un error de validación de alguna propiedad de un objeto validado.

Agregar Interfaces

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo de *Interface* llamado ***IModelValidator.cs*** en el directorio *Interfaces* y remplaza el contenido de la interface por el siguiente.

```
public interface IModelValidator<T>
{
    ValidationConstraint Constraint { get; }
    IEnumerable<ValidationError> Errors { get; }
    Task<bool> Validate(T model);
}
```

El código anterior define la estructura que deberán tener todas las clases utilizadas para validación. Una clase que valide un modelo ofrecerá un método *Validate* que reciba el modelo a validar y devolverá el valor `true` en caso de que no haya errores de validación. Si el valor devuelto no es `true`, la colección `Errors` tendrá la información sobre los errores encontrados en cada propiedad del modelo.

La propiedad `Constraint` permitirá indicar si la validación debe o no ejecutarse cuando haya errores previos de validación en algún otro validador previo del mismo modelo. Esto puede ser útil en el caso de tener un validador que valide los datos de entrada y otro que valide los datos de entrada contra valores de una base de datos. En caso de que los datos de entrada sean incorrectos, no tiene sentido validar contra la base de datos.

3. Importa los espacios de nombres requerido en el archivo *GlobalUsings.cs*.
4. Agrega un nuevo archivo de *Interface* llamado ***IModelValidatorHub.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IModelValidatorHub<T>
{
    Task<bool> Validate(T model);
    IEnumerable<ValidationError> Errors { get; }
}
```

El servicio que hemos definido expondrá la misma funcionalidad que un validador, pero sin el requerimiento de validación. La implementación de este servicio permitirá ejecutar el código de validación de cada uno de los validadores registrados de un determinado modelo.

5. Agrega un nuevo archivo de *interface* llamado ***IValidationRules.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IValidationRules<T, TProperty>
{
    IValidationRules<T, TProperty> NotEmpty(string errorMessage);
    IValidationRules<T, TProperty> NotNull(string errorMessage);
    IValidationRules<T, TProperty> Must(Func<TProperty, bool> predicate,
        string errorMessage);
    IValidationRules<T, TProperty> StopOnFirstFailure();
    IValidationRules<T, TProperty> GreaterThan<TValue>(
        TValue valueToCompare, string errorMessage)
```

```
        where TValue : TProperty, IComparable<TValue>, IComparable;
    IValidationRules<T, TProperty> Equal(
        Expression<Func<T, TProperty>> expression, string errorMessage);
    IValidationRules<T, string> Length(int length, string errorMessage);
    IValidationRules<T, string> MaximumLength(int length, string errorMessage);
    IValidationRules<T, string> MinimumLength(int length, string errorMessage);
    IValidationRules<T, string> EmailAddress(string errorMessage);
}
```

El código de la interface define las reglas de validación que deseamos implementar. El tipo del modelo a validar está definido en el parámetro de tipo ***T*** de la interface mientras que el tipo de la propiedad que deseamos validar está definido en el parámetro de tipo ***TProperty***.

Puedes notar que algunas de las reglas de validación se aplican exclusivamente a propiedades de tipo *string*.

6. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
7. Agrega un nuevo archivo de *interface* llamado ***ICollectionValidationRules.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface ICollectionValidationRules<T, TProperty>
{
    ICollectionValidationRules<T, TProperty> SetValidator(
        IModelValidator<TProperty> validator);
}
```

El código de la interface define una regla para las propiedades de tipos colección. Puedes notar que establecemos el validador que será utilizado para validar cada elemento de la colección.

8. Agrega un nuevo archivo de *interface* llamado ***IValidationService.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IValidationService<T>
{
    IValidationRules<T, TProperty> AddRuleFor<TProperty>(
        Expression<Func<T, TProperty>> expression);
    ICollectionValidationRules<T, TProperty> AddRuleForEach<TProperty>(
        Expression<Func<T, IEnumerable<TProperty>>> expression);
    Task<IEnumerable<ValidationError>> Validate(T model);
}
```

El servicio de validación es el componente principal que se encargará de evaluar las reglas de validación que se hayan especificado en implementaciones de *IModelvalidator<T>*. Permitirá agregar reglas de validación y realizar la validación de estas devolviendo una colección de errores en caso de que haya alguno.

Agregar código base de validación

1. Agrega un nuevo directorio llamado ***Abstractions*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***AbstractModelValidator.cs*** en el directorio *Abstractions* y remplaza el contenido de la clase por el siguiente.

```
public abstract class AbstractModelValidator<T>(
    IValidationService<T> validationService,
    ValidationConstraint constraint =
        ValidationConstraint.AlwaysValidate
    ) : IModelValidator<T>
{
    public ValidationConstraint Constraint => constraint;
    public IEnumerable<ValidationError> Errors { get; private set; }

    public async Task<bool> Validate(T model)
    {
        Errors = await validationService.Validate(model);
        return Errors == default;
    }

    protected IValidationRules<T, TProperty> AddRuleFor<TProperty>(
        Expression<Func<T, TProperty>> expression) =>
        validationService.AddRuleFor<TProperty>(expression);
    protected ICollectionValidationRules<T, TProperty> AddRuleForEach<TProperty>(
        Expression<Func<T, IEnumerable<TProperty>>> expression) =>
        validationService.AddRuleForEach<TProperty>(expression);

    public IValidationService<T> ValidatorService => validationService;
}
```

Esta clase contiene el código base que las clases que implementen validación deberán implementar. Puedes notar que solo estamos encapsulando la funcionalidad que expone *IValidationService<T>*. Las clases que hereden de esta clase podrán utilizar los métodos *AddRuleFor* y *AddRuleForEach* directamente de su clase base. También estamos exponiendo al servicio de validación, esta propiedad será útil cuando se intente validar a los elementos de una propiedad que almacene una colección.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. Agrega un nuevo archivo de clase llamado ***AbstractViewModelValidator.cs*** en el directorio *Abstractions* y remplaza el código de la clase por el siguiente.

```
public abstract class AbstractViewModelValidator<DtoType, ViewModelType>(
    IModelValidatorHub<DtoType> dtoModelValidatorHub,
    ValidationConstraint constraint) : IModelValidator<ViewModelType>
{
    public ValidationConstraint Constraint => constraint;
    public IEnumerable<ValidationError> Errors =>
        dtoModelValidatorHub.Errors;
```

```
    // En caso de que el ViewModel implemente el operador Explicit se puede
    // utilizar este método.
    // Si el ViewModel no implementa el operador Explicit, se podrá remplazar
    // (Override) este método en la clase que implemente esta clase.
    public virtual DtoType Cast(ViewModelType viewModel)
    {
        DtoType DtoModel = default;
        var ExplicitMethod = typeof(ViewModelType).GetMethod("op_Explicit");
        if (ExplicitMethod != null)
            DtoModel = (DtoType)ExplicitMethod.Invoke(
                viewModel, new object[] { viewModel });
        else
            throw new InvalidCastException();

        return DtoModel;
    }

    public Task<bool> Validate(ViewModelType model) =>
        dtoModelValidatorHub.Validate(Cast(model));
}
```

El código backend puede validar los datos de entrada implementando validación en los objetos *DTO*. Por otro lado, los datos de entrada que una aplicación frontend solicita al usuario pueden almacenarse en un *ViewModel* que también necesitará ser validado. Por lo tanto, el código frontend deberá utilizar la validación definida en el DTO y probablemente también pueda implementar validación en el *ViewModel*.

Para facilitar la validación del *ViewModel* hemos creado la clase *AbstractViewModelValidator* que contiene el código base que deberán implementar las clases que implementen validación de los *ViewModels*.

Puedes notar que la clase recibe una instancia del servicio de validación del DTO y devuelve el resultado que este devuelva.

El método *Cast* hará la conversión explicita del tipo *ViewModel* hacia el tipo *DTO*. En caso de que el *ViewModel* implemente el operador *Explicit*, se podrá utilizar el método *Cast para* convertir el tipo del *ViewModel* hacia el tipo del *DTO*. Si el *ViewModel* no implementa el operador *Explicit*, se podrá remplazar (*Override*) el método *Cast* en la clase que implemente la validación del *ViewModel*.

Implementar el Hub de validación predeterminado

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ModelValidatorHub.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class ModelValidatorHub<ModelType>(
    IEnumerable<IModelValidator<ModelType>> validators) :
    IModelValidatorHub<ModelType>
{
    public IEnumerable<ValidationError> Errors { get; private set; }
    public async Task<bool> Validate(ModelType model)
    {
        List<ValidationError> CurrentErrors = [];

        // Obtener los validadores que siempre deben evaluarse.
        var Validators = validators
            .Where(v => v.Constraint == ValidationConstraint.AlwaysValidate)
            .ToList();

        // Agregar los validadores que deben evaluarse cuando no haya errores
        // previos.
        Validators.AddRange(validators
            .Where(v => v.Constraint ==
                ValidationConstraint.ValidateIfThereAreNoPreviousErrors));

        foreach (var Validator in Validators)
        {
            if ((Validator.Constraint == ValidationConstraint.AlwaysValidate) ||
                !CurrentErrors.Any())
            {
                if (!await Validator.Validate(model))
                {
                    CurrentErrors.AddRange(Validator.Errors);
                }
            }
        }

        Errors = CurrentErrors;

        return !Errors.Any();
    }
}
```

Esta clase será la implementación predeterminada de *IModelValidatorHub<ModelType>* que permitirá ejecutar la validación de cada uno de los validadores de un modelo. El proceso de validación terminará dependiendo del requerimiento de validación que exponga el validador.

Antes de efectuar el proceso de validación, se ordenan en primer lugar aquellos validadores que deben validarse siempre y después aquellos que deben validarse solo si no hay errores previos. Por ejemplo, los validadores de bases de datos deberán ejecutarse solo si no hay errores previos de validación para evitar accesos no necesarios a la base de datos.

Agregar el contenedor de dependencias

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.

2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddDefaultModelValidatorHub(
        this IServiceCollection services)
    {
        services.TryAddScoped(typeof(IModelValidatorHub<>),
            typeof(ModelValidatorHub<>));

        return services;
    }

    public static IServiceCollection
        AddModelValidator<ModelType, ModelValidatorType>(
        this IServiceCollection services)
        where ModelValidatorType : class, IModelValidator<ModelType>
    {
        services.AddDefaultModelValidatorHub();
        services.AddScoped<IModelValidator<ModelType>, ModelValidatorType>();

        return services;
    }
}
```

   A través del método *AddDefaultModelValidatorHub* podremos registrar el Hub de validación predeterminado. Este método podrá ser invocado explícita o implícitamente a través del método *AddModelValidator*.

   El método *AddModelValidator* facilitará el registro de los servicios de validación de modelos *IModelValidator<ModelType>*.

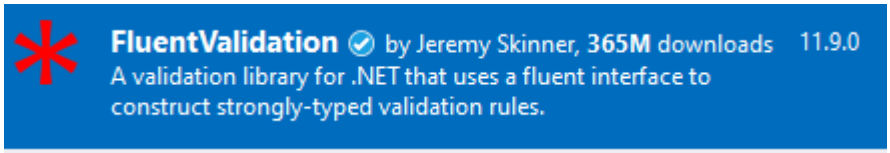
3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar el servicio de validación

Como comentamos previamente, para implementar la validación, utilizaremos la biblioteca *FluentValidation*, esta biblioteca quedará encapsulada en un nuevo proyecto, quienes utilicen el framework de validación no estarán acoplados a esta biblioteca, simplemente consumirán los servicios que hemos definido previamente.

1. Agrega un nuevo directorio llamado ***Libraries*** en el directorio ***4. Frameworks And Drivers***.
2. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.ValidationService.FluentValidation*** en el directorio *Libraries*.
3. Elimina el archivo ***Class1.cs***.

4. Agrega la referencia del proyecto *NorthWind.Validation.Entities*.
5. Agrega el paquete NuGet ***FluentValidation***.



Definir una implementación de AbstractValidator

FluentValidation proporciona la clase abstracta *AbstractValidator* para que a través de ella se puedan definir las reglas de validación, normalmente, las clases que implementan validación heredan de esta abstracción para acceder a su funcionalidad. Nosotros crearemos una clase que herede de *AbstractValidator* para instanciarla y utilizar a través de ella la funcionalidad de *AbstractValidator<T>* sin tener que heredarla en las clases que implementen validaciones. La clase estará definida como *internal* por lo que solo podrá ser accedida dentro del proyecto.

1. Agrega un nuevo archivo de clase llamado ***AbstractValidatorImplementation.cs*** en la raíz del proyecto y remplaza el contenido de la clase por el siguiente.

```
internal class AbstractValidatorImplementation<T> : AbstractValidator<T>
{
}
```

   Tres son los métodos que utilizaremos de *AbstractValidator* a través de esta implementación:

   - ***RuleFor*** para iniciar la definición de reglas sobre una propiedad.
   - ***RuleForEach*** para iniciar la definición reglas que serán invocadas en cada elemento de una propiedad de tipo colección.
   - ***ValidateAsync*** para realizar la validación.

Implementar IValidationService

Implementemos ahora el servicio de validación encapsulando una instancia de la implementación de *AbstractValidator<T>*.

1. Agrega un nuevo archivo de clase llamado ***FluentValidationService.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
internal class FluentValidationService<T> : IValidationService<T>
{
    internal readonly AbstractValidatorImplementation<T> Wrapper
```

```
        = new AbstractValidatorImplementation<T>();

    public IValidationRules<T, TProperty> AddRuleFor<TProperty>(
        Expression<Func<T, TProperty>> expression) =>
        new ValidationRules<T, TProperty>(Wrapper.RuleFor(expression));

    public ICollectionValidationRules<T, TProperty> AddRuleForEach<TProperty>(
        Expression<Func<T, IEnumerable<TProperty>>> expression) =>
        new CollectionValidationRules<T, TProperty>(
            Wrapper.RuleForEach(expression));

    public async Task<IEnumerable<ValidationError>> Validate(T model)
    {
        var Result = await Wrapper.ValidateAsync(model);
        IEnumerable<ValidationError> Errors = default;
        if (!Result.IsValid)
        {
            Errors = Result.Errors
                .Select(e => new ValidationError(
                    e.PropertyName, e.ErrorMessage));
        }

        return Errors;
    }
}
```

Puedes notar que el código de la clase simplemente encapsula la funcionalidad expuesta a través de la implementación de la clase abstracta *AbstractValidator<T>* de *FluentValidation*.

La propiedad *Wrapper* es declarada como *internal* para que pueda ser accedida en cualquier clase del mismo proyecto, específicamente, en la clase que implementará las reglas de validación de propiedades de tipo colección.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar las reglas de validación

1. Agrega un nuevo archivo de clase llamado ***ValidationRules.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
internal class ValidationRules<T, TProperty>(
    IRuleBuilderInitial<T, TProperty> ruleBuilderInitial) :
    IValidationRules<T, TProperty>
{
    // Utilizada para reglas sobre propiedades de tipo string
    IRuleBuilder<T, string> StringRuleBuilder =>
        (IRuleBuilder<T, string>)ruleBuilderInitial;

    // Utilizada para devolver el resultado de una regla en
    // propiedades de tipo string.
    IValidationRules<T, string> ThisAsStringValidationRules =>
```

```
        (IValidationRules<T, string>)this;

    public IValidationRules<T, TProperty> NotEmpty(string errorMessage)
    {
        ruleBuilderInitial.NotEmpty()
            .WithMessage(errorMessage);

        return this;
    }

    public IValidationRules<T, TProperty> NotNull(string errorMessage)
    {
        ruleBuilderInitial
            .NotNull()
            .WithMessage(errorMessage);

        return this;
    }

    public IValidationRules<T, TProperty> Must(Func<TProperty, bool> predicate,
        string errorMessage)
    {
        ruleBuilderInitial
            .Must(predicate)
            .WithMessage(errorMessage);

        return this;
    }

    public IValidationRules<T, TProperty> StopOnFirstFailure()
    {
        ruleBuilderInitial
            .Cascade(CascadeMode.Stop);

        return this;
    }

    public IValidationRules<T, TProperty> Equal(
        Expression<Func<T, TProperty>> expression, string errorMessage)
    {
        ruleBuilderInitial
            .Equal(expression)
            .WithMessage(errorMessage);

        return this;
    }

    public IValidationRules<T, TProperty> GreaterThan<TValue>(
        TValue valueToCompare, string errorMessage)
        where TValue : TProperty, IComparable<TValue>, IComparable
    {
        IRuleBuilder<T, TValue> Builder =
            (IRuleBuilder<T, TValue>)ruleBuilderInitial;

        Builder
            .GreaterThan(valueToCompare)
            .WithMessage(errorMessage);
```

```
        return this;
    }

    public IValidationRules<T, string> Length(
        int length, string errorMessage)
    {
        StringRuleBuilder
            .Length(length)
            .WithMessage(errorMessage);

        return ThisAsStringValidationRules;
    }

    public IValidationRules<T, string> MaximumLength(
        int length, string errorMessage)
    {
        StringRuleBuilder
            .MaximumLength(length)
            .WithMessage(errorMessage);

        return ThisAsStringValidationRules;
    }

    public IValidationRules<T, string> MinimumLength(
        int length, string errorMessage)
    {
        StringRuleBuilder
            .MinimumLength(length)
            .WithMessage(errorMessage);

        return ThisAsStringValidationRules;
    }

    public IValidationRules<T, string> EmailAddress(string errorMessage)
    {
        StringRuleBuilder
            .EmailAddress()
            .WithMessage(errorMessage);

        return ThisAsStringValidationRules;
    }
}
```

La clase *ValidationRules* implementa los miembros definidos en la interface *IValidationRules*. Examinando el código puedes notar que simplemente estamos encapsulando la funcionalidad de la biblioteca *FluentValidation* a través de la instancia *IRuleBuilderInitial* que es el constructor que inicia la cadena de reglas asociadas a una propiedad del modelo. Puedes consultar la documentación de *FluentValidation* para obtener información detallada sobre cada uno de los métodos de validación utilizados en el código anterior. Lo importante aquí es que podríamos encapsular una biblioteca distinta de terceros o propia sin afectar los otros proyectos de la solución.

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

3. Agrega un nuevo archivo de clase llamado ***CollectionValidationRules.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
internal class CollectionValidationRules<T, TProperty>(
    IRuleBuilderInitialCollection<T, TProperty> ruleBuilderInitialCollection) :
    ICollectionValidationRules<T, TProperty>
{
    public ICollectionValidationRules<T, TProperty> SetValidator(
        IModelValidator<TProperty> modelValidator)
    {
        var ModelValidator =
            modelValidator as AbstractModelValidator<TProperty>;
        var ValidationService =
            ModelValidator.ValidatorService as
            FluentValidationService<TProperty>;

        ruleBuilderInitialCollection
            .SetValidator(ValidationService.Wrapper);

        return this;
    }
}
```

La clase simplemente encapsula al objeto *IRuleBuilderInitialCollection* utilizado por *FluentValidation* para definir reglas de validación de colecciones.

El método *SetValidator* recibe una instancia de *IModelvalidator* y obtiene a través de este la instancia de *FluentValidationService* para obtener finalmente la instancia de la implementación de *AbstractValidator<T>*.

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs.*

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddValidationService(
        this IServiceCollection services)
    {
        services.AddScoped(typeof(IValidationService<>),
            typeof(FluentValidationService<>));
        return services;
    }
}
```

Puedes notar que estamos agregando un método que permita registrar la implementación predeterminada de ***IValidationService<T>***.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar las validaciones

Ahora que tenemos el framework de validación, es momento de crear el código de validación del caso de uso. Las validaciones de los datos de entrada tendrán que ser realizadas en el código backend y en el código frontend, por esa razón, implementaremos el código de validación a nivel empresarial, en el directorio `1. Enterprise Business Rules`.

Crear un proyecto para implementar las validaciones

1. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Sales.Validators.Entities*** en el directorio ***1. Enterprise Business Rules***.
2. Elimina el archivo *Class1.cs*.
3. Agrega una referencia del proyecto ***NorthWind.Validation.Entities***.
4. Agrega una referencia del proyecto ***NorthWind.Sales.Entities***.

Agregar un archivo de mensajes de error

Agreguemos un archivo con los mensajes de error de validación que utilizarán los validadores que crearemos.

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo ***Resources File*** llamado ***CreateOrderMessages.resx*** en el directorio *Resources*.
3. Mantén abierto el diseñador del archivo de recursos.

Agregar el validador de *CreateOrderDetailDto*

1. Agrega los siguientes valores en el diseñador del archivo de recursos.

Nombre	Valor
ProductIdGreaterThanZero	Debe especificar el identificador del producto.
QuantityGreaterThanZero	Debe especificar la cantidad ordenada del producto.
UnitPriceGreaterThanZero	Debe especificarse el precio del producto.

2. Agrega un nuevo directorio llamado ***CreateOrder*** en la raíz del proyecto.
3. Agrega un nuevo archivo de clase llamado ***CreateOrderDetailDtoValidator.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
internal class CreateOrderDetailDtoValidator :
    AbstractModelValidator<CreateOrderDetailDto>
{
    public CreateOrderDetailDtoValidator(
        IValidationService<CreateOrderDetailDto> validator) : base(validator)
    {
        AddRuleFor(d => d.ProductId)
            .GreaterThan(0, CreateOrderMessages.ProductIdGreaterThanZero);

        AddRuleFor<int>(d => d.Quantity)
            .GreaterThan(0, CreateOrderMessages.QuantityGreaterThanZero);


        AddRuleFor(d => d.UnitPrice)
            .GreaterThan<decimal>(0,
                CreateOrderMessages.UnitPriceGreaterThanZero);
    }
}
```

La clase que contiene el código de validación debe heredar de *AbstractModelValidator* y definir las reglas de validación en su constructor. La clase recibe una instancia del servicio de validación que pasa al constructor de la clase base.

Agregamos las reglas de validación de cada propiedad encadenando el método de validación especifico al método *AddRuleFor*. En este caso solo encadenamos un método de validación (*GreaterThan*), pero podemos encadenar más métodos. Cada método de validación especifica el mensaje de error que se deberá generar en caso de que la regla no se cumpla.

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el validador de CreateOrderDto

1. Agrega los siguientes valores en el diseñador del archivo de recursos.

Nombre	Valor
CustomerIdRequired	Debe proporcionar el identificador del cliente.
CustomerIdRequiredLength	La longitud del identificador debe ser 5.
ShipAddressRequired	Debe proporcionar la dirección de envío.
ShipAddressMaximumLength	La longitud máxima de la dirección es 60.
ShipCityRequired	Debe proporcionar la ciudad de envío.
ShipCityMinimumLength	Debe especificar al menos 3 caracteres del nombre de la ciudad.
ShipCityMaximumLength	La longitud máxima de la ciudad es 15.
ShipCountryRequired	Debe proporcionar el país de envío.

ShipCountryMinimumLength	Debe especificar al menos 3 caracteres del nombre del País.
ShipCountryMaximumLength	La longitud máxima del país es 15.
ShipPostalCodeMaximumLength	La longitud máxima del código postal es 10.
OrderDetailsRequired	Debe especificar los productos de la orden.
OrderDetailsNotEmpty	Debe especificar al menos un producto de la orden.

2. Agrega un nuevo archivo de clase llamado ***CreateOrderDtoValidator.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
internal class CreateOrderDtoValidator :
    AbstractModelValidator<CreateOrderDto>
{
    public CreateOrderDtoValidator(
        IValidationService<CreateOrderDto> validationService,
        IModelValidator<CreateOrderDetailDto> detailValidator) :
        base(validationService)
    {
        AddRuleFor(c => c.CustomerId)
            .NotEmpty(CreateOrderMessages.CustomerIdRequired)
            .Length(5, CreateOrderMessages.CustomerIdRequiredLength);

        AddRuleFor(c => c.ShipAddress)
            .NotEmpty(CreateOrderMessages.ShipAddressRequired)
            .MaximumLength(60, CreateOrderMessages.ShipAddressMaximumLength);

        AddRuleFor(c => c.ShipCity)
            .NotEmpty(CreateOrderMessages.ShipCityRequired)
            .MinimumLength(3, CreateOrderMessages.ShipCityMinimumLength)
            .MaximumLength(15, CreateOrderMessages.ShipCityMaximumLength);

        AddRuleFor(c => c.ShipCountry)
            .NotEmpty(CreateOrderMessages.ShipCountryRequired)
            .MinimumLength(3, CreateOrderMessages.ShipCountryMinimumLength)
            .MaximumLength(15, CreateOrderMessages.ShipCountryMaximumLength);

        AddRuleFor(c => c.ShipPostalCode)
            .MaximumLength(10, CreateOrderMessages.ShipPostalCodeMaximumLength);

        AddRuleFor(c => c.OrderDetails)
            .NotNull(CreateOrderMessages.OrderDetailsRequired)
            .NotEmpty(CreateOrderMessages.OrderDetailsNotEmpty);

        AddRuleForEach(c => c.OrderDetails)
            .SetValidator(detailValidator);
    }
}
```

Puedes notar que la clase recibe en el constructor el servicio de validación y el validador del modelo *CreateOrderDetailDto* que utiliza para establecer el validador de la propiedad *Orderdetails*.

En este caso, hemos encadenado distintas validaciones a las propiedades. También hemos encadenado el método de validación *SetValidator* al método *AddRuleForEach* para establecer la instancia del validador encargado de validar cada elemento de la colección *OrderDetails*.

Todo el código depende de nuestro framework de validación, el validador no está acoplado a *FluentValidation*. En cualquier momento, nuestro framework de validación podría remplazar la biblioteca *FluentValidation* por alguna otra herramienta propia o de terceros sin afectar el código de validación de los casos de uso.

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el código del archivo con el siguiente código.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddValidators(
        this IServiceCollection services)
    {
        services.AddModelValidator<CreateOrderDto,
            CreateOrderDtoValidator>();
        services.AddModelValidator<CreateOrderDetailDto,
            CreateOrderDetailDtoValidator>();

        return services;
    }
}
```

Puedes notar que estamos utilizando el método de extensión *AddModelValidator* que definimos en nuestro framework de validación para registrar las clases que implementan la validación.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar las dependencias en los proyectos IoC

Modifiquemos ahora los proyectos *IoC* para registrar la nueva implementación que hemos agregado a la solución.

1. Agrega la referencia del proyecto *NorthWind.Sales.Validators.Entities* en el proyecto *NorthWind.Sales.Backend.IoC*.
2. Agrega la referencia del proyecto *NorthWind.ValidationService.FluentValidation* en el proyecto *NorthWind.Sales.Backend.IoC*
3. Abre el archivo *DependencyContainer.cs* del proyecto ***NorthWind.Sales.Backend.IoC***.
4. Modifica el código del método *AddNorthWindSalesServices* para agregar el registro de los servicios de validación.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators();

    return services;
}
```

3. Agrega la referencia del proyecto *NorthWind.Sales.Validators.Entities* en el proyecto *NorthWind.Sales.Frontend.IoC*.
4. Agrega la referencia del proyecto *NorthWind.ValidationService.FluentValidation* en el proyecto *NorthWind.Sales.Frontend.IoC*
5. Abre el archivo *DependencyContainer.cs* del proyecto ***NorthWind.Sales.Frontend.IoC***.
6. Modifica el código del método *AddNorthWindSalesServices* para agregar el registro de los servicios de validación.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<HttpClient> configureHttpClient)
{
    services.AddWebApiGateways(configureHttpClient)
        .AddViewsServices()
        .AddValidationService()
        .AddValidators();

    return services;
```

```
}
```

Agregar una cláusula *Guard* de validación

Una cláusula *Guard* es simplemente un bloque de código que permite interrumpir la ejecución de un método a través de la evaluación de una condición. Si la evaluación no es exitosa, la ejecución del método es interrumpida ejecutando una sentencia *return* o lanzando una excepción.

Agreguemos ahora una cláusula *Guard* en el proyecto *NorthWind.Sales.Backend.BusinessObjects* que permita romper la ejecución de un método lanzando una excepción en caso de un validación no exitosa de un modelo**.**

1. Agrega la referencia del proyecto ***NorthWind.Validation.Entities*** en el proyecto ***NorthWind.Sales.Backend.BusinessObjects***.
2. Agrega un nuevo directorio llamado ***Guards*** en la raíz del proyecto.
3. Agrega un nuevo archivo de clase llamado ***GuardModel.cs*** en el directorio *Guards* y remplaza el código de la clase por el siguiente.

```
public static class GuardModel
{
    public static async Task AgainstNotValid<T>(
        IModelValidatorHub<T> modelValidatorHub, T model)
    {
        if (!await modelValidatorHub.Validate(model))
        {
            string Errors = string.Join(" ",
                modelValidatorHub.Errors
                .Select(e => $"{e.PropertyName}: {e.Message}"));
            throw new Exception(Errors);
        }

    }
}
```

   El método realiza la validación del modelo a través del Hub de validación. En caso de que haya un error de validación, la lista de errores será concatenada en una cadena y una excepción será lanzada.

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Modificar el Interactor del caso de uso

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el constructor primario de la clase para declarar la dependencia del servicio de validación.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository,
    IModelValidatorHub<CreateOrderDto> modelValidatorHub) :
    ICreateOrderInputPort
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. Agrega el siguiente código al inicio del método *Handle* para realizar la validación.

```
await GuardModel.AgainstNotValid(modelValidatorHub, orderDto);
```

La cláusula *Guard* lanzará una excepción en caso de que el modelo no sea válido. En este caso de uso utilizaremos la técnica de *lanzamiento de Excepción* para implementar el flujo alterno del caso de uso, en lecciones posteriores implementaremos el patrón ***Result*** para implementar el flujo alterno del caso de uso para manejo de errores de validación.

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Probar la funcionalidad con la API Web

1. Ejecuta la aplicación.

Probar con todos los datos correctos

1. En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

```
{
    "customerId": "ANTON",
    "shipAddress": "3 Sur 710",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 17,
            "quantity": 100
        }
    ]
}
```

Podrás ver un resultado mostrando el identificador de la orden creada.

Probar con 6 caracteres en el identificador del cliente

1. Intenta crear una orden especificando 6 caracteres en el identificador del cliente.

```
{
    "customerId": "ANTONX",
    "shipAddress": "3 Sur 710",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400",
    "orderDetails": [
```

```
        {
            "productId": 1,
            "unitPrice": 17,
            "quantity": 100
        }
    ]
}
```

Te será mostrado un resultado con el error de validación similar al siguiente.

Puedes notar que el mensaje de error de validación para la propiedad *CustomerId* es mostrado.

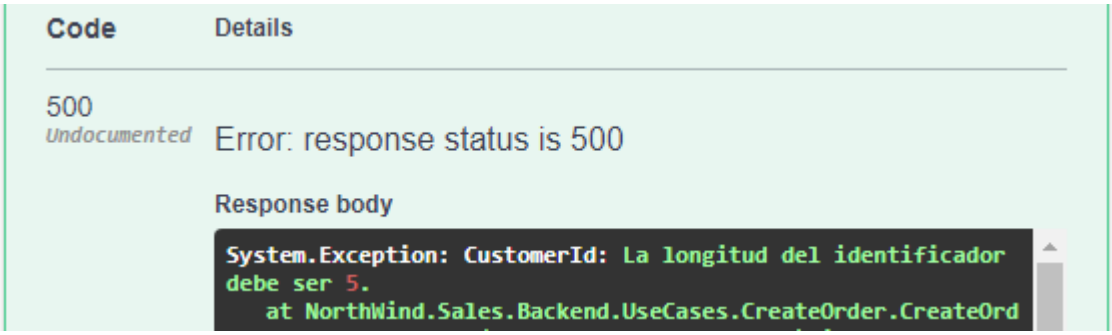
En una lección posterior implementaremos el manejo de excepciones para proporcionar al cliente de la Web API mensajes de error estándares de acuerdo con la especificación ***Problem Details for HTTP APIs*** descrita en https://tools.ietf.org/html/rfc7807.

Probar con 3 caracteres en el identificador del cliente

1. Intenta crear una orden especificando 3 caracteres en el identificador del cliente.

```
{
    "customerId": "ANT",
    "shipAddress": "3 Sur 710",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 17,
            "quantity": 100
        }
    ]
}
```

Te será mostrado un resultado con el error de validación similar al siguiente.

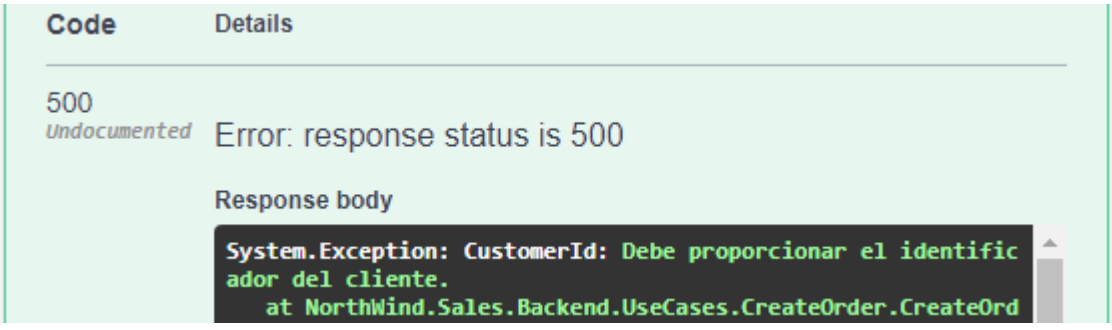


Probar sin el identificador del cliente

1. Intenta crear una orden si especificar el identificador del cliente.

```
{

    "shipAddress": "3 Sur 710",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 17,
            "quantity": 100
        }
    ]
}
```

Te será mostrado un resultado con el error de validación similar al siguiente.



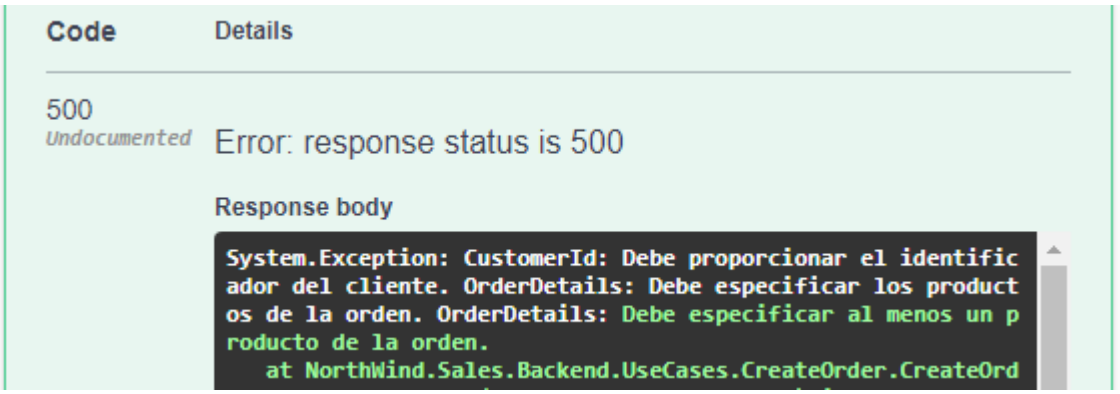
Probar sin el identificador de cliente y sin detalle

1. Intenta crear una orden si especificar el identificador del cliente y sin el detalle de la orden.

```
{
    "shipAddress": "3 Sur 710",
```

```
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400"
}
```

Te será mostrado un resultado con los errores de validación similar al siguiente.



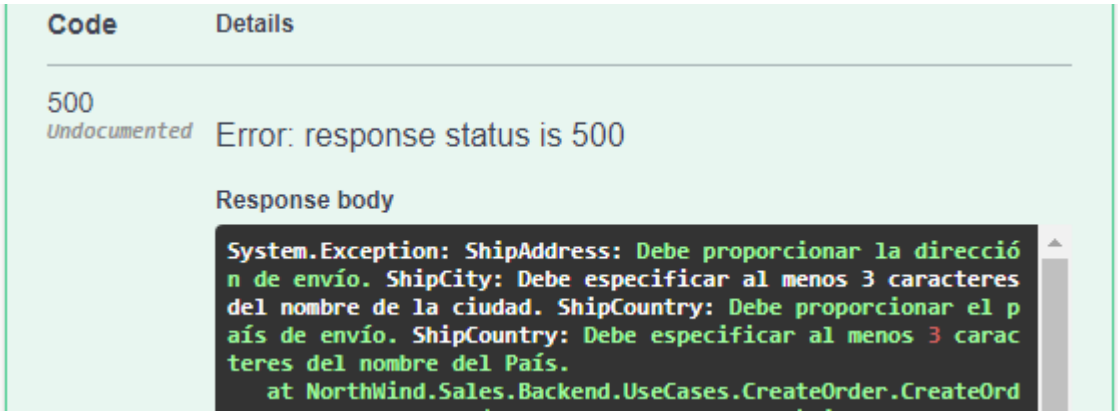
Puedes notar que se muestran los mensajes de error esperados.

Probar con errores en datos de envío

1. Intenta crear una orden con errores en los datos de envío.

```
{
    "customerId": "ANTON",
    "shipAddress": "",
    "shipCity": "Pu",
    "shipCountry": "",
    "shipPostalCode": "72400",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 17,
            "quantity": 100
        }
    ]
}
```

Te será mostrado un resultado con los errores de validación similar al siguiente.

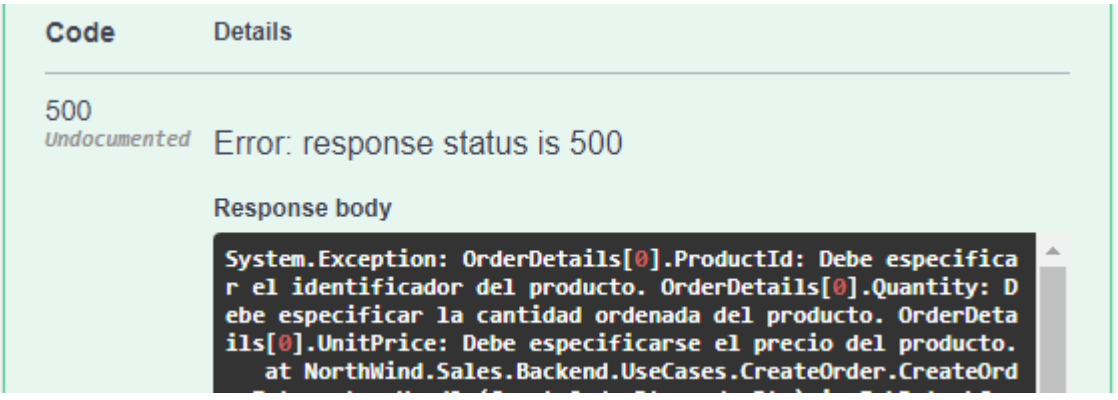


Probar con error en el detalle de la orden

1. Intenta crear una orden con errores en el detalle de la orden.

```
{
    "customerId": "ANTON",
    "shipAddress": "3 Sur 710",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72400",
    "orderDetails": [
        {
            "productId": 0,
            "unitPrice": 0,
            "quantity": 0
        }
    ]
}
```

Te será mostrado un resultado con los errores de validación similar al siguiente.



2. Regresa a Visual Studio y detén la ejecución.

Agregar requerimientos de validación con la base de datos

En algunas situaciones será necesario validar los datos de entrada tomando como referencia la información existente en la fuente de datos, por ejemplo, la existencia del identificador del cliente o el identificador del producto.

Modifiquemos nuestro caso de uso para agregar nuevos criterios de validación, los nuevos criterios serán los siguientes:

- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.
- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

Historia de Usuario: Crear orden de compra

"Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema valida los datos.
3. El sistema registra la orden de compra.

4. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Validaciones

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.
- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.
- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.

- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

Agregar objetos de negocio

Agreguemos los objetos de negocio que utilizaremos para implementar los nuevos requerimientos de validación.

Agregar un Value Object

Agreguemos una nueva clase que nos proporcione información sobre las unidades en existencia de un producto.

1. Agrega un nuevo archivo de clase llamado ***ProductUnitsInStock.cs*** en el directorio *ValueObjects* del proyecto *NorthWind.Sales.Backend.BusinessObjects* y remplaza el código de la clase por el siguiente.

```
public class ProductUnitsInStock(int productId, short unitsInStock)
{
    public int ProductId => productId;
    public short UnitsInStock => unitsInStock;
}
```

Agregar una interface de un nuevo Repositorio

Para implementar los nuevos requerimientos necesitaremos un repositorio, este puede ser un nuevo repositorio o podemos modificar el repositorio actual. En este ejercicio crearemos un nuevo repositorio para agrupar las consultas.

1. Agrega un nuevo archivo de *Interface* llamado ***IQueriesRepository.cs*** en el directorio *Interfaces\Repositories* del proyecto *NorthWind.Sales.Backend.BusinessObjects* y remplaza el código de la interface por el siguiente.

```
public interface IQueriesRepository
{

}
```

2. Agrega el siguiente código a la interface para definir un método que reciba el identificador de un cliente y devuelva su saldo actual.

```
Task<decimal?> GetCustomerCurrentBalance(string customerId);
```

   El valor del saldo actual será *null* cuando el identificador del cliente no exista.

Debido a que la única información que necesitamos es el saldo del cliente, no es necesario devolver una entidad que represente al cliente. El valor decimal es suficiente.

3. Agrega el siguiente código para definir un método que reciba una lista con los identificadores de producto a buscar y que devuelva sus respectivas existencias.

```
Task<IEnumerable<ProductUnitsInStock>>
    GetProductsUnitsInStock(IEnumerable<int> productIds);
```

El resultado devuelto tendrá únicamente las existencias de los productos existentes, si un producto no existe, su identificador no se encontrará en el resultado. Esto nos permitirá determinar cuáles de los productos proporcionados por el usuario no existen.

Debido a que lo único que necesitamos es conocer la cantidad existente de cada producto, no es necesario devolver entidades que representen productos.

El código de la interface será similar al siguiente.

```
public interface IQueriesRepository
{
    Task<decimal?> GetCustomerCurrentBalance(string customerId);
    Task<IEnumerable<ProductUnitsInStock>>
        GetProductsUnitsInStock(IEnumerable<int> productIds);
}
```

Implementar un nuevo repositorio

Implementemos ahora el nuevo repositorio en el proyecto *NorthWind.Sales.Backend.Repositories*.

Agregar las entidades de clientes y productos

Agreguemos las entidades que nos permitirán crear 2 nuevas tablas en la base de datos: ***Customers*** y ***Products***.

1. Agrega un nuevo archivo de clase llamado ***Customer.cs*** en el directorio ***Entities*** y remplaza el código de la clase por el siguiente.

```
public class Customer
{
    public string Id { get; set; }
    public string Name { get; set; }
    public decimal CurrentBalance { get; set; }
}
```

   Utilizaremos esta clase con el único fin de crear la tabla con los datos de los clientes.

2. Agrega un nuevo archivo de clase llamado ***Product.cs*** en el directorio ***Entities*** y remplaza el código de la clase por el siguiente.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public short UnitsInStock { get; set; }
    public decimal UnitPrice { get; set; }
}
```

   Utilizaremos esta clase con el único fin de crear la tabla con los datos de los *Productos*.

Agregar la abstracción del contexto de datos

1. Agrega un nuevo archivo de tipo *interface* llamado ***INorthWindSalesQueriesDataContext.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface INorthWindSalesQueriesDataContext
{
    IQueryable<Customer> Customers { get; }
    IQueryable<Product> Products { get; }
    Task<ReturnType> FirstOrDefaultAync<ReturnType>(
        IQueryable<ReturnType> queryable);
```

```
    Task<IEnumerable<ReturnType>> ToListAsync<ReturnType>(
        IQueryable<ReturnType> queryable);
}
```

La interface define las propiedades *Customer* y *Products* sobre las cuales se podrán realizar consultas de tipo *LINQ*. El método *FirstOrDefaultAync* permitirá obtener el primer elemento del resultado de una consulta mientras que el método *ToListAsync* permitirá obtener una lista de elementos del resultado de una consulta.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el repositorio

Implementemos ahora la nueva funcionalidad del repositorio.

1. Agrega un nuevo archivo de clase llamado ***QueriesRepository.cs*** en el directorio *Repositories* y remplaza el contenido de la clase por el siguiente.

```
internal class QueriesRepository(INorthWindSalesQueriesDataContext context) :
    IQueriesRepository
{
    public async Task<decimal?> GetCustomerCurrentBalance(
        string customerId)
    {
        var Queryable = context.Customers
            .Where(c => c.Id == customerId)
            .Select(c => new { c.CurrentBalance });

        var Result = await context.FirstOrDefaultAync(Queryable);

        return Result?.CurrentBalance;
    }

    public async Task<IEnumerable<ProductUnitsInStock>>
        GetProductsUnitsInStock(IEnumerable<int> productIds)
    {
        var Queryable = context.Products
            .Where(p => productIds.Contains(p.Id))
            .Select(p => new ProductUnitsInStock(
                p.Id, p.UnitsInStock));

        return await context.ToListAsync(Queryable);
    }
}
```

El método *GetCustomerCurrentBalance* realiza una consulta *LINQ* para obtener el saldo del cliente, en caso de que la consulta no devuelva resultado significará que el cliente no existe y en consecuencia se devolverá *null*.

El método *GetProductsUnitsInStock* realiza una consulta *LINQ* para obtener una lista de identificadores de productos con su existencia en almacén.

2. Importa el espacio de nombres requerido en el archivo *GlobaUsings.cs*.

Registrar el nuevo repositorio

1. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.Repositories*.
2. Agrega el siguiente código en el método *AddRepositories*.

```
public static IServiceCollection AddRepositories(
    this IServiceCollection services)
{
    services.AddScoped<ICommandsRepository, CommandsRepository>();
    services.AddScoped<IQueriesRepository, QueriesRepository>();

    return services;
}
```

Implementar el contexto de datos

Implementemos ahora el contexto de datos que utilizará el repositorio definido previamente. Realiza los siguientes pasos en el proyecto *NorthWind.Sales.Backend.DataContexts.EFCore*.

Agregar la configuración de las nuevas entidades

1. Agrega un nuevo archivo de clase llamado ***CustomerConfiguration.cs*** en el directorio ***Configurations*** y remplaza el código de la clase por el siguiente.

```
internal class CustomerConfiguration :
        IEntityTypeConfiguration<Customer>
{
    public void Configure(EntityTypeBuilder<Customer> builder)
    {
        builder
            .Property(c => c.Id)
            .HasMaxLength(5)
            .IsFixedLength();

        builder.Property(c => c.Name)
            .IsRequired()
            .HasMaxLength(40);

        builder.Property(c => c.CurrentBalance)
            .HasPrecision(8, 2);

        builder.HasData(
            new Customer
            {
                Id = "ALFKI",
                Name = "Alfreds Futterkiste",
                CurrentBalance = 0
            },
            new Customer
            {
                Id = "ANATR",
                Name = "Ana Trujillo Emparedados y helados",
                CurrentBalance = 0
            },
            new Customer
            {
                Id = "ANTON",
                Name = "Antonio Moreno Taquería",
                CurrentBalance = 100
            });
    }
}
```

Puedes notar que estamos configurando las propiedades ***Id***, ***Name*** y ***CurrentBalance*** de la entidad *Customer.* También estamos agregando datos de prueba de 3 clientes, uno de ellos con saldo mayor a cero.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
3. Agrega un nuevo archivo de clase llamado ***ProductConfiguration.cs*** en el directorio ***Configurations*** y remplaza el código de la clase por el siguiente.

```
internal class ProductConfiguration :
    IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> builder)
    {
        builder.Property(p => p.Name)
            .IsRequired()
            .HasMaxLength(40);

        builder.Property(p => p.UnitPrice)
            .HasPrecision(8, 2);

        builder.HasData(
            new Product
            {
                Id = 1,
                Name = "Chai",
                UnitPrice = 35,
                UnitsInStock = 20
            },
            new Product
            {
                Id = 2,
                Name = "Chang",
                UnitPrice = 55,
                UnitsInStock = 0
            },
            new Product
            {
                Id = 3,
                Name = "Aniseed Syrup",
                UnitPrice = 65,
                UnitsInStock = 20
            },
            new Product
            {
                Id = 4,
                Name = "Chef Anton's Cajun Seasoning",
                UnitPrice = 75,
                UnitsInStock = 40
            },
            new Product
            {
                Id = 5,
                Name = "Chef Anton's Gumbo Mix",
                UnitPrice = 50,
```

```
                UnitsInStock = 20
            });
    }
}
```

Puedes notar que estamos configurando las propiedades ***Name*** y ***UnitPrice*** de la entidad *Product.* También estamos agregando datos de prueba de 5 productos, uno de ellos sin unidades en existencia.

4. Abre el archivo ***OrderConfiguration.cs*** del directorio ***Configurations***.
5. Agrega el siguiente código al final del método *Configure* para definir la relación con la entidad *Customer*.

```
builder.HasOne<Customer>()
    .WithMany()
    .HasForeignKey(o => o.CustomerId);
```

6. Abre el archivo ***OrderDetailConfiguration.cs*** del directorio ***Configurations***.
7. Agrega el siguiente código al final del método *Configure* para definir la relación con la entidad *Product.*

```
builder.HasOne<Product>()
    .WithMany()
    .HasForeignKey(p => p.ProductId);
```

Modificar los contextos de datos

1. Abre el archivo *NorthWindContext.cs* del directorio *DataContexts**.***
2. Agrega el siguiente código para incorporar los *DbSet* de las nuevas entidades definidas.

```
public DbSet<Customer> Customers { get; set; }
public DbSet<Product> Products { get; set; }
```

Recuerda que el único propósito de ***NorthWindContext.cs*** es el de administrar la creación y modificación del esquema de la base de datos.

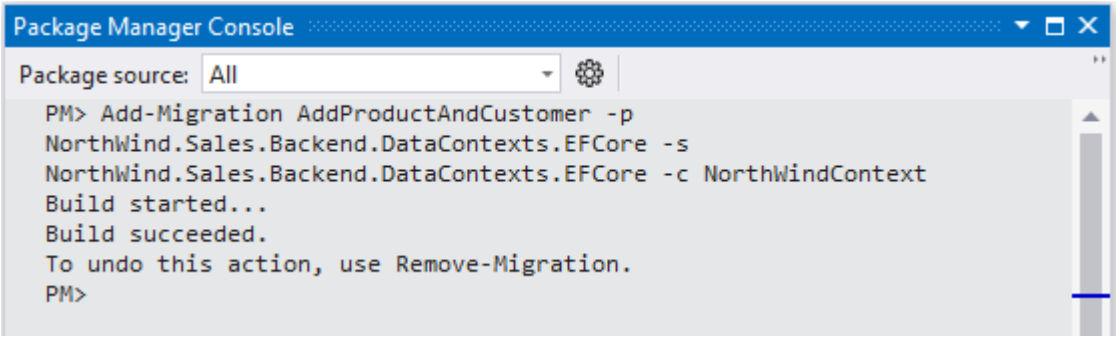
3. Abre el archivo *NorthWindSalesContext.cs* del directorio *DataContexts*.
4. Agrega el siguiente código para incorporar los *DbSet* de las nuevas entidades definidas.

```
public DbSet<Customer> Customers { get; set; }
public DbSet<Product> Products { get; set; }
```

Actualizar el esquema de la base de datos

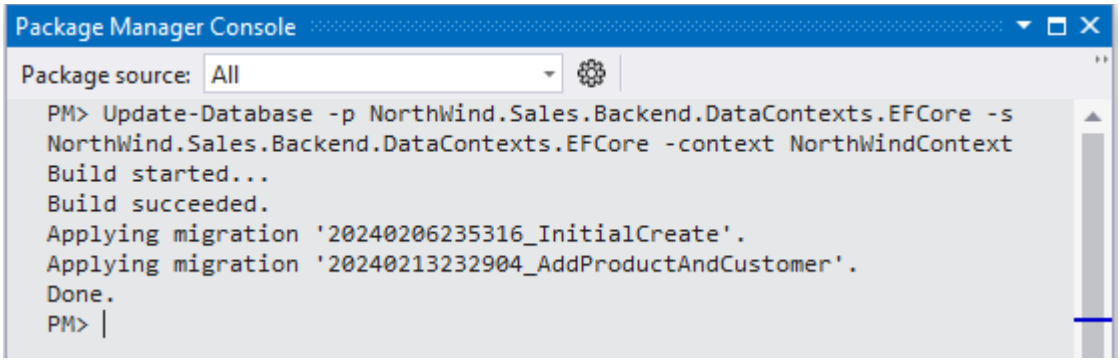
1. Elimina los datos de la tabla *Orders* de la base de datos *NorthWindDB* o si lo deseas, puedes eliminar la base de datos misma. Es importante eliminar los datos debido a que actualmente podríamos tener datos que violen las reglas de integridad establecidas por la relaciones entre entidades que hemos realizado.
2. Ejecuta el siguiente comando en la *Consola de Administración de Paquetes NuGet* (*Package Manager Console*) para crear la nueva migración y verifica que no haya errores.

```
Add-Migration AddProductAndCustomer -p
NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -c NorthWindContext
```

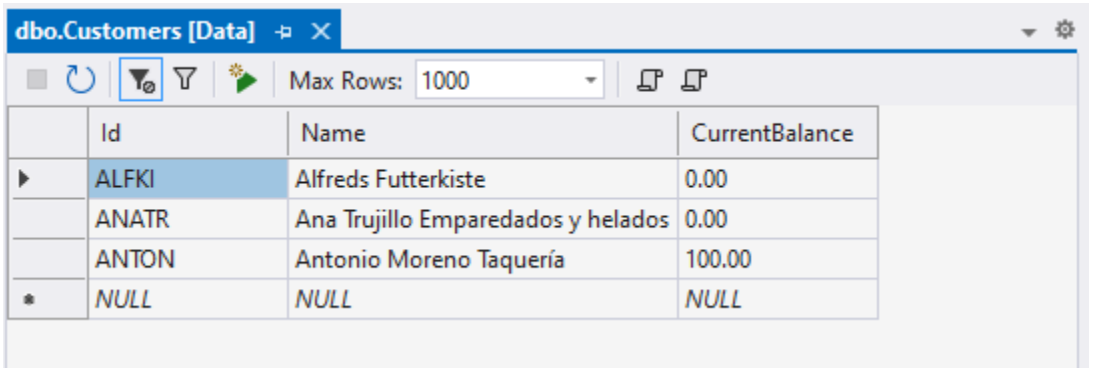


3. Ejecuta el siguiente comando para crear la base de datos y verifica que no haya errores.

```
Update-Database -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -context NorthWindContext
```



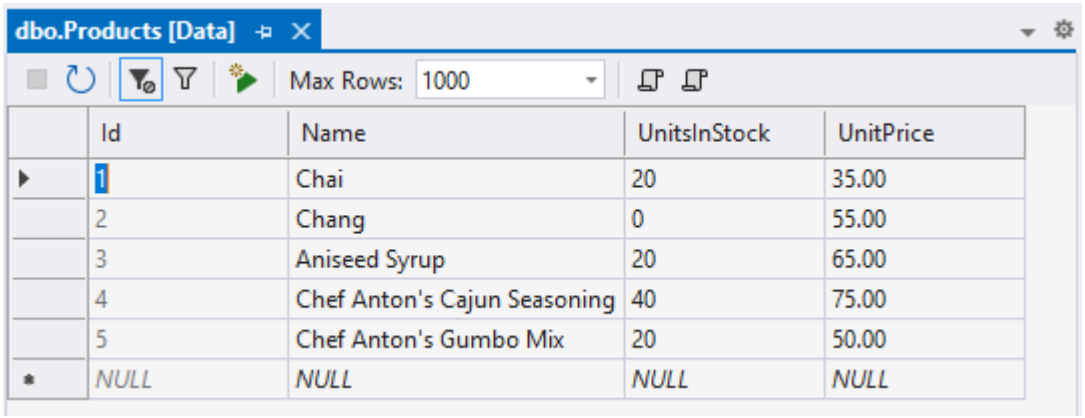
4. Verifica que las tablas ***Customers*** y ***Products*** tengan los datos de ejemplo.



dbo.Customers [Data]

Max Rows: 1000

Id	Name	CurrentBalance
ALFKI	Alfreds Futterkiste	0.00
ANATR	Ana Trujillo Emparedados y helados	0.00
ANTON	Antonio Moreno Taquería	100.00
NULL	*NULL*	*NULL*



dbo.Products [Data]

Max Rows: 1000

Id	Name	UnitsInStock	UnitPrice
1	Chai	20	35.00
2	Chang	0	55.00
3	Aniseed Syrup	20	65.00
4	Chef Anton's Cajun Seasoning	40	75.00
5	Chef Anton's Gumbo Mix	20	50.00
NULL	*NULL*	*NULL*	*NULL*

Implementar la interface de contexto de datos

1. Agrega un nuevo archivo de clase llamado ***NorthWindSalesQueriesDataContext.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindSalesQueriesDataContext :
    NorthWindSalesContext,
    INorthWindSalesQueriesDataContext
{
    public NorthWindSalesQueriesDataContext(IOptions<DBOptions> dbOptions)
        : base(dbOptions)
    {
        ChangeTracker.QueryTrackingBehavior =
            QueryTrackingBehavior.NoTracking;
    }

    public new IQueryable<Customer> Customers => base.Customers;

    public new IQueryable<Product> Products => base.Products;

    public Task<ReturnType> FirstOrDefaultAync<ReturnType>(
        IQueryable<ReturnType> queryable) =>
        queryable.FirstOrDefaultAsync();

    public async Task<IEnumerable<ReturnType>> ToListAsync<ReturnType>(
```

```
        IQueryable<ReturnType> queryable) =>
        await queryable.ToListAsync();
}
```

Registrar el nuevo contexto de datos

1. Abre el archivo *DependencyContainer.cs*.
2. Agrega el siguiente código en el método *AddDataContexts*.

```
public static IServiceCollection AddDataContexts(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions)
{
    services.Configure(configureDBOptions);

    services.AddScoped<INorthWindSalesCommandsDataContext,
    NorthWindSalesCommandsDataContext>();

    services.AddScoped<INorthWindSalesQueriesDataContext,
        NorthWindSalesQueriesDataContext>();


    return services;
}
```

Implementar la validación

Agreguemos ahora nuevos validadores en el proyecto *NorthWind.Sales.Backend.UseCases* que utilizaremos para implementar la validación sobre el medio de persistencia.

Agregar un archivo de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto *NorthWind.Sales.Backend.UseCases.*
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***CreateOrderMessages.resx*** en el directorio *Resources*.
3. Agrega los siguientes valores en el diseñador del archivo de recursos.

Nombre	Valor
CustomerIdNotFoundError	El identificador de cliente proporcionado no existe.
CustomerWithBalanceErrorTemplate	El cliente {0} tiene un adeudo pendiente de {1}.
ProductIdNotFoundErrorTemplate	El producto {0} no existe.
UnitsInStockLessThanQuantityErrorTemplate	Cantidad {0} de {1} no suficiente para producto {2}.

Agregar nuevos validadores

1. Agrega un nuevo archivo de *clase* llamado ***CreateOrderCustomerValidator.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
internal class CreateOrderCustomerValidator(
    IQueriesRepository repository) : IModelValidator<CreateOrderDto>
{
    readonly List<ValidationError> ErrorsField = [];
    public IEnumerable<ValidationError> Errors =>
        ErrorsField;
    public ValidationConstraint Constraint =>
        ValidationConstraint.ValidateIfThereAreNoPreviousErrors;

    public async Task<bool> Validate(CreateOrderDto model)
    {
        var CurrentBalance = await repository
            .GetCustomerCurrentBalance(model.CustomerId);

        if (CurrentBalance == null)
        {
            ErrorsField.Add(new ValidationError(
```

```
                nameof(model.CustomerId),
                CreateOrderMessages.CustomerIdNotFoundError));
        }
        else if (CurrentBalance > 0)
        {
            ErrorsField.Add(new ValidationError(
                nameof(model.CustomerId),
                string.Format(CreateOrderMessages
                .CustomerWithBalanceErrorTemplate,
                model.CustomerId, CurrentBalance)));
        }

        return !ErrorsField.Any();
    }
}
```

Puedes notar que el código de este validador no utiliza el código base del framework de validación, simplemente implementa la interface *IModelValidator*.

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.
3. Agrega un nuevo archivo de *clase* llamado ***CreateOrderProductValidator.cs*** en el directorio *CreateOrder* y remplaza el código de la clase por el siguiente.

```
internal class CreateOrderProductValidator(
    IQueriesRepository repository) : IModelValidator<CreateOrderDto>
{
    readonly List<ValidationError> ErrorsField = [];
    public IEnumerable<ValidationError> Errors =>
        ErrorsField;
    public ValidationConstraint Constraint =>
        ValidationConstraint.ValidateIfThereAreNoPreviousErrors;

    public async Task<bool> Validate(CreateOrderDto model)
    {
        // Obtener los Ids de productos con las cantidades requeridas.
        // Si hay productos repetidos, se suman las cantidades.
        IEnumerable<ProductUnitsInStock> RequiredQuantities =
            model.OrderDetails
            .GroupBy(d => d.ProductId)
            .Select(d => new ProductUnitsInStock(
                d.Key, (short)d.Sum(d => d.Quantity)));

        IEnumerable<int> ProductIds =
            RequiredQuantities.Select(d => d.ProductId);

        // Obtener las existencias en almacén.
        IEnumerable<ProductUnitsInStock> InStockQuantities =
            await repository.GetProductsUnitsInStock(ProductIds);

        // Realizar una operación Lef Join (Uno a muchos) para obtener una
        // colección de elementos:
        // {int ProductId, short Required, short? InStock}.
        // Cada elemento de la colección hará referencia al Id del producto
        // solicitado, la cantidad solicitada y la cantidad disponible
```

```
        // en almacén. Si el producto no existe en almacén, la propiedad
        // InStock tendrá null.
        var RequiredVsInStock = RequiredQuantities
        .GroupJoin(InStockQuantities,
            required => required.ProductId, inStock => inStock.ProductId,
            (oneRequired, manyInStock) => new { oneRequired, manyInStock })
        .SelectMany(groupResult =>
            groupResult.manyInStock.DefaultIfEmpty(), // null si no existe el Id
            (groupResult, singleInStock) =>
            new
            {
                groupResult.oneRequired.ProductId,
                Required = groupResult.oneRequired.UnitsInStock,
                InStock = singleInStock?.UnitsInStock
            });

        // Verificar que cada producto exista y que la cantidad
        // en existencia sea mayor o igual a la cantidad requerida.
        foreach (var Item in RequiredVsInStock)
        {
            if (!Item.InStock.HasValue)
            {
                // Obtener el nombre de la propiedad en el formato:
                // OrderDetails[x].ProductId
                string PropertyName =
                    GetPropertyNameWithIndex(model, Item.ProductId,
                    nameof(CreateOrderDetailDto.ProductId));

                ErrorsField.Add(new ValidationError(PropertyName,
                    string.Format(
                        CreateOrderMessages.ProductIdNotFoundErrorTemplate,
                        Item.ProductId)));
            }
            else
            {
                if (Item.InStock < Item.Required)
                {
                    // Obtener el nombre de la propiedad en el formato:
                    // OrderDetails[x].Quantity
                    string PropertyName =
                        GetPropertyNameWithIndex(model, Item.ProductId,
                        nameof(CreateOrderDetailDto.Quantity));

                    ErrorsField.Add(new ValidationError(PropertyName,
                        string.Format(CreateOrderMessages
                            .UnitsInStockLessThanQuantityErrorTemplate,
                            Item.Required, Item.InStock, Item.ProductId)));
                }
            }
        }

        return !ErrorsField.Any();
    }

    string GetPropertyNameWithIndex(CreateOrderDto model,
        int productId, string propertyName)
    {
        // Localizar el elemento con error de validación.
```

```
        var OrderDetail = model.OrderDetails
            .First(i => i.ProductId == productId);

        // Obtener su índice.
        var OrderDetaiIndex = model.OrderDetails.ToList()
            .IndexOf(OrderDetail);

        // Devolver el nombre de la propiedad incluyendo su índice.
        return string.Format("{0}[{1}].{2}",
            nameof(model.OrderDetails),
            OrderDetaiIndex,
            propertyName);
    }
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar los nuevos validadores

1. Agrega el siguiente código en el método *AddUseCasesServices* del archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.UseCases* para registrar los validadores creados.

```
services.AddModelValidator<CreateOrderDto,
    CreateOrderCustomerValidator>();

services.AddModelValidator<CreateOrderDto,
    CreateOrderProductValidator>();
```

El código del método será similar al siguiente.

```
public static IServiceCollection AddUseCasesServices(
    this IServiceCollection services)
{
    services.AddScoped<ICreateOrderInputPort,
        CreateOrderInteractor>();

    services.AddModelValidator<CreateOrderDto,
        CreateOrderCustomerValidator>();

    services.AddModelValidator<CreateOrderDto,
        CreateOrderProductValidator>();

    return services;
}
```

Probar la funcionalidad

Probemos ahora la nueva funcionalidad de validación.

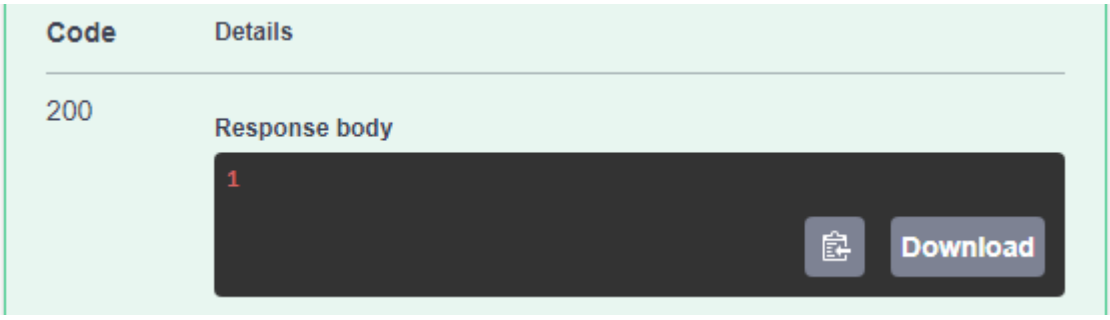
1. Ejecuta la aplicación.

Probar con todos los datos correctos

1. En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado similar al siguiente mostrando el identificador de la orden creada.

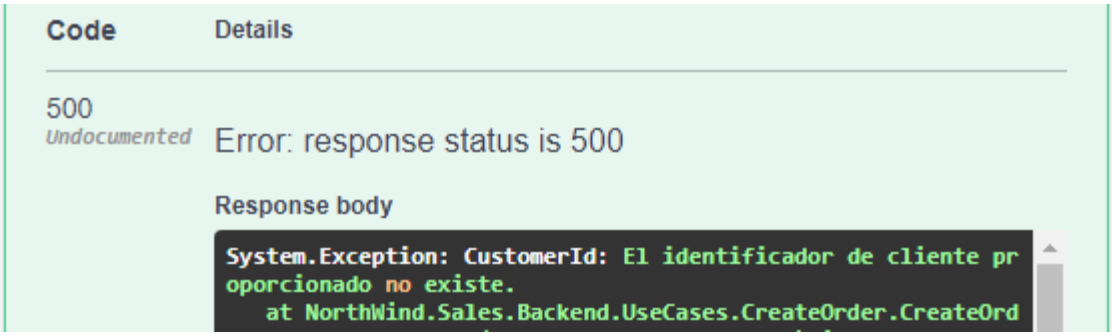


Probar con un cliente no existente

1. Intenta crear una orden con un identificador de cliente no existente.

```
{
    "customerId": "ALFKQ",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



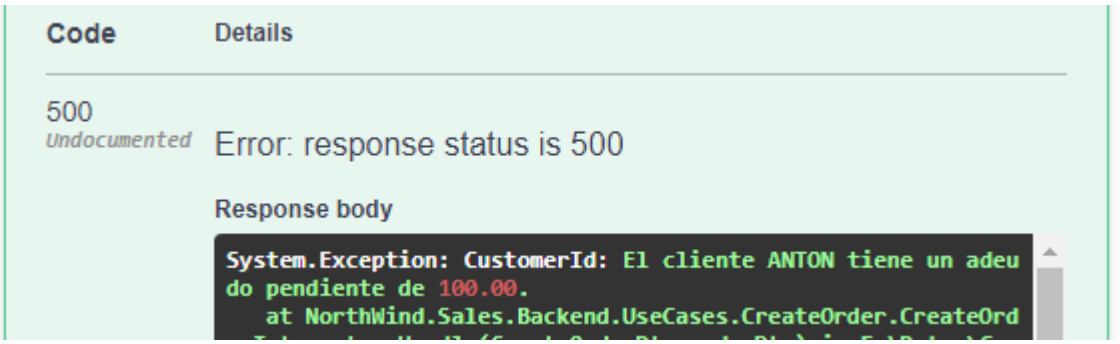
Probar con un cliente con saldo

1. Intenta crear una orden con un cliente que tiene saldo.

```
{
    "customerId": "ANTON",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
```

```
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



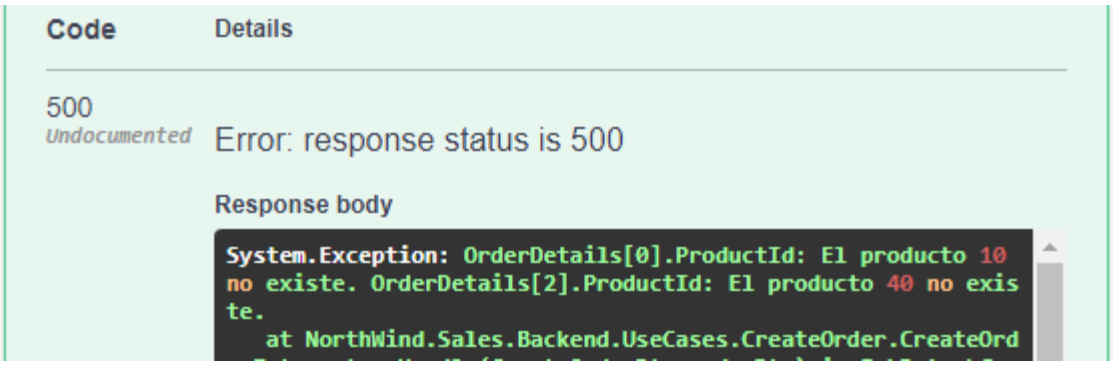
Probar con productos no existentes

1. Intenta crear una orden especificando identificadores de productos no existentes.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 10,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
```

```
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 40,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



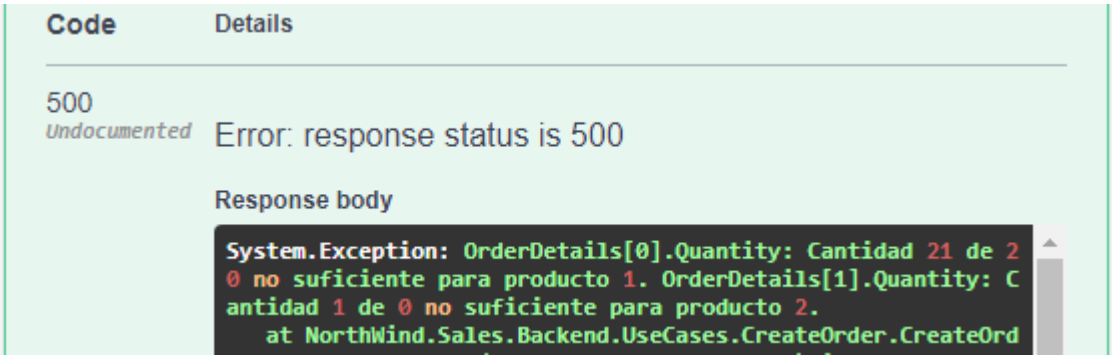
Probar con productos sin existencia

1. Intenta crear una orden con productos sin existencia.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 2,
            "unitPrice": 35,
            "quantity": 1
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
```

```
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        },
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 11
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



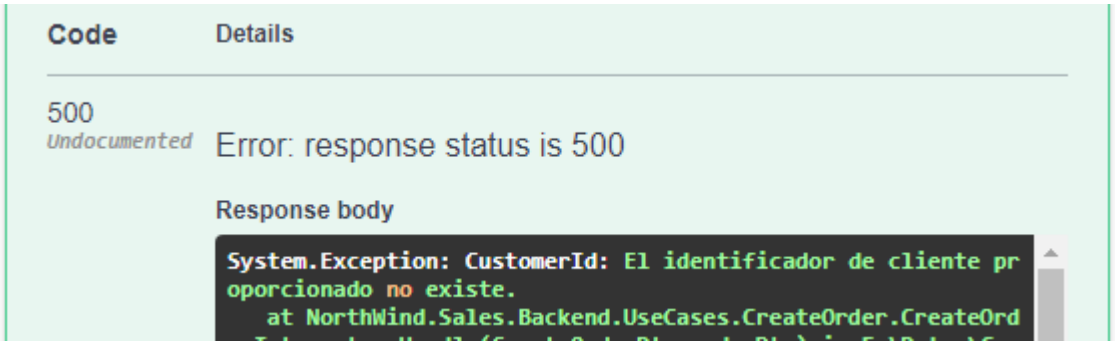
Probar con un cliente no existente y productos sin existencia

1. Intenta crear una orden con un cliente no existente y productos sin existencia.

```
{
    "customerId": "ALFKQ",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 2,
            "unitPrice": 35,
            "quantity": 1
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
```

```
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        },
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 11
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



Puedes notar que únicamente se realizó la validación del cliente y debido a que no fue exitosa la validación, el flujo de validación terminó y la validación de productos ya no fue realizada.

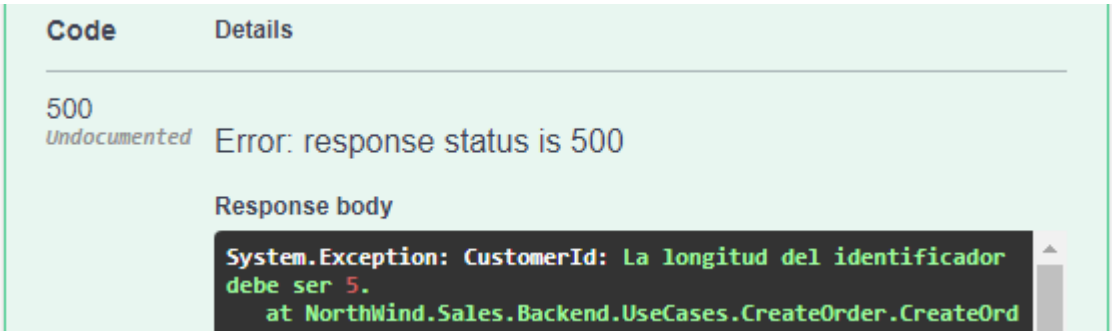
Probar con un identificador de cliente incorrecto y productos sin existencia

1. Intenta crear una orden con un identificador de cliente no valido y productos sin existencia.

```
{
    "customerId": "ALFK",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 2,
            "unitPrice": 35,
            "quantity": 1
        },
        {
```

```
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        },
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 11
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



Puedes notar que únicamente se realizó la validación del identificador del cliente y debido a que no fue exitosa la validación, el flujo de validación terminó y las validaciones sobre la existencia del cliente y de productos ya no fueron realizadas.

La validación está funcionando, aunque los mensajes de error no son muy amigables. En la siguiente lección implementaremos una técnica para manejo de excepciones.

2. Regresa a Visual Studio y detén la ejecución.

Implementar validación en la aplicación Blazor

El servidor ya está implementando la validación de la información requerida, implementemos ahora validación en la aplicación *Blazor WebAssembly* para disminuir el envío de información errónea a la API Web.

Crear un componente de validación

Un componente para validación en *Blazor* no necesita heredar de una clase específica para poder proporcionar validación, lo único que requiere es heredar de la clase base *ComponentBase*. Implementaremos un componente de validación en el proyecto *NorthWind.RazorComponents*.

1. Agrega una referencia del proyecto *NorthWind.Validation.Entities* en el proyecto *NorthWind.RazorComponents*.
2. Agrega un nuevo directorio llamado ***Validators*** en la raíz del proyecto *NorthWind.RazorComponents*.
3. Agrega un nuevo archivo de clase llamado ***ModelValidator.cs*** en el directorio *Validators* y modifica el código de la clase para que sea similar al siguiente.

```
public class ModelValidator<T> : ComponentBase
{

}
```

Puedes notar que estamos creando un componente genérico para validación. `T` representa el tipo del modelo que deseamos validar.

4. Agrega el siguiente código a la clase para definir un parámetro de cascada que será pasado a nuestro componente desde su contenedor padre ***<EditForm>***. Este parámetro cambiará su valor cada vez que el modelo del componente *EditForm* cambie.

```
[CascadingParameter]
EditContext EditContext { get; set; }
```

5. Agrega el siguiente código a la clase para definir un parámetro que reciba la instancia del servicio de validación.

```
[Parameter]
public IModelValidatorHub<T> Validator { get; set; }
```

6. Agrega el siguiente código a la clase para definir una variable que almacene los mensajes de validación para el *EditContext*.

```
ValidationMessageStore ValidationMessageStore;
```

7. Agrega el siguiente código a la clase para definir un método que obtenga el identificador de la propiedad que genera un error de validación. Este código es principalmente útil en las propiedades de tipo colección donde el nombre de propiedad con error incluye el nombre de la propiedad colección, el índice del elemento de la colección y el nombre de la propiedad que causó un error de validación.

```
FieldIdentifier GetFieldIdentifier(object model,
    string propertyName)
{
    char[] PropertyNameSeparators = new[] { '.', '[' };
    object NewModel = model;

    string PropertyPath = propertyName;
    int SeparatorIndex;
    string Token = null;

    do
    {
        SeparatorIndex =
            PropertyPath.IndexOfAny(PropertyNameSeparators);

        if (SeparatorIndex >= 0)
        {
            // Extraer la cadena que está antes del
            // índice encontrado.
            Token =
                PropertyPath.Substring(0, SeparatorIndex);

            // Eliminar la subcadena de la cadena.
            PropertyPath = PropertyPath
                .Substring(SeparatorIndex + 1);

            if (Token.EndsWith("]"))
            {
                // Se trata de un índice, p. ej.: "3]"
                // Extraer el valor del índice.
                Token = Token.Substring(0, Token.Length - 1);

                // Extraer la propiedad Item del modelo.
                var PropertyInfo =
                    NewModel.GetType().GetProperty("Item");

                // Obtener el tipo del Indexer.
                var IndexerType =
                    PropertyInfo.GetIndexParameters()[0]
                    .ParameterType;

                // Obtener el valor del Indexer, p. ej.: 3
```

```
                var IndexerValue =
                    Convert.ChangeType(Token, IndexerType);

                // Obtener el nuevo modelo.
                NewModel = PropertyInfo.GetValue(NewModel,
                    new object[] { IndexerValue });
            }
            else
            {
                // Es una propiedad normal.
                var PropertyInfo = NewModel.GetType()
                    .GetProperty(Token);
                NewModel = PropertyInfo.GetValue(NewModel);
            }

            Token = null;
        }
    } while (SeparatorIndex >= 0);

    return new FieldIdentifier(NewModel,
        Token ?? PropertyPath);
}
```

En el siguiente enlace se puede encontrar el código fuente que sirvió como base para implementar el código anterior.

FluentValidator.cs
https://gist.github.com/SteveSandersonMS/090145d7511c5190f62a409752c60d00#file-fluentvalidator-cs

8. Agrega el siguiente código para implementar un método que permita agregar errores al objeto que almacena los mensajes de errores de validación. Este método podrá ser utilizado para permitir agregar errores de validación que devuelva el código backend, por ejemplo, cuando el identificador del cliente no exista.

```
public void AddErrors(IEnumerable<ValidationError> errors)
{
    // Eliminar mensajes de validación existentes.
    ValidationMessageStore.Clear();

    // Agregar los errores de validación.
    foreach (var Error in errors)
    {
        var FieldIdentifier =
            GetFieldIdentifier(EditContext.Model, Error.PropertyName);
        ValidationMessageStore.Add(FieldIdentifier, Error.Message);
    }

    EditContext.NotifyValidationStateChanged();
}
```

Al agregar mensajes de error, creamos un ***FieldIdentifier*** para identificar el objeto que tiene un error. Después de agregar los mensajes de error, notificamos al *EditContext* para que sepa que el estado de validación ha cambiado.

9. Agrega el siguiente código a la clase para definir el manejador del evento *OnValidationRequested*. Este evento es disparado cuando todas las propiedades de ***EditContext.Model*** necesitan ser validadas. Esto ocurre cuando el usuario intenta hacer un "post" del *EditForm* para que *Blazor* pueda determinar si la entrada de datos es válida o no.

```
async void ValidationRequested(object sender,
    ValidationRequestedEventArgs args)
{
    // Validar el modelo.
    bool IsValid = await Validator.Validate((T)EditContext.Model);

    if (IsValid)
    {
        ValidationMessageStore.Clear();
        EditContext.NotifyValidationStateChanged();
    }
    else
    {
        AddErrors(Validator.Errors);
    }
}
```

10. Agrega el siguiente código a la clase para definir el manejador del evento ***OnFieldChanged*** del *EditContext*. Este evento ocurre cuando el usuario modifica el valor de una propiedad del *EditContext.Model* al editar algún componente derivado de ***InputBase<T>***.

```
async void FieldChanged(object sender, FieldChangedEventArgs e)
{
    // Eliminamos mensajes de error del campo modificado.
    ValidationMessageStore.Clear(e.FieldIdentifier);

    // Validar el modelo.
    bool IsValid = await Validator.Validate((T)EditContext.Model);

    if (!IsValid)
    {
        foreach (var Item in Validator.Errors)
        {
            var FieldIdentifier = GetFieldIdentifier(
                EditContext.Model, Item.PropertyName);

            // Agregamos únicamente mensajes de error de la propiedad modificada.
            if (FieldIdentifier.FieldName == e.FieldIdentifier.FieldName &&
                FieldIdentifier.Model == e.FieldIdentifier.Model)
            {
                ValidationMessageStore.Add(FieldIdentifier, Item.Message);
            }
```

```
        }
    }

    EditContext.NotifyValidationStateChanged();
}
```

11. Agrega el siguiente código a la clase para sobrescribir el método ***SetParametersAsync*** que es ejecutado cuando uno de los parámetros del componente cambie, incluyendo el parámetro de cascada *EditContext*.

```
public override async Task SetParametersAsync(ParameterView parameters)
{
    // Guardar una referencia de EditContext que tiene los valores
    // originales para poder determinar si hay cambios.
    // EditContext es creado por EditForm cuando EditForm.Model es modificado.
    EditContext PreviousEditContext = EditContext;

    // Establecer el valor de los parámetros, incluyendo el EditContext.
    await base.SetParametersAsync(parameters);

    // Verificar si el EditContext cambió.
    if (EditContext != PreviousEditContext)
    {
        // Crear un nuevo ValidationMessageStore.
        ValidationMessageStore = new ValidationMessageStore(EditContext);

        // Establecer los manejadores de eventos.
        EditContext.OnValidationRequested += ValidationRequested;
        EditContext.OnFieldChanged += FieldChanged;
    }
}
```

12. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Modificar el Gateway CreateOrder

Hagamos una modificación en el proyecto *NorthWind.Sales.Frontend.WebApiGateways.*

1. Abre el archivo ***CreateOrderGateway.cs***.
2. Modifica el código del método *CreateOrderAsync* para generar una excepción cuando la Web API retorne un resultado no exitoso.

```
public async Task<int> CreateOrderAsync(CreateOrderDto order)
{
    int OrderId = 0;
    var Response = await client.PostAsJsonAsync(
        Endpoints.CreateOrder, order);

    if (Response.IsSuccessStatusCode)
```

```
    {
        OrderId =
            await Response.Content.ReadFromJsonAsync<int>();
    }
    else
    {
        throw new HttpRequestException(
            await Response.Content.ReadAsStringAsync());
    }

    return OrderId;
}
```

Agregar un validador del ViewModel

1. Agrega un nuevo archivo de clase llamado ***CreateOrderViewModelValidator.cs*** en el directorio *ViewModels\CreateOrder* del proyecto ***NorthWind.Sales.Frontend.Views*** y remplaza el contenido de la clase por el siguiente.

```
internal class CreateOrderViewModelValidator(
    IModelValidatorHub<CreateOrderDto> validator) :
    AbstractViewModelValidator<CreateOrderDto, CreateOrderViewModel>(validator,
        ValidationConstraint.AlwaysValidate)
{
}
```

   La clase anterior utiliza *AbstractViewModelValidator* para devolver el estado de validación del DTO asociado *CreateOrderDto*.

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Modificar el ViewModel

Modifiquemos el *ViewModel* para incorporar el validador. Este validador será utilizado por el componente *CreateOrderComponent*.

1. Abre el archivo *CreateOrderViewModel.cs*.
2. Modifica la definición del constructor primario de la clase para declarar la dependencia del validador del *ViewModel*.

```
public class CreateOrderViewModel(ICreateOrderGateway gateway,
    IModelValidatorHub<CreateOrderViewModel> validator)
```

3. Agrega el siguiente código a la clase para definir una propiedad que almacene un instancia del validador del *ViewModel*.

```
public IModelValidatorHub<CreateOrderViewModel> Validator => validator;
```

Utilizar el componente de validación

1. Abre el archivo *CreateOrderComponent.razor* del proyecto *NorthWind.Sales.Frontend.Views*.
2. Agrega el siguiente código al inicio del componente *EditForm* para incorporar el componente de validación y el componente que muestra la lista de mensajes de validación.

```
<EditForm Model=Order OnValidSubmit="Order.Send">

    <ModelValidator Validator="Order.Validator"/>
    <ValidationSummary/>
```

3. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using NorthWind.RazorComponents.Validators
```

4. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *CustomerId*.

```
    <div class="form-group">
        <label for="customerId">
            @CreateOrderMessages.CustomerIdLabel
        </label>
        <InputText
            class="form-control" id="customerId"
            placeholder="@CreateOrderMessages.CustomerIdLabel"
            @bind-Value=Order.CustomerId/>
        <ValidationMessage For="() => Order.CustomerId"/>
    </div>
```

5. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *ShipAddress*.

```
    <div class="form-group">
        <label for="shipAddress">
            @CreateOrderMessages.ShipAddressLabel
        </label>
        <InputText
            class="form-control" id="shipAddress"
            placeholder="@CreateOrderMessages.ShipAddressLabel"
            @bind-Value=Order.ShipAddress />
        <ValidationMessage For="() => Order.ShipAddress"/>
    </div>
```

6. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *ShipCity*.

```
    <div class="form-group">
        <label for="shipCity">
            @CreateOrderMessages.ShipCityLabel
        </label>
        <InputText
            class="form-control" id="shipCity"
            placeholder="@CreateOrderMessages.ShipCityLabel"
            @bind-Value=Order.ShipCity />
        <ValidationMessage For="() => Order.ShipCity"/>
    </div>
```

7. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *ShipCountry*.

```
    <div class="form-group">
        <label for="shipCountry">
            @CreateOrderMessages.ShipCountryLabel
        </label>
        <InputText
            class="form-control" id="shipCountry"
            placeholder="@CreateOrderMessages.ShipCountryLabel"
            @bind-Value=Order.ShipCountry />
        <ValidationMessage For="() => Order.ShipCountry"/>
    </div>
```

8. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *ShipPostalCode*.

```
    <div class="form-group">
        <label for="shipPostalCode">
            @CreateOrderMessages.ShipPostalCodeLabel
        </label>
        <InputText
            class="form-control" id="shipPostalCode"
            placeholder="@CreateOrderMessages.ShipPostalCodeLabel"
            @bind-Value=Order.ShipPostalCode />
        <ValidationMessage For="() => Order.ShipPostalCode"/>
    </div>
```

9. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *ProductId.*

```
<div class="col">
    <InputNumber
        class="form-control"
        placeholder="@CreateOrderMessages.ProductIdLabel"
        @bind-Value="Item.ProductId" />
    <ValidationMessage For="() => Item.ProductId"/>
</div>
```

10. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *UnitPrice.*

```
<div class="col">
    <InputNumber
        class="form-control"
        placeholder="@CreateOrderMessages.UnitPriceLabel"
        @bind-Value="Item.UnitPrice" />
    <ValidationMessage For="() => Item.UnitPrice"/>
</div>
```

11. Agrega el siguiente código para mostrar el mensaje de validación de la propiedad *Quantity.*

```
<div class="col">
    <InputNumber
        class="form-control"
        placeholder="@CreateOrderMessages.QuantityLabel"
        @bind-Value="Item.Quantity" />
    <ValidationMessage For="() => Item.Quantity"/>
</div>
```

Registrar el validador

1. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Frontend.Views*.
2. Modifica el código del método *AddViewsServices* para registrar el validador del *ViewModel*.

```
public static IServiceCollection AddViewsServices(
    this IServiceCollection services)
{
    services.AddScoped<CreateOrderViewModel>();

    services.AddModelValidator<CreateOrderViewModel,
        CreateOrderViewModelValidator>();

    return services;
}
```

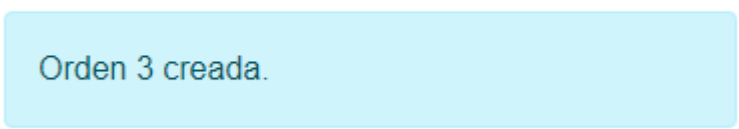
Probar la funcionalidad

1. Ejecuta la aplicación.
2. Haz clic en el enlace ***Crear orden*** de la aplicación *Blazor*.

Probar con todos los datos correctos

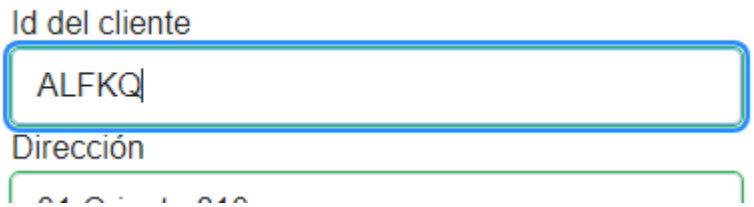
1. Intenta crear una orden proporcionando todos los datos correctos.

Podrás ver un resultado similar al siguiente.



Probar con un cliente no existente

1. Intenta crear una orden con un identificador de cliente no existente.



Podrás ver un resultado similar al siguiente mostrando el error de validación.

System.Exception: CustomerId: El identificador de cliente proporcionado no existe. at NorthWind.Sales.Backend.UseCases.CreateOrder.CreateOrderInteractor.Handle(CreateOrderDto orderDto) in

2. Haz clic en el botón ***Aceptar*** para regresar a la página de datos de la orden.

Probar con un cliente con saldo

1. Intenta crear una orden con un cliente que tiene saldo.

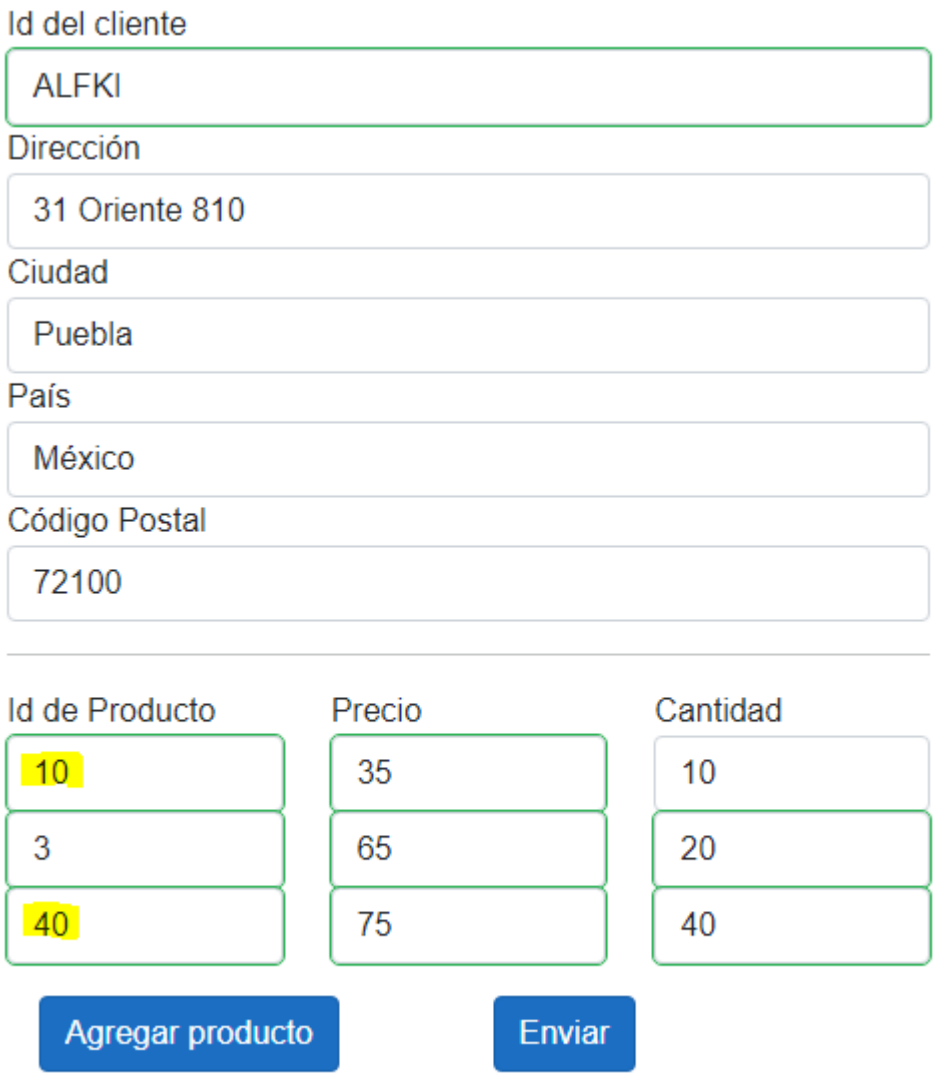
Podrás ver un resultado similar al siguiente mostrando el error de validación.

System.Exception: CustomerId: El cliente ANTON tiene un adeudo pendiente de 100.00. at NorthWind.Sales.Backend.UseCases.CreateOrder.CreateOrderInteractor.Handle(CreateOrderDto orderDto) in

2. Haz clic en el botón ***Aceptar*** para regresar a la página de datos de la orden.

Probar con productos no existentes

1. Intenta crear una orden especificando datos correctos, pero con identificadores de productos no existentes.



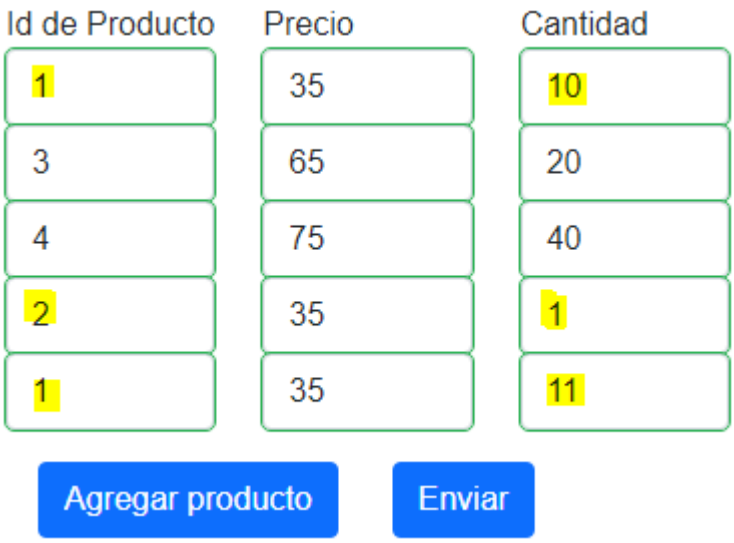
Podrás ver un resultado similar al siguiente mostrando el error de validación.

System.Exception: OrderDetails[0].ProductId: El producto 10 no existe. OrderDetails[2].ProductId: El producto 40 no existe. at NorthWind.Sales.Backend.UseCases.CreateOrder.CreateOrderInteractor.Handle(CreateOrderDto orderDto) in

2. Haz clic en el botón ***Aceptar*** para mostrar la página de datos de la orden.

Probar con productos sin existencia

1. Intenta crear una orden con productos sin existencia.



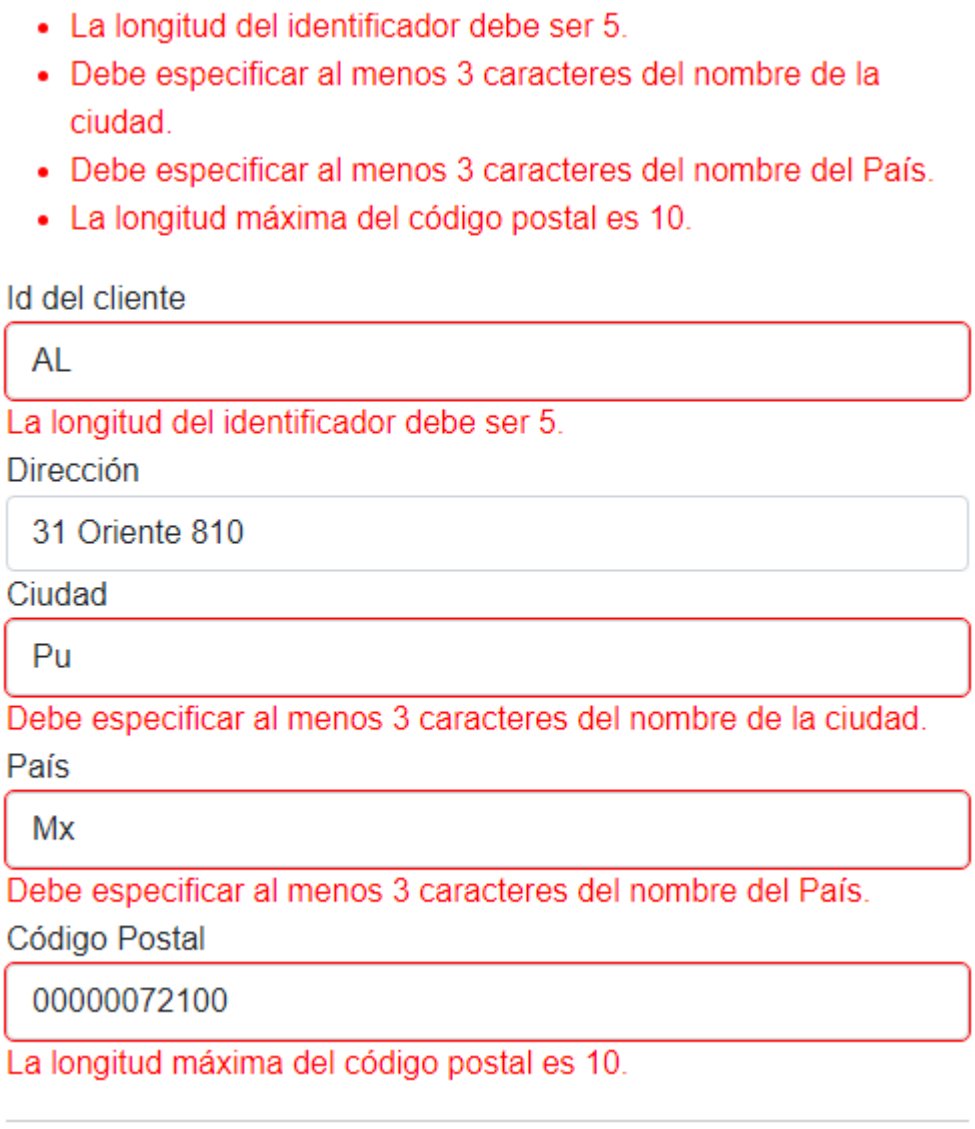
Podrás ver un resultado mostrando el error de validación.

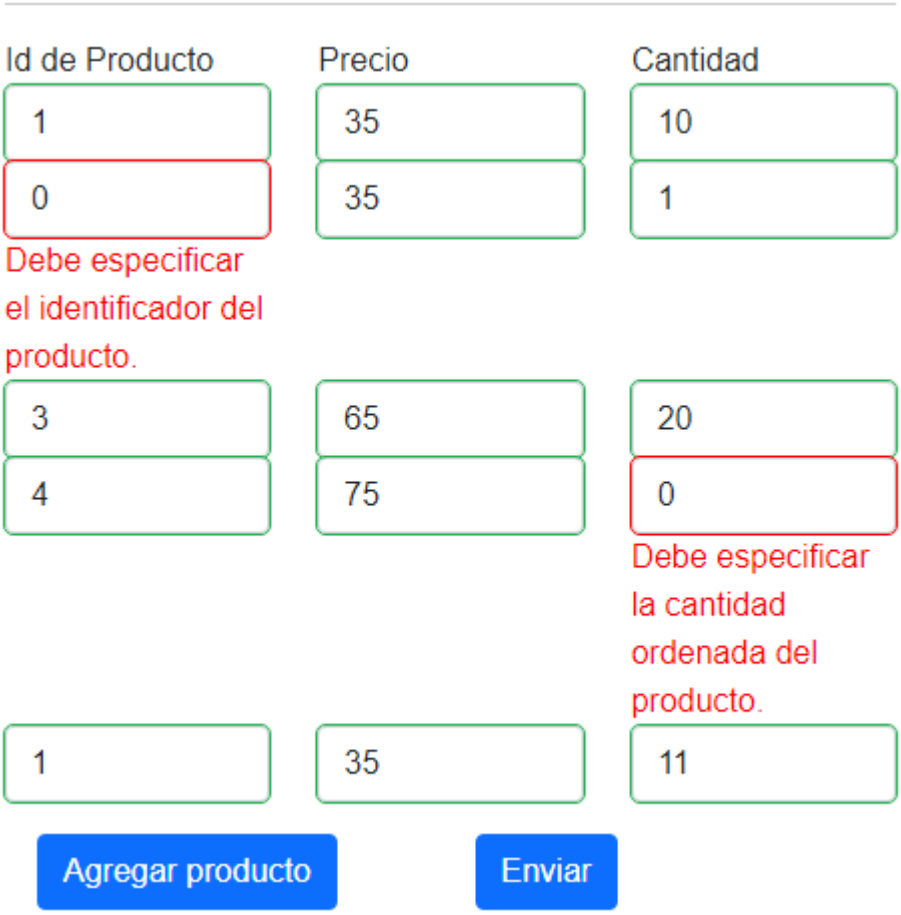
System.Exception: OrderDetails[0].Quantity: Cantidad 21 de 20 no suficiente para producto 1. OrderDetails[3].Quantity: Cantidad 1 de 0 no suficiente para producto 2. at NorthWind.Sales.Backend.UseCases.CreateOrder.CreateOrderInteractor.Handle(CreateOrderDto orderDto) in

2. Haz clic en el botón ***Aceptar*** para mostrar la página de datos de la orden.

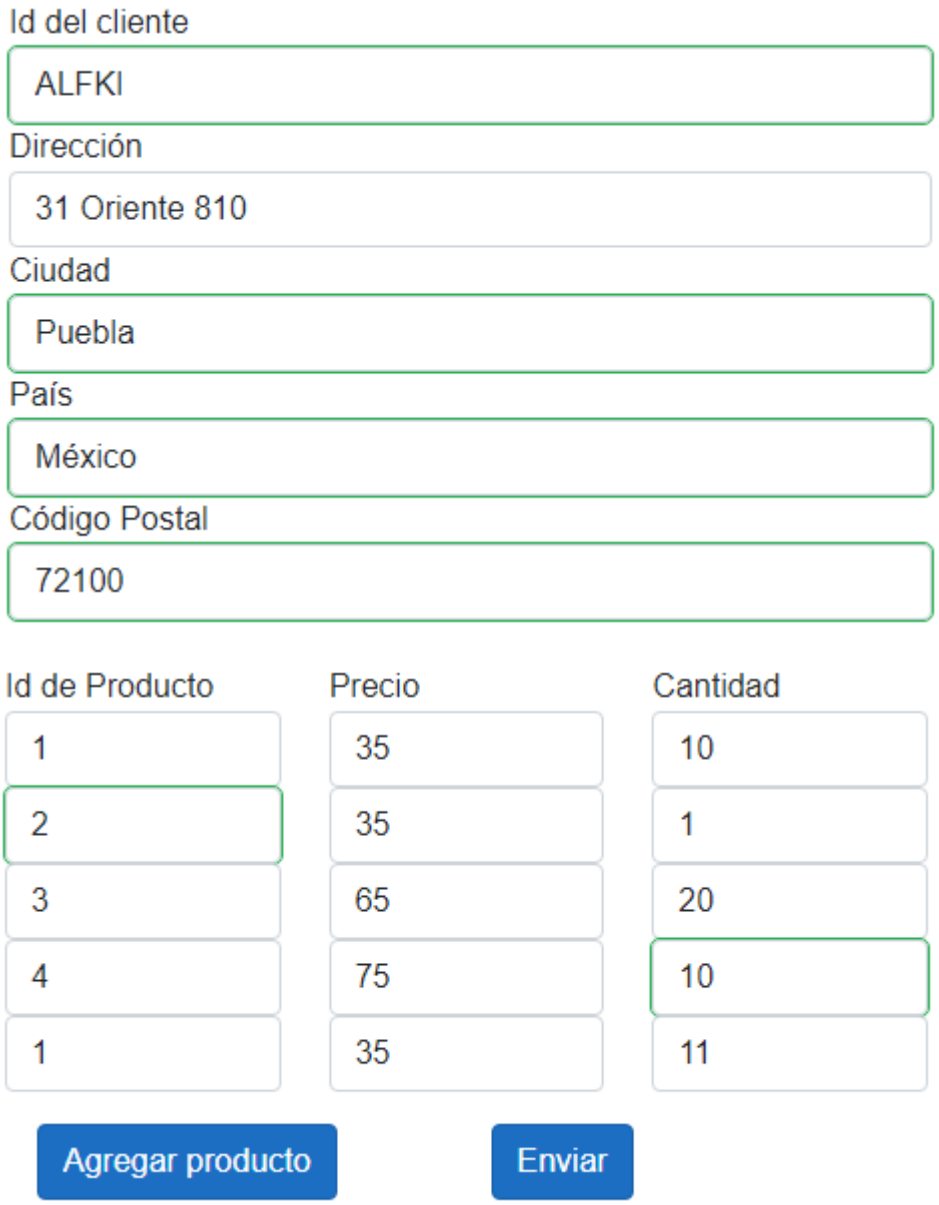
Probar con errores en los datos

1. Intenta crear una orden con errores en los datos. Podrás ver que se muestran los errores mientras modificas los datos sin que se envíe la información a la Web API.





2. Corrige los errores de datos, podrás ver que conforme vas corrigiendo, los mensajes de error van desapareciendo.



3. Regresa a Visual Studio y detén la ejecución.

Hemos creado un validador simple para *Blazor* siguiendo una arquitectura limpia de software. En la siguiente lección implementaremos un manejo de errores para mostrar información de error al usuario de una forma más amigable.

Práctica 8:

Manejo de excepciones

Cuando ejecutamos algún tipo de lógica de negocios y deseamos reportar alguna condición significativa, por ejemplo, para notificar al usuario que una determinada acción no pudo ser realizada, es común hacerlo lanzado algún tipo de excepción personalizada.

En esta práctica mostraremos una forma de manejar excepciones de forma centralizada para devolver a los clientes respuestas apropiadas implementando *Clean Architecture*.

Objetivos

En esta práctica los estudiantes:

- Crearán tipos de excepciones personalizadas para notificar condiciones de error.
- Implementarán el manejo de excepciones de forma centralizada en *ASP.NET Core*.

Implementar un framework de Excepciones

De la misma forma en que implementamos un framework de validación, implementaremos ahora un framework de *Excepciones* cuyo objetivo será el de facilitar el manejo de excepciones en el código backend, cualquier aplicación de backend podrá consumir este framework.

En este framework definiremos excepciones personalizadas generales y expondremos un componente para manejo global de excepciones personalizadas o de sistema que el código de la aplicación no haya manejado.

Crear un proyecto para el framework de Excepciones

1. Utiliza la plantilla *Class Library* para agregar un nuevo proyecto llamado ***NorthWind.Exceptions.Entities*** en el directorio ***1. Enterprise Business Rules***.
2. Elimina el archivo ***Class1.cs***.
3. Agrega el siguiente código al archivo del proyecto para agregar una referencia del framework ***Microsoft.AspNetCore.app*** que contiene ensamblados de soporte de aplicaciones ASP.NET Core.

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

Definir tipos de excepción personalizadas

Definamos algunas excepciones generales que nos permitan reportar ciertas condiciones de error, por ejemplo, errores de validación o errores de actualización al persistir los datos.

Definir la excepción ValidationException

Esta excepción será lanzada cuando se hayan encontrado errores de validación, tal como lo hacemos en el caso de uso "*Crear Orden de Compra*". En una lección posterior implementaremos el patrón *Result* para notificar errores de validación en lugar de lanzar excepciones.

1. Agrega un referencia del proyecto *NorthWind.Validation.Entities*.
2. Agrega un nuevo directorio llamado ***Exceptions*** en la raíz del proyecto.
3. Agrega un nuevo archivo de clase llamado ***ValidationException.cs*** en el directorio *Exceptions* y remplaza el código de la clase por el siguiente.

```
public class ValidationException : Exception
{
    public ValidationException() { }
    public ValidationException(string message) : base(message) { }
    public ValidationException(string message, Exception innerException)
        : base(message, innerException) { }

    public ValidationException(IEnumerable<ValidationError> errors) =>
        Errors = errors;
    public IEnumerable<ValidationError> Errors { get; }
}
```

La clase *ValidationException* nos permitirá encapsular la lista de errores de validación en la propiedad *Errors*.

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Definir la excepción UpdateException

Esta excepción será lanzada cuando se encuentren errores durante la persistencia de datos en la fuente de almacenamiento.

1. Agrega un nuevo archivo de clase llamado ***UpdateException.cs*** en el directorio *Exceptions* y remplaza el código de la clase por el siguiente.

```
public class UpdateException : Exception
{
    public UpdateException(){}

    public UpdateException(string message) : base(message){}

    public UpdateException(string message, Exception innerException)
        : base(message, innerException){}

    public UpdateException(Exception exception, IEnumerable<string> entities)
        : base(exception.Message, exception) =>
        Entities = entities;

    public IEnumerable<string> Entities { get; }

}
```

La clase *UpdateException* nos permitirá encapsular errores generados al persistir los datos en la fuente de datos. La propiedad *Entities* tendrá la lista de entidades que causaron la excepción.

Implementar manejadores de excepción

El formato de la respuesta que devolverá el manejador de la excepción estará basado en la especificación https://tools.ietf.org/html/rfc7807. Esta especificación define un "*detalle del problema*" como una forma estándar de representar errores en una respuesta HTTP y evitar la necesidad de definir nuevos formatos de respuesta de error para las APIs HTTP.

Agregar un método de extension de HttpContext

Agreguemos una extensión para el tipo `HttpContext` que nos permita escribir una instancia de *ProblemDetails* a la respuesta de una petición HTTP.

1. Agrega un nuevo directorio llamado ***Extensions*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***HttpContextExtensions.cs*** en el directorio *Extensions* y remplaza el código de la clase por el siguiente.

```
internal static class HttpContextExtensions
{
    public static async ValueTask WriteProblemDetailsAsync(
        this HttpContext context, ProblemDetails details)
    {
        // ProblemDetails tiene su propio content-type.
        context.Response.ContentType = "application/problem+json";

        // Código de respuesta HTTP a devolver.
        context.Response.StatusCode = details.Status.Value;

        // Serializar la respuesta.
        var Stream = context.Response.Body;
        await JsonSerializer.SerializeAsync(Stream, details);
    }
}
```

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar un archivo de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***ExceptionMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
ValidationExceptionTitle	Error en los datos de entrada.
ValidationExceptionDetail	Se encontraron uno o más errores de validación de datos.

UpdateExceptionTitle	Error de actualización.
UpdateExceptionDetail	Se encontraron uno o más errores de actualización de datos. Consulte al administrador.
UnhandledExceptionTitle	El servicio no se encuentra disponible.
UnhandledExceptionDetail	Consulte al administrador.

Implementar el manejador de la excepción ValidationException

1. Agrega un nuevo directorio llamado ***ExceptionHandlers*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ValidationExceptionHandler.cs*** en el directorio *ExceptionHandlers* y remplaza el código de la clase por el siguiente.

```
internal class ValidationExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext,
        Exception exception, CancellationToken cancellationToken)
    {
        bool Handled = false;
        if (exception is ValidationException Ex)
        {
            ProblemDetails Details = new ProblemDetails();
            Details.Status = StatusCodes.Status400BadRequest;
            Details.Type =
                "https://datatracker.ietf.org/doc/html/rfc7231#section-6.5.1";
            Details.Title = ExceptionMessages.ValidationExceptionTitle;
            Details.Detail = ExceptionMessages.ValidationExceptionDetail;
            Details.Instance =
                $"{nameof(ProblemDetails)}/{nameof(ValidationException)}";

            Details.Extensions.Add("errors", Ex.Errors);
            await httpContext.WriteProblemDetailsAsync(Details);
            Handled = true;
        }

        return Handled;
    }
}
```

Puedes notar que la clase implementa la interface `IExceptionHandler` definida en el espacio de nombres `Microsoft.AspNetCore.Diagnostics` a partir de .NET 8. *IExceptionHandler* representa una interfaz para manejar excepciones en aplicaciones ASP.NET Core. Las implementaciones de esta interface son utilizadas por el middleware de manejo de excepciones `ExceptionHandlerMiddleware`.

La interface *IExceptionHandler* define el método `TryHandleAsync` cuyo propósito es intentar manejar la excepción especificada de forma asincrónica dentro de la canalización de ASP.NET

Core. Las implementaciones de este método pueden proporcionar una lógica de control de excepciones personalizada para diferentes escenarios.

Si un manejador de excepciones maneja una solicitud, puede devolver `true` para detener el procesamiento. Si una excepción no es controlada por ningún manejador de excepciones, el control vuelve al comportamiento y las opciones predeterminadas del middleware. El manejador que hemos implementado manejará las excepciones de tipo `ValidationException` y por tal motivo el método `TryHandleAsync` devolverá `true` cuando se trate de una excepción de tipo *ValidationException*.

El método genera una instancia de *ProblemDetails* que es escrita como respuesta de la petición a través del método de extensión *WriteProblemDetailsAsync* que creamos previamente.

La propiedad ***Instance*** tiene una cadena con formato "***ProblemDetails/ValidationException***" que nos permitirá identificar los errores de validación que se hayan generado.

La propiedad ***Extensions*** de tipo *IDictionary* almacena un elemento de diccionario cuya llave es la cadena "`errors`" y su valor es el resultado de la validación que incluye los nombres de propiedades con sus mensajes de error.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar el manejador de la excepción UpdateException

1. Agrega un nuevo archivo de clase llamado ***UpdateExceptionHandler.cs*** en el directorio *ExceptionHandlers* y remplaza el código de la clase por el siguiente.

```
internal class UpdateExceptionHandler(ILogger<UpdateExceptionHandler> logger) :
    IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext,
        Exception exception, CancellationToken cancellationToken)
    {
        bool Handled = false;
        if (exception is UpdateException Ex)
        {
            ProblemDetails Details = new ProblemDetails();
            Details.Status = StatusCodes.Status500InternalServerError;
            Details.Type =
                "https://datatracker.ietf.org/doc/html/rfc7231#section-6.6.1";
            Details.Title = ExceptionMessages.UpdateExceptionTitle;
            Details.Detail = ExceptionMessages.UpdateExceptionDetail;
            Details.Instance =
                $"{nameof(ProblemDetails)}/{nameof(UpdateException)}";

            logger.LogError(exception,
```

```
                ExceptionMessages.UpdateExceptionTitle +
                ": " + string.Join(" ", Ex.Entities));

            await httpContext.WriteProblemDetailsAsync(Details);

            Handled = true;
        }

        return Handled;
    }
}
```

Cuando una excepción *UpdateException* sea generada, el detalle de la excepción se registrará a través del servicio *ILogger*.

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar un manejador de excepciones no manejadas

Agreguemos ahora un manejador de excepciones que procese aquellas excepciones que no tengan un manejador que implemente ***IExceptionHandler<T>***.

1. Agrega un nuevo archivo de clase llamado ***UnhandledExceptionHandler.cs*** en el directorio *ExceptionHandlers* y remplaza el código de la clase por el siguiente.

```
internal class UnhandledExceptionHandler(
    ILogger<UnhandledExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext,
        Exception exception, CancellationToken cancellationToken)
    {
        ProblemDetails Details = new ProblemDetails();
        Details.Status = StatusCodes.Status500InternalServerError;
        Details.Type =
            "https://datatracker.ietf.org/doc/html/rfc7231#section-6.6.1";
        Details.Title = ExceptionMessages.UnhandledExceptionTitle;
        Details.Detail = ExceptionMessages.UnhandledExceptionDetail;
        Details.Instance =
            $"{nameof(ProblemDetails)}/{exception.GetType()}";

        logger.LogError(exception,
            ExceptionMessages.UnhandledExceptionTitle);

        await httpContext.WriteProblemDetailsAsync(Details);

        return true;
    }
}
```

Si un manejador de excepciones maneja una solicitud, puede devolver `true` para detener el procesamiento. Si una excepción no es controlada por ningún manejador de excepciones, el control vuelve al comportamiento y las opciones predeterminadas del middleware. El manejador que hemos implementado manejará todas las excepciones y por tal motivo el método `TryHandleAsync` devuelve `true`.

Cuando registremos este manejador de excepciones, deberá ser registrado después de todos los posibles manejadores que implementen *IExceptionHandler*, de no hacerlo así, no dará oportunidad a que otros manejadores registrados puedan manejar excepciones.

5. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo con el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
}
```

2. Agrega el siguiente código a la clase para permitir el registro del manejador de excepciones de validación predeterminado.

```
public static IServiceCollection AddValidationExceptionHandler(
    this IServiceCollection services)
{
    services.AddExceptionHandler<ValidationExceptionHandler>();

    return services;
}
```

3. Agrega el siguiente código a la clase para permitir el registro del manejador de excepciones de actualización.

```
public static IServiceCollection AddUpdateExceptionHandler(
    this IServiceCollection services)
{
    services.AddExceptionHandler<UpdateExceptionHandler>();

    return services;
}
```

4. Agrega el siguiente código a la clase para permitir el registro del manejador de excepciones no manejadas.

```
public static IServiceCollection AddUnhandledExceptionHandler(
    this IServiceCollection services)
{
    services.AddExceptionHandler<UnhandledExceptionHandler>();

    return services;
}
```

   Este método deberá ser invocado después de haber registrado todos los manejadores de excepción personalizada. Es importante recordar que el orden de registro es importante. Debido a que el manejador *UnhandledExceptionHandler* devuelve siempre `true`, los demás manejadores que hubieran sido registrados después de haber registrado este manejador, nunca se ejecutarían.

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Remplazar las excepciones actuales por los tipos personalizados

Remplacemos ahora las excepciones de nuestra aplicación por los nuevos tipos de excepción que hemos creado.

Lanzar la excepción de validación

1. En el proyecto *NorthWind.Sales.Backend.BusinessObjects*, agrega una referencia del proyecto *NorthWind.Exceptions.Entities*.
2. Abre el archivo *GuardModel.cs* del proyecto *NorthWind.Sales.Backend.BusinessObjects*.
3. Localiza el siguiente bloque de código.

```
if (!await modelValidatorHub.Validate(model))
{
    string Errors = string.Join(" ",
        modelValidatorHub.Errors
        .Select(e => $"{e.PropertyName}: {e.Message}"));
    throw new Exception(Errors);
}
```

4. Remplaza el código anterior por el siguiente.

```
if (!await modelValidatorHub.Validate(model))
    throw new ValidationException(modelValidatorHub.Errors);
```

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Lanzar la excepción de actualización

Hagamos algunos cambios en el proyecto *NorthWind.Sales.Backend.DataContexts.EFCore*.

1. Agrega un nuevo directorio llamado ***Guards*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***GuardDBContext.cs*** en el directorio *Guards* y remplaza el código de la clase por el siguiente.

```
internal static class GuardDBContext
{
    public static async Task AgainstSaveChangesErrorAsync(DbContext context)
    {
        try
        {
            await context.SaveChangesAsync();
        }
        catch (DbUpdateException ex)
```

```
        {
            throw new UpdateException(ex,
                ex.Entries.Select(e => e.Entity.GetType().Name));
        }
        catch (Exception)
        {
            throw;
        }
    }
}
```

Puedes notar que estamos atrapando la excepción ***DbUpdateException*** que puede generarse al intentar guardar la información. En este caso lanzamos una excepción *UpdateException*.

También estamos atrapando cualquier otro tipo de excepción, en cuyo caso, simplemente relanzamos la excepción.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. Abre el archivo *NorthWindSalesCommandsDataContext.cs*.
5. Modifica el código del método *SaveChangesAsync* para que sea similar al siguiente.

```
public async Task SaveChangesAsync() =>
    await GuardDBContext.AgainstSaveChangesErrorAsync(this);
```

6. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar los servicios

1. Agrega el siguiente código en el método ***AddNorthWindSalesServices*** del archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler();

    return services;
}
```

Con este código estamos registrando los manejadores de excepciones personalizadas y el manejador de excepciones no manejadas.

2. Abre el archivo *Startup.cs* del proyecto *NorthWind.Sales.WebApi*.
3. Agrega el siguiente código al inicio del método *ConfigureWebApplication*.

```
app.UseExceptionHandler(builder =>{ });
```

Puedes notar que el middleware tiene la capacidad de ejecutar un delegado después de que haya ejecutado los manejadores de excepción. Esto es necesario para que los manejadores de excepción puedan ser ejecutados y para que al mismo tiempo podamos agregar algún delegado adicional para manejo de excepciones.

El código del método *ConfigureWebApplication* será similar al siguiente.

```
public static WebApplication ConfigureWebApplication(
    this WebApplication app)
{
    app.UseExceptionHandler(builder => { });

    // Habilitar el middleware para servir el documento
    // JSON generado y la interfaz UI de Swagger en el
    // ambiente de desarrollo.
    if (app.Environment.IsDevelopment())
    {
        app.UseSwagger();
```

```
        app.UseSwaggerUI();
    }

    // Registrar los endpoints de la aplicación
    app.MapNorthWindSalesEndpoints();

    // Agregar el Middleware CORS
    app.UseCors();

    return app;
}
```

Para obtener información adicional sobre el manejo de excepciones en ASP.NET Core, pueden consultarse los siguientes enlaces:

Handle errors in ASP.NET Core
https://learn.microsoft.com/aspnet/core/fundamentals/error-handling

How to handle errors in Minimal API apps
https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/handle-errors

Middleware order
https://learn.microsoft.com/aspnet/core/fundamentals/middleware#middleware-order

Probar la funcionalidad con la API Web

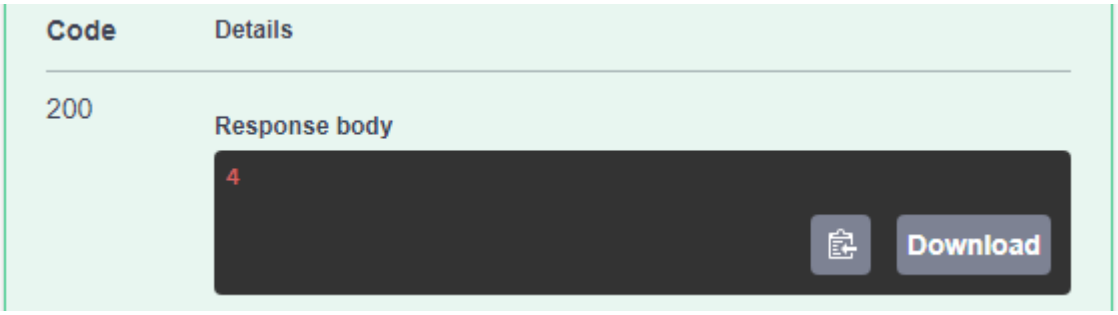
1. Ejecuta la aplicación Web Api.

Probar con todos los datos correctos

1. En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el identificador de la orden creada.

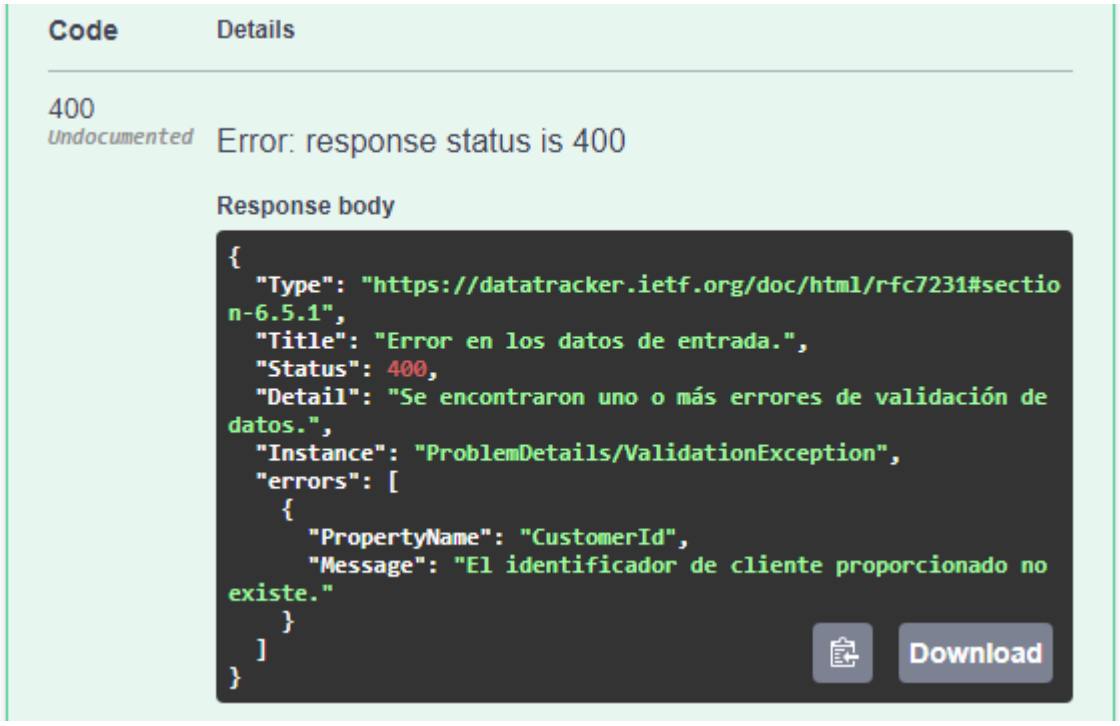


Probar con un cliente no existente

1. Intenta crear una orden con un identificador de cliente no existente.

```
{
    "customerId": "ALFKQ",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación en un formato personalizado.

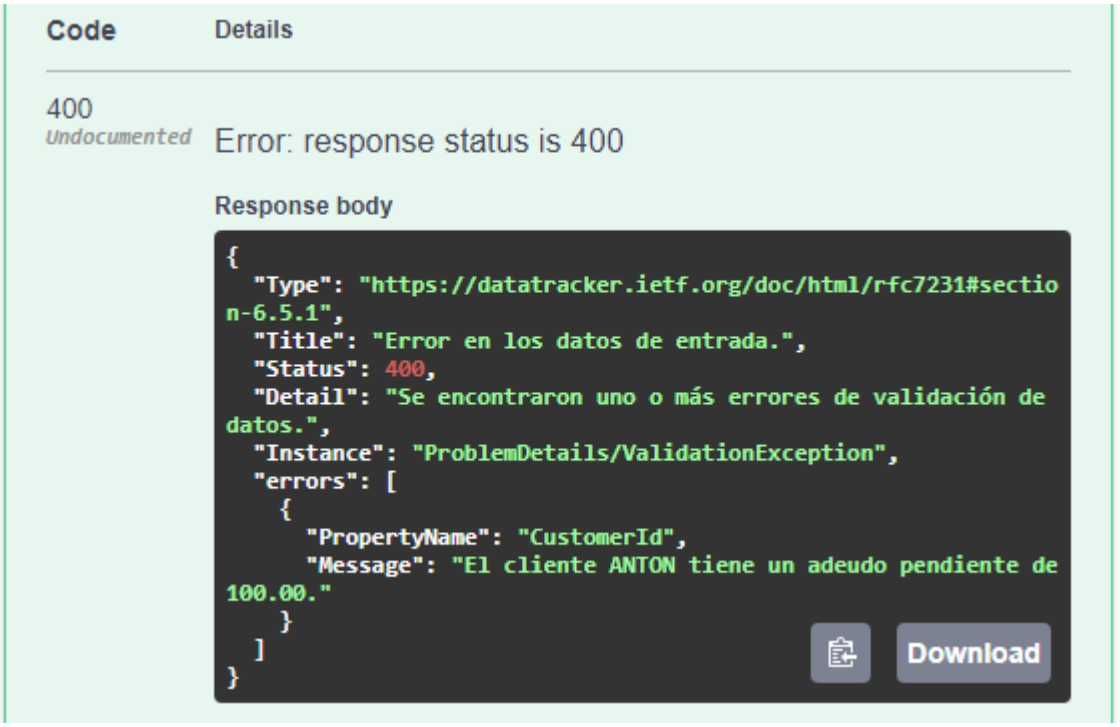


Probar con un cliente con saldo

1. Intenta crear una orden con un cliente que tiene saldo.

```
{
    "customerId": "ANTON",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.

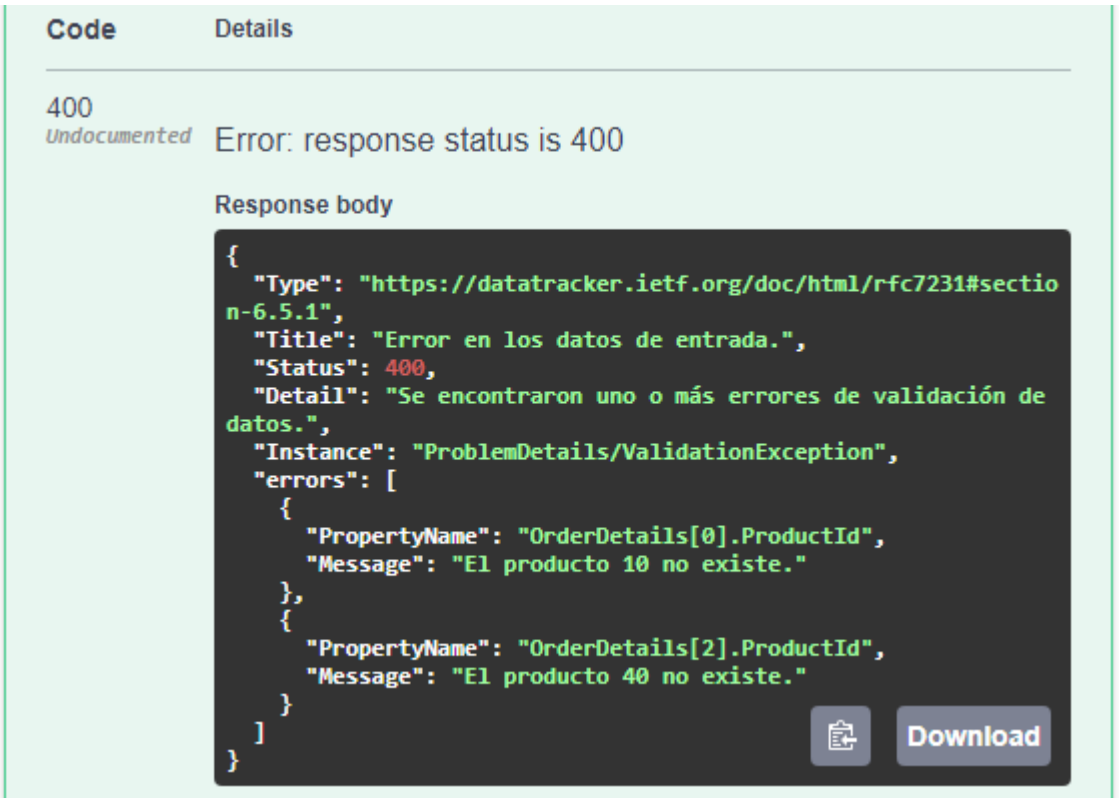


Probar con productos no existentes

1. Intenta crear una orden especificando identificadores de productos no existentes.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 10,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 40,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



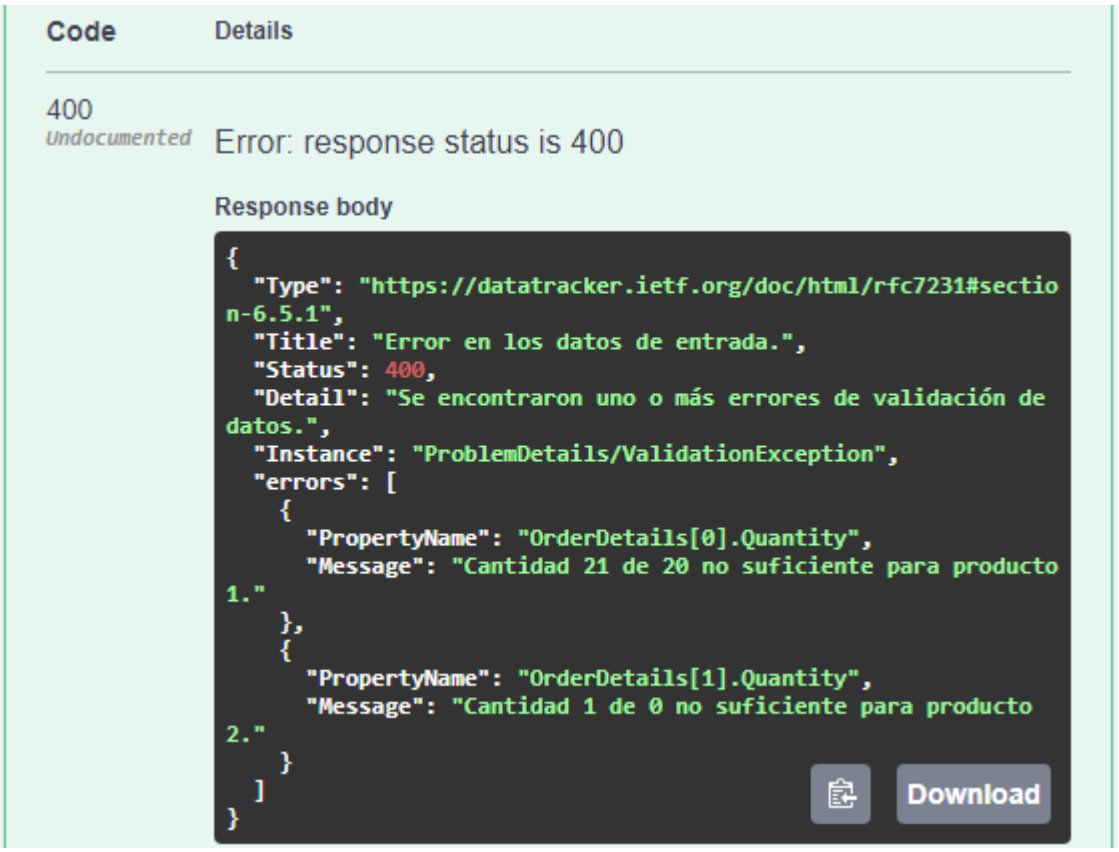
Probar con productos sin existencia

1. Intenta crear una orden con productos sin existencia.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 2,
            "unitPrice": 35,
            "quantity": 1
        },
        {
            "productId": 3,
            "unitPrice": 65,
```

```
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        },
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 11
        }
    ]
}
```

Podrás ver un resultado mostrando el error de validación.



2. Regresa a Visual Studio y detén la ejecución de la aplicación.

Probar con un error general

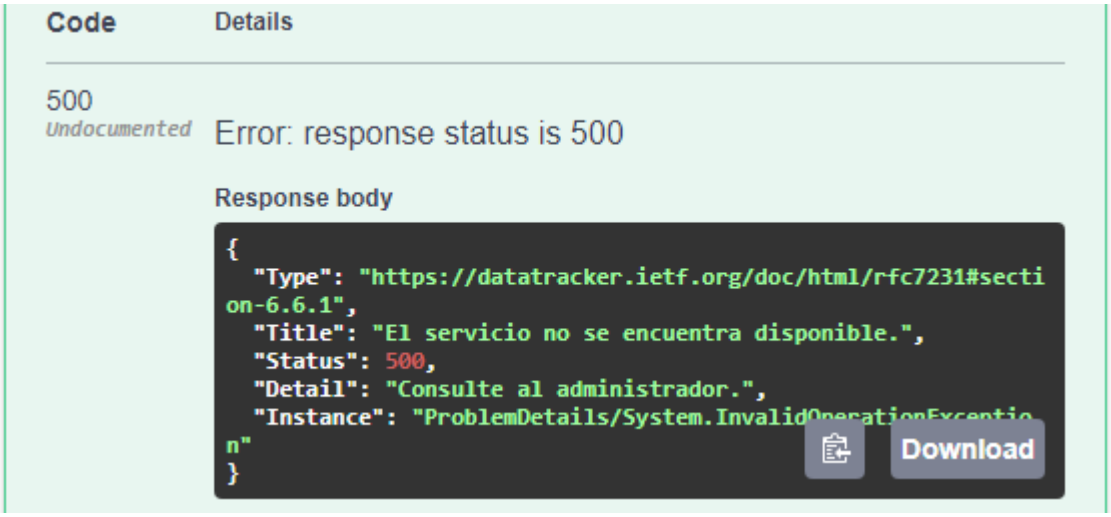
1. Modifica el nombre de la base de datos en la cadena de conexión del archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi* para provocar una excepción general.

2. Ejecuta la aplicación.

   En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        },
        {
            "productId": 3,
            "unitPrice": 65,
            "quantity": 20
        },
        {
            "productId": 4,
            "unitPrice": 75,
            "quantity": 40
        }
    ]
}
```

   Podrás ver un resultado mostrando el error.



   En la *Consola* y en la ventana *Output* de Visual Studio podrás ver el detalle de la excepción similar al siguiente.

```
fail: NorthWind.Exceptions.Entities.ExceptionHandlers.UnhandledExce
ptionHandler[0]
      El servicio no se encuentra disponible.
      System.InvalidOperationException: An exception has been raise
d that is likely due to a transient failure. Consider enabling tran
sient error resiliency by adding 'EnableRetryOnFailure' to the 'Use
SqlServer' call.
       ---> Microsoft.Data.SqlClient.SqlException (0x80131904): Can
not open database "NorthWindDBX" requested by the login. The login
failed.
```

3. Regresa a Visual Studio y detén la ejecución.
4. Modifica el nombre de la base de datos en la cadena de conexión del archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi* para acceder a la base de datos correcta.

Probar un error de actualización

1. Localiza el siguiente código en el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.

```
OrderAggregate Order =
    OrderAggregate.From(orderDto);
```

2. Agrega el siguiente código después del código anterior para provocar un error de actualización de datos.

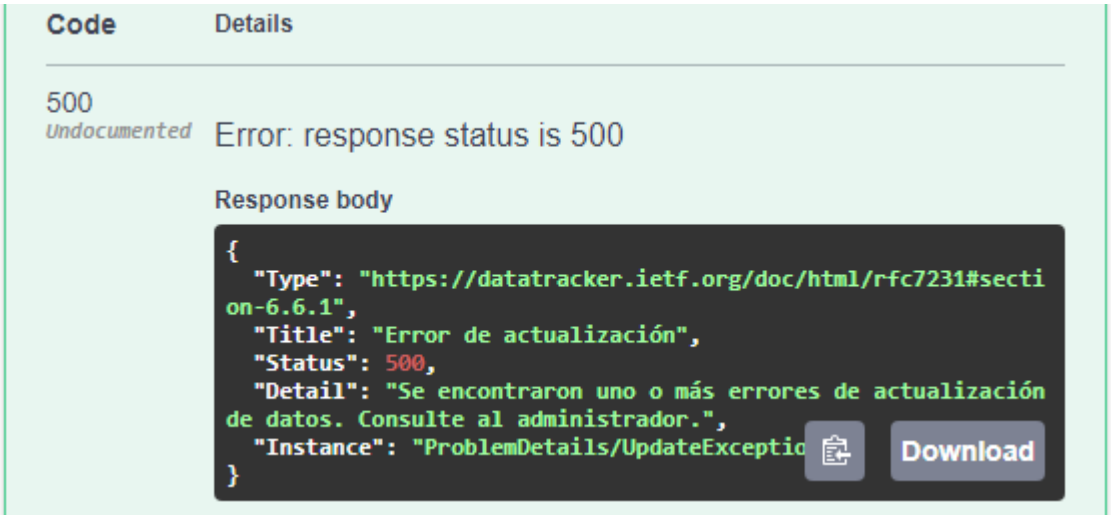
```
Order.CustomerId = "";
```

3. Ejecuta la aplicación.
4. En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

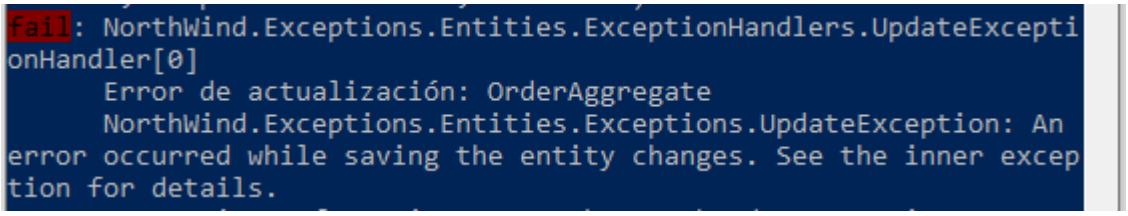
```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        }
    ]
```

```
}
```

Podrás ver un resultado mostrando el error.



En la *Consola* y la ventana *Output* de Visual Studio podrás ver el detalle de la excepción.



5. Regresa a Visual Studio y detén la ejecución.
6. Elimina la siguiente línea de código del archivo *CreateOrderInteractor.cs*.

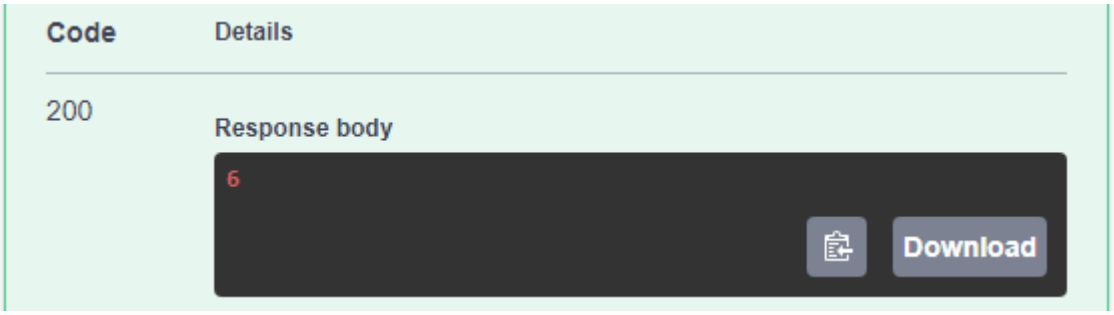
```
Order.CustomerId = "";
```

7. Ejecuta la aplicación.
8. En la página *Index* de la aplicación *Web API* intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
```

```
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        }
    ]
}
```

Podrás ver un resultado mostrando el número de orden creada.



9. Regresa a Visual Studio y detén la ejecución.

Incorporar el manejo de excepciones en la aplicación Blazor

Incorporemos ahora el manejo de excepciones a la aplicación Blazor.

Agregar un manejador de mensajes de respuestas HTTP

Cuando realizamos peticiones HTTP podemos obtener respuestas con éxito y sin éxito. Para poder determinar la respuesta que la API Web nos devuelve, por ejemplo, errores de validación, podemos crear un manejador de mensajes de respuestas HTTP (*Delegating Handler*) y realizar las acciones que nos interesen.

Para poder reutilizar el código en otras aplicaciones que realicen peticiones Web, implementaremos el manejador de mensajes en un proyecto independiente.

1. Utiliza la plantilla ***Class Library*** para agregar a la solución un nuevo proyecto llamado ***NorthWind.HttpDelegatingHandlers*** en el directorio ***1. Enterprise Business Rules***.
2. Elimina el archivo *Class1.cs*.
3. Agrega una referencia del proyecto *NorthWind.Validation.Entities*. El manejador de mensajes que crearemos nos permitirá obtener los mensajes de error de las validaciones.
4. Agrega un nuevo archivo de clase llamado ***ExceptionDelegatingHandler.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
public class ExceptionDelegatingHandler : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request, CancellationToken cancellationToken)
    {
        HttpResponseMessage Response =
            await base.SendAsync(request, cancellationToken);

        if (!Response.IsSuccessStatusCode)
        {
            string ErrorMessage = await Response.Content.ReadAsStringAsync();
            string Source = null;
            string Message = null;
            IEnumerable<ValidationError> Errors = null;
            bool IsValidProblemDetails = false;

            switch (Response.StatusCode)
            {
                case HttpStatusCode.Unauthorized:
                case HttpStatusCode.Forbidden:
                    ErrorMessage =
                        $"{(int)Response.StatusCode} {Response.ReasonPhrase}";
                    break;
                default:
                    try
```

```
                {
                    var ContentType = Response.Content.Headers
                        .ContentType.MediaType;
                    var JsonResponse = JsonSerializer
                        .Deserialize<JsonElement>(ErrorMessage);

                    // Verificar si el tipo de contenido de la respuesta es
                    // ProblemDetails y verificar si contiene el valor
                    // "instance".
                    if (ContentType == "application/problem+json" &&
                        TryGetProperty(JsonResponse, "instance",
                        out JsonElement InstanceValue))
                    {
                        // Intentar extraer la información de ProblemDetails.
                        // El valor de instance debe tener el formato
                        // "ProblemDetails/<Tipo-de-Excepción>".
                        string Value = InstanceValue.ToString();
                        if (Value.ToLower().StartsWith("problemdetails/"))
                        {
                            Source = Value;
                            if (TryGetProperty(JsonResponse, "title",
                                out var TitleValue))
                            {
                                Message = TitleValue.ToString();
                            }

                            if (TryGetProperty(JsonResponse, "detail",
                                out var DetailValue))
                            {
                                Message = $"{Message} {DetailValue}";
                            }

                            // ¿Contiene errores de validación?
                            if (TryGetProperty(JsonResponse, "errors",
                                out JsonElement ErrorsValue))
                            {
                                Errors = ErrorsValue.Deserialize
                                    <IEnumerable<ValidationError>>(
                                    new JsonSerializerOptions
                                    {
                                        PropertyNameCaseInsensitive = true
                                    });
                            }

                            IsValidProblemDetails = true;
                        }
                    }
                }
                catch { }
                break;
            }

            if (!IsValidProblemDetails)
            {
                Message = ErrorMessage;
                Source = null;
                Errors = null;
            }
```

```
            var Ex = new HttpRequestException(
                Message, null, Response.StatusCode);

            Ex.Source = Source;
            if (Errors != null) Ex.Data.Add("Errors", Errors);
            throw Ex;
        }

        return Response;
    }

    // Obtener el valor de una propiedad de un objeto JsonElement
    // ignorando mayúsculas/minúsculas.
    bool TryGetProperty(JsonElement element, string propertyName,
        out JsonElement Value)
    {
        bool Found = false;
        Value = default;

        var Property = element.EnumerateObject()
            .FirstOrDefault(e => string.Compare(e.Name,
            propertyName, StringComparison.OrdinalIgnoreCase) == 0);

        if (Property.Value.ValueKind != JsonValueKind.Undefined)
        {
            Value = Property.Value;
            Found = true;
        }

        return Found;
    }
}
```

El manejador que hemos creado lanza una excepción *HttpRequestException* cuando la respuesta recibida no sea exitosa, adicionalmente, en caso de que la respuesta contenga errores de validación, estos serán almacenados en la propiedad *Data* del objeto *HttpRequestException*.

5. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias.

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza su contenido por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddExceptionDelegatingHandler(
        this IServiceCollection services)
    {
```

```
        services.TryAddTransient<ExceptionDelegatingHandler>();

        return services;
    }
}
```

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Modificar el Gateway de creación de orden

Hagamos unos cambios en el proyecto *NorthWind.Sales.Frontend.WebApiGateways.*

1. Agrega una referencia del proyecto *NorthWind.HttpDelegatingHandlers* en el proyecto *NorthWind.Sales.Frontend.WebApiGateways*.
2. Abre el archivo *DependencyContainer.cs*.
3. Modifica el código del método ***AddWebApiGateways*** para incorporar el manejador de mensajes de respuesta HTTP creado. El código será similar al siguiente.

```
public static IServiceCollection AddWebApiGateways(
    this IServiceCollection services,
    Action<HttpClient> configureHttpClient)
{
    services.AddExceptionDelegatingHandler();
    services.AddHttpClient<ICreateOrderGateway,
        CreateOrderGateway>(configureHttpClient)
        .AddHttpMessageHandler<ExceptionDelegatingHandler>();

    return services;
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
5. Abre el archivo *CreateOrderGateway.cs*.
6. Modifica el código del método *CreateOrderAsync* para que sea similar al siguiente.

```
public async Task<int> CreateOrderAsync(CreateOrderDto order)
{
    var Response = await client.PostAsJsonAsync(
        Endpoints.CreateOrder, order);

    return await Response.Content.ReadFromJsonAsync<int>();
}
```

En caso de que la petición no haya sido exitosa, el manejador HTTP que incorporamos lanzará una excepción, por lo tanto, el código de la sentencia `return` será ejecutado únicamente

cuando la respuesta haya sido exitosa, no es necesario realizar alguna validación como la que teníamos previamente.

Modificar el ViewModel

Hagamos ahora unos cambios en el proyecto *NorthWind.Sales.Frontend.Views*.

1. Abre el archivo *CreateOrderViewModel.cs*.
2. Agrega el siguiente código en la clase *CreateOrderViewModel*.

```
public ModelValidator<CreateOrderViewModel>
    ModelValidatorComponentReference { get; set; }
```

   Esta propiedad almacenará una instancia del componente de validación *ModelValidator*.

3. Modifica el código del método *Send* para que sea similar al siguiente.

```
public async Task Send()
{
    InformationMessage = "";
    try
    {
        var OrderId = await gateway.CreateOrderAsync(
            (CreateOrderDto)this);
        InformationMessage = string.Format(
            CreateOrderMessages.CreatedOrderTemplate, OrderId);
    }
    catch (HttpRequestException ex)
    {
        if (ex.Data.Contains("Errors"))
        {
            IEnumerable<ValidationError> Errors =
                ex.Data["Errors"] as IEnumerable<ValidationError>;
            ModelValidatorComponentReference.AddErrors(Errors);
        }
        else
        {
            throw;
        }
    }
}
```

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Modificar el componente CreateOrderComponent

1. Abre el archivo *CreateOrderComponent.razor*.

2. Localiza el siguiente código.

```
<ModelValidator Validator="Order.Validator"/>
```

3. Remplaza el código anterior por el siguiente.

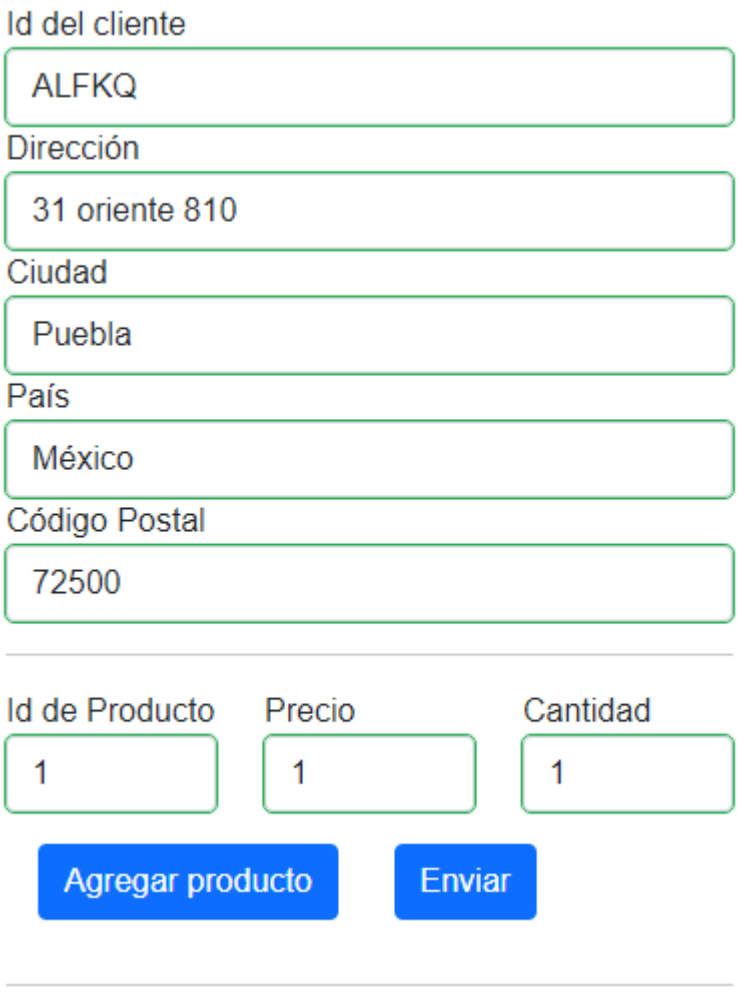
```
<ModelValidator Validator="Order.Validator"
    @ref=Order.ModelValidatorComponentReference/>
```

Probar la funcionalidad de la aplicación Blazor

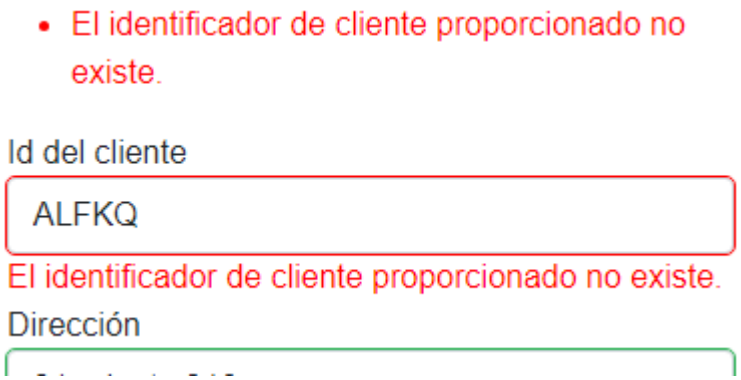
1. Ejecuta la aplicación Web Api y Blazor.

Probar con un cliente no existente

1. En la aplicación Blazor proporciona los siguientes datos para crear una orden con un cliente no existente.

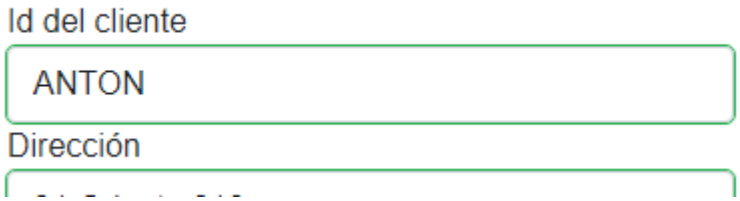


Podrás ver un resultado indicando que el identificador del cliente proporcionado no existe.

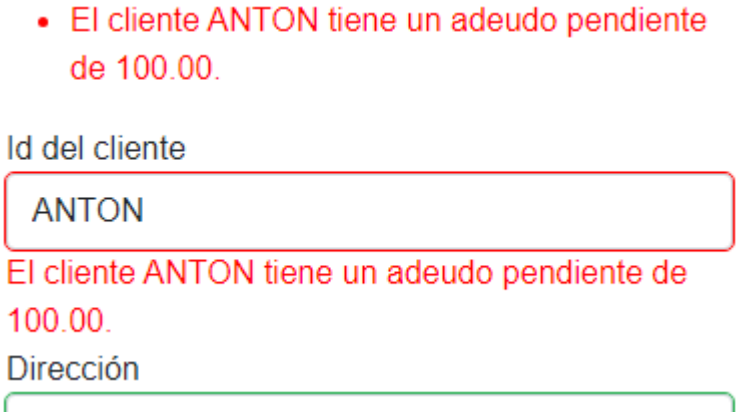


Probar con un cliente con saldo

1. Intenta crear una orden con un cliente que tiene saldo.

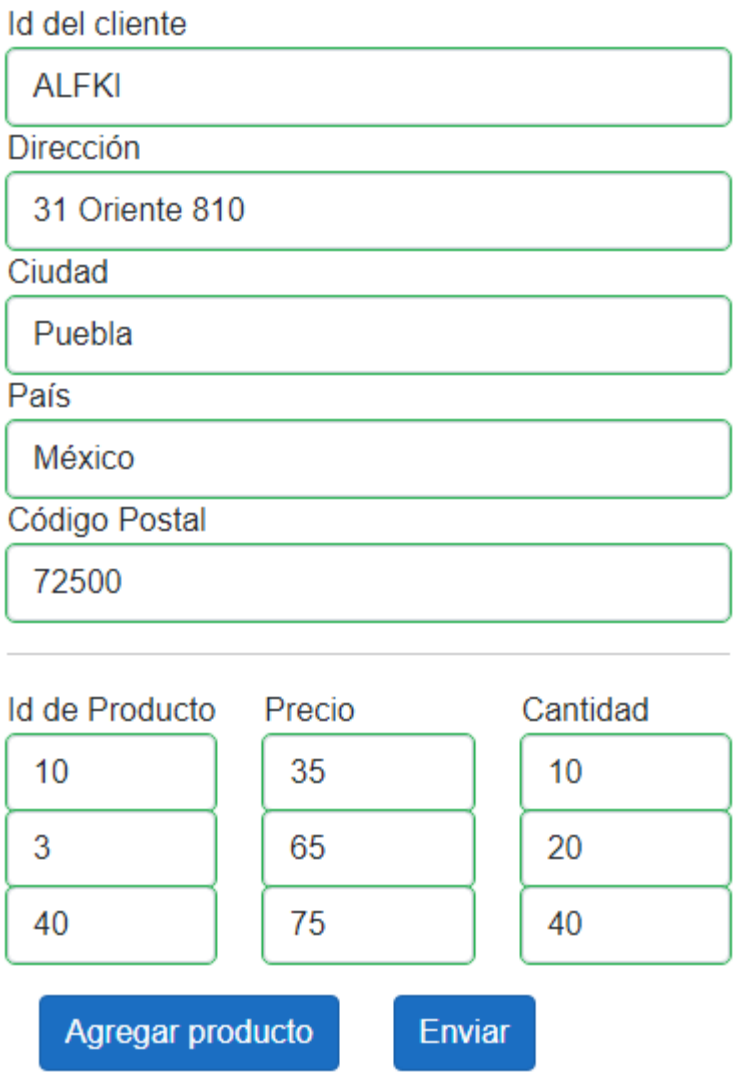


Podrás ver un resultado mostrando el error de validación.

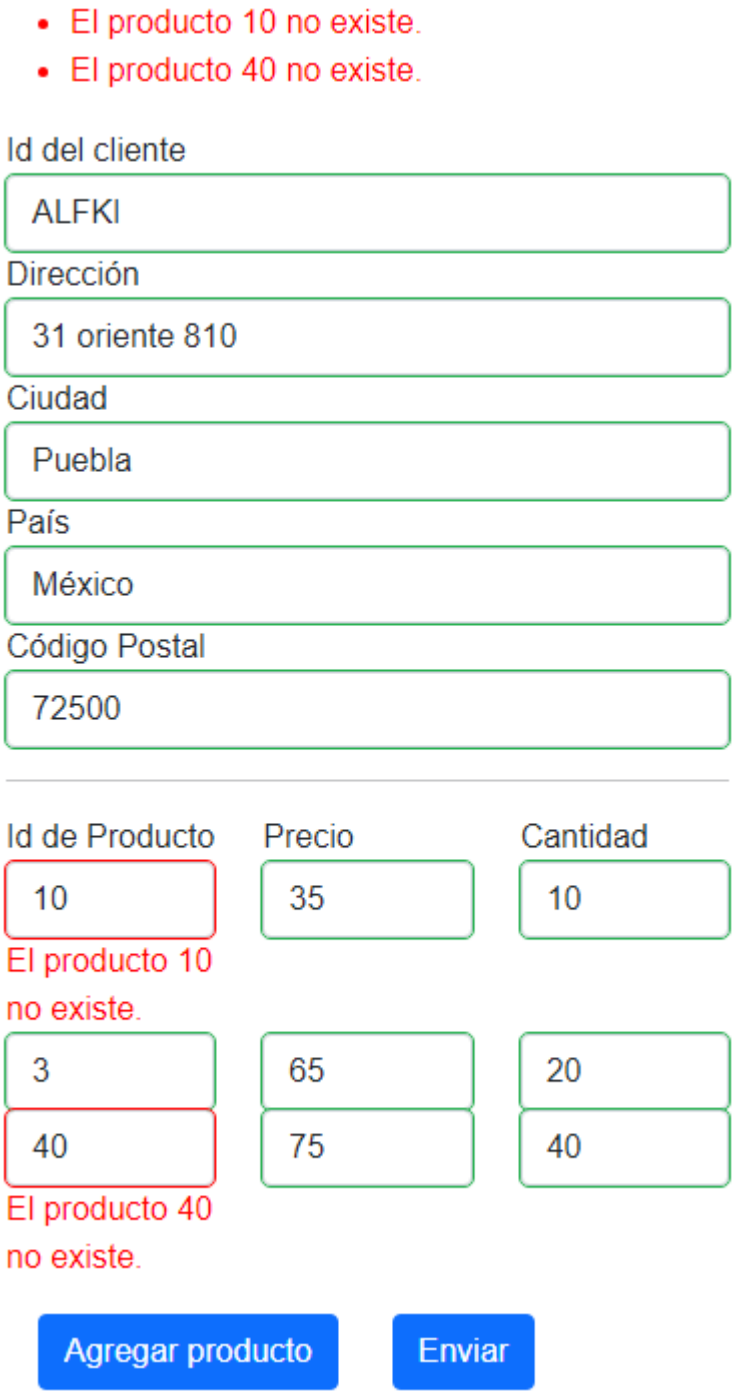


Probar con productos no existentes

1. Intenta crear una orden especificando identificadores de productos no existentes.

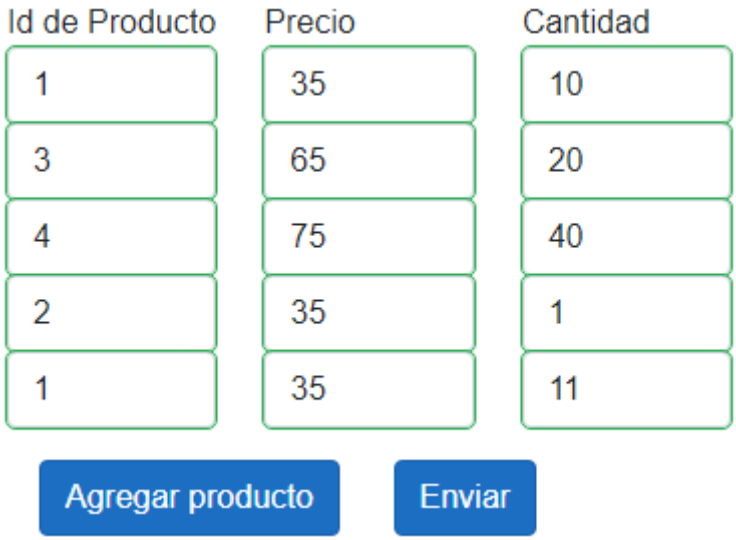


Podrás ver un resultado mostrando los errores de validación.

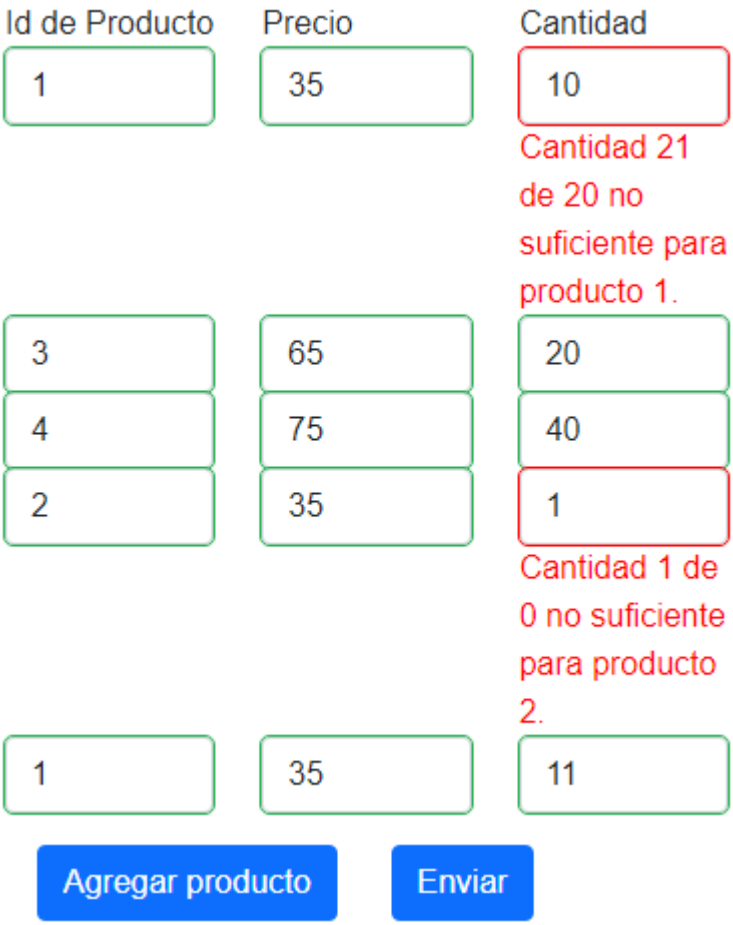


Probar con productos sin existencia

1. Intenta crear una orden con productos sin existencia.



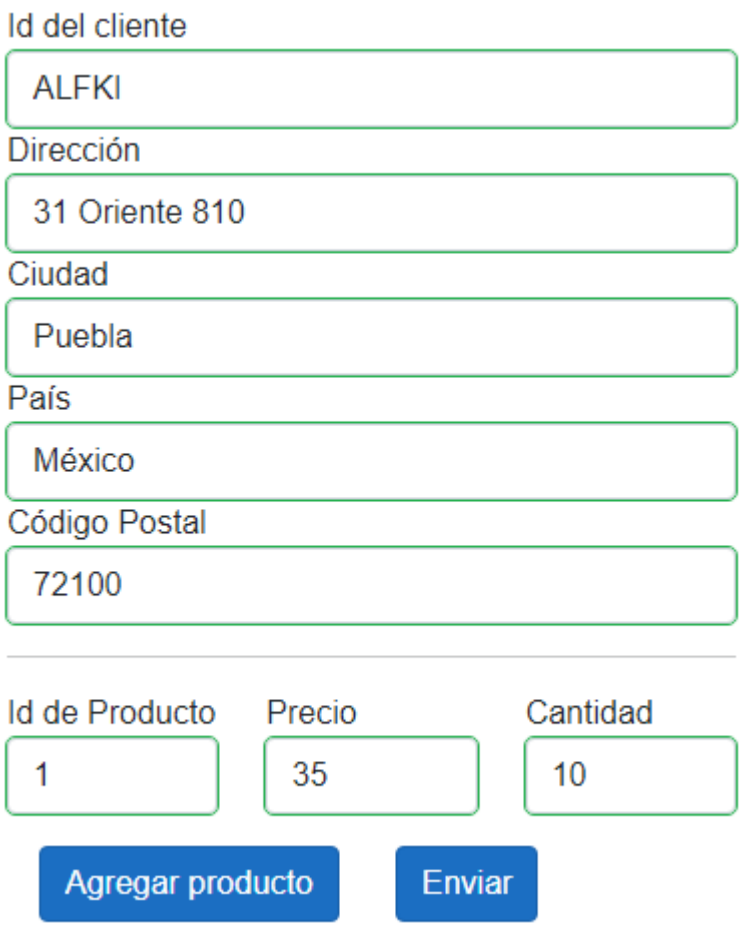
Podrás ver un resultado mostrando los errores de validación.



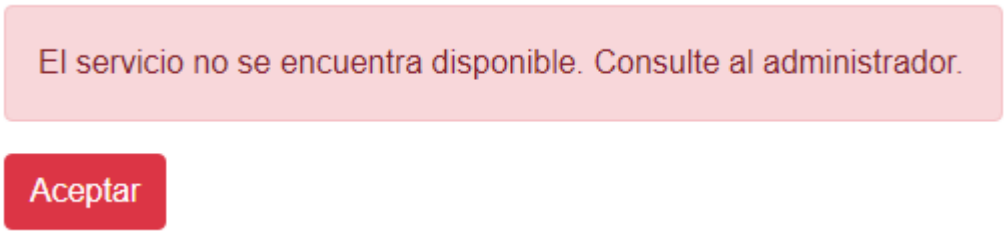
2. Regresa a Visual Studio y detén la ejecución.

Probar con un error general

1. Modifica el nombre de la base de datos en la cadena de conexión del archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi* para provocar una excepción general.
2. Ejecuta la aplicación.
3. Intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.



Podrás ver un resultado mostrando el error.



En la *Consola* y en la ventana *Output* de Visual Studio podrás ver el detalle de la excepción.

4. Regresa a Visual Studio y detén la ejecución.
5. Modifica el nombre de la base de datos en la cadena de conexión del archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi* para acceder a la base de datos correcta.

Probar un error de actualización

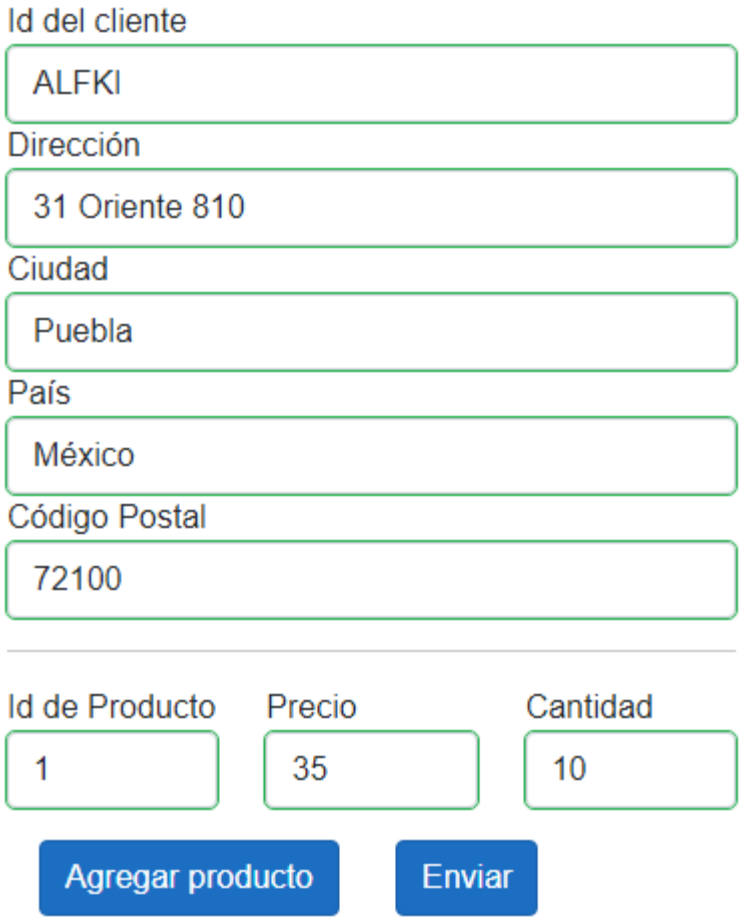
1. Localiza el siguiente código en el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.

```
OrderAggregate Order =
    OrderAggregate.From(orderDto);
```

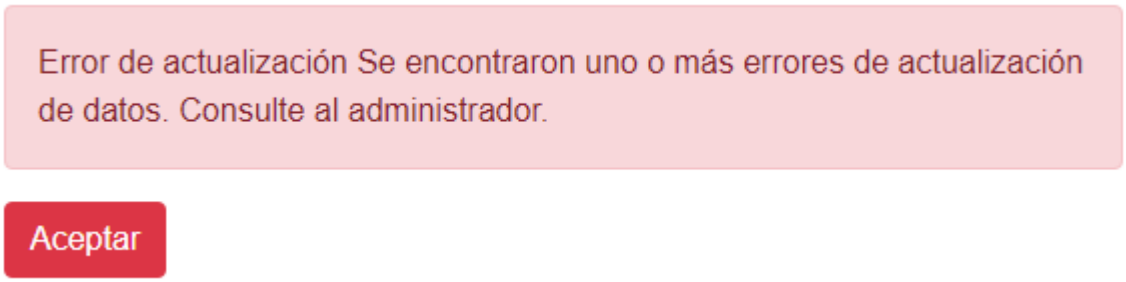
2. Agrega el siguiente código después del código anterior para provocar un error de actualización de datos.

```
Order.CustomerId = "";
```

3. Ejecuta la aplicación.
4. Intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.



Podrás ver un resultado mostrando el error.

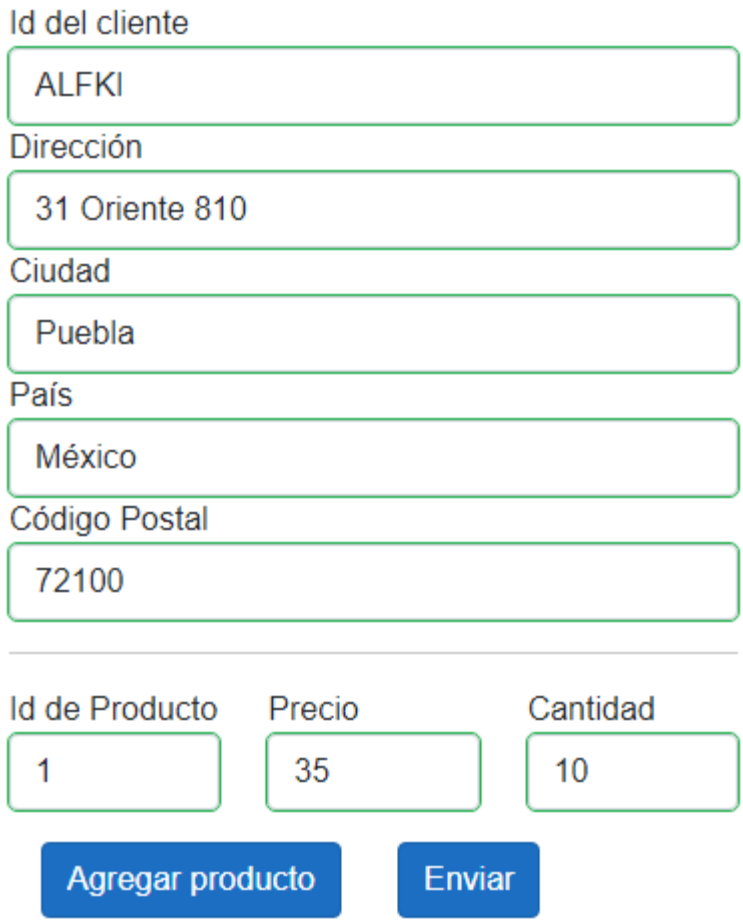


En la Consola y la ventana *Output* de Visual Studio podrás ver el detalle de la excepción.

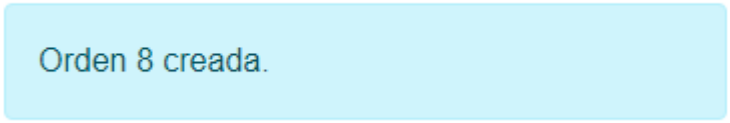
5. Regresa a Visual Studio y detén la ejecución.
6. Elimina la siguiente línea de código del archivo *CreateOrderInteractor.cs*.

```
Order.CustomerId = "";
```

7. Ejecuta la aplicación.
8. intenta crear una orden proporcionando los siguientes datos que cumplen con las reglas de validación.



Podrás ver un resultado mostrando el número de orden creada.



La validación está funcionando adecuadamente y los mensajes de error ahora son más amigables.

Práctica 9:

Eventos de dominio

Utilizamos *Eventos de dominio* para implementar explícitamente los efectos secundarios de los cambios que suceden dentro del dominio. En otras palabras y utilizando la terminología *DDD*, utilizamos *Eventos de dominio* para implementar explícitamente los efectos secundarios de los cambios en los *Agregados*.

En esta práctica presentaremos una introducción a los *Eventos de dominio* implementando una arquitectura limpia.

Objetivos

Al finalizar esta práctica los estudiantes contarán con las habilidades y conocimientos para:

- Describir qué es un *Evento de dominio*.
- Implementar *Eventos de dominio* en aplicaciones con *Clean Architecture*.

Introducción

Cuando hablamos de *Clean Architecture*, *DDD* o *Microservicios* nos encontramos frecuentemente con el concepto *Evento*.

Un *Evento* es algo que ha ocurrido en el pasado. Un *Evento de dominio* es algo que ha ocurrido en el dominio y que queremos que otras partes del dominio se enteren. Las partes notificadas normalmente reaccionan de alguna forma a los eventos.

Un beneficio importante de los *Eventos de dominio* es que los efectos secundarios de los cambios pueden ser expresados explícitamente. Por ejemplo, "*Cuando una orden especial sea creada, deberá enviarse un correo al administrador*". En este ejemplo, el efecto secundario de agregar una "*orden especial*" es enviar un correo al administrador.

Al utilizar *Eventos de dominio* hacemos que el concepto sea expresado explícitamente ya que siempre existirá un *Evento de dominio* (*DomainEvent*) y al menos un *Manejador del Evento de dominio* (*DomainEventHandler*).

Por ejemplo, en la aplicación *NorthWind.Sales*, cuando una orden especial sea creada se deberá disparar el evento *SpecialOrderCreatedEvent* que será procesado con el manejador de evento *SendEMailWhenSpecialOrderCreatedEventHandler.*

En resumen, los *Eventos de dominio* nos ayudan a expresar, explícitamente, las reglas de dominio en función del lenguaje generalizado proporcionado por los expertos de dominio. Además, los *Eventos de dominio* permiten una mejor separación de responsabilidades entre las clases dentro del mismo dominio.

En algunas circunstancias, los eventos pueden comportarse como en una transacción de base de datos, donde todas las operaciones relacionadas con un evento de dominio finalizan correctamente o ninguna lo hace.

Los *Eventos de dominio* son similares a los eventos de estilo de mensajería, con una diferencia importante. Con la mensajería real, las colas de mensajes, los agentes de mensajes o un bus de servicio con *AMQP* (*Advanced Message Queuing Protocol*), un mensaje siempre se envía de forma asíncrona y se comunica entre procesos y equipos. Esto es útil para integrar varios contextos delimitados, microservicios o incluso otras aplicaciones. Sin embargo, con los eventos de dominio, nos interesa generar un evento desde la operación de dominio que se está ejecutando actualmente y queremos que los efectos secundarios ocurran dentro del mismo dominio.

Los *Eventos de dominio* y sus efectos secundarios (las acciones que son realizadas por los manejadores de eventos) se deben producir casi de inmediato, por lo general en el mismo proceso (*in-process*) y dentro del mismo dominio. Por tanto, los eventos de dominio pueden ser síncronos o asíncronos. Pero los *Eventos de integración* siempre deben ser asíncronos.

Eventos de dominio Vs Eventos de integración

Semánticamente, los *Eventos de dominio* y los *Eventos de integración* son lo mismo: notificaciones sobre algo que acaba de ocurrir. Sin embargo, su implementación debe ser diferente. Los *Eventos de dominio* son simplemente mensajes insertados en un distribuidor de eventos de dominio, que se puede implementar como un mediador en memoria basado en un contenedor de *IoC* o cualquier otro método.

Por otro lado, el propósito de los *Eventos de integración* es propagar las transacciones confirmadas y actualizaciones a subsistemas adicionales, independientemente de que sean otros microservicios, contextos delimitados o incluso aplicaciones externas. Por tanto, solo se deben producir si la entidad es persistida correctamente, de otra forma será como si toda la operación nunca se hubiera producido.

Como mencionamos antes, los eventos de integración deben estar basados en la comunicación asíncrona entre varios microservicios (otros contextos delimitados) o incluso sistemas o aplicaciones externas.

Por tanto, la interfaz de bus de eventos necesita una infraestructura que permita la comunicación entre procesos (*inter-process*) y distribuida entre servicios potencialmente remotos. Se pueden basar en un bus de servicio comercial, colas, una base de datos compartida que se use como un buzón o cualquier otro sistema de mensajería distribuida e, idealmente, basado en inserciones (*push*).

Agregar nuevos requerimientos al caso de uso

Agreguemos una regla de negocio a nuestro caso de uso *Crear Orden de Compra* para poder ejemplificar el uso de *Eventos de Dominio*. La nueva regla de negocio indica que cuando una *Orden especial* sea creada, deberá enviarse un correo electrónico al administrador de la empresa.

Historia de Usuario: Crear orden de compra

"*Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén*".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema valida los datos.
3. El sistema registra la orden de compra.
4. ***Cuando el número de productos de la orden sea mayor de 3, el sistema enviará un correo electrónico de notificación de "Orden especial creada" al administrador de la empresa***.
5. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Validaciones

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.
- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.
- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.
- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

El único cambio que hemos incorporado al caso de uso es el paso 4 del flujo primario:

Cuando el número de productos de la orden sea mayor de 3, el sistema enviará un correo electrónico de notificación de "Orden especial creada" al administrador de la empresa.

Definir la abstracción para el servicio de envío de correo

El nuevo requerimiento indica que cuando una orden especial sea creada deberá enviarse un correo al administrador del sistema. Para cumplir con el requerimiento de envío de correo definiremos una interface que deberá implementar un servicio de envío de correos.

La interface que definiremos para implementar el servicio de correos no será de uso exclusivo de la aplicación que estamos desarrollando, sino que podrá ser utilizada por diversas aplicaciones de la empresa. Por esta razón, podemos definir esta interface en un proyecto a nivel empresarial para que pueda ser reutilizada por diversas aplicaciones.

Crear un nuevo proyecto

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto a la solución llamado `NorthWind.Entities` en el directorio `1. Enterprise Business Rules`.
2. Elimina el archivo ***Class1.cs***.
3. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
4. Agrega un nuevo archivo de tipo interface llamado ***IMailService.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IMailService
{
    Task SendMailToAdministrator(string subject, string body);
}
```

Por simplicidad, la interface *IMailService* define únicamente un método para enviar correos al administrador. La interface puede ser modificada para poder enviar mensajes a cualquier destinatario o incluso para poder enviar archivos adjuntos o mensajes con formato personalizado.

Mover la interface IUnitOfWork

Actualmente tenemos definida la interface ***IUnitOfWork*** que al igual que la interface *IMailService*, no es de uso exclusivo de la aplicación que estamos desarrollando, por lo tanto, podemos moverla hacia el nuevo proyecto.

1. Mueve el archivo *IUnitOfWork.cs* ubicado en el directorio *Interfaces\Common* del proyecto *NorthWind.Sales.Backend.BusinessObjects* hacia el directorio *Interfaces* del nuevo proyecto.
2. Actualiza el nuevo espacio de nombres de la interface *IUnitWork*. El código del archivo *IUnitOfWork.cs* será similar al siguiente.

```
namespace NorthWind.Entities.Interfaces;
public interface IUnitOfWork
{
    Task SaveChanges();
}
```

3. Elimina el directorio *Interfaces\Common* de *NorthWind.Sales.Backend.BusinessObjects*.
4. En el proyecto *NorthWind.Sales.Backend.BusinessObjects,* agrega una referencia del proyecto *NorthWind.Entities*.
5. Abre el archivo *GlobalUsings.cs* del proyecto *NorthWind.Sales.Backend.BusinessObjects* y elimina la siguiente línea de código con el espacio de nombres que muestra un error.

```
global using NorthWind.Sales.Backend.BusinessObjects.Interfaces.Common;
```

6. Agrega el siguiente código para importar el espacio de nombres actualizado.

```
global using NorthWind.Entities.Interfaces;
```

7. Compila la solución y verifica que no haya errores.

Definir las entidades de eventos

Definamos las entidades que utilizaremos para el manejo de *Eventos de dominio*. Debido a que las entidades que definiremos para el manejo de eventos podrán ser utilizadas por cualquier aplicación de la empresa, definiremos estas abstracciones en un proyecto independiente en la capa de *Reglas de Negocio Empresariales*. Antes de iniciar, si lo deseas, puedes realizar un respaldo de la solución actual para que conserves el estado de los cambios.

Crear el nuevo proyecto

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto a la solución llamado `NorthWind.Events.Entities` en el directorio `1. Enterprise Business Rules`.
2. Elimina el archivo ***Class1.cs***.

Definir la interface IDomainEvent

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Interface* llamado ***IDomainEvent.cs*** en el directorio *Interfaces* y modifica el código de la interface por el siguiente.

```
public interface IDomainEvent
{
}
```

La interface no define ningún miembro, solo será utilizada para indicar que una clase representa un evento de dominio. En términos de C#, un *Evento de dominio* es simplemente una estructura o clase que almacena datos como un *DTO*, con toda la información relacionada con lo que acaba de suceder en el dominio.

Definir la interface IDomainEventHandler

1. Agrega un nuevo archivo de tipo *Interface* llamado ***IDomainEventHandler.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IDomainEventHandler<EventType>
    where EventType : IDomainEvent
{
    Task Handle(EventType eventTypeInstance);
}
```

La interface permitirá implementar manejadores de evento para tipos de eventos de dominio específicos. El parámetro de tipo de la interface genérica especifica que el tipo debe ser un tipo *IDomainEvent*.

Definir la interface IDomainEventHub

Definamos ahora una interface que permita al implementador recibir un evento para después lanzarlo a los manejadores de ese tipo de evento.

1. Agrega un nuevo archivo de tipo *interface* llamado ***IDomainEventHub.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IDomainEventHub<EventType>
    where EventType : IDomainEvent
{
    Task Raise(EventType eventTypeInstance);
}
```

La interface nos permitirá implementar el servicio que funcione como un distribuidor de eventos.

Implementar el distribuidor de eventos

Los *Eventos de dominio* son simplemente mensajes insertados en un distribuidor de eventos de dominio que se puede implementar como un mediador en memoria basado en un contenedor de *IoC* o cualquier otro método.

Implementemos el servicio para distribución de eventos en el proyecto *NorthWind.Events.Entities*.

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***DomainEventHub.cs*** en el directorio *Services* y modifica el código de la clase para que sea similar al siguiente.

```
internal class DomainEventHub<EventType>(
    IEnumerable<IDomainEventHandler<EventType>> eventHandlers) :
    IDomainEventHub<EventType>
    where EventType : IDomainEvent
{
    public async Task Raise(EventType eventTypeInstance)
    {
        foreach (var Handler in eventHandlers)
        {
            await Handler.Handle(eventTypeInstance);
        }
    }
}
```

   Puedes notar que la clase genérica *DomainEventHub* recibe a través de su constructor primario una colección de los manejadores para el tipo de evento especificado en el parámetro de tipo de la clase.

   Para disparar un evento de dominio simplemente tendremos que invocar el método ***Raise*** que a través de un ciclo notificará a los manejadores de evento de forma asíncrona. El método finalizará después de que todos lo manejadores de evento hayan terminado su ejecución.

   Con esta implementación, podríamos fácilmente adaptar su funcionalidad para manejo de eventos de integración donde podríamos invocar cada manejador de evento en una tarea independiente sin esperar a que finalice su ejecución.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

Agreguemos ahora el contenedor de dependencias en el proyecto *NorthWind.Events.Entities.*

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y modifica su contenido para que sea similar al siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddEventServices(
        this IServiceCollection services)
    {
        services.AddScoped(typeof(IDomainEventHub<>),
            typeof(DomainEventHub<>));

        return services;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. En el proyecto *NorthWind.Sales.Backend.BusinessObjects*, agrega una referencia del proyecto *NorthWind.Events.Entities*.
5. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
6. Modifica el código del método ***AddNorthWindSalesServices*** para registrar el nuevo servicio. El código del método será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices();

    return services;
}
```

Implementar el evento

1. Agrega un nuevo archivo de clase llamado ***SpecialOrderCreatedEvent.cs*** en el directorio *CreateOrder* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el código de la clase *SpecialOrderCreatedEvent* para que sea similar al siguiente.

```
internal class SpecialOrderCreatedEvent(
    int orderId, int productsCount) : IDomainEvent
{
    public int OrderId => orderId;
    public int ProductsCount => productsCount;
}
```

Cuando una orden especial sea creada se notificará por correo al administrador indicándole el número de orden y la cantidad de productos de esta.

Puedes notar que un *Evento de dominio* es simplemente una estructura que almacena toda la información relacionada con lo que acaba de suceder en el dominio.

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el manejador de evento

1. Abre el archivo de recursos *CreateOrderMessages.resx* del directorio *Resources* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Agrega los siguientes valores al archivo de recursos.

Nombre	Valor
SendEmailSubject	Orden especial creada
SendEmailBodyTemplate	Se ha creado la orden {0} con {1} productos.

3. Agrega un nuevo archivo llamado ***SendEMailWhenSpecialOrderCreatedEventHandler.cs*** en el directorio *CreateOrder* del proyecto *NorthWind.Sales.Backend.UseCases* y remplaza el código de la clase con el siguiente.

```
internal class SendEMailWhenSpecialOrderCreatedEventHandler(
    IMailService mailService) :
    IDomainEventHandler<SpecialOrderCreatedEvent>
{
    public Task Handle(SpecialOrderCreatedEvent createdOrder)
    {
        return mailService.SendMailToAdministrator(
            CreateOrderMessages.SendEmailSubject,
            string.Format(CreateOrderMessages.SendEmailBodyTemplate,
            createdOrder.OrderId, createdOrder.ProductsCount));
    }
}
```

La clase recibe a través del constructor una instancia del servicio de correo. El método *Handle* simplemente invoca al método *SendMailToAdministrator* con un mensaje indicando el identificador de la orden creada y el número de productos de la orden.

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar el manejador de evento

1. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el código del método *AddUseCasesServices* para registrar el nuevo servicio.

```
public static IServiceCollection AddUseCasesServices(
    this IServiceCollection services)
{
    services.AddScoped<ICreateOrderInputPort,
        CreateOrderInteractor>();

    services.AddModelValidator<CreateOrderDto,
```

```
        CreateOrderCustomerValidator>();

    services.AddModelValidator<CreateOrderDto,
        CreateOrderProductValidator>();

    services.AddScoped<IDomainEventHandler<SpecialOrderCreatedEvent>,
        SendEMailWhenSpecialOrderCreatedEventHandler>();

    return services;
}
```

Implementar el servicio de envío de correo

El servicio de envío de correo funcionará como un *Gateway* para enviar información del dominio hacia un servidor de correo en el exterior.

1. Agrega un nuevo directorio llamado ***EmailGateways*** en el directorio ***Gateways*** del directorio ***3. Interface Adapters***.
2. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Sales.Backend.SmtpGateways*** en el directorio *EmailGateways*.
3. Elimina el archivo ***Class1.cs***.
4. Agrega una referencia del proyecto *NorthWind.Sales.Backend.BusinessObjects*.

Agregar una clase para almacenar las opciones de envio de correo

1. Agrega un nuevo directorio llamado ***Options*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***SmtpOptions.cs*** en el directorio *Options* y remplaza el contenido de la clase por el siguiente.

```
public class SmtpOptions
{
    public const string SectionKey = nameof(SmtpOptions);

    public string SmtpHost { get; set; }
    public int SmtpHostPort { get; set; }
    public string SmtpUserName { get; set; }
    public string SmtpPassword { get; set; }
    public string SenderEmail { get; set; }
    public string AdministratorEmail { get; set; }
}
```

Implementar la interface IMailService

1. Agrega un nuevo archivo de clase llamado ***MailService.cs*** en la raíz del proyecto.
2. Remplaza el código de la clase *MailService* por el siguiente.

```
internal class MailService(IOptions<SmtpOptions> smtpOptions,
    ILogger<MailService> logger) : IMailService
{
    public async Task SendMailToAdministrator(
        string subject, string body)
    {
        try
        {
            MailMessage Message =
                new MailMessage(smtpOptions.Value.SenderEmail,
```

```
                smtpOptions.Value.AdministratorEmail);
            Message.Subject = subject;
            Message.Body = body;

            SmtpClient Client = new SmtpClient(
                smtpOptions.Value.SmtpHost,
                smtpOptions.Value.SmtpHostPort)
            {
                Credentials = new NetworkCredential(
                    smtpOptions.Value.SmtpUserName,
                    smtpOptions.Value.SmtpPassword),
                EnableSsl = true
            };

            await Client.SendMailAsync(Message);
        }
        catch (Exception ex)
        {
            logger.LogError(ex, ex.Message);
        }
    }
}
```

Puedes notar que, en caso de generarse una excepción, simplemente escribimos en el *log* la excepción generada, pero no detenemos la ejecución. Si las reglas de negocio requieren detener la ejecución, podríamos lanzar una excepción personalizada en lugar de escribir a *log*.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto *NorthWind.Sales.Backend.SmtpGateways* y remplaza el contenido del archivo con el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddMailServices(
        this IServiceCollection services,
        Action<SmtpOptions> configureSmtpOptions)
    {
        services.AddSingleton<IMailService, MailService>();
        services.Configure(configureSmtpOptions);

        return services;
    }
}
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Actualizar la nueva dependencia en el proyecto IoC

1. Agrega una referencia del proyecto *NorthWind.Sales.Backend.SmtpGateways* en el proyecto *NorthWind.Sales.Backend.IoC*.
2. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
3. Modifica el código del método ***AddNorthWindSalesServices*** para registrar el nuevo servicio. El código del método será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions);

    return services;
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Actualizar la nueva dependencia en el proyecto Web Api

1. Abre el archivo *Startup.cs* del proyecto *NorthWind.Sales.WebApi*.
2. Localiza el siguiente código que ahora está mostrando un error.

```
builder.Services.AddNorthWindSalesServices(dbOptions =>
    builder.Configuration.GetSection(DBOptions.SectionKey)
    .Bind(dbOptions));
```

3. Remplaza el código anterior por el siguiente.

```
builder.Services.AddNorthWindSalesServices(
    dbOptions =>
        builder.Configuration.GetSection(DBOptions.SectionKey)
        .Bind(dbOptions),
    smtpOptions =>
```

```
        builder.Configuration.GetSection(SmtpOptions.SectionKey)
        .Bind(smtpOptions));
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Establecer los valores de configuración del servidor SMTP

Para poder probar el servicio de correo, será necesario establecer los datos de configuración del servidor SMTP. Si lo deseas, puedes utilizar servicios de prueba de servidores *SMTP* tal como el que ofrece ***Mailtrap***: https://mailtrap.io/.

1. Abre el archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi*.
2. Agrega la información del servidor *SMTP* utilizando los datos de tu servidor *SMTP*. El código será similar al siguiente.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "DBOptions": {
    "ConnectionString": "Server=(localdb)\\mssqllocaldb;Database=NorthWindDB"
  },
  "SmtpOptions": {
    "SmtpHost": "sandbox.smtp.mailtrap.io",
    "SmtpHostPort": 465,
    "SmtpUserName": "1a72ffa4a9cec3",
    "SmtpPassword": "5ccb3732f43450",
    "SenderEmail": "no-replay@northwind.com",
    "AdministratorEmail": "admin@northwind.com"
  }
}
```

Modificar el Interactor del caso de uso

Finalmente, modifiquemos el código del *Interactor* del caso de uso para determinar si debe lanzarse el evento al crear una orden de compra.

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el constructor primario de la clase para agregar la nueva dependencia.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository,
    IModelValidatorHub<CreateOrderDto> modelValidatorHub,
    IDomainEventHub<SpecialOrderCreatedEvent> domainEventHub) :
    ICreateOrderInputPort
```

3. Agrega el siguiente código al final del método *Handle* para implementar la lógica para lanzar el evento.

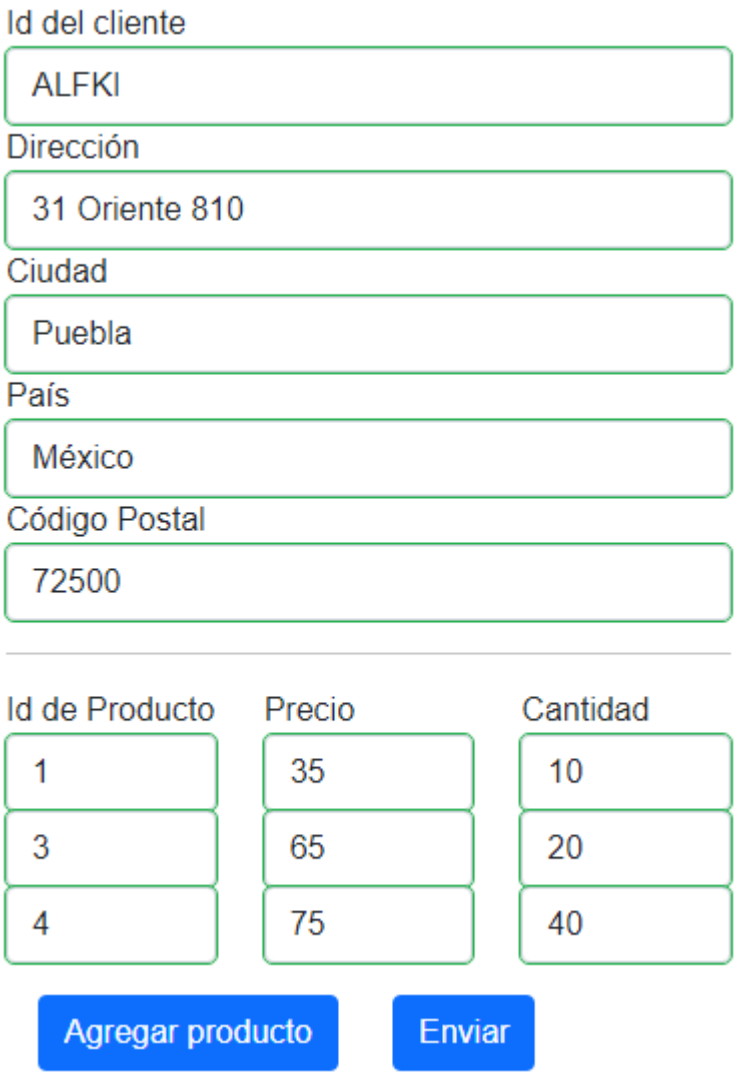
```
if (Order.OrderDetails.Count > 3)
{
    await domainEventHub.Raise(
        new SpecialOrderCreatedEvent(
            Order.Id, Order.OrderDetails.Count));
}
```

Probar la funcionalidad

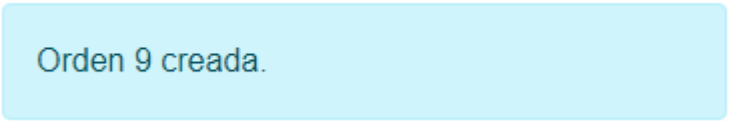
1. Ejecuta la aplicación.

Probar con una orden no mayor de 3 productos

1. Utiliza los siguientes datos para crear una orden con 3 productos.



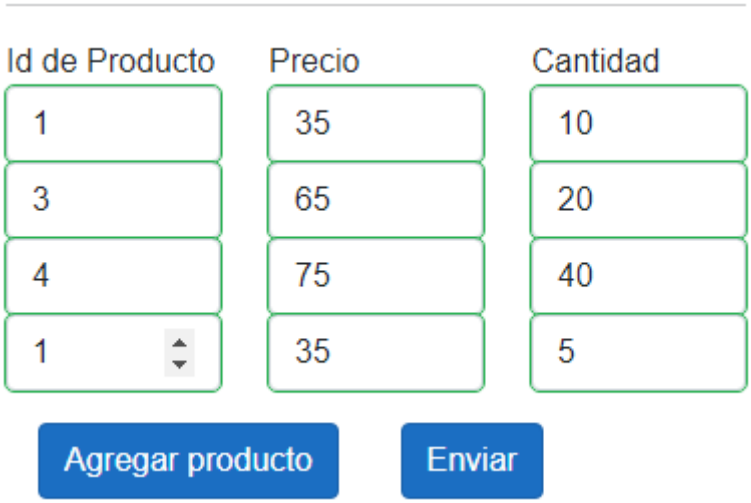
2. Podrás ver un resultado mostrando el identificador de la orden similar al siguiente.



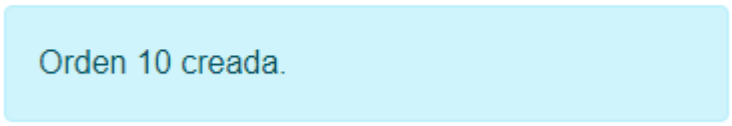
También puedes verificar que el correo no se haya enviado debido a que no es una orden con más de 3 productos.

Probar con una orden con productos duplicados

1. Utiliza los siguientes datos para crear una orden con productos repetidos.



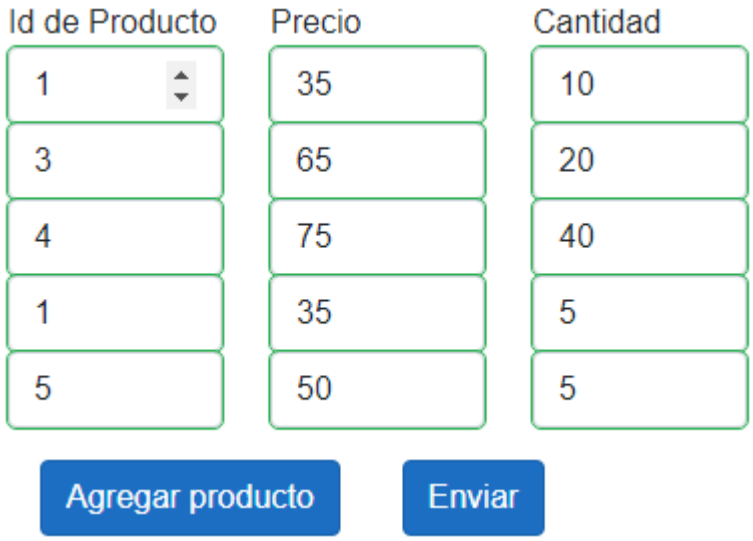
2. Podrás ver un resultado mostrando el identificador de la orden similar al siguiente.



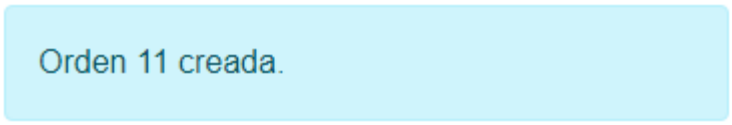
También puedes verificar que el correo no se haya enviado debido a que no es una orden con más de 3 productos.

Probar con una orden con más de 3 productos

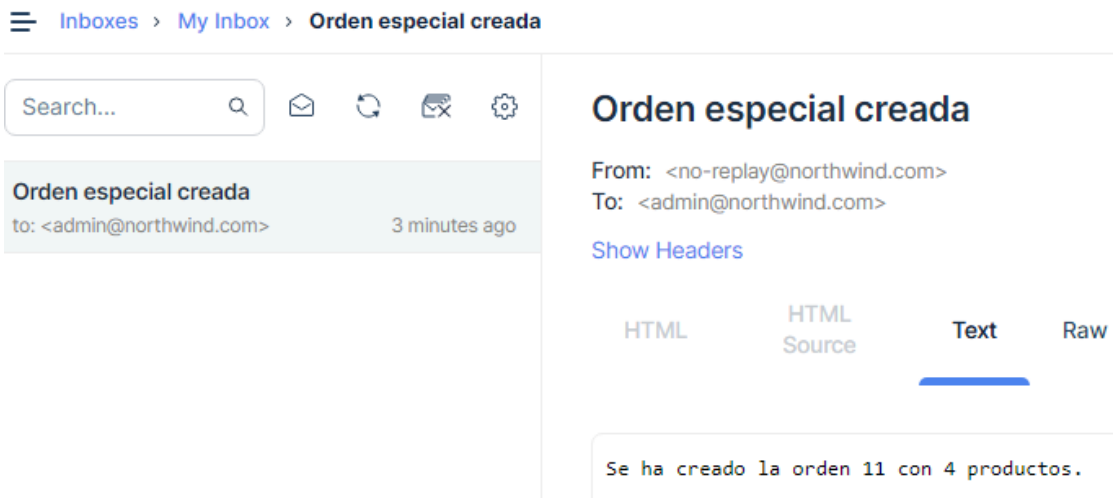
1. Utiliza los siguientes datos para crear una orden con más de 3 productos.



3. Podrás ver un resultado mostrando el identificador de la orden similar al siguiente.



Debido a que la orden tiene más de 3 productos, debió haberse enviado un correo. Si utilizas el servicio *Mailtrap* podrás ver un correo similar al siguiente.



4. Regresa a Visual Studio y detén la ejecución.

La aplicación implementa ahora el nuevo requerimiento para enviar un correo cuando una orden especial sea creada. Este requerimiento lo hemos implementado a través de un *Evento de dominio*.

Práctica 10:

Registro de sucesos (Logging)

En la mayoría de los casos es necesario llevar un registro de sucesos (*Logging*) que se producen durante la ejecución de una aplicación. Podemos diferenciar 2 tipos de registros:

- ***Registros del estado de la aplicación***. Estos registros almacenan información del estado de una aplicación en el momento de una falla técnica. Esta información nos ayuda a diagnosticar y dar solución a errores en el funcionamiento de la aplicación.
- ***Registros del dominio.*** Estos registros nos permiten llevar un seguimiento de las distintas acciones que suceden en el dominio, por ejemplo, la creación de una orden, la eliminación de un cliente, etc.

En prácticas anteriores hemos implementado el registro del estado de la aplicación utilizando el servicio nativo de *Logging (ILogger)* de .NET, en esta práctica ejemplificaremos los registros de dominio agregando nuevos requerimientos al caso de uso *Crear orden de compra*.

Objetivos

Al finalizar esta práctica los estudiantes contarán con las habilidades y conocimientos para:

- Implementar registro (*Logging*) para dar seguimiento a las actividades realizadas en una aplicación.

Agregar nuevos requerimientos al caso de uso

Modifiquemos nuestro caso de uso para poder ejemplificar la implementación de *Registro de sucesos* (*Logging*). Los nuevos requerimientos son los siguientes:

- Después de validar los datos de entrada, el sistema deberá registrar la acción "*Inicio de creación de orden de compra*" con fines de auditoría.
- Después de crear la orden de compra, el sistema deberá registrar la acción "*Orden de compra <Número de Orden> creada.*" con fines de auditoría.

Historia de Usuario: Crear orden de compra

"Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema valida los datos.
3. **El sistema registra la acción "*Inicio de creación de orden de compra*" con fines de auditoría.**
4. El sistema registra la orden de compra.
5. **El sistema registra la acción "*Orden de compra <Número de Orden> creada.*" con fines de auditoría.**
6. Cuando el número de productos de la orden sea mayor de 3, el sistema enviará un correo electrónico de notificación de "Orden especial creada" al administrador de la empresa.

7. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Validaciones

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.
- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.
- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.

- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

Los cambios del caso de uso se encuentran en el paso 3 y 5 del flujo primario.

Definir las entidades para el registro de sucesos

Con el fin de implementar los registros de sucesos del dominio, agreguemos nuevas entidades. Estas entidades podrán ser utilizadas por cualquier aplicación por lo que serán agregadas en la capa de *Reglas de Negocio Empresariales*. Antes de iniciar, si lo deseas, puedes realizar un respaldo de la solución actual para que conserves el estado de los cambios.

Crear un nuevo proyecto

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto a la solución llamado `NorthWind.DomainLogs.Entities` en el directorio `1. Enterprise Business Rules`.
2. Elimina el archivo ***Class1.cs***.

Agregar la clase DomainLog

1. Agrega un nuevo directorio llamado ***ValueObjects*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***DomainLog.cs*** en el directorio *ValueObjects* y remplaza el contenido de la clase con el siguiente.

```
public class DomainLog(string information)
{
    public DateTime DateTime => DateTime.Now;
    public string Information => information;
}
```

En nuestro caso, la información de un registro de dominio incluirá únicamente la fecha y hora actual del sistema, así como la descripción de este.

Agregar la abstracción IDomainLogger

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo de *Interface* llamado ***IDomainLogger.cs*** en el directorio *Interfaces* y modifica el código de la interface por el siguiente.

```
public interface IDomainLogger
{
    Task LogInformation(DomainLog log);
}
```

3. Importa el espacio de nombres requerido en el archivo GlobalUsings.cs

Agregar la abstracción IDomainLogsRepository

1. Agrega una referencia del proyecto *NorthWind.Entities*. Este proyecto define la interface *IUnitOfWork* que necesitaremos para la nueva abstracción.
2. Agrega un nuevo archivo de tipo *interface* llamado ***IDomainLogsRepository.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface IDomainLogsRepository : IUnitOfWork
{
    Task Add(DomainLog log);
}
```

   Hemos definido un repositorio para poder persistir los registros de sucesos del dominio. El repositorio implementa la interface *IUnitOfWork*.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el servicio IDomainLogger

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***DomainLogger.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class DomainLogger(IDomainLogsRepository repository) : IDomainLogger
{
    public async Task LogInformation(DomainLog log)
    {
        await repository.Add(log);
        await repository.SaveChanges();
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y modifica su contenido para que sea similar al siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
```

```
public static class DependencyContainer
{
    public static IServiceCollection AddDomainLogsServices(
        this IServiceCollection services)
    {
        services.AddScoped<IDomainLogger, DomainLogger>();

        return services;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. En el proyecto *NorthWind.Sales.Backend.BusinessObjects*, agrega una referencia del proyecto *NorthWind.DomainLogs.Entities*.
5. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
6. Modifica el código del método ***AddNorthWindSalesServices*** para registrar el nuevo servicio. El código del método será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices();

    return services;
}
```

Implementar la nueva funcionalidad requerida

Modifiquemos ahora el código de la aplicación para implementar la nueva funcionalidad requerida.

Agregar mensajes en el archivo de recursos

1. Agrega la siguiente información en el archivo de recursos ***CreateOrderMessages.resx*** del proyecto *NorthWind.Sales.Backend.UseCases*.

Nombre	Valor
StartingPurchaseOrderCreation	Inicio de creación de orden de compra
PurchaseOrderCreatedTemplate	Orden de compra {0} creada.

Registrar las acciones de creación de orden

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el constructor primario de la clase para recibir la dependencia ***IDomainLogger***.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository,
    IModelValidatorHub<CreateOrderDto> modelValidatorHub,
    IDomainEventHub<SpecialOrderCreatedEvent> domainEventHub,
    IDomainLogger domainLogger) : ICreateOrderInputPort
```

3. Localiza el siguiente código en el método *Handle*.

```
await GuardModel.AgainstNotValid(modelValidatorHub, orderDto);
```

4. Agrega el siguiente código justo después del código anterior.

```
await domainLogger.LogInformation(new DomainLog(
    CreateOrderMessages.StartingPurchaseOrderCreation));
```

5. Localiza el siguiente código en el método *Handle*.

```
await repository.SaveChanges();
```

6. Agrega el siguiente código justo después de la línea de código anterior.

```
await domainLogger.LogInformation(new DomainLog(string.Format(
    CreateOrderMessages.PurchaseOrderCreatedTemplate, Order.Id)));
```

7. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar el repositorio para el registro de sucesos del dominio

Implementemos ahora el repositorio que es utilizado para persistir la información de los sucesos del dominio.

Agregar la entidad DomainLog

1. Agrega un nuevo archivo de clase llamado ***DomainLog.cs*** en el directorio *Entities* del proyecto *NorthWind.Sales.Backend.Repositories* y remplaza el código de la clase por el siguiente.

```
public class DomainLog
{
    public int Id { get; set; }
    public DateTime CreatedDate { get; set; }
    public string Information { get; set; }
}
```

Agregar la interface para el contexto de datos del repositorio

1. Agrega un nuevo archivo de tipo *Interface* llamado ***INorthWindDomainLogsDataContext.cs*** en el directorio *Interfaces* del proyecto *NorthWind.Sales.Backend.Repositories* y remplaza el código de la interface por el siguiente.

```
public interface INorthWindDomainLogsDataContext
{
    Task AddLogAsync(Entities.DomainLog log);
    Task SaveChangesAsync();
}
```

Implementar el repositorio

1. Agrega un nuevo archivo de clase llamado ***DomainLogsRepository.cs*** en el directorio *Repositories* del proyecto *NorthWind.Sales.Backend.Repositories* y remplaza el código de la clase por el siguiente.

```
internal class DomainLogsRepository(
    INorthWindDomainLogsDataContext context) : IDomainLogsRepository
{
    public async Task Add(DomainLogs.Entities.ValueObjects.DomainLog log)
    {
        await context.AddLogAsync(new Entities.DomainLog
        {
            CreatedDate = log.DateTime,
            Information = log.Information
```

```
        });
    }

    public async Task SaveChanges() =>
        await context.SaveChangesAsync();
}
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar el repositorio

1. Agrega el siguiente código en el método *AddRepositories* del archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.Repositories* para registrar el servicio.

```
services.AddScoped<IDomainLogsRepository, DomainLogsRepository>();
```

Implementar el contexto de datos del repositorio

Trabajemos ahora con el proyecto *NorthWind.Sales.Backend.DataContexts.EFCore* para implementar el contexto de datos del repositorio.

Modificar la clase DbOptions

En este ejercicio, utilizaremos una nueva base de datos para almacenar los registros de sucesos de dominio, aunque también podría ser posible utilizar el contexto de datos *NorthWindContext* para utilizar la misma base de datos de la aplicación.

1. Abre el archivo *DBOptions.cs* del directorio *Options*.
2. Agrega el siguiente código para poder almacenar el valor de la cadena de conexión de la base de datos donde se agregarán los registros de sucesos del dominio.

```
public string DomainLogsConnectionString { get; set; }
```

Crear el nuevo contexto de datos

Agreguemos ahora el contexto de datos que será utilizado para persistir los registros de dominio.

1. Agrega un nuevo archivo de clase llamado ***NorthWindDomainLogsContext.cs*** en el directorio *DataContexts* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindDomainLogsContext(IOptions<DBOptions> dbOptions)
    : DbContext
{
    public DbSet<DomainLog> DomainLogs { get; set; }
    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            dbOptions.Value.DomainLogsConnectionString);
    }
}
```

Crear una fabrica para la creación del contexto

1. Agrega un nuevo archivo de clase llamado ***NorthWindDomainLogsContextFactory.cs*** en el directorio *DataContexts* y remplaza el contenido de la clase por el siguiente.

```
internal class NorthWindDomainLogsContextFactory :
    IDesignTimeDbContextFactory<NorthWindDomainLogsContext>
```

```
{
    public NorthWindDomainLogsContext CreateDbContext(string[] args)
    {
        IOptions<DBOptions> DbOptions =
            Microsoft.Extensions.Options.Options.Create(
            new DBOptions
            {
                DomainLogsConnectionString =
                    "Server=(localdb)\\mssqllocaldb;Database=NorthWindLogsDB"
            });

        return new NorthWindDomainLogsContext(DbOptions);
    }
}
```

Podemos indicar a las herramientas de *Entity Framework Core* la forma de crear el *DbContext* a través de la creación de un *generador de tiempo de diseño.* El generador es una clase que implementa la interface ***IDesignTimeDbContextFactory<TContext>***.

Si una clase que implementa esta interface se encuentra en el mismo proyecto que el *DbContext* derivado o en el proyecto de inicio de la aplicación, las herramientas omiten las otras formas de crear el *DbContext* y usan el generador en tiempo de diseño en su lugar.

Un generador en tiempo de diseño puede ser especialmente útil si necesitamos configurar el *DbContext* de forma diferente para el tiempo de diseño que para el tiempo de ejecución.

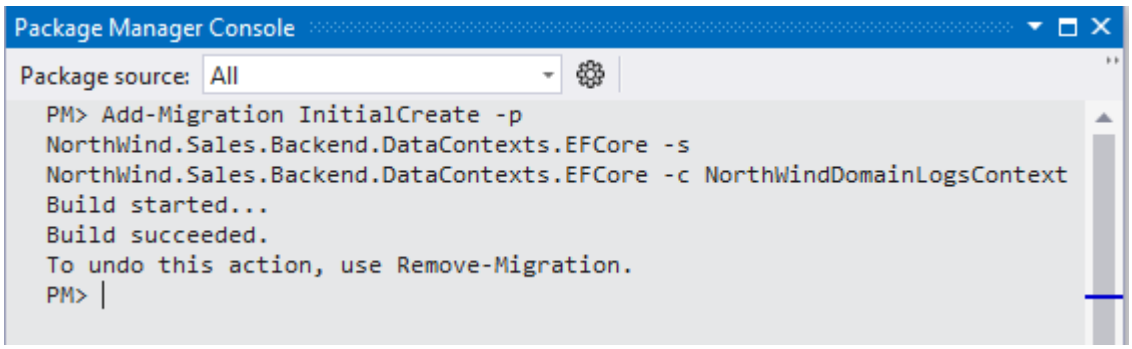
Puedes notar que estamos construyendo una instancia de ***IOptions<DBOptions>*** utilizando la cadena de conexión a la nueva base de datos. Tendremos que agregar ese valor en el archivo de configuración de la Web Api.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear la migración

1. Ejecuta el siguiente comando en la consola de administración de paquetes NuGet para crear la migración inicial del nuevo contexto.

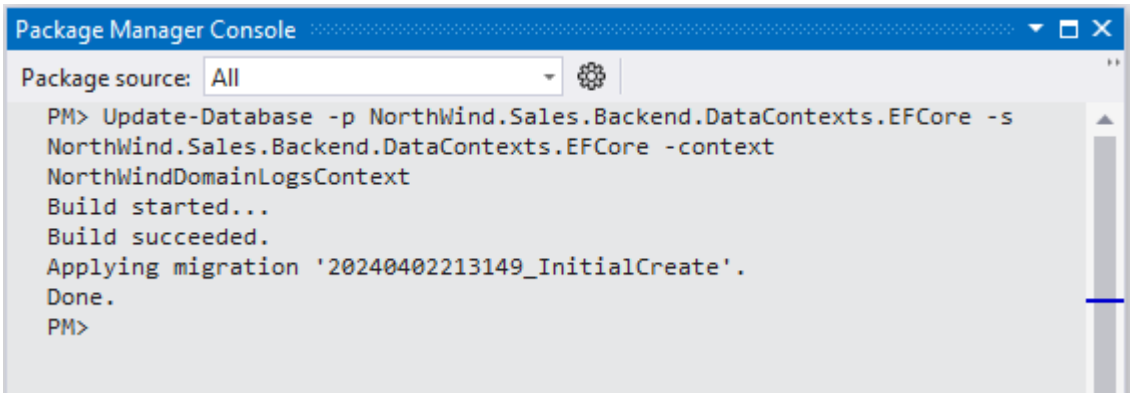
```
Add-Migration InitialCreate -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -c NorthWindDomainLogsContext
```



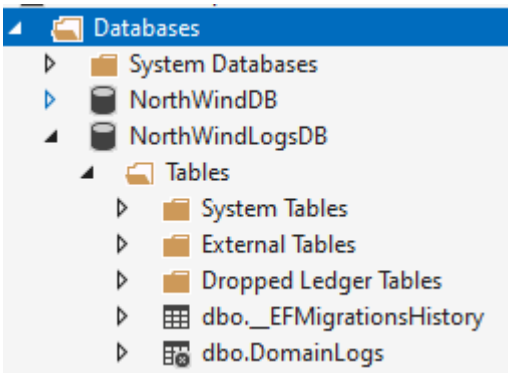
Crear la base de datos

1. Ejecuta el siguiente comando para crear la nueva base de datos y verifica que no haya errores.

```
Update-Database -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -context NorthWindDomainLogsContext
```



2. Verifica que la nueva base de datos haya sido creada.



Implementar el contexto de datos del repositorio

Implementemos ahora el contexto de datos del repositorio que será utilizado para persistir los registros de dominio.

1. Agrega un nuevo archivo de clase llamado ***NorthWindDomainLogsDataContext.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindDomainLogsDataContext(IOptions<DBOptions> dbOptions) :
    NorthWindDomainLogsContext(dbOptions),
    INorthWindDomainLogsDataContext
{
    public async Task AddLogAsync(DomainLog log) =>
        await AddAsync(log);

    public async Task SaveChangesAsync() =>
        await GuardDBContext.AgainstSaveChangesErrorAsync(this);
}
```

Registrar el servicio

1. Agrega el siguiente código al método *AddDataContexts* del archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.DataContexts.EFCore* para registrar el nuevo servicio implementado.

```
services.AddScoped<INorthWindDomainLogsDataContext,
    NorthWindDomainLogsDataContext>();
```

Agregar la cadena de conexión

1. Abre el archivo ***appsettings.json*** del proyecto *NorthWind.Sales.WebApi*.
2. Modifica el elemento *DbOptions* para agregar el valor de la nueva cadena de conexión.

```
"DBOptions": {
  "ConnectionString": "Server=(localdb)\\mssqllocaldb;Database=NorthWindDB",
  "DomainLogsConnectionString":
"Server=(localdb)\\mssqllocaldb;Database=NorthWindLogsDB"
},
```

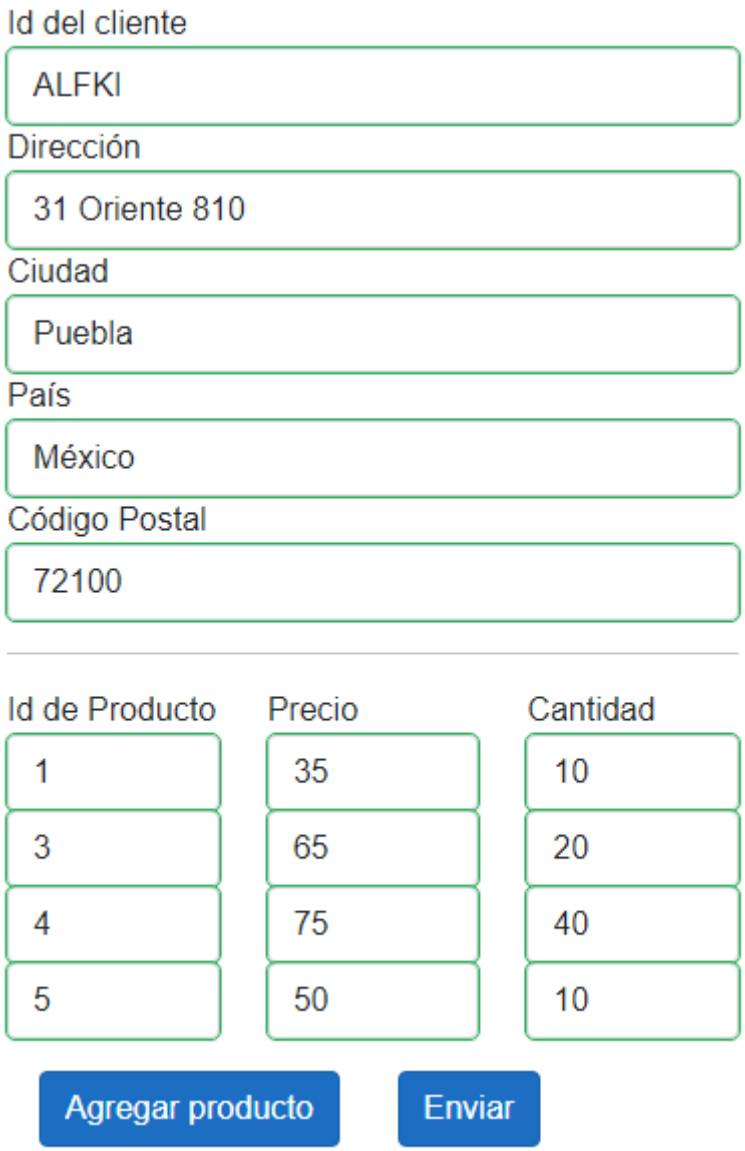
Probar la funcionalidad

Finalmente, probemos la nueva funcionalidad.

1. Ejecuta la aplicación.

Crear una orden

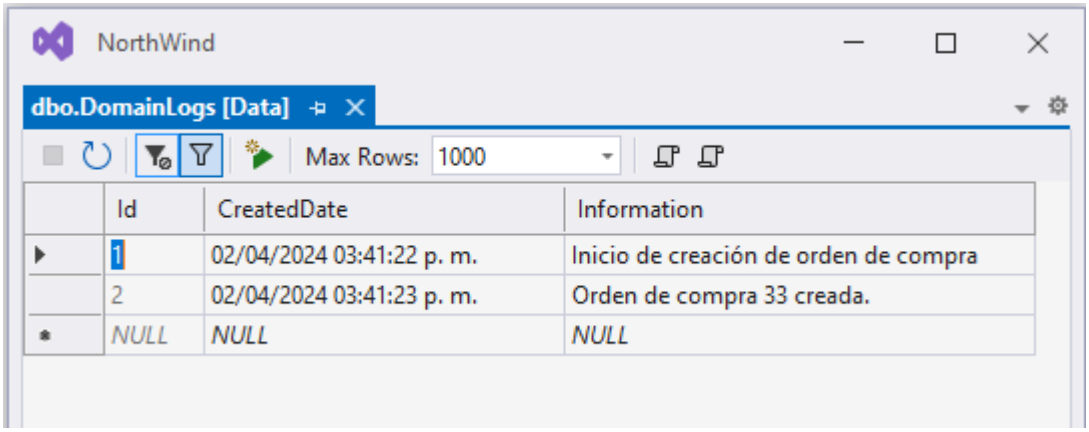
1. Proporciona los siguientes datos para crear una orden.



2. Podrás ver un resultado mostrando el identificador de la orden creada similar al siguiente.

Orden 33 creada.

3. Regresa a Visual y detén la ejecución.
4. Verifica que la tabla *DomainLogs* de la base de datos *NorthWindLogsDB* tenga los registros de creación de orden.



Hemos implementado el registro de sucesos de dominio. Para el caso de los sucesos de la aplicación, hemos utilizado el servicio predeterminado de .NET (*ILogging*), sin embargo, podemos también utilizar cualquier servicio de terceros. Si deseamos escribir los mensajes de log del estado de la aplicación a un archivo, podríamos implementar un proveedor de logs personalizado.

Los siguientes enlaces proporcionan información detallada del servicio de ***Logging*** de .NET, así como la creación de proveedores de *logs* personalizados.

Logging in .NET
https://docs.microsoft.com/en-us/dotnet/core/extensions/logging

Implement a custom logging provider in .NET
https://docs.microsoft.com/en-us/dotnet/core/extensions/custom-logging-provider

Práctica 11:

Manejo de transacciones

Cuando implementamos el patrón *Repository* junto con el patrón *Unit Of Work*, hacemos que nuestras operaciones de persistencia de datos sean realizadas de forma transaccional. En algunos casos, cuando realizamos operaciones de persistencia de datos utilizando múltiples repositorios, podría ser necesario implementar un manejo de transacciones a nivel de caso de uso envolviendo múltiples repositorios.

En esta práctica implementaremos un escenario de manejo de transacciones a nivel de caso de uso.

Objetivos

Al finalizar esta práctica los estudiantes contarán con las habilidades y conocimientos para:

- Implementar manejo de transacciones a nivel de caso de uso envolviendo uno o más repositorios.

Agregar un nuevo requerimiento al caso de uso

Modifiquemos nuestro caso de uso para poder ejemplificar la implementación de manejo de transacciones a nivel de caso de uso. El nuevo requerimiento es el siguiente:

Si no es posible crear una orden, por ejemplo, cuando no se pueda enviar el correo al administrador, deberá cancelarse la creación de la orden y deberá registrarse la acción "Creación de orden cancelada", indicando de ser posible el identificador de la orden que haya sido cancelada.

Este será el nuevo caso de uso donde solo hemos agregado un flujo alterno.

Historia de Usuario: Crear orden de compra

"Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. El sistema valida los datos.
3. El sistema registra la acción "*Inicio de creación de orden de compra*" con fines de auditoría.
4. El sistema registra la orden de compra.
5. El sistema registra la acción "*Orden de compra <Número de Orden> creada.*" con fines de auditoría.
6. Cuando el número de productos de la orden sea mayor de 3, el sistema enviará un correo electrónico de notificación de "Orden especial creada" al administrador de la empresa.

7. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Flujo alterno: Error al crear la orden

1. El proceso de la solicitud es cancelado.
2. El sistema registra la acción "*Creación de orden cancelada*" indicando de ser posible, el identificador de la orden que haya sido cancelada.
3. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.

Validaciones

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.

- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.
- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.
- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

El cambio del caso de uso se encuentra en el flujo alterno "***Error al crear la orden"***.

Implementar las entidades para manejo de Transacciones

Empecemos por definir las entidades para el manejo de transacciones en un nuevo proyecto. Este proyecto definirá entidades que podrán ser consumidas por diferentes aplicaciones, por lo tanto, crearemos el proyecto en la capa de reglas de negocio empresariales. Antes de iniciar, si lo deseas, puedes realizar un respaldo de la solución actual para que conserves el estado de los cambios.

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Transactions.Entities*** en el directorio `1. Enterprise Business Rules`.
2. Elimina el archivo ***Class1.cs***.
3. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
4. Agrega un nuevo archivo de tipo *Interface* llamado ***IDomainTransaction.cs*** en el directorio Interfaces y remplaza el código de la interface por el siguiente.

```
public interface IDomainTransaction
{
    void BeginTransaction();
    void CommitTransaction();
    void RollbackTransaction();
}
```

El método *BeginTransaction* nos permitirá definir el inicio de un bloque de código que será tratado como una transacción.

El método *CommitTransaction* nos permitirá aceptar los cambios realizados en la transacción.

El método *RollbackTransaction* nos permitirá cancelar una transacción.

Implementar el servicio para manejo de transacciones

Para implementar el manejo de transacciones en aplicaciones .NET podemos hacer uso de la infraestructura ***System.Transactions***. Esta infraestructura nos proporciona un modelo de programación explicita basado en la clase ***Transaction*** y nos proporciona un modelo de programación implícita basado en la clase ***TransactionScope*** en el cual las transacciones son administradas automáticamente por la infraestructura.

Microsoft recomienda crear transacciones implícitas mediante la clase *TransactionScope* de modo que el ambiente del contexto de la transacción sea administrado automáticamente.

El servicio que implementaremos para manejo de transacciones estará basado en el uso de la clase *TransactionScope*.

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***DomainTransaction.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class DomainTransaction : IDomainTransaction, IDisposable
{
}
```

3. Agrega el siguiente código a la clase *DomainTransaction* para declarar una variable que nos permitirá almacenar una instancia de la clase *TransactionScope*.

```
TransactionScope TransactionScope;
```

4. Agrega el siguiente código a la clase *DomainTransaction* para implementar el método *BeginTransaction*.

```
public void BeginTransaction()
{
    TransactionManager.ImplicitDistributedTransactions = true;
    TransactionScope = new TransactionScope(
        TransactionScopeOption.Required,
        new TransactionOptions
        {
            IsolationLevel = IsolationLevel.ReadCommitted
        },
        TransactionScopeAsyncFlowOption.Enabled);
}
```

El código anterior utiliza la clase `TransactionManager` que contiene propiedades estáticas y métodos que se utilizan para obtener información sobre las opciones de transacción predeterminadas.

La propiedad `ImplicitDistributedTransactions` establece un valor que indica si el uso de las APIs ***System.Transactions*** que requieren escalamiento a una transacción distribuida podrán hacerlo. El valor `true` asignado, indica que requerimos una transacción distribuida ya que estamos afectando 2 bases de datos distintas. El valor predeterminado de esta propiedad es `false`. Actualmente, en .NET 8, el soporte de transacciones distribuidas está limitado a la plataforma Windows. Si deseamos ejecutar el código en Linux, podemos evitar el uso de transacciones distribuidas utilizando una sola base de datos en lugar de 2 como lo tenemos actualmente en este ejercicio.

La instancia de *TransactionScope* es utilizada para ejecutar comandos garantizando que estos puedan aceptarse o cancelarse como una sola unidad de trabajo.

El valor ***TransactionScopeOption.Required*** indica que se requiere una transacción para el ámbito de código. Un ambiente de transacción es utilizado en caso de que este exista. Si no existe un ambiente de transacción, uno nuevo es creado. *Required* es el valor predeterminado.

A través del parámetro *TransactionOptions* podemos describir las opciones de la transacción cuando una nueva transacción es creada. La propiedad *IsolationLevel* nos permite especificar el nivel de aislamiento de la transacción. El valor *ReadCommitted* indica que los datos volátiles no pueden ser leídos (datos modificados en otra transacción que pueden ser cancelados), esto garantiza que únicamente se puedan leer los datos que han sido aceptados (*commited*).

El argumento *TransactionScopeAsyncFlowOption.Enabled* habilita el flujo de la transacción entre hilos (*threads*) diferentes, útil en operaciones asíncronas donde participan varios hilos de ejecución.

5. Agrega el siguiente código para implementar el método *CommitTransaction*.

```
public void CommitTransaction()
{
    TransactionScope.Complete();
    Dispose();
}
```

El método *Complete* del *TransactionScope* acepta la transacción. Si el método *Complete* nunca es invocado, cuando el método *Dispose* sea ejecutado la transacción será cancelada. Más adelante implementaremos el método *Dispose*.

6. Agrega el siguiente código para implementar el método *RollbackTransaction*.

```
public void RollbackTransaction()
{
    Dispose();
}
```

El método *RollbackTransaction* es similar al método *CommitTransaction* con la diferencia de que no estamos invocando al método *Complete* para que al invocar al método *Dispose*, la transacción sea cancelada.

7. Agrega el siguiente código para implementar el método *Dispose* que se encargará de liberar recursos de la transacción.

```
public void Dispose()
{
    TransactionScope?.Dispose();
}
```

8. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar el servicio

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y modifica su contenido para que sea similar al siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddTransactionServices(
        this IServiceCollection services)
    {
        services.AddTransient<IDomainTransaction, DomainTransaction>();

        return services;
    }
}
```

3. Importa los espacio de nombres requeridos en el archivo *GlobalUsings.cs*.
4. En el proyecto *NorthWind.Sales.Backend.BusinessObjects*, agrega una referencia del proyecto *NorthWind.Transactions.Entities*.
5. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.

6. Modifica el código del método ***AddNorthWindSalesServices*** para registrar el nuevo servicio. El código del método será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices()
        .AddTransactionServices();

    return services;
}
```

Implementar el nuevo requerimiento

Agregar un mensaje al archivo de de recursos

1. Agrega la siguiente información en el archivo de recursos ***CreateOrderMessages.resx*** del proyecto *NorthWind.Sales.Backend.UseCases*.

Nombre	Valor
OrderCreationCancelledTemplate	Creación de orden {0} cancelada.

Modificar el interactor del caso de uso

Modifiquemos el Interactor del caso de uso para ejemplificar el manejo de transacciones.

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el código del constructor primario de la clase para recibir la instancia de *IDomainTransaction*.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository,
    IModelValidatorHub<CreateOrderDto> modelValidatorHub,
    IDomainEventHub<SpecialOrderCreatedEvent> domainEventHub,
    IDomainLogger domainLogger,
    IDomainTransaction domainTransaction) :
    ICreateOrderInputPort
```

3. Localiza la siguiente línea de código.

```
OrderAggregate Order =
    OrderAggregate.From(orderDto);
```

4. Agrega el siguiente código después del código anterior.

```
try
{
    // Iniciar la transacción
    domainTransaction.BeginTransaction();
```

5. Agrega el siguiente código justo antes del final del método *Handle*.

```
    // Aceptar la transacción
    domainTransaction.CommitTransaction();
```

```
}
```

6. Agrega el siguiente código después del código anterior para verificar si se ha generado una excepción.

```
catch
{
    // Cancelar la transacción
    domainTransaction.RollbackTransaction();
    string Information = string.Format(
        CreateOrderMessages.OrderCreationCancelledTemplate,
        Order.Id);
    await domainLogger.LogInformation(new DomainLog(Information));
    throw;
}
```

El código completo del método *Handle* será similar al siguiente.

```
public async Task Handle(CreateOrderDto orderDto)
{
    await GuardModel.AgainstNotValid(modelValidatorHub, orderDto);

    await domainLogger.LogInformation(new DomainLog(
        CreateOrderMessages.StartingPurchaseOrderCreation));


    OrderAggregate Order = OrderAggregate.From(orderDto);

    try
    {
        // Iniciar la transacción
        domainTransaction.BeginTransaction();

        await repository.CreateOrder(Order);

        await repository.SaveChanges();
        await domainLogger.LogInformation(new DomainLog(string.Format(
            CreateOrderMessages.PurchaseOrderCreatedTemplate, Order.Id)));

        await outputPort.Handle(Order);

        if (Order.OrderDetails.Count > 3)
        {
            await domainEventHub.Raise(
                new SpecialOrderCreatedEvent(
                    Order.Id, Order.OrderDetails.Count));
        }

        // Aceptar la transacción
        domainTransaction.CommitTransaction();
    }
    catch
    {
        // Cancelar la transacción
```

```
            domainTransaction.RollbackTransaction();
            string Information = string.Format(
                CreateOrderMessages.OrderCreationCancelledTemplate,
                Order.Id);
            await domainLogger.LogInformation(new DomainLog(Information));
            throw;
        }
    }
```

Puedes notar que hemos agregado un bloque *try-catch* para atrapar excepciones. El código que podría generar una excepción los hemos delimitado entre *BeginTransaction* y *CommitTransaction*.

En caso de que haya una excepción, invocamos al método *RollbackTransaction* para cancelar la creación de la orden y el registro de orden creada.

Por simplicidad no estamos atrapando excepciones dentro del bloque *catch*.

7. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Modificar el servicio MailService

Modifiquemos el código del servicio *MailService* para generar una excepción cuando el correo no pueda ser enviado.

1. Abre el archivo *MailService.cs* del proyecto *NorthWind.Sales.Backend.SmtpGateways*.
2. Modifica el código del bloque catch para que sea similar al siguiente.

```
catch (Exception ex)
{
    logger.LogError(ex, ex.Message);
    throw;
}
```

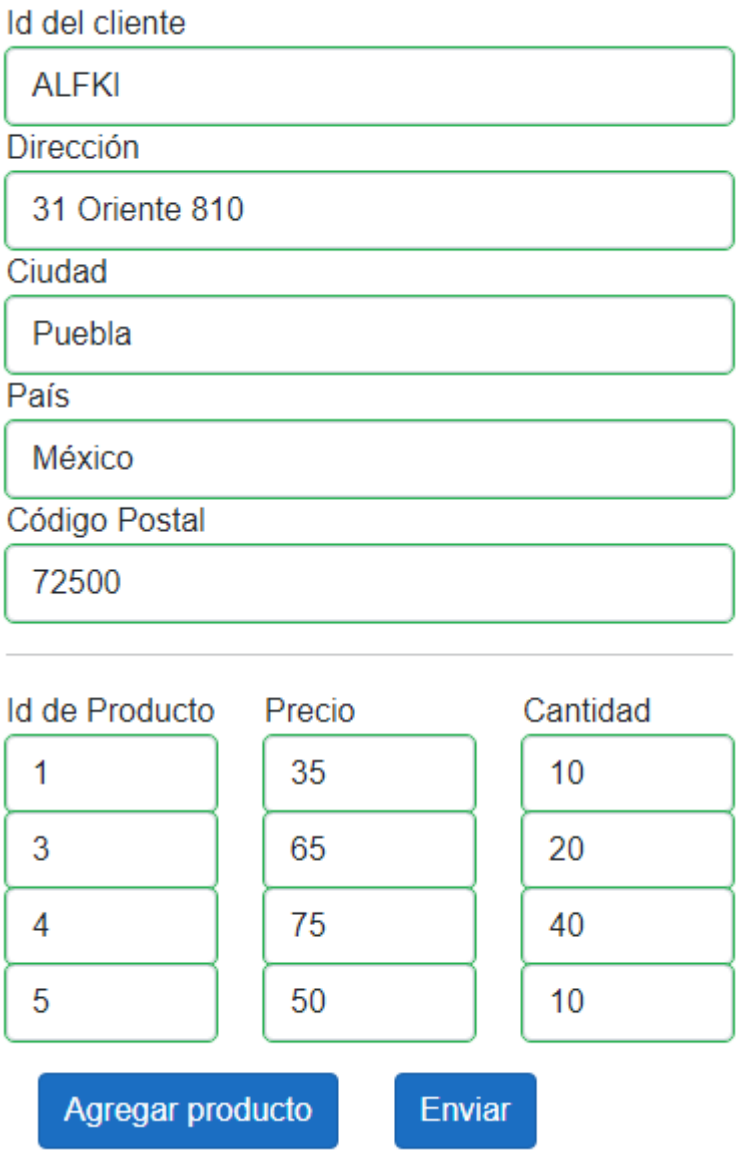
Ahora, cuando no pueda enviarse el correo, se dejará pasar la excepción.

Probar la funcionalidad

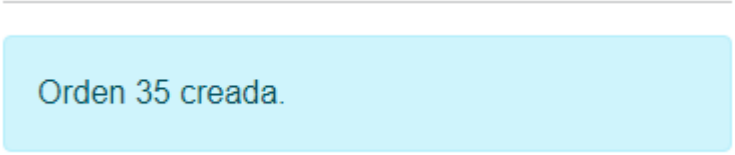
1. Ejecuta la aplicación.

Crear una orden con más de 3 productos

1. Utiliza los siguientes datos para crear una orden con más de 3 productos.



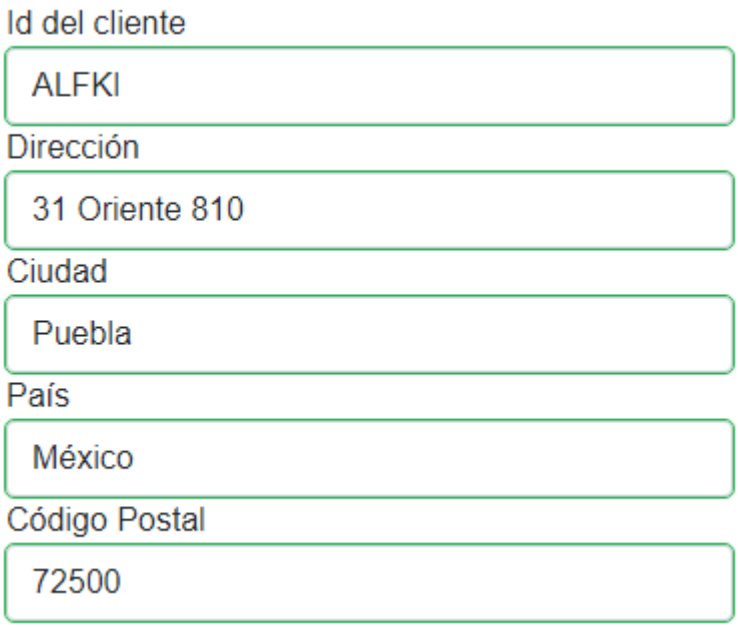
4. Podrás ver un resultado mostrando el identificador de la orden similar al siguiente.

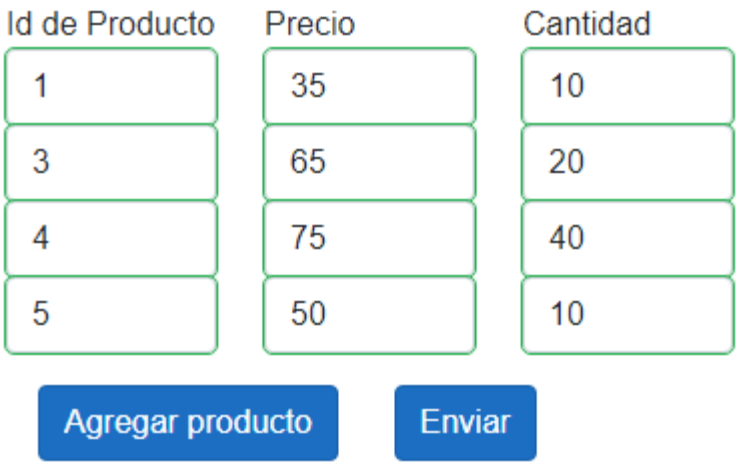


5. Regresa a Visual Studio y detén la ejecución.

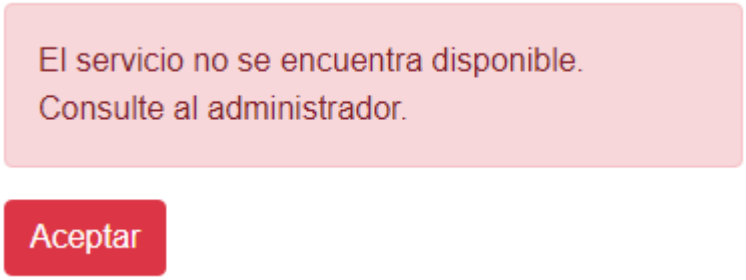
Simular error de envío de correo

1. Establece un valor incorrecto de "*SmtpUserName*" en el archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi*.
2. Ejecuta la aplicación.
3. Utiliza los siguientes datos para crear una orden con más de 3 productos.

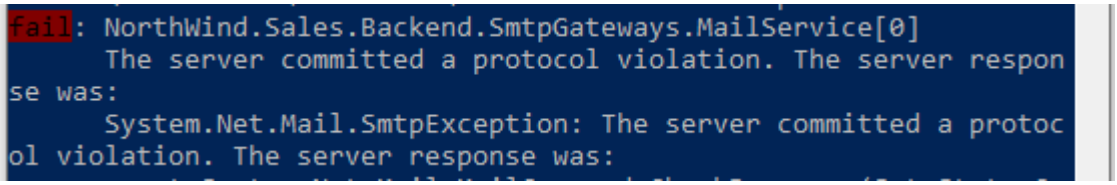




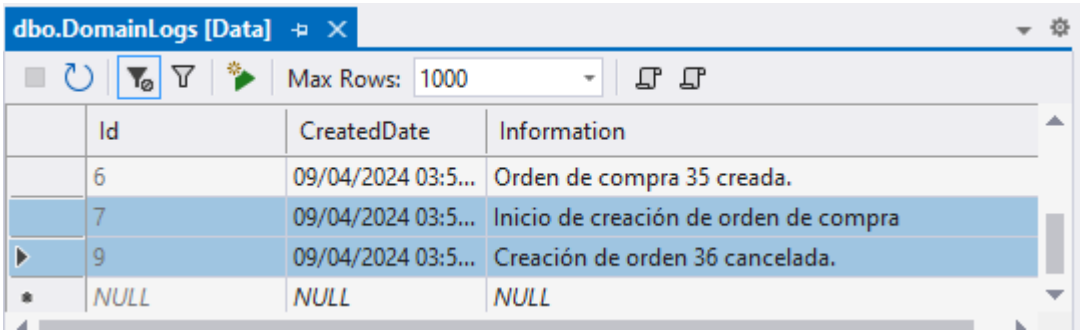
4. Podrás ver un resultado similar al siguiente.



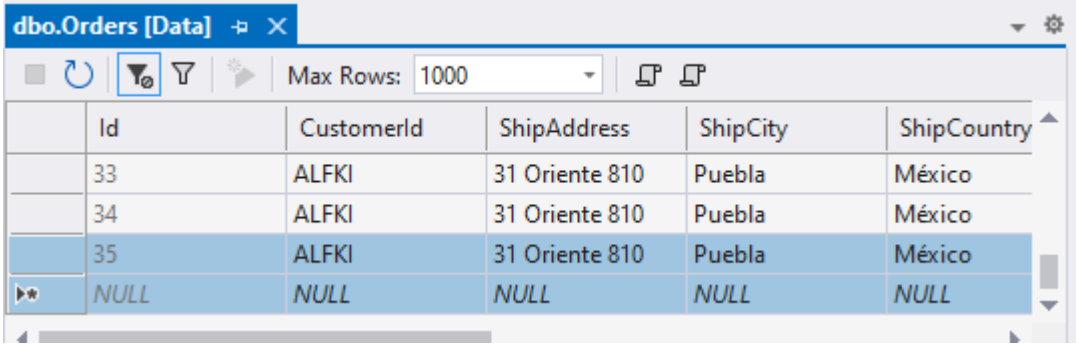
5. En la consola podrás encontrar un mensaje similar al siguiente.



6. En la tabla *DomainLogs* de la base de datos podrás encontrar los registros indicando que la creación de la orden fue cancelada.



7. La tabla *Orders* de la base de datos no habrá registrado la orden cancelada.



Id	CustomerId	ShipAddress	ShipCity	ShipCountry
33	ALFKI	31 Oriente 810	Puebla	México
34	ALFKI	31 Oriente 810	Puebla	México
35	ALFKI	31 Oriente 810	Puebla	México
NULL	NULL	NULL	NULL	NULL

8. Regresa a Visual Studio y detén la ejecución.
9. Corrige el valor "*SmtpUserName*" en el archivo *appsettings.json* del proyecto *NorthWind.Sales.WebApi*.

Hemos ejemplificado una forma de implementar transacciones de dominio utilizando la infraestructura de transacciones de .NET.

Los siguientes enlaces proporcionan más información sobre el uso de transacciones en aplicaciones .NET.

TransactionScope Class
https://docs.microsoft.com/en-us/dotnet/api/system.transactions.transactionscope

Implementing an Implicit Transaction using Transaction Scope
https://docs.microsoft.com/en-us/dotnet/framework/data/transactions/implementing-an-implicit-transaction-using-transaction-scope

Writing a Transactional Application
https://docs.microsoft.com/en-us/dotnet/framework/data/transactions/writing-a-transactional-application

TransactionManager Class
https://learn.microsoft.com/dotnet/api/system.transactions.transactionmanager

Práctica 12:

El patrón Specification

Otro de los patrones comúnmente utilizados en arquitecturas orientadas al dominio, es el patrón de diseño ***Specification***.

En esta práctica exploraremos el patrón de diseño *Specification* y la forma en que podemos utilizarlo al implementar una arquitectura limpia.

Objetivos

Al finalizar esta práctica, los estudiantes contarán con las habilidades y conocimientos para:

- Describir el patrón *Specification*.
- Implementar el patrón *Specification* en aplicaciones .NET con *Clean Architecture*.

Introducción

El patrón ***Specification*** es un patrón de diseño de software que nos permite encapsular reglas de negocio en una sola unidad: *la Especificación* (*Specification*). Una vez encapsuladas, las reglas de negocio pueden ser reutilizadas en diferentes partes del código y además pueden ser modificadas fácilmente. El patrón *Specification* se utiliza con frecuencia en el contexto del diseño orientado al dominio.

En el dominio, una *Especificación* (*Specification*) recibe una entidad como parámetro y nos dice si satisface o no las reglas de negocio que encapsula. Las reglas de negocio no son más que una serie de condiciones que ciertas entidades deben cumplir.

El patrón ***Specification*** es explicado detalladamente en el documento "***Specifications***" elaborado por *Eric Evans* y *Martin Fowler*.

Specifications
https://martinfowler.com/apsupp/spec.pdf

Implementar el patrón Specification

En la aplicación que estamos desarrollando hemos establecido una condición para que una orden de compra sea considerada como especial: ***la orden de compra debe contener más de 3 productos distintos***.

Si en distintas partes de la aplicación tuviéramos que utilizar está lógica para verificar si una orden de compra es considerada como especial, cuando la condición cambiara tendríamos que realizar modificaciones en varias partes del código de la aplicación. Nos convendría centralizar está lógica en un solo lugar: *una especificación*.

Modifiquemos el código de la aplicación para implementar el patrón *Specification*.

Agregar la abstracción

1. Agrega un nuevo directorio llamado ***Abstractions*** en la raíz del proyecto *NorthWind.Entities*.
2. Agrega un nuevo archivo de clase llamado ***Specification.cs*** en el directorio *Abstractions* y remplaza el código de la clase por el siguiente.

```
public abstract class Specification<T>
{
    public abstract Expression<Func<T, bool>> ConditionExpression { get; }
    public bool IsSatisfiedBy(T entityInstance)
    {
        Func<T, bool> ExpressionDelegate = ConditionExpression.Compile();
        return ExpressionDelegate(entityInstance);
    }
}
```

   El propósito de esta clase es el de proporcionar el código base para verificar que una entidad cumple con una condición. La clase es genérica para validar cualquier tipo de entidad.

   La clase expone la propiedad ***ConditionExpression*** de solo lectura que deberá ser implementada por las clases que la hereden. Las clases que la hereden podrán establecer la lógica de negocios a través de esta propiedad.

   La clase también expone el método *IsSatisfiedBy* que recibe como parámetro la instancia de la entidad que se desea validar. El código del método obtiene el delegado compilando la expresión y finalmente devuelve el resultado de la ejecución del delegado.
3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear la especificación

1. En la raíz del proyecto *NorthWind.Sales.Backend.BusinessObjects*, agrega un nuevo directorio llamado ***Specifications***.
2. Agrega un nuevo archivo de clase llamado ***SpecialOrderSpecification.cs*** en el directorio *Specifications* y remplaza el código de la clase por el siguiente.

```
public class SpecialOrderSpecification : Specification<OrderAggregate>
{
    public override Expression<Func<OrderAggregate, bool>>
        ConditionExpression =>
            order => order.OrderDetails.Count > 3;
}
```

   Puedes notar que la clase únicamente establece el valor de la propiedad *ConditionExpression* con la lógica de negocios para determinar si una orden es especial o no.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Utilizar la especificación

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Localiza el siguiente código en el método *Handle*.

```
if (Order.OrderDetails.Count > 3)
{
    await domainEventHub.Raise(
        new SpecialOrderCreatedEvent(
            Order.Id, Order.OrderDetails.Count));
}
```

3. Modifica el código anterior para que sea similar al siguiente.

```
if (new SpecialOrderSpecification().IsSatisfiedBy(Order))
{
    await domainEventHub.Raise(
        new SpecialOrderCreatedEvent(
            Order.Id, Order.OrderDetails.Count));
}
```

   Puedes notar que ahora estamos utilizando la lógica de negocios de verificación encapsulada en la especificación. Si la lógica cambia, simplemente podremos modificar la especificación.

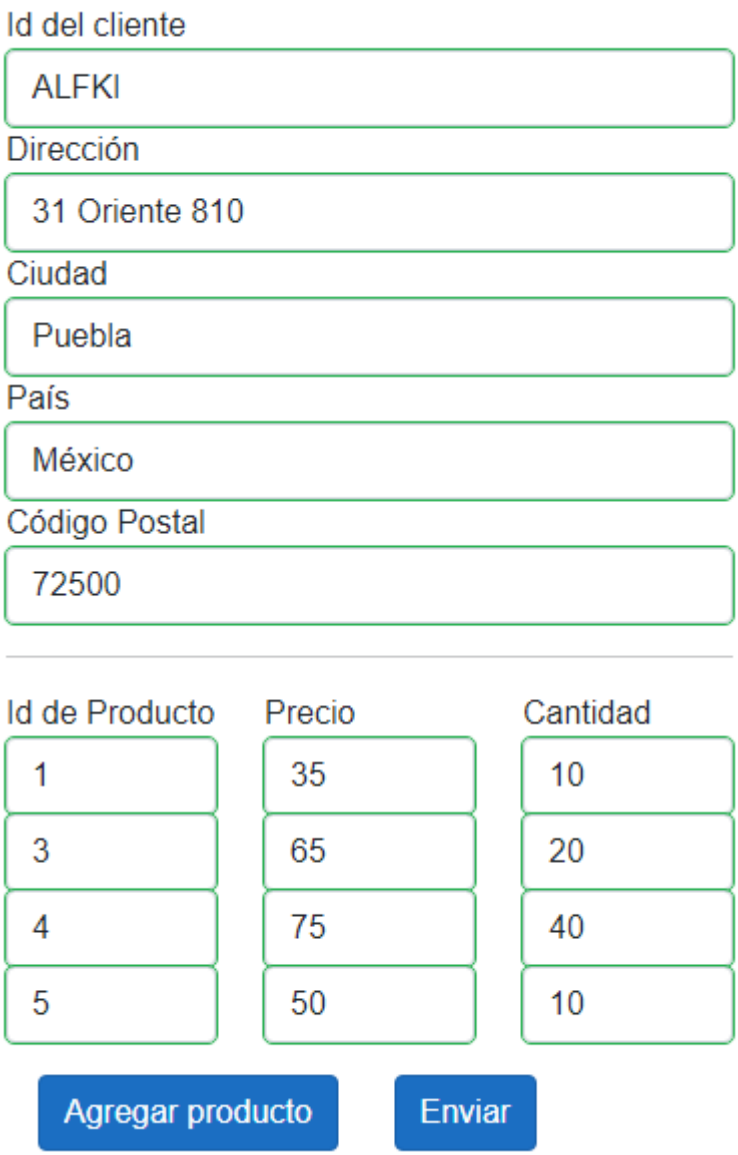
4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Probar la funcionalidad

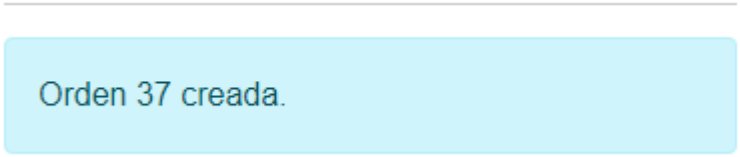
1. Ejecuta la aplicación

Crear una orden con más de 3 productos

1. Utiliza los siguientes datos para crear una orden con más de 3 productos.



2. Podrás ver un resultado mostrando el identificador de la orden similar al siguiente.



Debido a que la orden tiene más de 3 productos, debió haberse enviado un correo. Si utilizas el servicio *Mailtrap* podrás ver un correo similar al siguiente.

3. Regresa a Visual Studio y detén la ejecución.

La aplicación funciona como era esperado implementado ahora el patrón *Specification* para centralizar la lógica de negocios de verificación de una orden especial. Si la condición cambiara, solo tendríamos que modificar el archivo *SpecialOrderSpecification.cs* y todas las líneas de código que utilicen la misma lógica de negocios no tendrían que ser modificadas.

Práctica 13:

Implementar requerimientos de Autenticación y autorización

Para ejemplificar la implementación de autenticación y autorización en aplicaciones .NET con *Clean Architecture*, en esta práctica agregaremos nuevos requerimientos al sistema que estamos desarrollando para especificar que solo usuarios autenticados estarán autorizados para poder crear órdenes de compra.

Objetivos

Al finalizar esta práctica los estudiantes:

- Habrán implementado lógica de negocio para que únicamente usuarios autorizados puedan crear órdenes de compra.

Agregar nuevos requerimientos al caso de uso

Modifiquemos nuestro caso de uso para poder ejemplificar la implementación de autenticación y autorización. Los nuevos requerimientos serán los siguientes:

- Solo usuarios autenticados podrán crear órdenes.
- Al crear una orden se deberá registrar el usuario que la crea.
- Si el usuario no está autenticado se deberá cancelar el proceso de creación de la orden.

Historia de Usuario: Crear orden de compra

"Como usuario del sistema, deseo poder crear una orden de compra para solicitar productos del almacén".

Datos de entrada

- Identificador del cliente
- Dirección de envío
- Ciudad de envío
- País de envío
- Código postal de envío
- Lista de productos incluyendo:
    - Identificador del producto
    - Precio
    - Cantidad

Flujo primario

1. El usuario envía la solicitud "*Crear orden de compra*" con los datos de entrada.
2. ***El sistema verifica que el usuario esté autenticado.***
3. El sistema valida los datos.
4. El sistema registra la acción "*Inicio de creación de orden de compra*" con fines de auditoría.
5. El sistema registra la orden de compra.
6. El sistema registra la acción "*Orden de compra <Número de Orden> creada.*" con fines de auditoría.

7. Cuando el número de productos de la orden sea mayor de 3, el sistema enviará un correo electrónico de notificación de "Orden especial creada" al administrador de la empresa.
8. El sistema confirma al usuario que su solicitud ha sido procesada notificándole el número de la orden creada.

Flujo alterno: Error de validación

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Flujo alterno: Error al crear la orden

1. El proceso de la solicitud es cancelado.
2. El sistema registra la acción "*Creación de orden cancelada*" indicando de ser posible, el identificador de la orden que haya sido cancelada.
3. El sistema muestra el error al usuario.

Flujo alterno: El usuario no está autenticado

1. El procesamiento de la solicitud es cancelado.
2. El sistema muestra el error al usuario.

Consideraciones

- NorthWind maneja 4 tipos de transporte de mercancías: *Marítimo*, *Aéreo*, *Ferroviario* y *Terrestre*. El tipo de transporte predeterminado es *Terrestre*.
- NorthWind maneja 2 formas para especificar descuentos: Mediante porcentaje y mediante cantidades absolutas. El descuento predeterminado de una compra es del 10%.
- La fecha de la orden de compra corresponderá a la fecha y hora en la que sea creada.
- Si en la orden se especifican productos con el mismo identificador de producto, solo se agregará un producto con ese identificador y la cantidad registrada será la suma de las cantidades de los productos con el mismo identificador.
- **Solo usuarios autenticados podrán crear órdenes.**
- **Al crear una orden se deberá registrar el usuario que la crea.**
- **Si el usuario no está autenticado se deberá cancelar el proceso de creación de la orden.**

Validaciones

- El identificador del cliente es requerido y debe ser de 5 caracteres alfanuméricos.
- La dirección de envío es requerida y debe ser de una longitud máxima de 60 caracteres alfanuméricos.
- La ciudad de envío es requerida y debe tener una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El país de envío es requerido y debe ser de una longitud mínima de 3 y máxima de 15 caracteres alfanuméricos.
- El código Postal es opcional con una longitud máxima de 10 caracteres alfanuméricos.
- Debe especificarse al menos un producto en la orden.
- De cada producto especificado en la orden, será requerido el identificador del producto, la cantidad y el precio.
- El identificador del cliente debe existir en la base de datos.
- El cliente no debe tener adeudos para que se pueda crear la orden de compra.
- El identificador de producto debe existir en la base de datos.
- La cantidad en existencia del producto solicitado debe ser mayor o igual a la cantidad solicitada del producto.

Los cambios del caso de uso se encuentran en:

- El paso 2 del flujo primario.
- El flujo alterno "***El usuario no está autenticado***".
- Las 3 últimas consideraciones.

Definir la abstracción

Empecemos por definir la abstracción que nos permitirá obtener información del usuario, para nuestro ejemplo, solo requeriremos el estado de autenticación, el nombre de la cuenta de usuario y su nombre completo. En otros escenarios podríamos requerir información adicional como los roles asignados o alguna otra información importante.

Antes de iniciar, si lo deseas, puedes realizar un respaldo de la solución actual para que conserves el estado de los cambios.

1. Agrega un nuevo archivo de tipo *Interface* llamado ***IUserService.cs*** en el directorio *Interfaces* del proyecto *NorthWind.Entities* y remplaza el código de la interface por el siguiente.

```
public interface IUserService
{
    bool IsAuthenticated { get; }
    string UserName { get; }
    string FullName { get; }
}
```

2. Agrega un nuevo directorio llamado ***Guards*** en la raíz del proyecto *NorthWind.Entities*.
3. Agrega un nuevo archivo de clase llamado ***GuardUser.cs*** en el directorio *Guards* y remplaza el código de la clase por el siguiente.

```
public static class GuardUser
{
    public static void AgainstUnauthenticated(IUserService userService)
    {
        if (!userService.IsAuthenticated)
            throw new UnauthorizedAccessException();
    }
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Modificar la entidad DomainLog

Modifiquemos ahora la clase *DomainLog.cs* para incluir el nombre del usuario.

1. Abre el archivo *DomainLog.cs* del proyecto *NorthWind.DomainLogs.Entities*.
2. Modifica el código de la clase para agregar la propiedad ***UserName***.

```
public class DomainLog(string information, string userName)
{
    public DateTime DateTime => DateTime.Now;
    public string Information => information;
    public string UserName => userName;
}
```

Modificar el Interactor del caso de uso

Modifiquemos ahora el código del Interactor del caso de uso para implementar el nuevo requerimiento.

1. Abre el archivo *CreateOrderInteractor.cs* del proyecto *NorthWind.Sales.Backend.UseCases*.
2. Modifica el constructor primario de la clase para solicitar la dependencia *IUserService*.

```
internal class CreateOrderInteractor(
    ICreateOrderOutputPort outputPort,
    ICommandsRepository repository,
    IModelValidatorHub<CreateOrderDto> modelValidatorHub,
    IDomainEventHub<SpecialOrderCreatedEvent> domainEventHub,
    IDomainLogger domainLogger,
    IDomainTransaction domainTransaction,
    IUserService userService) :
    ICreateOrderInputPort
```

3. Agrega el siguiente código al inicio del método *Handle* para verificar si el usuario se encuentra autenticado.

```
GuardUser.AgainstUnauthenticated(userService);
```

Con el código anterior al inicio del método *Handle*, garantizamos que cuando un usuario no se encuentre autenticado, el flujo del proceso sea cancelado lanzando una excepción indicando un acceso no autorizado.

4. Localiza el siguiente código dentro del método *Handle*.

```
await domainLogger.LogInformation(
    new DomainLog(CreateOrderMessages.StartingPurchaseOrderCreation));
```

5. Remplaza el código anterior por el siguiente para poder registrar el usuario que está creando la orden de compra.

```
await domainLogger.LogInformation(
    new DomainLog(CreateOrderMessages.StartingPurchaseOrderCreation,
    userService.UserName));
```

6. Localiza el siguiente código dentro del método *Handle*.

```
await domainLogger.LogInformation(
    new DomainLog(string.Format(
```

```
        CreateOrderMessages.PurchaseOrderCreatedTemplate,
        Order.Id)));
```

7. Remplaza el código anterior por el siguiente para poder registrar el usuario que está creando la orden de compra.

```
await domainLogger.LogInformation(
    new DomainLog(string.Format(
        CreateOrderMessages.PurchaseOrderCreatedTemplate,
        Order.Id), userService.UserName));
```

8. Localiza el siguiente código dentro del bloque *catch* del método *Handle*.

```
await domainLogger.LogInformation(new DomainLog(Information));
```

9. Remplaza el código anterior por el siguiente para poder registrar el usuario que está creando la orden de compra.

```
await domainLogger.LogInformation(
    new DomainLog(Information, userService.UserName));
```

10. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

En este momento, el código del Interactor ya implementa el nuevo requerimiento solicitado.

Implementar la interface IUserService

1. Utiliza la plantilla ***Class Library*** para agregar a la solución un nuevo proyecto llamado ***NorthWind.UserServices*** en el directorio *Libraries* del directorio `4. Frameworks And Drivers`.
2. Elimina el archivo ***Class1.cs***.
3. Agrega el siguiente código en el archivo de proyecto.

```
<ItemGroup>
      <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

4. Agrega una referencia del proyecto *NorthWind.Entities*.
5. Agrega un nuevo archivo de clase llamado ***UserService.cs*** en la raíz del proyecto y modifica el código de la clase para que sea similar al siguiente.

```
internal class UserService(IHttpContextAccessor contextAccesor) : IUserService
{
    public bool IsAuthenticated =>
        contextAccesor.HttpContext.User.Identity.IsAuthenticated;

    public string UserName =>
        contextAccesor.HttpContext.User.Identity.Name;

    public string FullName =>
        contextAccesor.HttpContext.User.Claims
        .Where(c => c.Type == "FullName")
        .Select(c => c.Value).FirstOrDefault();
}
```

En el código anterior, ***IHttpContextAccessor*** proporciona el acceso al contexto actual HTTP (*HttpContext*).

El objeto ***HttpContext*** encapsula toda la información específica HTTP de una petición HTTP individual.

El objeto ***User*** de tipo ***ClaimsPrincipal***, almacena la información del usuario de la petición actual.

El objeto *ClaimsPrincipal* soporta múltiples identidades basadas en *Claims*.

La propiedad *Identity* del objeto *User* representa la identidad primaria del usuario autenticado.

La propiedad ***IsAuthenticated*** contiene un valor booleano que indica si el usuario actual de la petición está autenticado o no.

La propiedad ***Name*** del objeto *Identity* representa el nombre de usuario cuando este está autenticado.

Obtenemos el nombre completo del usuario a través del *claim* de tipo "*FullName*" que deberá venir como parte de la colección de *Claims* del usuario. La información del usuario que incluye este tipo de *Claim*, deberá ser generada durante el proceso de inicio de sesión.

6. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Para obtener información adicional sobre cómo obtener acceso a *HttpContext* en ASP.NET Core, puede consultarse el siguiente enlace:

Access HttpContext in ASP.NET Core
https://learn.microsoft.com/aspnet/core/fundamentals/http-context

Registrar el servicio

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el código de la clase por el siguiente.

```
public static class DependencyContainer
{
    public static IServiceCollection AddUserServices(
        this IServiceCollection services)
    {
        services.AddHttpContextAccessor();
        services.AddSingleton<IUserService, UserService>();

        return services;
    }
}
```

El código anterior registra las implementaciones predeterminadas de los servicios *IHttpContextAccessor* e *IUserService* respectivamente.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
3. En el proyecto *NorthWind.Sales.Backend.IoC*, agrega una referencia del proyecto *NorthWind.UserServices*.
4. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC.*
5. Modifica el código del método *AddNorthWindSalesServices* para registrar el nuevo servicio.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions)
{
```

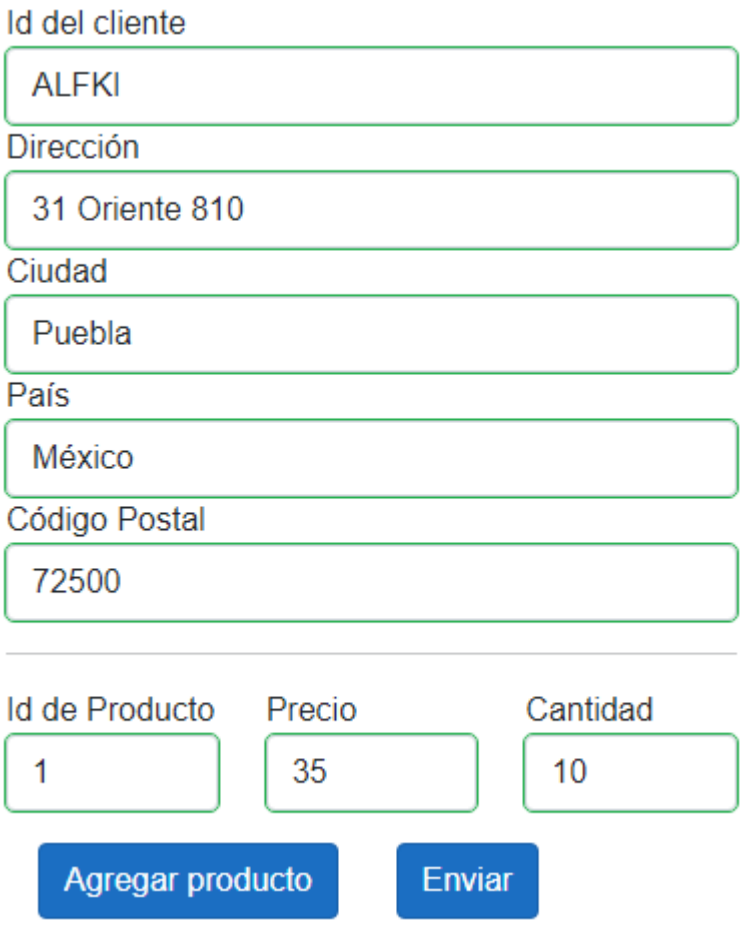
```
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices()
        .AddTransactionServices()
        .AddUserServices();

    return services;
}
```

Probar la funcionalidad

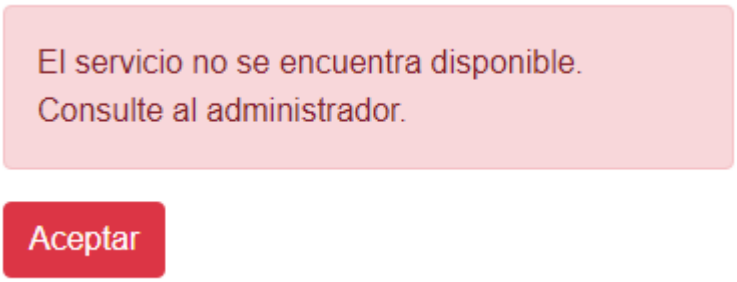
Hemos implementado la lógica de negocios que da solución al requerimiento de verificación de un usuario autenticado. En este momento, solo los usuarios autenticados podrán crear órdenes.

Aunque *IUserService* nos proporciona información sobre el usuario autenticado, aún no tenemos el código que establece la identidad del usuario, por lo tanto, *IUserService* siempre devolverá *false* cuando solicitemos el valor de la propiedad *IsAuthenticated*. Probemos este escenario.

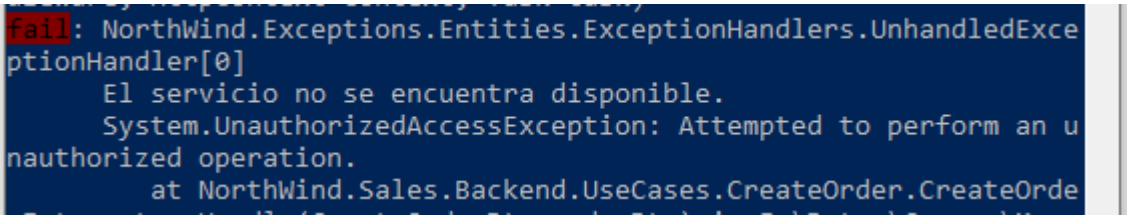
1. Ejecuta la aplicación.
2. Intenta crear una nueva orden utilizando los siguientes datos.



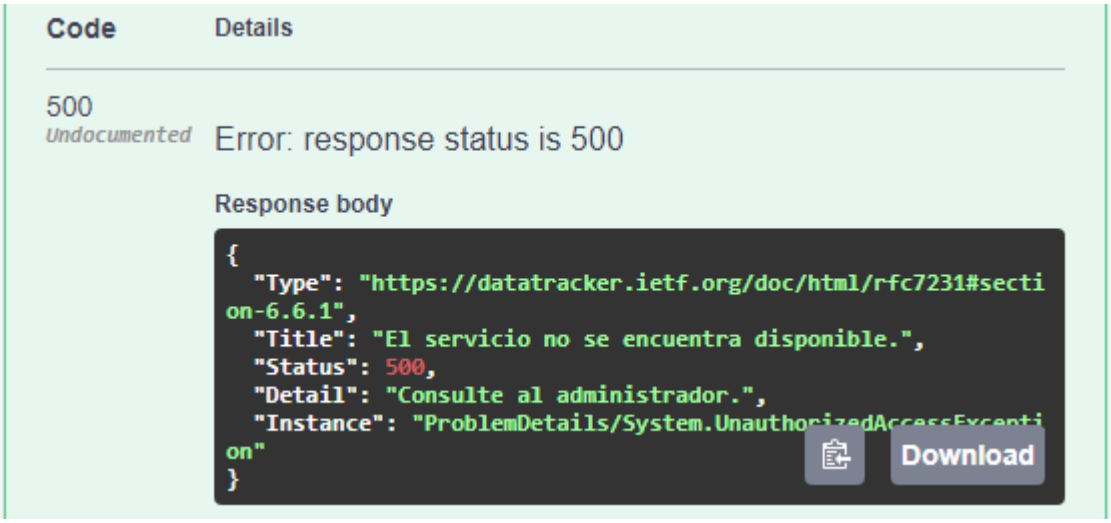
3. Podrás ver un resultado similar al siguiente.



4. La *Consola* mostrará un mensaje similar al siguiente indicando que se ha intentado realizar una operación no autorizada.



Si intentas crear una orden desde la página */swagger/index.html* de la Web Api, podrás ver un mensaje similar al siguiente.



5. Regresa a Visual Studio y detén la ejecución.

Agregar un manejador de excepción

Agreguemos un manejador para la excepción de acceso no autorizado que devuelva un código de estado ***HTTP 401 Unauthorized*** en lugar de devolver el valor *500*.

Agregar mensajes en el archivo de recursos

1. Agrega la siguiente información en el archivo ***ExceptionMessages.resx*** del directorio *Resources* del proyecto *NorthWind.Exceptions.Entities*.

Nombre	Valor
UnauthorizedAccessExceptionTitle	Acceso no autorizado.
UnauthorizedAccessExceptionDetail	El acceso al recurso solicitado está restringido a usuarios autorizados.

Agregar el manejador de la excepción

1. Agrega un nuevo archivo de clase llamado *UnauthorizedAccessExceptionHandler.cs* en el directorio *ExceptionHandlers* del proyecto *NorthWind.Exceptions.Entities* y remplaza el código de la clase por el siguiente.

```
internal class UnauthorizedAccessExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext httpContext,
    Exception exception, CancellationToken cancellationToken)
    {
        bool Handled = false;
        if (exception is UnauthorizedAccessException Ex)
        {
            ProblemDetails Details = new ProblemDetails();
            Details.Status = StatusCodes.Status401Unauthorized;
            Details.Type =
                "https://datatracker.ietf.org/doc/html/rfc7235#section-3.1";
            Details.Title = ExceptionMessages.UnauthorizedAccessExceptionTitle;
            Details.Detail = ExceptionMessages.UnauthorizedAccessExceptionDetail;
            Details.Instance =

$"{nameof(ProblemDetails)}/{nameof(UnauthorizedAccessException)}";

            await httpContext.WriteProblemDetailsAsync(Details);
            Handled = true;
        }

        return Handled;
    }
}
```

Registrar el manejador de Excepción

1. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Exceptions.Entities*.
2. Agrega el siguiente código a la clase.

```
public static IServiceCollection AddUnauthorizedAccessExceptionHandler(
    this IServiceCollection services)
{
    services.AddExceptionHandler<UnauthorizedAccessExceptionHandler>();

    return services;
}
```

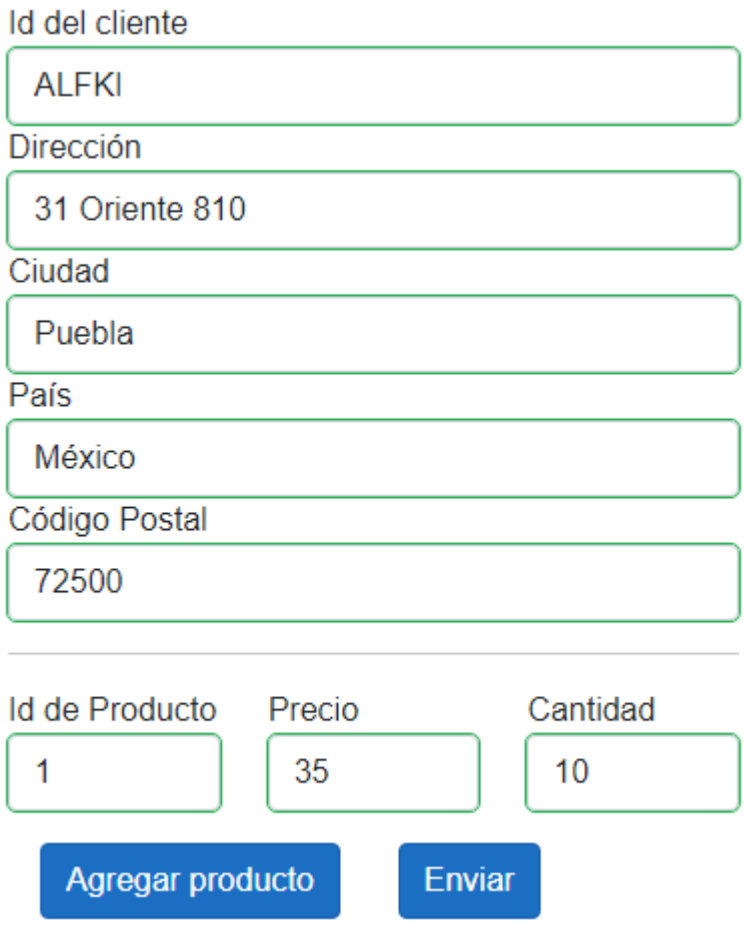
3. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
4. Agrega el siguiente código al método *AddNorthWindSalesServices* para registrar el nuevo manejador de excepciones.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnauthorizedAccessExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices()
        .AddTransactionServices()
        .AddUserServices();

    return services;
}
```

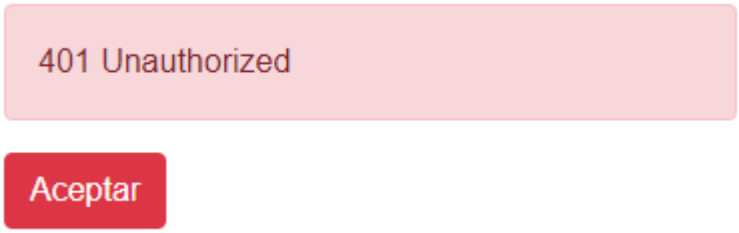
NOTA: recuerda que el registro de los manejadores de excepción debe realizarse antes del registro `AddUnhandledExceptionHandler()`.

Probar la funcionalidad

1. Ejecuta la aplicación.
2. Intenta crear una nueva orden utilizando los siguientes datos.



3. Podrás ver un resultado similar al siguiente.



4. Regresa a Visual Studio y detén la ejecución.

Actualizar el repositorio

Modifiquemos ahora el repositorio para incluir el nombre del usuario al registrar una actividad en la base de datos.

Modificar la entidad DomainLog

1. Abre el archivo ***DomainLog.cs*** que se encuentra en el directorio *Entities* del proyecto *NorthWind.Sales.Backend.Repositories*.
2. Agrega el siguiente código al final de la clase *DomainLog* para definir una nueva propiedad que almacene el nombre del usuario.

```
public string UserName { get; set; }
```

Modificar el repositorio

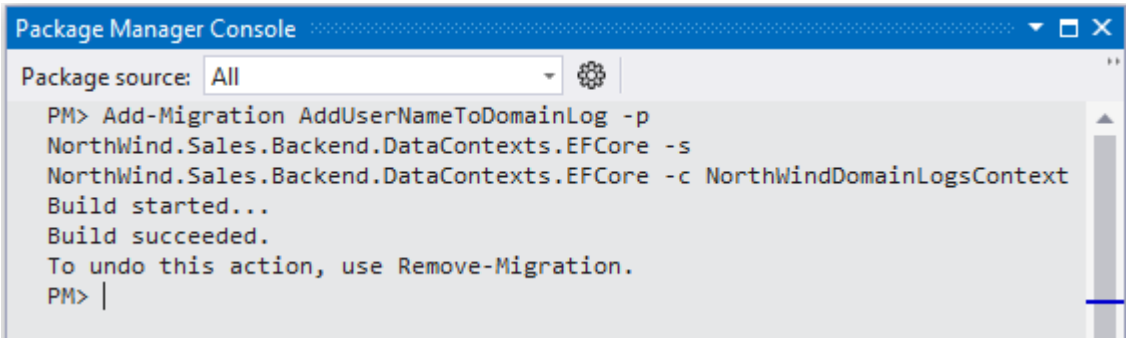
1. Abre el archivo *DomainLogsRepository.cs* que se encuentra en el directorio *Repositories* del proyecto *NorthWind.Sales.Backend.Repositories*.
2. Modifica el código del método *Add* para establecer el valor de la propiedad *UserName*.

```
public async Task Add(DomainLogs.Entities.ValueObjects.DomainLog log)
{
    await context.AddLogAsync(new Entities.DomainLog
    {
        CreatedDate = log.DateTime,
        Information = log.Information,
        UserName = log.UserName
    });
}
```

Crear la migración

1. Ejecuta el siguiente comando en la consola de administración de paquetes NuGet para crear la nueva migración.

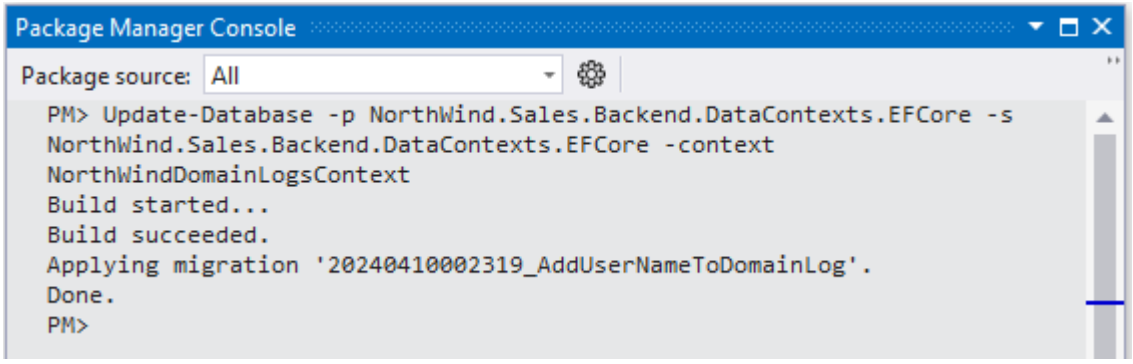
```
Add-Migration AddUserNameToDomainLog -p
NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -c NorthWindDomainLogsContext
```



Actualizar el esquema de la base de datos

1. Ejecuta el siguiente comando para crear la nueva base de datos y verifica que no haya errores.

```
Update-Database -p NorthWind.Sales.Backend.DataContexts.EFCore -s
NorthWind.Sales.Backend.DataContexts.EFCore -context NorthWindDomainLogsContext
```



Probar la funcionalidad

Probemos ahora la funcionalidad simulando un caso donde el usuario se encuentra autenticado.

Implementar un servicio de usuarios falso

1. Agrega un nuevo archivo de clase llamado ***UserServiceFake.cs*** en la raíz del proyecto *NorthWind.UserServices* y modifica el código de la clase para que sea similar al siguiente.

```
internal class UserServiceFake : IUserService
{
    public bool IsAuthenticated => true;
    public string UserName => "user@northwind.com";
    public string FullName => "Usuario de prueba";
}
```

   Esta implementación nos permitirá simular un usuario autenticado.

Registrar el servicio

1. Abre el archivo *DependencyContainer.cs* del mismo proyecto.
2. Localiza el siguiente código en el método *AddBusinessObjectsServices* .

```
services.AddSingleton<IUserService, UserService>();
```

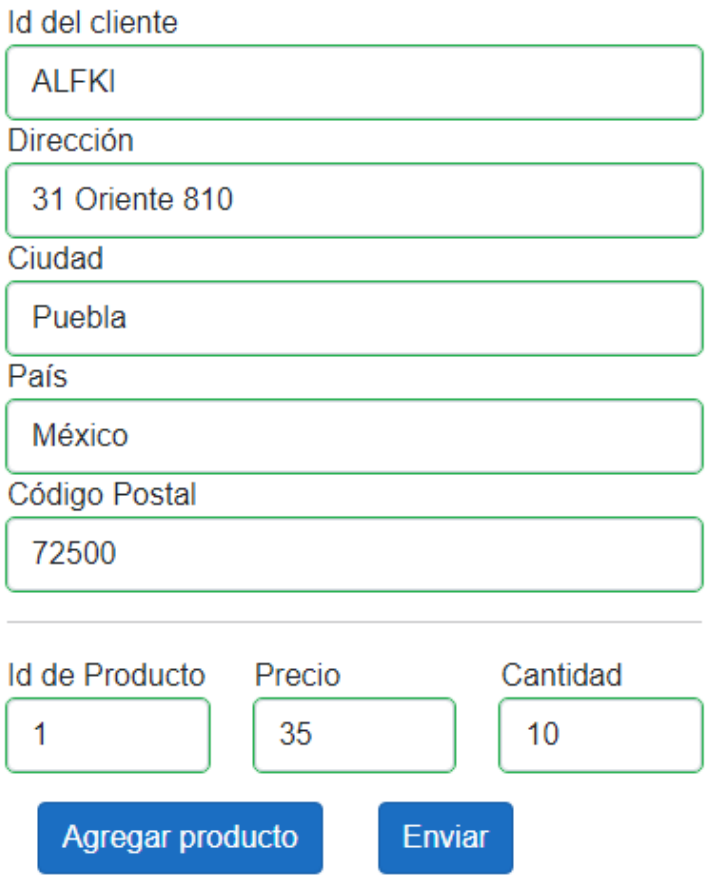
3. Modifica el código anterior para que sea similar al siguiente.

```
services.AddSingleton<IUserService, UserServiceFake>();
```

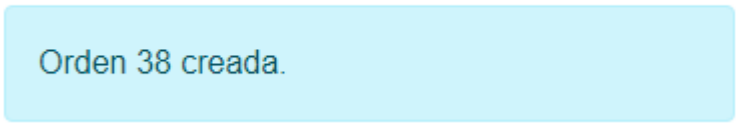
   Puedes notar que estamos registrando el servicio falso de pruebas.

Probar la funcionalidad con el servicio falso

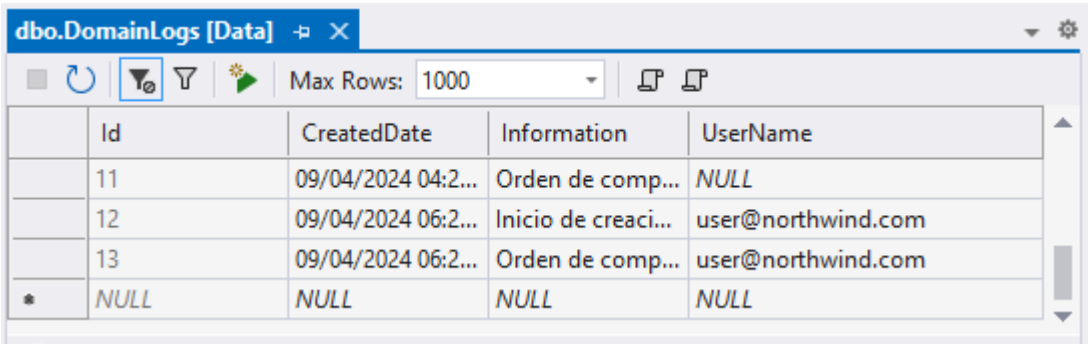
1. Ejecuta la aplicación.
2. Intenta crear una nueva orden utilizando los siguientes datos.



3. Podrás ver un resultado con el identificador de la orden de compra similar al siguiente.



La tabla ***DomainLogs*** de la base de datos deberá mostrar los nuevos registros creados que ahora incluyen el nombre del usuario que ha creado la orden.



Id	CreatedDate	Information	UserName
11	09/04/2024 04:2...	Orden de comp...	NULL
12	09/04/2024 06:2...	Inicio de creaci...	user@northwind.com
13	09/04/2024 06:2...	Orden de comp...	user@northwind.com
NULL	NULL	NULL	NULL

4. Regresa a Visual Studio y detén la ejecución.

Práctica 14:

Implementar el registro de usuarios

Para que un usuario pueda crear órdenes de compra será necesario permitir la creación de cuentas de usuarios para que estos puedan posteriormente autenticarse y ser autorizados.

Para ofrecer la funcionalidad de administración de usuarios, implementaremos un framework de membresía que llamaremos *NortWind.Membership*. Este framework de membresía básico nos permitirá registrar y autenticar usuarios utilizando como herramienta la funcionalidad que ofrece *ASP.NET Core Identity* pero, con la opción de utilizar alguna otra herramienta propia o de terceros.

El sistema de membresía ofrecerá componentes de backend y componentes de frontend para aplicaciones *Blazor*.

En esta práctica implementaremos los componentes de backend y los componentes de frontend para resolver el nuevo caso de uso de registro de usuarios.

Objetivos

Al finalizar esta práctica los estudiantes:

- Agregarán funcionalidad para crear cuentas de usuario utilizando como herramienta *ASP.NET Core Identity*.
- Implementarán el enfoque *Result* para la devolución de errores de validación de datos.

Caso de uso: Registrar usuario

Agreguemos un nuevo caso de uso a la solución para implementar el registro de usuarios.

Historia de usuario: Registrar usuario

"*Como usuario del sistema, necesito registrar mis credenciales para poder crear órdenes de compra*".

Datos de entrada

- Correo
- Clave de acceso
- Nombre
- Apellidos

Flujo primario

1. El usuario envía la solicitud "*Registrar usuario*" con los datos de entrada.
2. El sistema registra los datos del usuario.
3. El sistema confirma al usuario el registro de la información.

Flujo alterno: Error al registrar la información.

1. El procesamiento de la solicitud es cancelado.
2. El sistema notifica el error al usuario.

Validaciones

- *Nombre* es un dato requerido.
- *Apellidos* es un dato requerido.
- *Correo* es un dato requerido con las siguientes características:
    - Debe tener un formato válido de correo electrónico.
    - No debe existir el correo en la fuente de datos.
- *Clave de acceso* es un dato requerido con las siguientes características:
    - Longitud mínima 6.
    - Debe incluir al menos un carácter minúscula.

- Debe incluir al menos un carácter mayúscula.
- Debe incluir al menos un dígito.
- Debe incluir al menos un símbolo no alfanumérico.

Agregar el código para manejo de errores

Para manejar errores de validación utilizaremos la técnica de la devolución de un *Resultado* en lugar de una *Excepción*, esto permitirá ejemplificar otra opción para el manejo de flujos de ejecución alternos.

1. Utiliza la plantilla ***Class Library*** para agregar a la solución un nuevo proyecto llamado ***NorthWind.Result.Entities*** en el directorio ***1. Enterprise Business Rules***.
2. Elimina el archivo *Class1.cs*.
3. Agrega un nuevo archivo de clase llamado ***ResultErrorOnly.cs*** en la raíz del proyecto y remplaza el contenido de la clase por el siguiente.

```
public class Result<ErrorResultType>
{
}
```

   Esta clase permitirá devolver el error de una operación cuando solo se desee verificar si una operación se completó exitosamente o se generó un error.

4. Agrega el siguiente código a la clase para almacenar el valor del error generado.

```
public ErrorResultType ErrorValue { get; private set; }
```

5. Agrega el siguiente código a la clase para indicar si hay o no algún error.

```
public bool HasError { get; private set; }
```

6. Agrega el siguiente código que deberá ser utilizado para construir una instancia de ***Result*** indicando que no hay algún error.

```
public Result()
{
    HasError = false;
}
```

7. Agrega el siguiente código que deberá ser utilizado para construir una instancia de ***Result*** indicando que hay un error.

```
public Result(ErrorResultType errorValue)
{
    HasError = true;
    ErrorValue = errorValue;
```

```
}
```

8. Agrega el siguiente código que permitirá ejecutar alguna acción cuando haya o no haya algún error.

```
public void HandleResult(
    Action<ErrorResultType> whenHasErrorAction,
    Action whenIsSuccessAction = null)
{
    if (HasError)
        whenHasErrorAction(ErrorValue);
    else
        whenIsSuccessAction?.Invoke();
}
```

9. Agrega un nuevo archivo de clase llamado ***ResultSuccessAndError.cs*** en la raíz del proyecto y remplaza el contenido de la clase por el siguiente.

```
public class Result<SuccessResultType, ErrorResultType> :
    Result<ErrorResultType>
{
}
```

Esta clase permitirá devolver el resultado exitoso de una operación o el error de esta.

10. Agrega el siguiente código a la clase para almacenar el resultado satisfactorio de una operación.

```
public SuccessResultType SuccessValue { get; private set; }
```

11. Agrega el siguiente código a la clase para crear una instancia de ***Result*** y establecer el valor del resultado exitoso de una operación.

```
public Result(SuccessResultType successValue) : base()
{
    SuccessValue = successValue;
}
```

Al invocar al constructor predeterminado de la clase base, se establece la propiedad ***HasError*** como ***false***, esto indicará que no hay un error.

12. Agrega el siguiente código para crear una instancia de ***Result*** y establecer el valor del error como resultado de una operación no exitosa.

```
public Result(ErrorResultType errorValue) : base(errorValue)
{
}
```

El código anterior simplemente pasa el error a la clase base.

13. Agrega el siguiente código que permitirá ejecutar alguna acción cuando haya o no haya algún error.

```
public void HandleResult(
    Action<SuccessResultType> whenIsSuccessAction,
    Action<ErrorResultType> whenHasErrorAction)
{
    if (HasError)
        whenHasErrorAction(ErrorValue);
    else
        whenIsSuccessAction(SuccessValue);
}
```

14. Compila el proyecto y verifica que no haya errores.

Utilizaremos este código durante el proceso de creación y autenticación de usuarios para manejar las posibles condiciones de error.

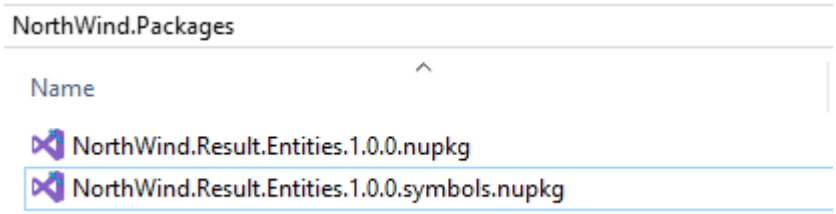
Crear paquetes NuGet para compartir código

Para implementar el nuevo framework de membresía necesitaremos parte del código que hemos definido en la capa `1. Enterprise Business Rules` de la solución *NorthWind*. Compartiremos este código creando paquetes NuGet que alojaremos en un directorio local.

1. Crea un directorio llamado ***NorthWind.Packages*** en tu disco local. Si lo deseas, puedes utilizar otro nombre u otra ubicación que sea accesible desde tu equipo de desarrollo.
2. Abre una ventana de línea de comandos.
3. Desplázate al directorio raíz del proyecto ***NorthWind.Result.Entities***.
4. Ejecuta el siguiente comando para crear el paquete NuGet y el paquete de símbolos con código fuente para facilitar el proceso de depuración. Reemplaza `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

5. Verifica que los paquetes hayan sido creado.



Para obtener información completa y recomendaciones sobre la creación de paquetes NuGet pueden consultarse los siguientes enlaces:

Create a NuGet package with the dotnet CLI
https://learn.microsoft.com/nuget/create-packages/creating-a-package-dotnet-cli

Quickstart: Create and publish a package with the dotnet CLI
https://learn.microsoft.com/nuget/quickstart/create-and-publish-a-package-using-the-dotnet-cli

Quickstart: Create and publish a NuGet package using Visual Studio (Windows only)
https://learn.microsoft.com/nuget/quickstart/create-and-publish-a-package-using-visual-studio

Package versioning
https://learn.microsoft.com/nuget/concepts/package-versioning

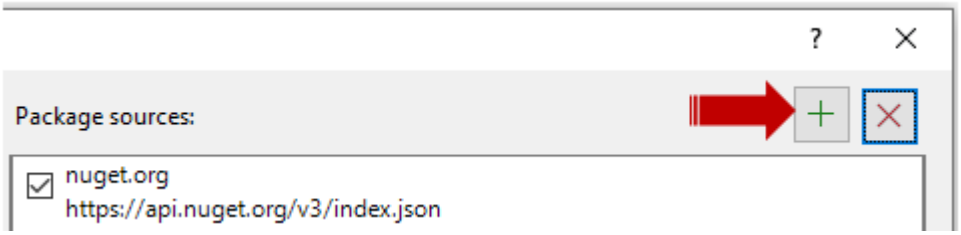
6. Repite el proceso anterior para crear los paquetes NuGet de los siguientes proyectos:

    a. NorthWind.Validation.Entities.
    b. NorthWind.RazorComponents.
    c. NorthWind.HttpDelegatingHandlers.

Agregar una fuente de paquetes NuGet en Visual Studio

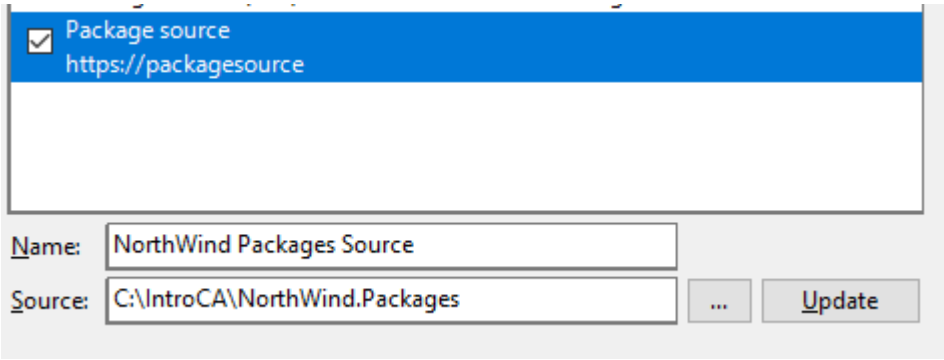
De manera predeterminada, los paquetes NuGet se instalan desde el origen `nuget.org`. Para poder utilizar los paquetes NuGet que hemos creado será necesario establecer un nuevo origen.

1. Abre Visual Studio y selecciona la opción ***Tools > NuGet Package Manager > Package Manager Settings*** del menú de Visual Studio. Se mostrará la venta ***Options***.
2. En la ventana ***Options*** selecciona ***Package Sources***. Se mostrarán los orígenes actuales.
3. Haz clic en el botón ***+*** ubicado en el extremo superior derecho de la ventana *Options*.



Se agregará un nuevo origen ***Package Source***.

4. Escribe ***NorthWind Packages Source*** en el cuadro de texto ***Name***.
5. Establece el directorio donde se ubican los paquetes *NuGet* en el cuadro de texto ***Source***. La información será similar a la siguiente.



6. Haz clic en el botón ***Update***.
7. Haz clic en ***OK*** para finalizar.

Implementar la funcionalidad backend del framework de membresía

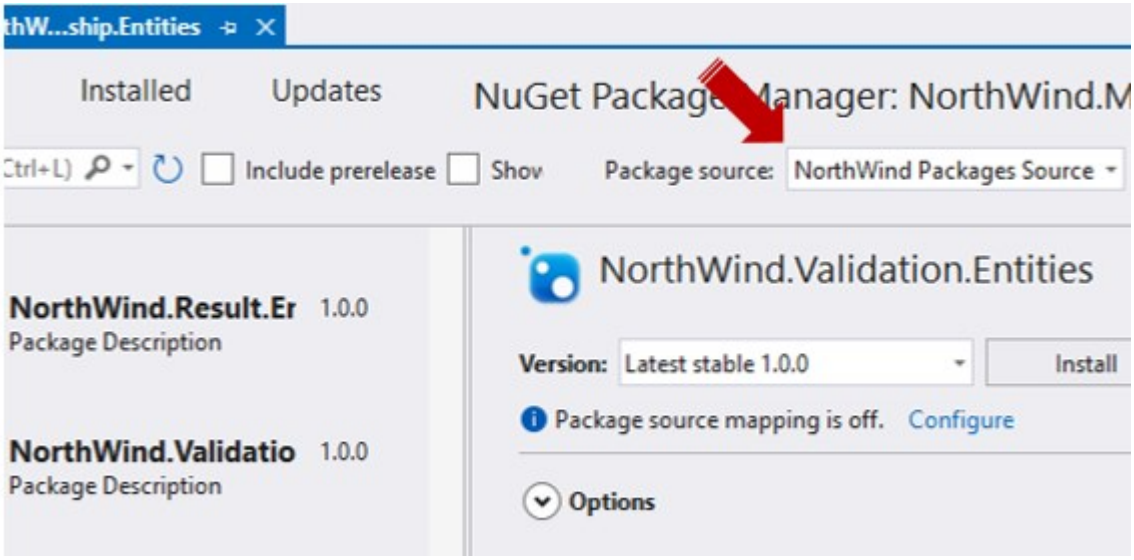
El framework de membresía que implementaremos contará con código backend y código frontend. El código backend expondrá los endpoints para crear y autenticar usuarios mientras que el código frontend expondrá el código y las vistas para crear y autenticar usuarios.

Con el fin de ejemplificar otra forma de estructurar el código en una aplicación implementando *Clean Architecture*, implementaremos el framework de membresía en una solución independiente.

Agregar entidades empresariales

Empecemos por crear las entidades que serán compartidas por el código backend y el código frontend.

1. Utiliza la plantilla ***Class Library*** para crear un nuevo proyecto llamado `NorthWind.Membership.Entities` en una solución llamada `NorthWind.Membership`.
2. Elimina el archivo ***Class1.cs***.
3. Agrega el paquete NuGet ***NorthWind.Validation.Entities*** al proyecto. Recuerda indicar el origen en la lista desplegable de orígenes.



Agregar el DTO para registro de usuario

1. Agrega un nuevo directorio llamado ***Dtos*** en la raíz del nuevo proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *Dtos*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationDto.cs*** en el directorio *UserRegistration* y remplaza el código de la clase *UserRegistrationDto* por el siguiente.

```
public class UserRegistrationDto(string email, string password,
    string firstName, string lastName)
{
    public string Email => email;
    public string Password => password;
    public string FirstName => firstName;
    public string LastName => lastName;
}
```

Agregar un archivo de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserRegistrationMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
RequiredFirstNameErrorMessage	El nombre es requerido.
RequiredLastNameErrorMessage	El apellido es requerido.
RequiredEmailErrorMessage	El correo es requerido.
InvalidEmailErrorMessage	El formato de correo no es válido.
RequiredPasswordErrorMessage	La contraseña es requerida.
PasswordTooShortErrorMessage	Se requiere al menos 6 caracteres en la contraseña.
PasswordRequiresLowerErrorMessage	Se requiere al menos un caracter minúscula en la contraseña.
PasswordRequiresUpperErrorMessage	Se requiere al menos un caracter mayúscula en la contraseña.
PasswordRequiresDigitErrorMessage	Se requiere al menos un dígito en la contraseña.
PasswordRequiresNonAlphanumericErrorMessage	Se requiere al menos un caracter no alfanumérico en la contraseña.

Implementar la validación de los datos de registro de usuario

1. Agrega un nuevo directorio llamado ***Validators*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *Validators*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationDtoValidator.cs*** en el directorio *UserRegistration* y remplaza el contenido de la clase por el siguiente.

```
internal class UserRegistrationDtoValidator :
```

```
    AbstractModelValidator<UserRegistrationDto>
{
    public UserRegistrationDtoValidator(
        IValidationService<UserRegistrationDto> validationService) :
        base(validationService)
    {
        AddRuleFor(u => u.FirstName)
            .NotEmpty(UserRegistrationMessages.RequiredFirstNameErrorMessage);

        AddRuleFor(u => u.LastName)
            .NotEmpty(UserRegistrationMessages.RequiredLastNameErrorMessage);

        AddRuleFor(u => u.Email)
            .NotEmpty(UserRegistrationMessages.RequiredEmailErrorMessage)
            .EmailAddress(UserRegistrationMessages.InvalidEmailErrorMessage);

        AddRuleFor(u => u.Password)
            .StopOnFirstFailure()
            .NotEmpty(UserRegistrationMessages.RequiredPasswordErrorMessage)
            .MinimumLength(6,
                UserRegistrationMessages.PasswordTooShortErrorMessage)
            .Must(p => p.Any(c => char.IsLower(c)),
                UserRegistrationMessages.PasswordRequiresLowerErrorMessage)
            .Must(p => p.Any(c => char.IsUpper(c)),
                UserRegistrationMessages.PasswordRequiresUpperErrorMessage)
            .Must(p => p.Any(c => char.IsDigit(c)),
                UserRegistrationMessages.PasswordRequiresDigitErrorMessage)
            .Must(p => p.Any(c => !char.IsLetterOrDigit(c)),
                UserRegistrationMessages
                    .PasswordRequiresNonAlphanumericErrorMessage);
    }
}
```

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar el validador

1. Agrega el paquete NuGet ***Microsoft.Extensions.DependencyInjection.Abstractions*** al proyecto.
2. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddMembershipValidators(
        this IServiceCollection services)
    {
        services.AddModelValidator<UserRegistrationDto,
            UserRegistrationDtoValidator>();

        return services;
    }
```

```
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar la clase para definir los endpoints

1. Agrega un directorio llamado ***ValueObjects*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***Endpoints.cs*** en el directorio *ValueObjects* y remplaza el contenido de la clase por el siguiente.

```
public class Endpoints
{
    public const string Register = $"/user/{nameof(Register)}";
}
```

Crear el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Entities*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Implementar el código del backend

Implementemos ahora el código del backend necesario para implementar la funcionalidad del registro de usuarios.

1. Utiliza la plantilla ***Class Library*** para agregar a la solución un nuevo proyecto llamado ***NorthWind.Membership.Backend.Core***.
2. Elimina el archivo **Class1.cs**.
3. Agrega el paquete NuGet *NorthWind.Membership.Entities*.
4. Agrega el paquete NuGet *NorthWind.Result.Entities*.
5. Agrega el siguiente código en el archivo del proyecto.

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.App"/>
```

```
</ItemGroup>
```

Definir los objetos de negocio

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *Interfaces*.
3. Agrega un nuevo archivo de tipo *Interface* llamado ***IUserRegistrationInputPort.cs*** en el directorio *UserRegistration* y modifica el código de la interface para que sea similar al siguiente.

```
internal interface IUserRegistrationInputPort
{
    Task Handle(UserRegistrationDto userData);
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
5. Agrega un nuevo archivo de tipo *Interface* llamado ***IUserRegistrationOutputPort.cs*** en el directorio *UserRegistration* y modifica el código de la interface para que sea similar al siguiente.

```
internal interface IUserRegistrationOutputPort
{
    IResult Result { get; }
    Task Handle(Result<IEnumerable<ValidationError>> userRegistrationResult);
}
```

El *OutputPort* recibe el resultado del registro del usuario. En caso de que haya errores, deberá crear una respuesta apropiada (*IResult*) que indique los errores expuestos por el tipo *Result*.

6. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Definamos ahora una abstracción que nos permita implementar la funcionalidad requerida para la administración y almacenamiento de cuentas de usuario.

7. Agrega un nuevo directorio llamado ***Common*** en el directorio *Interfaces*.
8. Agrega un nuevo archivo de tipo *Interface* llamado ***IMembershipService.cs*** en el directorio *Common* y remplaza el código de la interface por el siguiente.

```
public interface IMembershipService
{
    Task<Result<IEnumerable<ValidationError>>> Register(
        UserRegistrationDto userData);
```

```
}
```

El método *Register* recibe los datos del nuevo usuario a registrar y devuelve una instancia del tipo *Result* encapsulando una posible lista de errores encontrados durante el registro.

Implementar el caso de uso

1. Agrega un nuevo directorio llamado ***UseCases*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *UseCases*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationInteractor.cs*** en el directorio *UserRegistration* y remplaza el código de la clase por el siguiente.

```
internal class UserRegistrationInteractor(
    IMembershipService membershipService,
    IUserRegistrationOutputPort presenter,
    IModelValidatorHub<UserRegistrationDto> validationService)
    : IUserRegistrationInputPort
{
    public async Task Handle(UserRegistrationDto userData)
    {
        Result<IEnumerable<ValidationError>> Result;

        if (!await validationService.Validate(userData))
        {
            Result = new Result<IEnumerable<ValidationError>>(
                validationService.Errors);
        }
        else
        {
            Result = await membershipService.Register(userData);
        }

        await presenter.Handle(Result);
    }
}
```

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar los servicios

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddMembershipCoreServices(
```

```
        this IServiceCollection services)
    {

        services.AddMembershipValidators();
        services.AddDefaultModelValidatorHub();

        services.AddScoped<IUserRegistrationInputPort,
            UserRegistrationInteractor>();

        return services;
    }
}
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar un archivo de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserRegistrationMessages.resx*** en el directorio *Resources* con la siguiente información.

Nombre	Valor
UserRegistrationErrorTitle	Error en los datos de entrada.
UserRegistrationErrorDetail	Se encontraron uno o más errores de validación de datos.

Agregar métodos de extensión

1. Agrega un nuevo directorio llamado ***Extensions*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***ValidationErrorsExtensions.cs*** en el directorio *Extensions* y remplaza el código de la clase por el siguiente.

```
internal static class ValidationErrorsExtensions
{
    public static ProblemDetails ToProblemDetails(
        this IEnumerable<ValidationError> errors,
        string title, string detail, string instance)
    {
        ProblemDetails Details = new ProblemDetails();
        Details.Status = StatusCodes.Status400BadRequest;
        Details.Type =
            "https://datatracker.ietf.org/doc/html/rfc7231#section-6.5.1";
        Details.Title = title;
        Details.Detail = detail;
        Details.Instance =
            $"{nameof(ProblemDetails)}/{instance}";
        Details.Extensions.Add("errors", errors);
```

```
        return Details;
    }
}
```

El código anterior devuelve una instancia de `ProblemDetails` a partir de los errores de validación.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el Presenter

1. Agrega un nuevo directorio llamado ***Presenters*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *Presenters*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationPresenter.cs*** en el directorio *UserRegistration* y remplaza el código de la clase por el siguiente.

```
internal class UserRegistrationPresenter :
    IUserRegistrationOutputPort
{
    public IResult Result { get; private set; }

    public Task Handle(
        Result<IEnumerable<ValidationError>> userRegistrationResult)
    {
        userRegistrationResult.HandleResult(
            errors =>
                Result = Results.Problem(
                    errors.ToProblemDetails(
                        UserRegistrationMessages.UserRegistrationErrorTitle,
                        UserRegistrationMessages.UserRegistrationErrorDetail,
                        nameof(UserRegistrationPresenter))),
            () =>
                Result = Results.Ok()
            );

        return Task.CompletedTask;
    }
}
```

En caso de que haya errores de validación, el código anterior almacena en ***Result*** los mensajes de error encapsulados en una instancia de *ProblemDetails*. En caso de que no haya errores de validación, se almacena el estado *HTTP 200 OK*.

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar el Presenter

1. Abre el archivo *DependencyContainer.cs*.
2. Agrega el siguiente código al método *AddMembershipCoreServices*.

```
services.AddScoped<IUserRegistrationOutputPort,
    UserRegistrationPresenter>();
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar el Controlador

1. Agrega un nuevo directorio llamado ***Controllers*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *Controllers*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationController.cs*** en el directorio *UserRegistration* y remplaza el código del archivo por el siguiente.

```
namespace Microsoft.AspNetCore.Builder;
internal static class UserRegistrationController
{
    public static WebApplication UseUserRegistrationController(
        this WebApplication app)
    {
        app.MapPost(Endpoints.Register,
             async (UserRegistrationDto userData,
                    IUserRegistrationInputPort inputPort,
                    IUserRegistrationOutputPort presenter) =>
             {
                 await inputPort.Handle(userData);

                 return presenter.Result;
             }
            );

        return app;
    }
}
```

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar el endpoint

1. Agrega un nuevo archivo de clase llamado ***EndpointsContainer.cs*** en la raíz del proyecto y modifica el contenido del archivo para que sea similar al siguiente.

```
namespace Microsoft.AspNetCore.Builder;
public static class EndpointsContainer
{
    public static WebApplication UseMembershipEndpoints(this WebApplication app)
    {
        app.UseUserRegistrationController();

        return app;
    }
}
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear el paquete NuGet

4. Abre una ventana de línea de comandos.
5. Desplázate al directorio del proyecto *NorthWind.Membership.Backend.Core*.
6. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Implementar el servicio de administración de usuarios

Para implementar el servicio de administración de usuarios, utilizaremos ***ASP.NET Core Identity***. *ASP.NET Core Identity* proporciona un framework para administrar y almacenar cuentas de usuario en aplicaciones *ASP.NET Core*. Utilizaremos *ASP.NET Core Identity* como una simple herramienta que posteriormente podríamos cambiar por algún otro código para implementar la funcionalidad requerida para la administración y almacenamiento de cuentas de usuario.

1. Utiliza la plantilla ***Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Membership.Backend.AspNetIdentity***.
2. Elimina el archivo *Class1.cs*.
3. Agrega el paquete NuGet ***Microsoft.EntityFrameworkCore.SqlServer*** al proyecto. Utilizaremos *Entity Framework Core* para almacenar las cuentas de usuarios.
4. Agrega el paquete NuGet ***Microsoft.EntityFrameworkCore.Tools*** al proyecto.
5. Agrega el paquete NuGet ***Microsoft.AspNetCore.Identity.EntityFrameworkCore*** al proyecto. Este paquete contiene el proveedor de *ASP.NET Core Identity* que utiliza *Entity Framework Core*.
6. Agrega el paquete NuGet *NorthWind.Membership.Backend.Core* al proyecto.

Crear la clase para la identidad del usuario

1. Agrega un nuevo directorio llamado ***Entities*** en la raíz del nuevo proyecto.
2. Agrega un nuevo archivo de clase llamado ***NorthWindUser.cs*** en el directorio *Entities* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindUser : IdentityUser
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

   Puedes notar que solo estamos definiendo las propiedades para almacenar el nombre y apellidos del usuario. Las propiedades para el correo y la contraseña se encuentran definidas en la clase *IdentityUser*.
3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar la clase para opciones de base de datos

1. Agrega un nuevo directorio llamado ***Options*** en la raíz del proyecto.

2. Agrega un nuevo archivo de clase llamado ***MembershipDBOptions.cs*** en el directorio *Options* y remplaza el contenido de la clase por el siguiente.

```
public class MembershipDBOptions
{
    public const string SectionKey = nameof(MembershipDBOptions);
    public string ConnectionString { get; set; }
}
```

Agregar el contexto de datos

Utilizaremos una base de datos nueva exclusivamente para almacenar los datos de las cuentas de usuarios. Esta base de datos será utilizada por *ASP.NET Core Identity*.

1. Agrega un nuevo directorio llamado ***DataContexts*** en la raíz del nuevo proyecto.
2. Agrega un nuevo archivo de clase llamado ***NorthWindMembershipContext.cs*** en el directorio *DataContexts* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindMembershipContext(
    IOptions<MembershipDBOptions> dbOptions)
    : IdentityDbContext<NorthWindUser>
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            dbOptions.Value.ConnectionString);

        base.OnConfiguring(optionsBuilder);
    }
}
```

   El contexto que estamos creando hereda de la clase ***IdentityDbContext*** que es la clase base para el contexto de datos de *Entity Framework* utilizado por *ASP.NET Core Identity*. El parámetro del tipo *IdentityDbContext* representa al tipo derivado de *IdentityUser*.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Crear la fábrica del contexto de datos

Para crear las migraciones de *Entity Framework Core*, utilizaremos una fábrica.

1. Agrega un nuevo archivo de clase llamado ***NorthWindMembershipContextFactory.cs*** en el directorio *DataContexts* y remplaza el código de la clase por el siguiente.

```
internal class NorthWindMembershipContextFactory :
    IDesignTimeDbContextFactory<NorthWindMembershipContext>
{
    public NorthWindMembershipContext CreateDbContext(string[] args)
    {
        IOptions<MembershipDBOptions> DbOptions =
            Microsoft.Extensions.Options.Options.Create(
                new MembershipDBOptions()
                {
                    ConnectionString =
                    "Server=(localdb)\\mssqllocaldb;Database=NorthWindUsersDB"
                });
        return new NorthWindMembershipContext(DbOptions);
    }
}
```

Utilizaremos esta clase en tiempo de diseño por lo que no representa ningún problema utilizar los datos de la cadena de conexión como código duro.

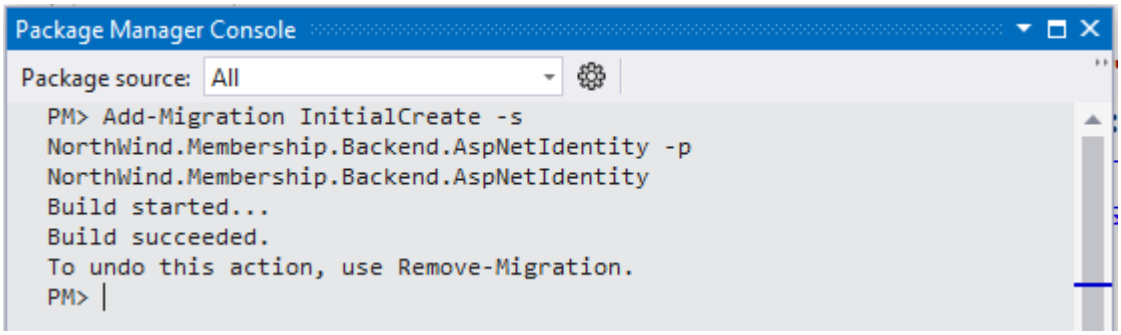
Puedes notar que estamos utilizando una nueva base de datos llamada *NorthWindUsersDB*.

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear la migración

1. Ejecuta el siguiente comando en la consola de administración de paquetes NuGet para crear la migración de *Entity Framework Core*.

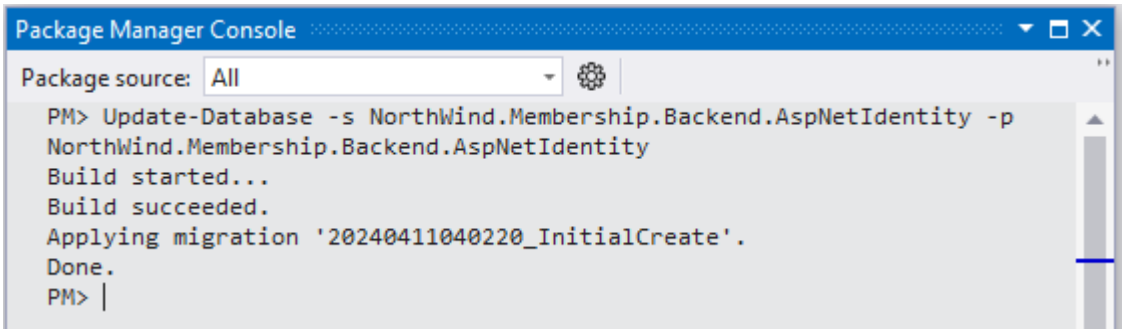
```
Add-Migration InitialCreate -s NorthWind.Membership.Backend.AspNetIdentity -p
NorthWind.Membership.Backend.AspNetIdentity
```



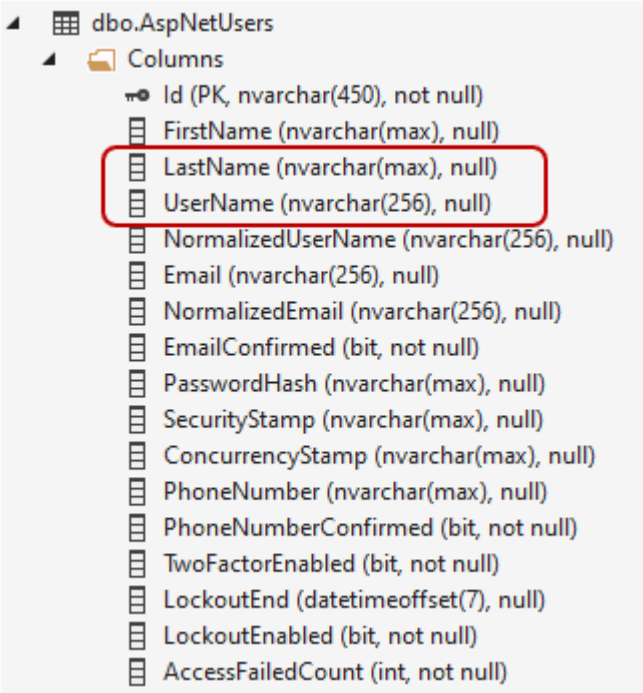
Crear la base de datos

1. Ejecuta el siguiente comando en la consola de administración de paquetes NuGet para crear la base de datos de usuarios.

```
Update-Database -s NorthWind.Membership.Backend.AspNetIdentity -p
NorthWind.Membership.Backend.AspNetIdentity
```



En este momento la base de datos de usuarios ya se encuentra creada. Puedes notar que la tabla ***AspNetUsers*** contiene los campos que definimos a través de las propiedades de la clase *NorthWindUser* junto con otros campos adicionales.



Agregar un archivo de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***Messages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
DuplicateUserNameErrorMessage	El nombre de usuario proporcionado ya se encuentra registrado.

Agregar un manejador de mensajes de error

Al registrar un usuario con *ASP.NET Core Identity*, este podría devolvernos errores que mapearemos a mensajes de error personalizados.

1. Agrega un nuevo directorio llamado ***Extensions*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***IdentityErrorExtensions.cs*** en el directorio *Extensions* y remplaza el código de la clase por el siguiente.

```
internal static class IdentityErrorExtensions
{
    public static IEnumerable<ValidationError> ToValidationErrors(
    this IEnumerable<IdentityError> errors)
    {
        List<ValidationError> Result = [];

        foreach (var Error in errors)
        {
            switch (Error.Code)
            {
                case nameof(IdentityErrorDescriber.DuplicateUserName):
                    Result.Add(new ValidationError(
                        nameof(UserRegistrationDto.Email),
                        Messages.DuplicateUserNameErrorMessage));
                    break;
                default:
                    Result.Add(new ValidationError(
                        Error.Code, Error.Description));
                    break;
            }
        }

        return Result;
    }
}
```

   Por el momento solo estamos manejando el mensaje de error que se genera cuando una cuenta de usuario ya se encuentra registrada.
3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar IMembershipService

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.

2. Agrega un nuevo archivo de clase llamado ***MembershipService.cs*** en el directorio *Services* y remplaza el código de la clase por el siguiente.

```
internal class MembershipService(UserManager<NorthWindUser> manager) :
    IMembershipService
{
    public async Task<Result<IEnumerable<ValidationError>>> Register(
        UserRegistrationDto userData)
    {
        Result<IEnumerable<ValidationError>> Result;

        var User = new NorthWindUser
        {
            UserName = userData.Email,
            Email = userData.Email,
            FirstName = userData.FirstName,
            LastName = userData.LastName
        };

        var CreateResult =
            await manager.CreateAsync(User, userData.Password);

        if (CreateResult.Succeeded)
        {
            // Resultado sin errores
            Result = new Result<IEnumerable<ValidationError>>();
        }
        else
        {
            // Resultado con errores
            Result = new Result<IEnumerable<ValidationError>>(
                CreateResult.Errors.ToValidationErrors());
        }

        return Result;
    }
}
```

El código anterior utiliza una instancia de *UserManager* que proporciona la API de *ASP.NET Core Identity* para administrar usuarios en el medio de persistencia.

El valor *UserName* es el valor que el usuario deberá proporcionar para iniciar sesión, en este caso, es su correo electrónico.

El método ***CreateAsync*** de *UserManager* crea el usuario especificado en el medio de persistencia con la clave de acceso proporcionada y devuelve un tipo *IdentityResult*.

La propiedad *Succeeded* de *IdentityResult* contiene un valor booleano que indica si la operación fue exitosa o no. La propiedad *Errors* de *IdentityResult* contiene instancias de los errores que hayan ocurrido durante la operación.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Registrar las dependencias

1. Agrega un nuevo archivo de clase llamado ***DependencyContainer.cs*** en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddMembershipIdentityServices(
        this IServiceCollection services,
        Action<MembershipDBOptions> configureMembershipDBOptions)
    {
        services.AddDbContext<NorthWindMembershipContext>();

        // Configurar Identity
        services.AddIdentityCore<NorthWindUser>()
            .AddEntityFrameworkStores<NorthWindMembershipContext>();

        services.AddScoped<IMembershipService, MembershipService>();
        services.Configure(configureMembershipDBOptions);

        return services;
    }
}
```

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Crear el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Backend.AspNetIdentity*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Incorporar la funcionalidad backend de membresía

1. Abre la solución *NorthWind*.
2. Agrega el paquete NuGet *NorthWind.Membership.Backend.AspNetIdentity* al proyecto *NorthWind.Sales.Backend.IoC*.
3. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
4. Modifica el código del método *AddNorthWindSalesServices* para registrar los nuevos servicios. El código será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions,
    Action<MembershipDBOptions> configureMembershipDBOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnauthorizedAccessExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices()
        .AddTransactionServices()
        .AddUserServices()
        .AddMembershipCoreServices()
        .AddMembershipIdentityServices(configureMembershipDBOptions);

    return services;
}
```

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
6. Abre el archivo *EndpointsContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
7. Agrega el siguiente código al método *MapNorthWindSalesEndpoints* para incorporar los endpoints del framework de membresía.

```
public static WebApplication MapNorthWindSalesEndpoints(
    this WebApplication app)
{
    app.UseCreateOrderController();
    app.UseMembershipEndpoints();

    return app;
```

```
}
```

Actualizar el proyecto Web API

Realicemos algunos cambios en el proyecto *NorthWind.Sales.WebApi* para incorporar la nueva funcionalidad.

Modificar el registro de los servicios

1. Abre el archivo *Startup.cs*.
2. Localiza el siguiente código.

```
builder.Services.AddNorthWindSalesServices(
    dbOptions =>
        builder.Configuration.GetSection(DBOptions.SectionKey)
        .Bind(dbOptions),
    smtpOptions =>
        builder.Configuration.GetSection(SmtpOptions.SectionKey)
        .Bind(smtpOptions));
```

3. Modifica el código anterior por el siguiente para proporcionar el argumento faltante.

```
builder.Services.AddNorthWindSalesServices(
    dbOptions =>
        builder.Configuration.GetSection(DBOptions.SectionKey)
        .Bind(dbOptions),
    smtpOptions =>
        builder.Configuration.GetSection(SmtpOptions.SectionKey)
        .Bind(smtpOptions),
    membershipDBOptions =>
        builder.Configuration.GetSection(MembershipDBOptions.SectionKey)
        .Bind(membershipDBOptions));
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar la nueva cadena de conexión

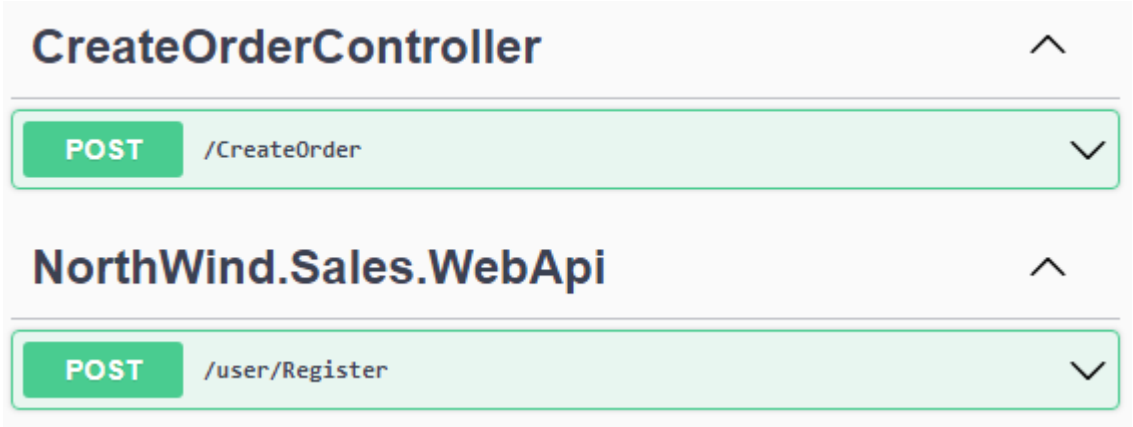
1. Abre el archivo *appsettings.json*.
2. Agrega el siguiente código para definir la cadena de conexión para la base de datos de usuarios.

```
"MembershipDBOptions": {
  "ConnectionString": "Server=(localdb)\\mssqllocaldb;Database=NorthWindUsersDB"
}
```

Probar la funcionalidad

Probemos ahora la nueva funcionalidad que nos permitirá registrar nuevos usuarios.

1. Ejecuta la aplicación. Podrás ver el nuevo endpoint que hemos agregado.

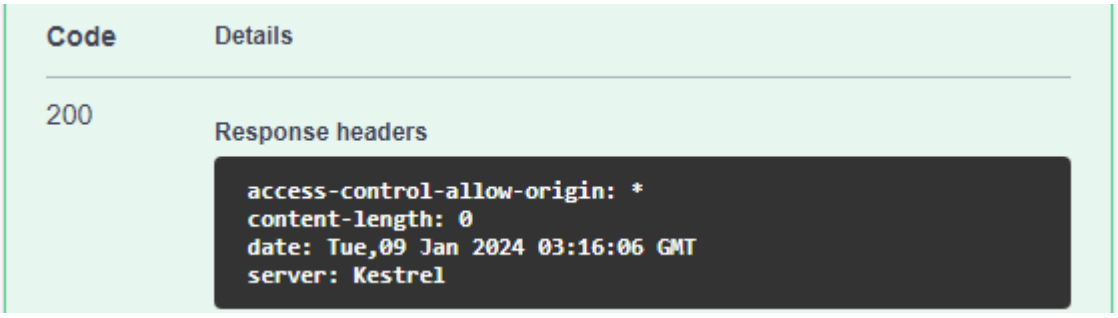


Probar con datos correctos

1. Intenta registrar un usuario con los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "Mm1234567$",
    "firstName": "Maria",
    "lastName": "Sanders"
}
```

2. Podrás ver un resultado indicando que la operación fue exitosa.



En la tabla ***AspNetUsers*** de la base de datos de usuarios podrás ver la información agregada del nuevo usuario registrado.

dbo.AspNetUsers [Data]

Max Rows: 1000

	Id	FirstName	LastName	UserName	NormalizedUs...	E
▶	3924770c-297f-...	Maria	Sanders	msanders@northwind.com	MSANDERS@N...	m
*	NULL	NULL	NULL	NULL	NULL	N

Probar con una cuenta duplicada

1. Intenta crear un usuario con el mismo correo que registraste previamente.

   Se mostrará un error similar al siguiente.

Code Details

400
Undocumented

Error: response status is 400

Response body


```
{
  "type": "https://datatracker.ietf.org/doc/html/rfc7231#section-6.5.1",
  "title": "Error en los datos de entrada.",
  "status": 400,
  "detail": "Se encontraron uno o más errores de validación de datos.",
  "instance": "ProblemDetails/UserRegistrationPresenter",
  "errors": [
    {
      "propertyName": "Email",
      "message": "El nombre de usuario proporcionado ya se encuentra registrado."
    }
  ]
}
```

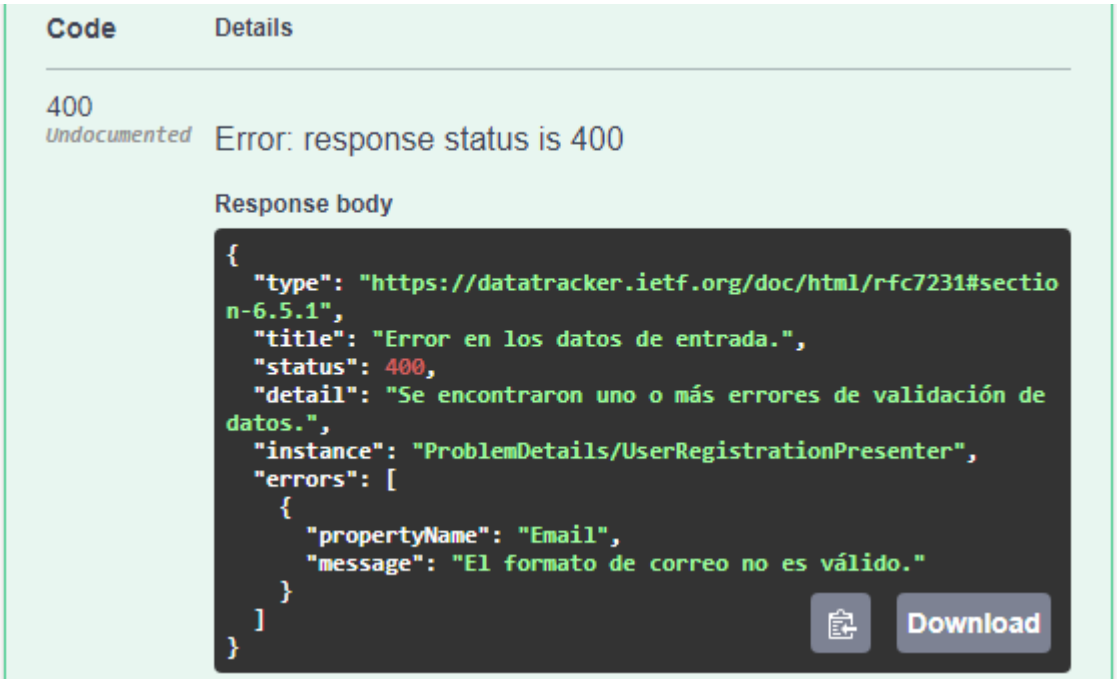

Download

Probar con un correo no válido

1. Intenta crear un usuario con los siguientes datos.


```
{
    "email": "msanders ",
    "password": "Mm1234567$",
    "firstName": "Maria",
    "lastName": "Sanders"
}
```

Podrás ver un error similar al siguiente.

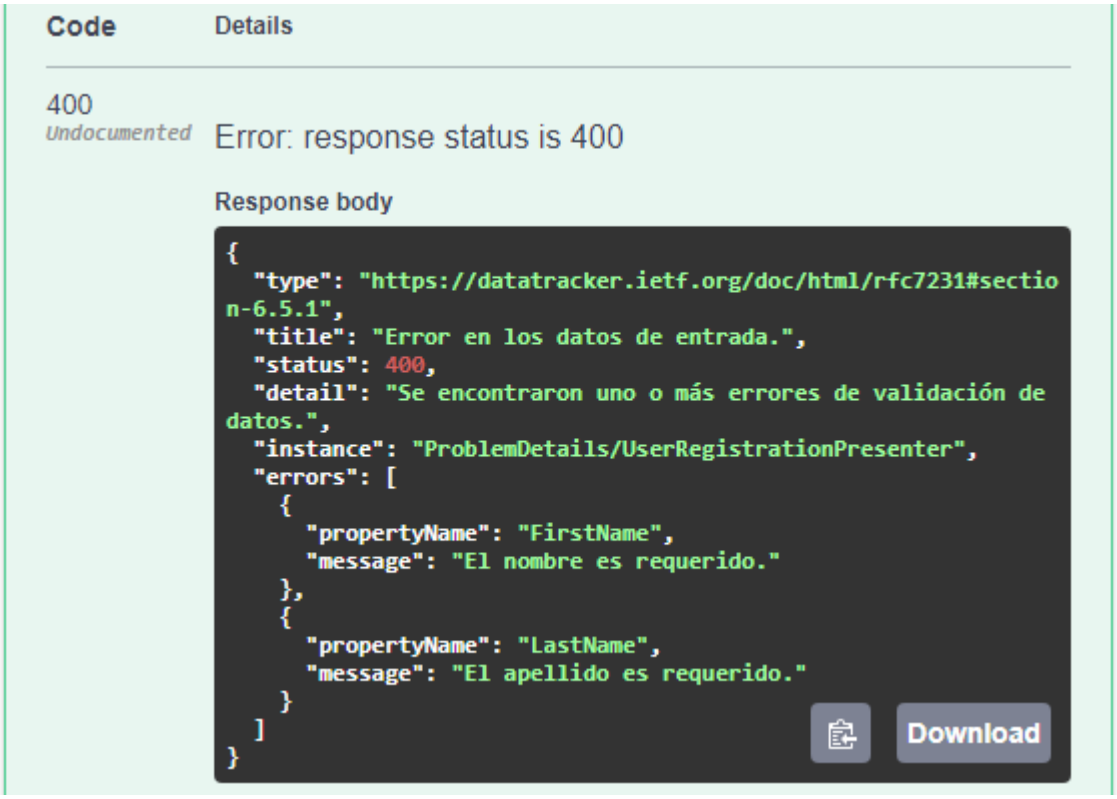


Probar con error en FirstName y LastName

1. Intenta crear un usuario con los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "Mm1234567$"
}
```

Podrás ver un resultado de error similar al siguiente.

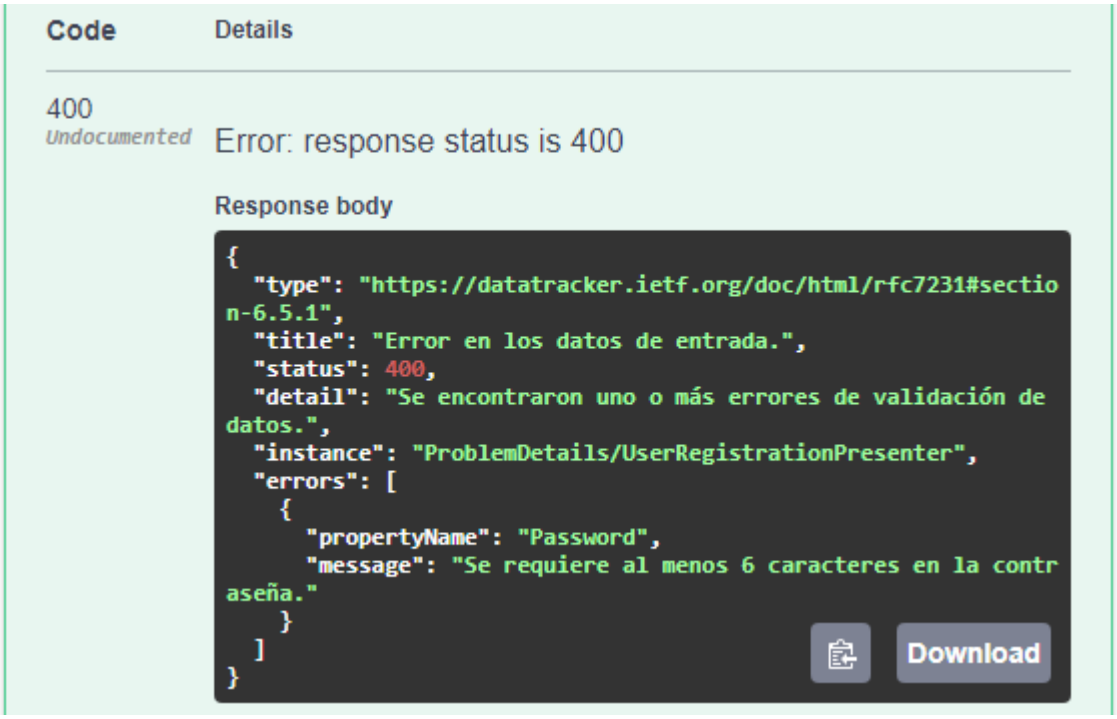


Probar con una clave de acceso no válida

1. Intenta registrar un usuario con los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "12345",
    "firstName": "Maria",
    "lastName": "Sanders"
}
```

Se mostrará un error similar al siguiente.



2. Regresa a Visual Studio y detén la ejecución.

Implementar la funcionalidad frontend del framework de membresía

Implementemos ahora el código frontend del framework de membresía para el registro de usuarios.

1. Utiliza la plantilla de proyecto ***Razor Class Library*** para agregar un nuevo proyecto llamado ***NorthWind.Membership.Frontend.RazorViews*** a la solución ***NorthWind.Membership*** sin seleccionar el soporte de páginas y vistas.
2. Elimina los siguientes archivos:
    a. Component1.razor
    b. ExampleJsInterop.cs
    c. wwwroot\background.png
    d. wwwroot\exampleJsInterop.js
3. Agrega el paquete NuGet *NorthWind.RazorComponents* al proyecto.
4. Agrega el paquete NuGet *NorthWind.Membership.Entities*.
5. Agrega el paquete NuGet *NorthWind.HttpDelegatingHandlers* al proyecto.

Crear el Gateway para los endpoints de administración de usuarios

Agreguemos el *Gateway* que permitirá a la aplicación *Blazor* consumir los endpoints de registro e inicio de sesión de usuarios.

1. Agrega un nuevo directorio llamado ***WebApiGateways*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***MembershipGateway.cs*** en el directorio *WebApiGateways* y remplaza el código de la clase por el siguiente.

```
public class MembershipGateway(HttpClient client)
{
    public async Task RegisterAsync(UserRegistrationDto userData) =>
        await client.PostAsJsonAsync(Endpoints.Register, userData);
}
```

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar el contenedor de dependencias

1. Agrega el paquete NuGet ***Microsoft.Extensions.Http*** al proyecto.
2. Agrega un nuevo archivo de clase llamado *DependencyContainer.cs* en la raíz del proyecto y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class DependencyContainer
{
    public static IServiceCollection AddMembershipServices(
        this IServiceCollection services,
        Action<HttpClient> configureHttpClient)
    {
        services.AddExceptionDelegatingHandler();
        services.AddHttpClient<MembershipGateway>(configureHttpClient)
            .AddHttpMessageHandler<ExceptionDelegatingHandler>();


        return services;
    }
}
```

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar archivos de recursos

1. Agrega un nuevo directorio llamado ***Resources*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserRegistrationMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
ConfirmPasswordErrorMessage	La contraseña y la confirmación no coinciden.
RegisteredUserMessage	Usuario registrado exitosamente.
FirstNameLabel	Nombre
LastNameLabel	Apellidos
PasswordConfirmationLabel	Confirmar contraseña
RegisterButtonText	Registrar

3. Agrega un nuevo archivo de tipo *Resources File* llamado ***CommonMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
OK	Aceptar
EmailLabel	Correo
PasswordLabel	Contraseña

Agregar el ViewModel

1. Agrega un nuevo directorio llamado ***ViewModels*** en la raíz del proyecto.
2. Agrega un nuevo directorio llamado ***UserRegistration*** en el directorio *ViewModels*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationViewModel.cs*** en el directorio *UserRegistration* y remplaza el código de la clase por el siguiente.

```
public class UserRegistrationViewModel(
    MembershipGateway gateway,
    IModelValidatorHub<UserRegistrationViewModel> validator)
{
    public IModelValidatorHub<UserRegistrationViewModel> Validator =>
        validator;

    public ModelValidator<UserRegistrationViewModel>
        ModelValidatorComponentReference
    { get; set; }

    public string InformationMessage { get; private set; }

    #region Datos de entrada
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
    public string Password { get; set; }
    public string PasswordConfirmation { get; set; }
    #endregion

    public async Task Register()
    {
        try
        {
            InformationMessage = "";
            await gateway.RegisterAsync((UserRegistrationDto)this);
            InformationMessage =
                UserRegistrationMessages.RegisteredUserMessage;
        }
        catch (HttpRequestException ex)
        {
            if (ex.Data.Contains("Errors"))
            {
                IEnumerable<ValidationError> Errors =
                    ex.Data["Errors"] as IEnumerable<ValidationError>;
                ModelValidatorComponentReference.AddErrors(Errors);
            }
            else
            {
                throw;
            }
        }
    }

    public static explicit operator UserRegistrationDto(
        UserRegistrationViewModel model) =>
```

```
        new UserRegistrationDto(
            model.Email, model.Password,
            model.FirstName, model.LastName);
}
```

Utilizaremos esta clase para que el usuario pueda proporcionar la información de la cuenta a crear.

4. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar la validación

1. Agrega un nuevo archivo de clase llamado ***UserRegistrationViewModelDtoValidator.cs*** en el directorio *UserRegistration* y remplaza el código de la clase por el siguiente.

```
internal class UserRegistrationViewModelDtoValidator(
    IModelValidatorHub<UserRegistrationDto> validator) :
    AbstractViewModelValidator<UserRegistrationDto, UserRegistrationViewModel>(
        validator, ValidationConstraint.AlwaysValidate)
{
}
```

La clase anterior simplemente encapsula el validador del DTO *UserRegistrationDto* asociado.

2. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.
3. Agrega un nuevo archivo de clase llamado *UserRegistrationViewModelValidator.cs* en el directorio *UserRegistration* y remplaza el código de la clase por el siguiente.

```
internal class UserRegistrationViewModelValidator :
    AbstractModelValidator<UserRegistrationViewModel>
{
    public UserRegistrationViewModelValidator(
        IValidationService<UserRegistrationViewModel> validationService) :
        base(validationService, ValidationConstraint.AlwaysValidate)
    {
        AddRuleFor(c => c.PasswordConfirmation)
            .Equal(c => c.Password,
                        UserRegistrationMessages.ConfirmPasswordErrorMessage);
    }
}
```

Con este validador, simplemente estamos agregando validación adicional para la contraseña y la confirmación de la contraseña.

Registrar las dependencias

1. Agrega el siguiente código en el método *AddMembershipServices* del archivo *DependencyContainer.cs* para registrar el *ViewModel*, los validadores de *Membership.Entities* y los nuevos validadores que hemos creado en este proyecto.

```
services.AddScoped<UserRegistrationViewModel>();

services.AddMembershipValidators();
services.AddModelValidator<UserRegistrationViewModel,
    UserRegistrationViewModelDtoValidator>();
services.AddModelValidator<UserRegistrationViewModel,
    UserRegistrationViewModelValidator>();
```

2. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear el componente de registro

1. Agrega un nuevo directorio llamado ***Components*** en la raíz del proyecto.
2. Agrega un nuevo componente razor llamado ***UserRegistrationComponent.razor*** en el directorio *Components*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationComponent.razor.cs*** en el directorio *Components* y remplaza el código de la clase por el siguiente.

```
public partial class UserRegistrationComponent
{
    [Parameter]
    public UserRegistrationViewModel ViewModel { get; set; }

}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
5. Remplaza el código del archivo *UserRegistrationComponent.razor* por el siguiente para definir la interfaz de usuario que permita al usuario proporcionar la información de la cuenta a crear.

```
<EditForm Model=ViewModel OnValidSubmit=ViewModel.Register>
      <ModelValidator Validator="ViewModel.Validator"
            @ref=ViewModel.ModelValidatorComponentReference />
      <ValidationSummary/>

      <div class="form-group">
            <label for="FirstName">
                  @UserRegistrationMessages.FirstNameLabel
          </label>
            <InputText
```

```
                id="FirstName"
                class="form-control"
                placeholder=
                        "@UserRegistrationMessages.FirstNameLabel"
                         @bind-Value=ViewModel.FirstName />
        <ValidationMessage For="@(() => ViewModel.FirstName)" />
</div>

<div class="form-group">
        <label for="LastName">
                @UserRegistrationMessages.LastNameLabel
        </label>
        <InputText
                id="LastName"
                class="form-control"
                placeholder=
                        "@UserRegistrationMessages.LastNameLabel"
                         @bind-Value=ViewModel.LastName />
        <ValidationMessage For="@(() => ViewModel.LastName)" />
</div>

<div class="form-group">
        <label for="Email">
                @CommonMessages.EmailLabel
        </label>
        <InputText
                id="Email"
                class="form-control"
                placeholder="@CommonMessages.EmailLabel"
                          @bind-Value=ViewModel.Email />
        <ValidationMessage For="@(() => ViewModel.Email)" />
</div>

<div class="form-group">
        <label for="Password">
                @CommonMessages.PasswordLabel
        </label>
        <InputText
                id="Password"
                class="form-control"
                placeholder="@CommonMessages.PasswordLabel"
                type="password"
                          @bind-Value=ViewModel.Password />
        <ValidationMessage For="@(() => ViewModel.Password)" />
</div>

<div class="form-group">
        <label for="PasswordConfirmation">
                @UserRegistrationMessages.PasswordConfirmationLabel
        </label>
        <InputText
                id="PasswordConfirmation"
                class="form-control"
                placeholder=
                   "@UserRegistrationMessages.PasswordConfirmationLabel"
            type="password"
                          @bind-Value=ViewModel.PasswordConfirmation />
        <ValidationMessage For="@(() => ViewModel.PasswordConfirmation)" />
```

```
        </div>

        <div class="row p-3">
                <div class="col">
                        <button class="btn btn-primary">
                                @UserRegistrationMessages.RegisterButtonText
                        </button>
                </div>
        </div>
</EditForm>

<hr />

@if (!string.IsNullOrWhiteSpace(ViewModel.InformationMessage))
{
        <div class="alert alert-info">
                @ViewModel.InformationMessage
        </div>
}
```

6. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using Microsoft.AspNetCore.Components.Forms
@using NorthWind.RazorComponents.Validators
```

Agregar la página de registro

1. Agrega un nuevo directorio llamado ***Pages*** en la raíz del proyecto.
2. Agrega un nuevo componente razor llamado ***UserRegistrationPage.razor*** en el directorio *Pages*.
3. Agrega un nuevo archivo de clase llamado ***UserRegistrationPage.razor.cs*** en el directorio *Pages* y remplaza el código de la clase por el siguiente.

```
public partial class UserRegistrationPage
{
        [Inject]
        UserRegistrationViewModel ViewModel { get; set; }

        ErrorBoundary ErrorBoundaryRef;

        void Recover()
        {
                ErrorBoundaryRef?.Recover();
        }
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

5. Remplaza el contenido del archivo ***UserRegistrationPage.razor*** por el siguiente para implementar la interfaz de usuario de la página de registro de usuarios.

```
@page "/user/register"

<ErrorBoundary @ref=ErrorBoundaryRef>
      <ChildContent>
            <UserRegistrationComponent ViewModel=ViewModel/>
      </ChildContent>
      <ErrorContent>
            <ExceptionDisplay Exception=context />
            <button class="btn btn-danger" @onclick=Recover>
                  @CommonMessages.OK
            </button>
      </ErrorContent>
</ErrorBoundary>
```

Puedes notar que el componente está definiendo una página que responderá al endpoint ***/user/register***.

6. Agrega el siguiente código en el archivo *_Imports.razor* para importar los espacios de nombres requeridos.

```
@using NorthWind.Membership.Frontend.RazorViews.Components
@using NorthWind.RazorComponents.Exceptions
```

Agregar una clase de soporte

1. Agrega un nuevo directorio llamado ***Extensions*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***MembershipPages.cs*** en el directorio *Extensions* y remplaza el contenido de la clase con el siguiente.

```
public static class MembershipPages
{
      public static Assembly Assembly =>
            typeof(MembershipPages).Assembly;
}
```

Este código facilitará a la aplicación cliente la incorporación del ensamblado que contiene las páginas del framework de membresía.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Frontend.RazorViews*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Incorporar la funcionalidad frontend de membresía

1. Abre la solución *NorthWind*.
2. Agrega el paquete NuGet ***NorthWind.Membership.Frontend.RazorViews*** al proyecto ***NorthWind.Sales.Frontend.IoC***.
3. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Frontend.IoC*.
4. Agrega el siguiente código al método *AddNorthWindSalesServices* para registrar los servicios de *Membership*.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<HttpClient> configureHttpClient,
    Action<HttpClient> configureMembershipHttpClient)
{
    services.AddWebApiGateways(configureHttpClient)
        .AddViewsServices()
        .AddValidationService()
        .AddValidators()
        .AddMembershipServices(configureMembershipHttpClient);

    return services;
}
```

Actualizar el proyecto NorthWind.Sales.BlazorClient

Hagamos algunas modificaciones en el proyecto *NorthWind.Sales.BlazorClient*.

1. Abre el archivo *Program.cs*.
2. Localiza el siguiente código.

```
builder.Services.AddNorthWindSalesServices(client =>
{
      client.BaseAddress =
      new Uri(builder.Configuration["WebApiAddress"]);
});
```

3. Modifica el código anterior por el siguiente.

```
builder.Services.AddNorthWindSalesServices(client =>
{
      client.BaseAddress =
      new Uri(builder.Configuration["WebApiAddress"]);
},
membershipHttpClient =>
{
    membershipHttpClient.BaseAddress =
```

```
    new Uri(builder.Configuration["WebApiAddress"]);
}
);
```

5. Abre el archivo ***App.razor***.
6. Modifica el bloque ***@code*** para registrar el ensamblado que contiene la página de registro de usuarios. El código será similar al siguiente.

```
@code {
    Assembly[] AdditionalAssemblies =>
        new Assembly[]
        {
            typeof(NorthWind.Sales.Frontend.Views.Pages.CreateOrder)
            .Assembly,
            MembershipPages.Assembly
        };
}
```

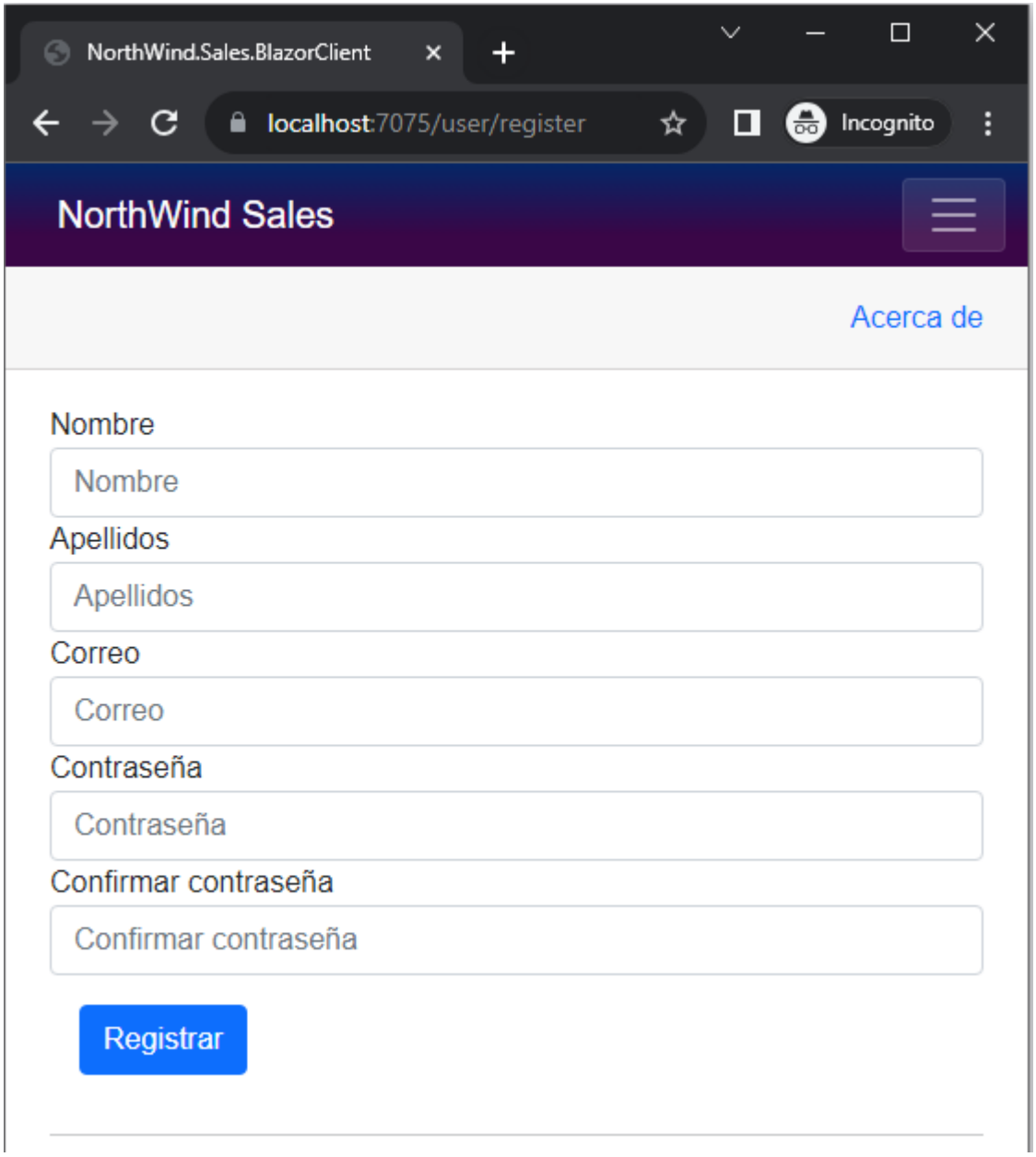
7. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres de la clase *MembershipPages*.

```
@using NorthWind.Membership.Frontend.RazorViews.Extensions
```

Probar la funcionalidad

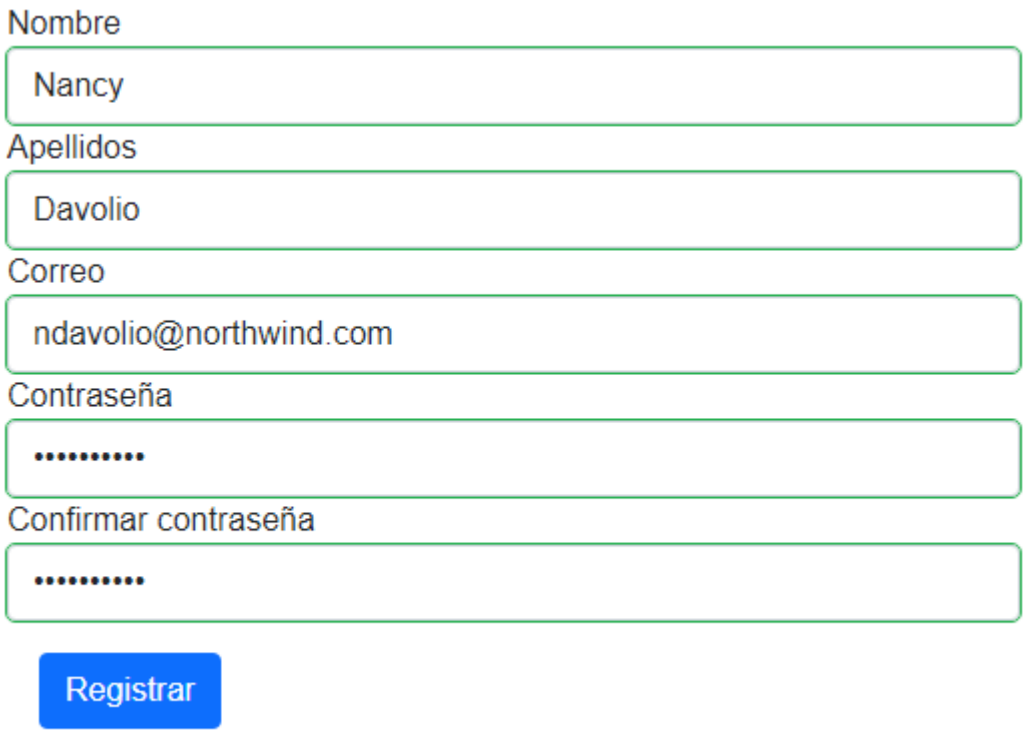
Podemos ahora probar la funcionalidad de registro de usuarios.

1. Ejecuta la aplicación.
2. Navega a la Uri ***/user/register*** de la aplicación *Blazor*. Podrás ver una página similar a la siguiente.

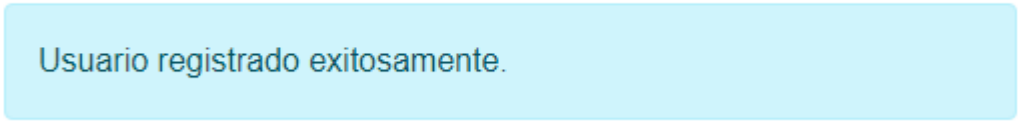


Probar con datos correctos

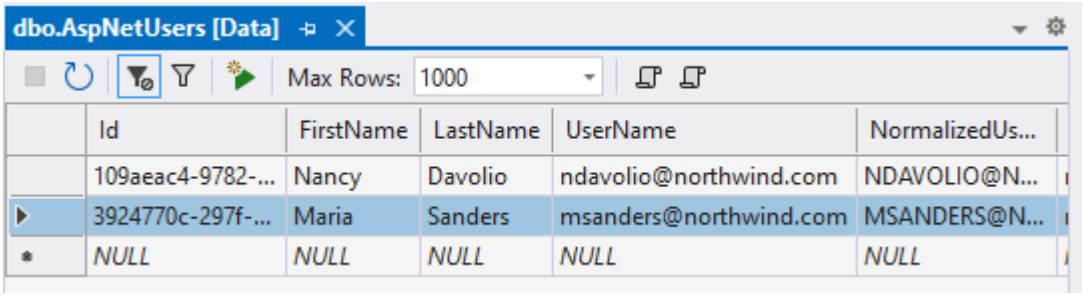
1. Proporciona los datos requeridos para crear una cuenta de usuario y haz clic en el botón ***Registrar***.



Podrás ver un resultado similar al siguiente.

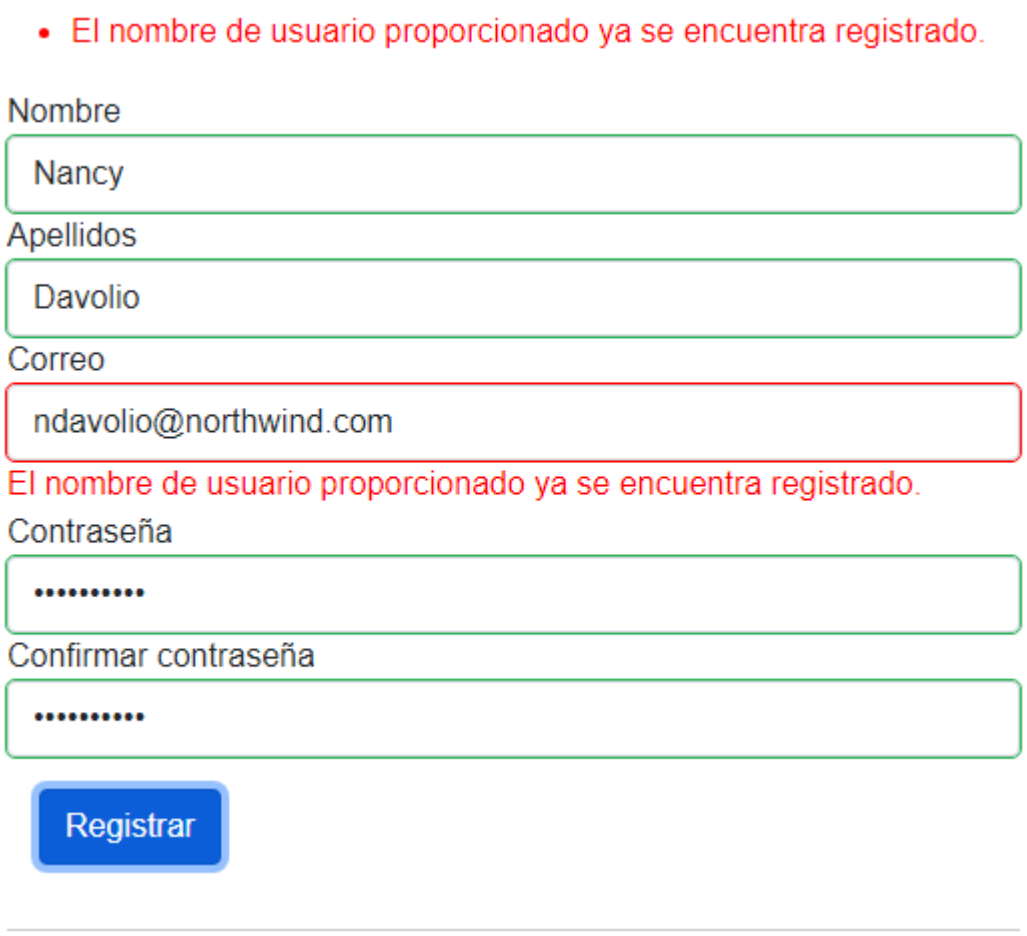


2. Examina el contenido de la tabla ***AspNetUsers*** de la base de datos de usuarios. Podrás ver la información del usuario registrado.



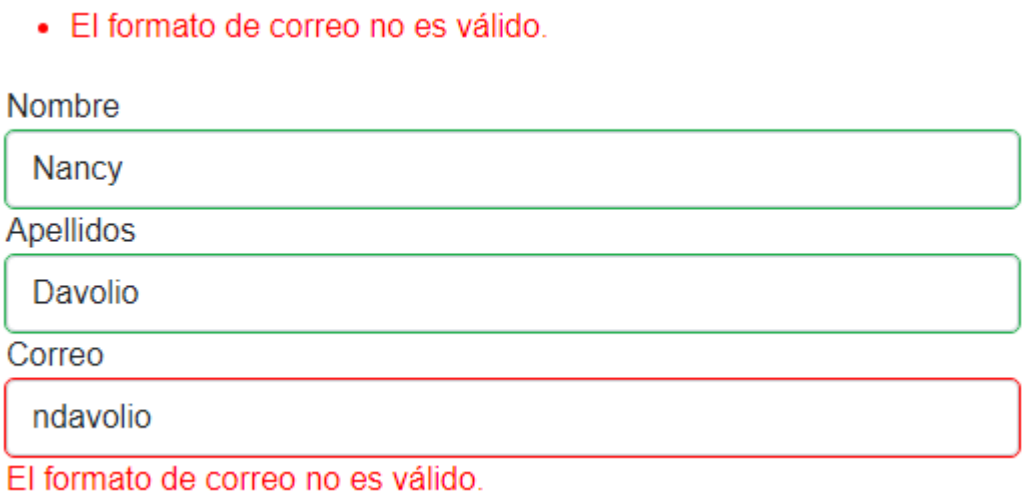
Probar con cuenta duplicada

1. Intenta crear un nuevo usuario con los mismos datos. Podrás ver un resultado similar al siguiente indicando el error.



Probar con un correo no válido

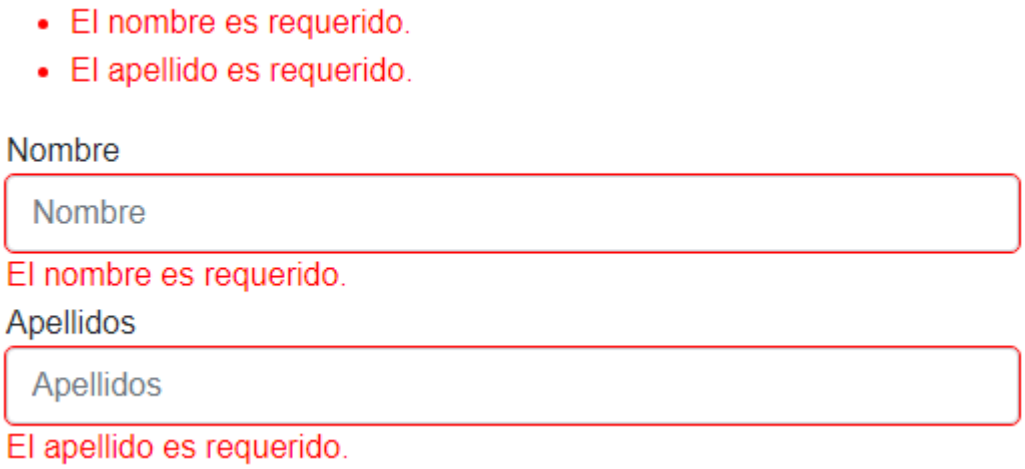
1. Intenta crear un usuario con un formato de correo no válido. Podrás ver un resultado similar al siguiente.



Puedes notar que la validación se realiza en el navegador, no se envía la información al backend.

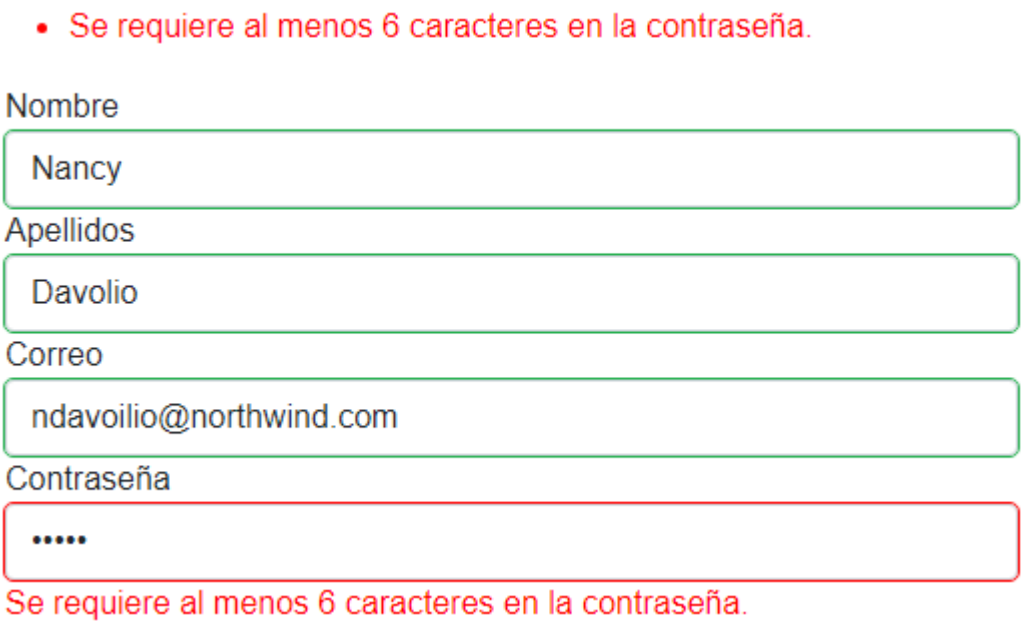
Probar con error en FirstName y LastName

1. Intenta crear un usuario sin especificar nombre y apellidos. Podrás ver un resultado similar al siguiente.



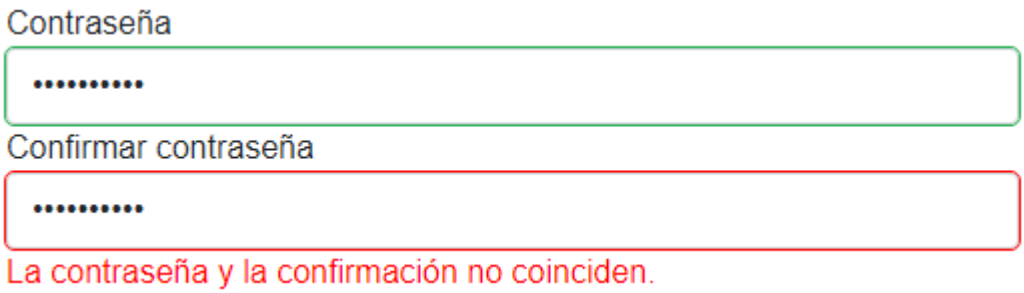
Probar con una clave de acceso no válida

1. Intenta registrar un usuario con una clave de acceso no válida. Podrás ver un resultado similar al siguiente.



Probar con confirmación de contraseña incorrecta

1. Intenta registrar un usuario proporcionando contraseñas que no coincidan. Podrás ver un resultado similar al siguiente.



2. Regresa a Visual Studio y detén la ejecución.

Hemos implementado el caso de uso que nos permite registrar usuarios en la aplicación. Podemos continuar ahora con la funcionalidad para inicio de sesión y autenticación de usuarios.

Práctica 15:

Implementar el inicio de sesión

Para que un usuario pueda crear órdenes de compra, primero deberá iniciar sesión proporcionando sus credenciales para poder ser autenticado y autorizado.

En esta práctica implementaremos un nuevo caso de uso para implementar el inicio de sesión de usuarios.

Objetivos

Al finalizar esta práctica los estudiantes:

- Agregarán funcionalidad para implementar el inicio de sesión de usuarios utilizando el sistema de membresía *ASP.NET Core Identity*.

Caso de uso: Inicio de sesión

Agreguemos un nuevo caso de uso a la solución para implementar el inicio de sesión de usuarios.

Historia de usuario: Inicio de sesión

"*Como usuario del sistema, necesito iniciar sesión para poder ser autenticado y autorizado para poder crear órdenes de compra*".

Datos de entrada

- Correo
- Clave de acceso

Flujo primario

1. El usuario envía la solicitud "*Inicio de sesión*" con los datos de entrada.
2. El sistema verifica que el correo y contraseña correspondan a un usuario registrado.
3. El sistema devuelve un token de acceso al usuario.

Flujo alterno: Error de validación de datos.

1. El procesamiento de la solicitud es cancelado.
2. El sistema notifica el error al usuario.

Validaciones

- *Correo* es un dato requerido y debe tener un formato válido de correo electrónico.
- *Clave de acceso* es un dato requerido.

Implementar la funcionalidad backend del framework de membresía

Empecemos por implementar los componentes backend del framework de membresía.

Agregar nuevas entidades empresariales

Agreguemos en el proyecto *NorthWind.Membership.Entities* las entidades de negocio empresariales que utilizaremos para implementar el nuevo caso de uso.

Agregar DTOs

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *Dtos* del proyecto.
2. Agrega un nuevo archivo de clase llamado ***UserCredentialsDto.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
public class UserCredentialsDto(string email, string password)
{
    public string Email => email;
    public string Password => password;
}
```

   Utilizaremos esta clase para recibir las credenciales del usuario.

3. Agrega un nuevo archivo de clase llamado ***TokensDto.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
public class TokensDto(string accessToken)
{
    public string AccessToken => accessToken;
}
```

   Utilizaremos esta clase para devolver un token de acceso con la información del usuario autenticado.

Agregar una ruta de Endpoint

1. Abre el archivo ***Endpoints.cs*** del directorio *ValueObjects*.
2. Agrega el siguiente código al final de la clase *Endpoints*.

```
public const string Login = $"/user/{nameof(Login)}";
```

Agregar un archivo de recursos

1. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserLoginMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
RequiredEmailErrorMessage	El correo es requerido.
InvalidEmailErrorMessage	El formato de correo no es válido.
RequiredPasswordErrorMessage	La contraseña es requerida.

Agregar la clase de validación

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *Validators*.
2. Agrega un nuevo archivo de clase llamado ***UserCredentialsDtoValidator.cs*** en el directorio *UserLogin* y remplaza el contenido de la clase por el siguiente.

```
internal class UserCredentialsDtoValidator :
    AbstractModelValidator<UserCredentialsDto>
{
    public UserCredentialsDtoValidator(
        IValidationService<UserCredentialsDto> validationService) :
        base(validationService)
    {
        AddRuleFor(u => u.Email)
            .NotEmpty(UserLoginMessages.RequiredEmailErrorMessage)
            .EmailAddress(UserLoginMessages.InvalidEmailErrorMessage);

        AddRuleFor(u => u.Password)
            .NotEmpty(UserLoginMessages.RequiredPasswordErrorMessage);
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar el validador

1. Abre el archivo *DependencyContainer.cs*.
2. Agrega el siguiente código en el método *AddMembershipValidators* para registrar el nuevo validador.

```
services.AddModelValidator<UserCredentialsDto,
    UserCredentialsDtoValidator>();
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Actualizar el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Entities*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Agregar objetos de negocio

Agreguemos ahora en el proyecto *NorthWind.Membership.Backend.Core* los objetos de negocio que serán utilizados en el código backend.

Reinstalar el paquete NorthWind.Membership.Entities

1. Expande el nodo ***Dependencies*** del proyecto *NorthWind.Membership.Backend.Core* en la ventana ***Solution Explorer***.
2. Expande el elemento ***Packages***.
3. Selecciona la opción ***Open Folder in File Explorer*** del menú contextual del paquete *NorthWind.Membership.Entities* para abrir el explorador de Windows. Se mostrarán los archivos en cache del paquete.
4. Elimina los archivos del directorio.
5. Cierra el explorador de Windows y regresa a Visual Studio.
6. Elimina y vuelve a agregar el paquete *NorthWind.Membership.Entities*.

Agregar un DTO

1. Agrega un nuevo directorio llamado ***Dtos*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***UserDto.cs*** en el directorio *Dtos* y remplaza el código de la clase *UserDto* por el siguiente.

```
public class UserDto(string email, string firstName, string lastName)
{
    public string Email => email;
    public string FirstName => firstName;
```

```
    public string LastName => lastName;
}
```

Utilizaremos esta estructura de datos para obtener la información del usuario que ha iniciado sesión.

Modificar IMembershipService

Modifiquemos la interface *IMembershipService* para definir el método que nos permitirá buscar un usuario utilizando las credenciales proporcionadas.

1. Abre el archivo *IMembershipService.cs* ubicado en el directorio *Interfaces\Common*.
2. Agrega el siguiente código al final de la interface.

```
Task<UserDto> GetUserByCredentials(UserCredentialsDto userData);
```

El método *GetUserByCredentials* devolverá los datos del usuario autenticado en una instancia del tipo *UserDto*. Sin embargo, si el usuario no puede ser autenticado, devolverá ***null***.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Definir el InputPort

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *Interfaces* del proyecto.
2. Agrega un nuevo archivo de tipo *Interface* llamado ***IUserLoginInputPort.cs*** en el directorio *UserLogin*.
3. Remplaza el código de la interface por el siguiente.

```
internal interface IUserLoginInputPort
{
    Task Handle(UserCredentialsDto userData);
}
```

Definir el OutputPort

1. Agrega un nuevo archivo de tipo *Interface* llamado ***IUserLoginOutputPort.cs*** en el directorio *UserLogin*.
2. Remplaza el código de la interface por el siguiente.

```
internal interface IUserLoginOutputPort
```

```
{
    IResult Result { get; }
    Task Handle(Result<UserDto, IEnumerable<ValidationError>> userLoginResult);
}
```

Implementar el Controlador

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *Controllers*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginController.cs*** en el directorio *UserLogin* y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.AspNetCore.Builder;
internal static class UserLoginController
{
    public static WebApplication UseUserLoginController(
        this WebApplication app)
    {
        app.MapPost(Endpoints.Login,
            async (UserCredentialsDto userCredentials,
            IUserLoginInputPort inputPort,
            IUserLoginOutputPort presenter) =>
            {
                await inputPort.Handle(userCredentials);
                return presenter.Result;
            });

        return app;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar el endpoint

1. Abre el archivo *EndpointsContainer.cs*.
2. Agrega el siguiente código en el método *UseMembershipEndpoints* para registrar el nuevo endpoint.

```
app.UseUserLoginController();
```

Agregar un archivo de recursos

1. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserLoginMessages.resx*** en el directorio *Resources* con la siguiente información.

Nombre	Valor
InvalidUserCredentialsErrorMessage	Las credenciales de usuario proporcionadas son incorrectas.

Implementar InputPort

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *UseCases*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginInteractor.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
internal class UserLoginInteractor(
    IMembershipService membershipService,
    IUserLoginOutputPort presenter,
    IModelValidatorHub<UserCredentialsDto> validationService)
    : IUserLoginInputPort
{
    public async Task Handle(UserCredentialsDto userData)
    {
        Result<UserDto, IEnumerable<ValidationError>> Result;

        if (!await validationService.Validate(userData))
        {
            Result = new(validationService.Errors);
        }
        else
        {
            var User = await membershipService.GetUserByCredentials(userData);
            if (User == null)
            {
                Result = new(
                    [new ValidationError(
                        nameof(userData.Password),
                        UserLoginMessages
                        .InvalidUserCredentialsErrorMessage)
                    ]);
            }
            else
            {
                Result = new(User);
            }
        }

        await presenter.Handle(Result);
    }
}
```

Registrar el InputPort

1. Abre el archivo *DependencyContainer.cs*.

2. Agrega el siguiente código en el método *AddMembershipCoreServices* para registrar la implementación del interactor del caso de uso.

```
services.AddScoped<IUserLoginInputPort,
    UserLoginInteractor>();
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Implementar la funcionalidad del Presenter

Agreguemos ahora el código para implementar la funcionalidad del *OutputPort*. Esto incluirá el código necesario para generar el token de autorización del usuario.

Agregar un archivo de recursos

1. Agrega los siguientes valores en el archivo ***UserLoginMessages.resx*** del directorio *Resources*.

Nombre	Valor
UserLoginErrorTitle	Error de inicio de sesión.
UserLoginErrorDetail	El correo o la contraseña son incorrectos.

Agregar opciones del token

1. Agrega un nuevo directorio llamado ***Options*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***JwtOptions.cs*** en el directorio *Options* y remplaza el contenido del archivo por el siguiente.

```
public class JwtOptions
{
    public const string SectionKey = nameof(JwtOptions);
    public string SecurityKey { get; set; }
    public string ValidIssuer { get; set; }
    public string ValidAudience { get; set; }
    public int ExpireInMinutes { get; set; }
}
```

Implementar el servicio de Tokens

1. Agrega el paquete NuGet ***Microsoft.IdentityModel.JsonWebTokens*** al proyecto. Este paquete incluye los tipos que proporcionan soporte para crear, serializar y validar *Tokens Web JSON* (*JSON Web Tokens*).

2. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
3. Agrega un nuevo archivo de clase llamado ***JwtService.cs*** en el directorio *Services* y remplaza el contenido de la clase por el siguiente.

```
internal class JwtService(IOptions<JwtOptions> options)
{
}
```

4. Agrega el siguiente código a la clase para definir un método que obtenga la información necesaria para firmar el token.

```
SigningCredentials GetSigningCredentials()
{
    var Key = Encoding.UTF8.GetBytes(
        options.Value.SecurityKey);

    var Secret = new SymmetricSecurityKey(Key);

    return new SigningCredentials(Secret,
        SecurityAlgorithms.HmacSha256);
}
```

La llave para firmar el token estará almacenada como una cadena de caracteres de la cual obtendremos sus bytes para crear el *Secret*. El algoritmo que utilizaremos para firmar el token será *HMACSHA256*.

5. Agrega el siguiente código a la clase para definir el método que devuelva la lista de *Claims* con la información de usuario que será incluida en el token.

```
List<Claim> GetClaims(UserDto userDto) =>
    [
        new Claim(ClaimTypes.Name, userDto.Email),
        new Claim("FullName",
            $"{userDto.FirstName} {userDto.LastName}")
    ];
```

Puedes notar que aquí estamos creando el *Claim FullName* que almacenará el nombre completo del usuario.

6. Agrega el siguiente código a la clase para definir el método que devuelva la información que será utilizada para crear el token de seguridad.

```
SecurityTokenDescriptor GetTokenDescriptor(
    SigningCredentials signingCredentials,
    List<Claim> claims) =>
```

```
        new SecurityTokenDescriptor
        {
            Subject = new ClaimsIdentity(claims),
            Issuer = options.Value.ValidIssuer,
            Audience = options.Value.ValidAudience,
            Expires = DateTime.UtcNow.AddMinutes(
                options.Value.ExpireInMinutes),
            SigningCredentials = signingCredentials
        };
```

7. Agrega el siguiente código a la clase para permitir devolver el token asociado al usuario.

```
public string GetToken(UserDto userData)
{
    var SigningCredentials = GetSigningCredentials();
    var Claims = GetClaims(userData);
    var TokenDescriptor = GetTokenDescriptor(SigningCredentials, Claims);

    var TokenHandler = new JsonWebTokenHandler();
    return TokenHandler.CreateToken(TokenDescriptor);
}
```

8. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Implementar IUserLoginOutputPort

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *Presenters*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginPresenter.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
internal class UserLoginPresenter(JwtService service) : IUserLoginOutputPort
{
    public IResult Result { get; private set; }

    public Task Handle(
        Result<UserDto, IEnumerable<ValidationError>> userLoginResult)
    {
        userLoginResult.HandleResult(
            userDto =>
            {
                Result = Results.Ok(new TokensDto(
                    service.GetToken(userDto)));
            },
            errors =>
            {
                Result = Results.Problem(
                    errors.ToProblemDetails(
                        UserLoginMessages.UserLoginErrorTitle,
                        UserLoginMessages.UserLoginErrorDetail,
                        nameof(UserLoginPresenter)));
```

```
            });

        return Task.CompletedTask;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar las dependencias

1. Abre el archivo *DependencyContainer.cs*.
2. Modifica el código del método *AddMembershipCoreServices* para registrar las nuevas dependencias. El código será similar al siguiente.

```
public static IServiceCollection AddMembershipCoreServices(
    this IServiceCollection services,
    Action<JwtOptions> configureJwtOptions)
{

    services.AddMembershipValidators();
    services.AddDefaultModelValidatorHub();

    services.AddScoped<IUserRegistrationInputPort,
        UserRegistrationInteractor>();

    services.AddScoped<IUserLoginInputPort,
        UserLoginInteractor>();

    services.AddScoped<IUserRegistrationOutputPort,
        UserRegistrationPresenter>();


    services.AddScoped<IUserLoginOutputPort,
        UserLoginPresenter>();

    services.AddSingleton<JwtService>();

    services.Configure(configureJwtOptions);


    return services;
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Actualizar el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Backend.Core*.

3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Implementar el servicio de administración de usuarios

Trabajemos ahora en el proyecto ***NorthWind.Membership.Backend.AspNetIdentity*** para implementar la nueva funcionalidad requerida del servicio de administración de usuarios.

Reinstalar el paquete NorthWind.Membership.Backend.Core

1. En la ventana ***Solution Explorer,*** expande el nodo ***Dependencies*** del proyecto *NorthWind.Membership.Backend.AspNetIdentity*.
2. Expande el elemento ***Packages***.
3. Selecciona la opción ***Open Folder in File Explorer*** del menú contextual del paquete *NorthWind.Membership.Backend.Core* para abrir el explorador de Windows. Se mostrarán los archivos en cache del paquete.
4. Elimina los archivos del directorio.
5. Cierra el explorador de Windows y regresa a Visual Studio.
6. Elimina y vuelve a agregar el paquete *NorthWind.Membership.Backend.Core*.

Agregar la funcionalidad para inicio de sesión

1. Abre el archivo *MembershipService.cs*.
2. Agrega el siguiente código al final de la clase *MembershipService*.

```
public async Task<UserDto> GetUserByCredentials(
    UserCredentialsDto userData)
{
    UserDto FoundUser = null;

    var User = await manager.FindByNameAsync(userData.Email);
    if(User != null &&
        await manager.CheckPasswordAsync(User, userData.Password))
    {
        FoundUser = new UserDto(User.UserName, User.FirstName,
            User.LastName);
    }

    return FoundUser;
}
```

El código anterior utiliza el método *FindByNameAsync* de *UserManager* para buscar un usuario con el correo especificado. En caso de que se encuentre un usuario con el correo especificado, utilizamos el método *CheckPasswordAsync* para verificar que la clave de acceso proporcionada sea la correcta.

Si el correo y la clave de acceso son correctas, devolvemos el correo (*UserName*), nombre y apellidos del usuario.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Actualizar el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Backend.AspNetIdentity*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Incorporar la funcionalidad backend de membresía

Trabajemos ahora en el proyecto ***NorthWind.Sales.Backend.IoC*** de la solución NorthWind.

Reinstalar el paquete NorthWind.Membership.Backend.AspNetIdentity

1. Abre la solución *NorthWind*.
2. En la ventana ***Solution Explorer,*** expande el nodo ***Dependencies*** del proyecto *NorthWind.Sales.Backend.IoC*.
3. Expande el elemento ***Packages***.
4. Selecciona la opción ***Open Folder in File Explorer*** del menú contextual del paquete *NorthWind.Membership.Backend.AspNetIdentity* para abrir el explorador de Windows. Se mostrarán los archivos en cache del paquete.
5. Elimina los archivos del directorio.
6. Cierra el explorador de Windows y regresa a Visual Studio.
7. Elimina y vuelve a agregar el paquete *NorthWind.Membership.Backend.AspNetIdentity*.

Actualizar el contenedor de dependencias

1. Abre el archivo *DependencyContainer.cs* del proyecto *NorthWind.Sales.Backend.IoC*.
2. Modifica el código del método *AddNorthWindSalesServices* para registrar los nuevos servicios. El código será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<DBOptions> configureDBOptions,
    Action<SmtpOptions> configureSmtpOptions,
    Action<MembershipDBOptions> configureMembershipDBOptions,
    Action<JwtOptions> configureJwtOptions)
{
    services.AddUseCasesServices()
        .AddRepositories()
        .AddDataContexts(configureDBOptions)
        .AddPresenters()
        .AddValidationService()
        .AddValidators()
        .AddValidationExceptionHandler()
        .AddUpdateExceptionHandler()
        .AddUnauthorizedAccessExceptionHandler()
        .AddUnhandledExceptionHandler()
        .AddEventServices()
        .AddMailServices(configureSmtpOptions)
        .AddDomainLogsServices()
        .AddTransactionServices()
        .AddUserServices()
```

```
        .AddMembershipCoreServices(configureJwtOptions)
        .AddMembershipIdentityServices(configureMembershipDBOptions);

    return services;
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Actualizar el proyecto Web API

Realicemos algunos cambios en el proyecto *NorthWind.Sales.WebApi* para incorporar la nueva funcionalidad.

Modificar el registro de los servicios

1. Abre el archivo *Startup.cs*.
2. Localiza el siguiente código.

```
builder.Services.AddNorthWindSalesServices(
    dbObtions =>
        builder.Configuration.GetSection(DBOptions.SectionKey)
        .Bind(dbObtions),
    smtpOptions =>
        builder.Configuration.GetSection(SmtpOptions.SectionKey)
        .Bind(smtpOptions),
    membershipDBOptions =>
        builder.Configuration.GetSection(MembershipDBOptions.SectionKey)
        .Bind(membershipDBOptions));
```

3. Modifica el código anterior por el siguiente para proporcionar el argumento faltante.

```
builder.Services.AddNorthWindSalesServices(
    dbObtions =>
        builder.Configuration.GetSection(DBOptions.SectionKey)
        .Bind(dbObtions),
    smtpOptions =>
        builder.Configuration.GetSection(SmtpOptions.SectionKey)
        .Bind(smtpOptions),
    membershipDBOptions =>
        builder.Configuration.GetSection(MembershipDBOptions.SectionKey)
        .Bind(membershipDBOptions),
    jwtOptions =>
        builder.Configuration
        .GetSection(JwtOptions.SectionKey)
        .Bind(jwtOptions));
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar la configuración para el Token Web JSON

1. Abre el archivo *appsettings.json*.
2. Agrega el siguiente código para definir los valores de configuración para el *Token Web JSON*.

```
"JwtOptions": {
  "SecurityKey": "KeyMustBeAtLeast32CharactersLong",
  "ValidIssuer": "NorthWindSales",
  "ValidAudience": "NorthWindSalesClient",
  "ExpireInMinutes": 1440
}
```

El valor de *SecurityKey* debe ser de al menos 32 caracteres.

ValidIssuer contiene el valor que identifica al emisor del token

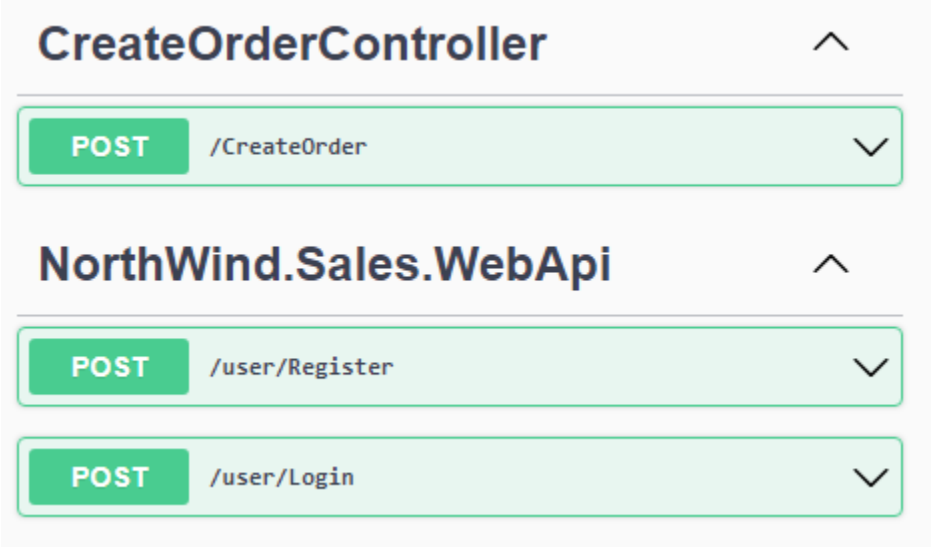
ValidAudience contiene el valor que identifica la audiencia para quién fue creado el token.

ExpireInMinutes contiene el valor del tiempo de validez del token expresado en minutos.

Probar la funcionalidad

Probemos ahora la nueva funcionalidad que permitirá obtener el token de acceso a partir de las credenciales de un usuario.

1. Ejecuta la aplicación. Podrás ver el nuevo endpoint que hemos agregado.

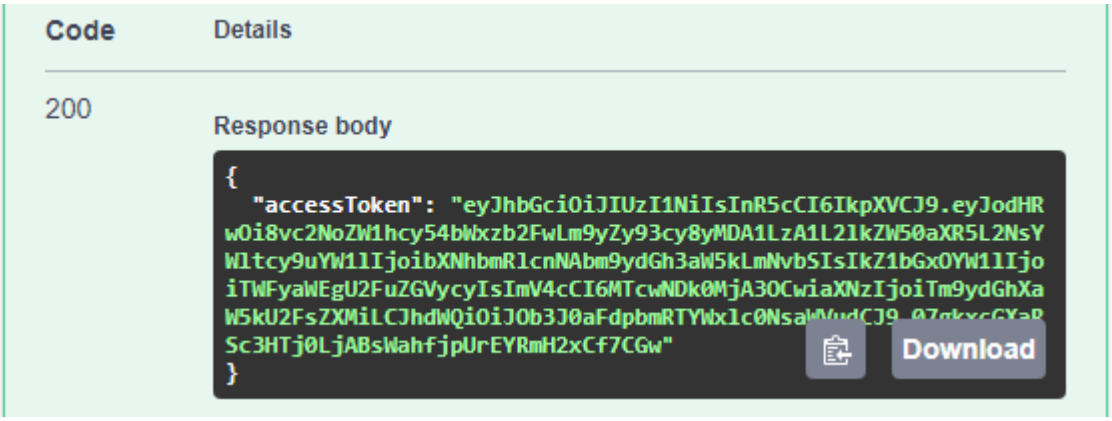


Probar con datos correctos

1. En la página *Index* de la aplicación Web API intenta iniciar sesión utilizando los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "Mm1234567$"
}
```

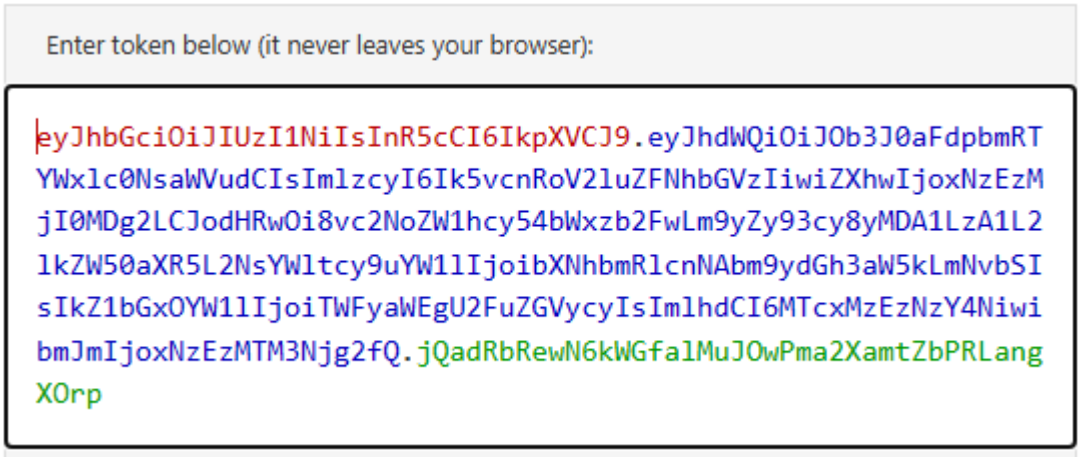
Podrás ver un resultado similar al siguiente mostrando el token devuelto.



2. Copia el valor del token devuelto sin incluir las comillas del principio y el final.
3. Pega el valor copiado en la siguiente página.

   https://jwt.ms

2. Podrás ver un resultado similar al siguiente.



Decoded Token | Claims


```
{
  "alg": "HS256",
  "typ": "JWT"
}.{
  "aud": "NorthWindSalesClient",
  "iss": "NorthWindSales",
  "exp": 1713224086,

"http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name":
"msanders@northwind.com",
  "FullName": "Maria Sanders",
  "iat": 1713137686,
  "nbf": 1713137686
}.[Signature]
```

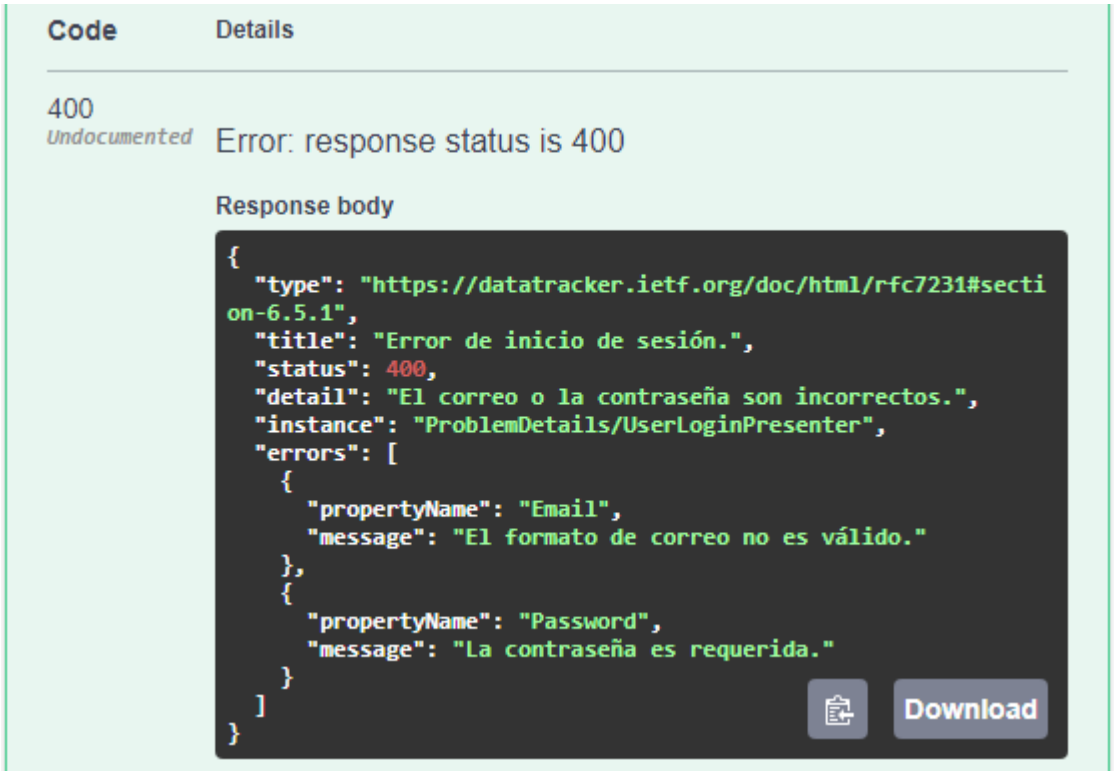

Puedes notar que se incluye en el token el correo del usuario y su nombre completo junto con información adicional del Token.

Probar con datos incorrectos

1. En la página *Index* de la aplicación Web API intenta iniciar sesión utilizando los siguientes datos.

```
{
    "email": "msanders ",
    "password": ""
}
```

Podrás ver un resultado similar al siguiente.

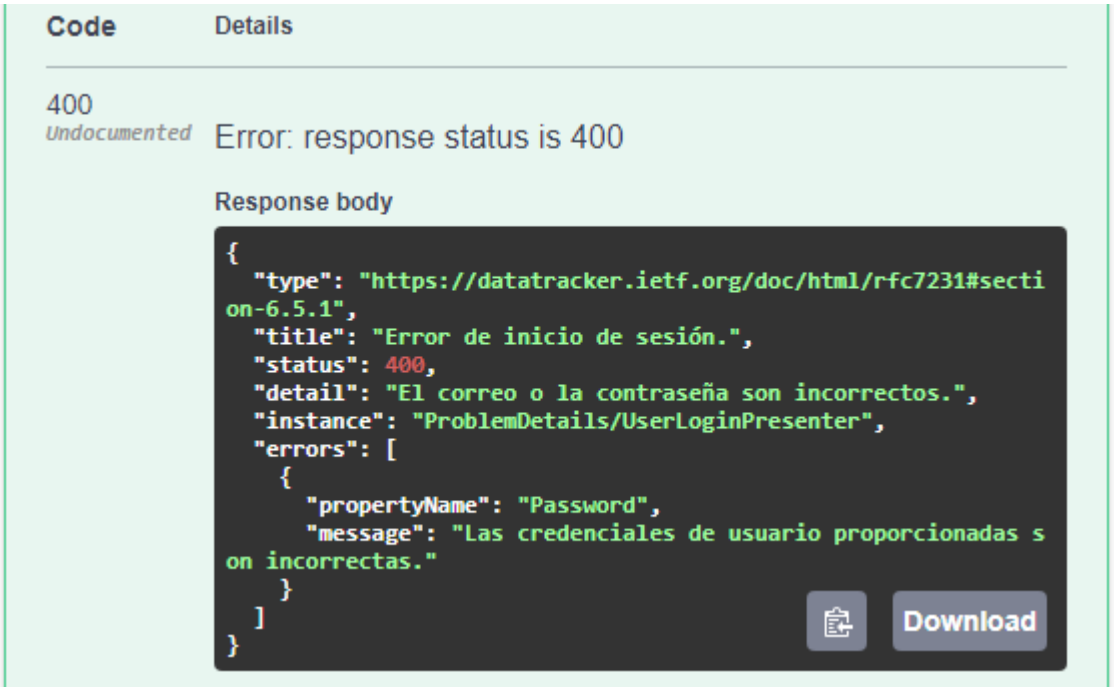


Probar con contraseña incorrecta

1. En la página *Index* de la aplicación Web API intenta iniciar sesión utilizando los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "12345"
}
```

Podrás ver un resultado similar al siguiente.



2. Regresa a Visual Studio y detén la ejecución.

Implementar la funcionalidad frontend del framework de membresía

Implementemos ahora el código para el inicio de sesión de la aplicación *Blazor* en el proyecto *NorthWind.Membership.Frontend.RazorViews*.

Modificar el Gateway de los endpoints de administración de usuarios

1. Abre la solución *NorthWind.Membership*.
2. Abre el archivo *MembershipGateway.cs*.
3. Agrega el siguiente código a la clase.

```
public async Task<TokensDto> LoginAsync(UserCredentialsDto userCredentials)
{
    var Response = await client.PostAsJsonAsync(Endpoints.Login,
        userCredentials);
    return await Response.Content.ReadFromJsonAsync<TokensDto>();
}
```

4. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar un archivo de recursos

1. Agrega un nuevo archivo de tipo *Resources File* llamado ***UserLoginMessages.resx*** en el directorio *Resources* con el siguiente contenido.

Nombre	Valor
LoginButtonText	Iniciar sesión

Agregar el ViewModel

1. Agrega un nuevo directorio llamado ***UserLogin*** en el directorio *ViewModels*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginViewModel.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
public class UserLoginViewModel(
    MembershipGateway gateway,
    IModelValidatorHub<UserLoginViewModel> validator)
{
    public IModelValidatorHub<UserLoginViewModel> Validator => validator;

    public ModelValidator<UserLoginViewModel> ModelValidatorComponentReference
    { get; set; }
```

```
    public event Action OnLogin;

    #region Datos de entrada
    public string Email { get; set; }
    public string Password { get; set; }
    #endregion

    public async Task Login()
    {
        try
        {
            TokensDto Tokens =
                await gateway.LoginAsync((UserCredentialsDto)this);
            Console.WriteLine(Tokens.AccessToken);
            OnLogin?.Invoke();
        }
        catch (HttpRequestException ex)
        {
            if (ex.Data.Contains("Errors"))
            {
                var Errors = ex.Data["Errors"] as
                    IEnumerable<ValidationError>;
                ModelValidatorComponentReference.AddErrors(Errors);
            }
            else
            {
                throw;
            }
        }
    }

    public static explicit operator UserCredentialsDto(
        UserLoginViewModel model) =>
        new UserCredentialsDto(model.Email, model.Password);
}
```

Utilizaremos esta clase para que el usuario pueda proporcionar sus credenciales de inicio de sesión. Puedes notar que estamos definiendo el evento ***OnLogin*** que será lanzado cuando se obtenga una respuesta exitosa de la Web API indicando que las credenciales proporcionadas por el usuario son correctas. La API devolverá el token de acceso que podrá ser utilizado para autenticar al usuario en la aplicación *Blazor* y para poder ser utilizado al acceder a los recursos de la Web API. Esta funcionalidad será implementada en un ejercicio posterior.

Agregar la validación

1. Agrega un nuevo archivo de clase llamado ***UserLoginViewModelDtoValidator.cs*** en el directorio *UserLogin* y remplaza el código de la clase por el siguiente.

```
internal class UserLoginViewModelDtoValidator(
    IModelValidatorHub<UserCredentialsDto> validator) :
```

```
    AbstractViewModelValidator<UserCredentialsDto, UserLoginViewModel>(
        validator, ValidationConstraint.AlwaysValidate)
{
}
```

La clase anterior simplemente encapsula el validador del DTO *UserCredentialsDto* asociado.

Registrar las dependencias

1. Abre el archivo *DependencyContainer.cs*.
2. Agrega el siguiente código en el método *AddMembershipServices* para registrar el ViewModel y el validador.

```
services.AddScoped<UserLoginViewModel>();
services.AddModelValidator<UserLoginViewModel,
    UserLoginViewModelDtoValidator>();
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Crear el componente de inicio de sesión

1. Agrega un nuevo componente razor llamado ***UserLoginComponent.razor*** en el directorio *Components*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginComponent.razor.cs*** en el directorio *Components* y remplaza el código de la clase por el siguiente.

```
public partial class UserLoginComponent
{
      [Parameter]
      public UserLoginViewModel ViewModel { get; set; }
}
```

3. Remplaza el código del archivo *UserLoginComponent.razor* por el siguiente para definir la interfaz de usuario que permita al usuario proporcionar sus datos de inicio de sesión.

```
<EditForm Model="ViewModel" OnValidSubmit="ViewModel.Login">
      <ModelValidator Validator="ViewModel.Validator"
            @ref=ViewModel.ModelValidatorComponentReference />
      <ValidationSummary />

      <div class="form-group">
            <label for="Email">
                  @CommonMessages.EmailLabel
            </label>
```

```
            <InputText id="Email"
                   class="form-control"
                   placeholder="@CommonMessages.EmailLabel"
                   @bind-Value=ViewModel.Email />
            <ValidationMessage For="@(() => ViewModel.Email)" />
      </div>

      <div class="form-group">
            <label for="Password">
                   @CommonMessages.PasswordLabel
            </label>
            <InputText id="Password"
                   class="form-control"
                   placeholder="@CommonMessages.PasswordLabel"
                   type="password"
                   @bind-Value=ViewModel.Password />
            <ValidationMessage For="@(() => ViewModel.Password)" />
      </div>

      <div class="row p-3">
            <div class="col">
                   <button class="btn btn-primary">
                         @UserLoginMessages.LoginButtonText
                   </button>
            </div>
      </div>
</EditForm>
```

Agregar la página de inicio de sesión

1. Agrega un nuevo componente razor llamado ***UserLoginPage.razor*** en el directorio *Pages*.
2. Agrega un nuevo archivo de clase llamado ***UserLoginPage.razor.cs*** en el directorio *Pages* y remplaza el código de la clase por el siguiente.

```
public partial class UserLoginPage
{
    // Plantilla de ruta de la página
    const string RouteTemplate = "/user/login";

    [Inject]
    UserLoginViewModel ViewModel { get; set; }

    [Inject]
    NavigationManager NavigationManager { get; set; }


    ErrorBoundary ErrorBoundaryRef;

    void Recover()
    {
        ErrorBoundaryRef?.Recover();
    }

    protected override void OnInitialized()
```

```
    {
        ViewModel.OnLogin += OnLogin;
    }

    void OnLogin()
    {
        if (NavigationManager.Uri.EndsWith(RouteTemplate))
        {
            // Si se navegó explícitamente a la página 'user/login',
            // después de iniciar sesión navegar a la raíz.
            NavigationManager.NavigateTo("");
        }
        else
        {
            // El usuario fue dirigido a la página '/user/login',
            // navegar a la página original
            NavigationManager.NavigateTo(NavigationManager.Uri);
        }
    }
}
```

3. Remplaza el contenido del archivo *UserLoginPage.razor* por el siguiente para implementar la interfaz de usuario de la página de inicio de sesión.

```
@attribute [Route(RouteTemplate)]

<ErrorBoundary @ref=ErrorBoundaryRef>
    <ChildContent>
        <UserLoginComponent ViewModel="ViewModel"/>
    </ChildContent>
    <ErrorContent>
        <ExceptionDisplay Exception="context"/>
        <button class="btn btn-danger" @onclick=Recover>
            @CommonMessages.OK
        </button>
    </ErrorContent>
</ErrorBoundary>
```

En este ejemplo, especificamos la plantilla de ruta del componente mediante el atributo ***Route.*** La plantilla de ruta la hemos definido con la constante *RouteTemplate*.

Actualizar el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Frontend.RazorViews*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Incorporar la funcionalidad frontend de membresía

Trabajemos ahora en el proyecto ***NorthWind.Sales.Frontend.IoC*** de la solución *NorthWind*.

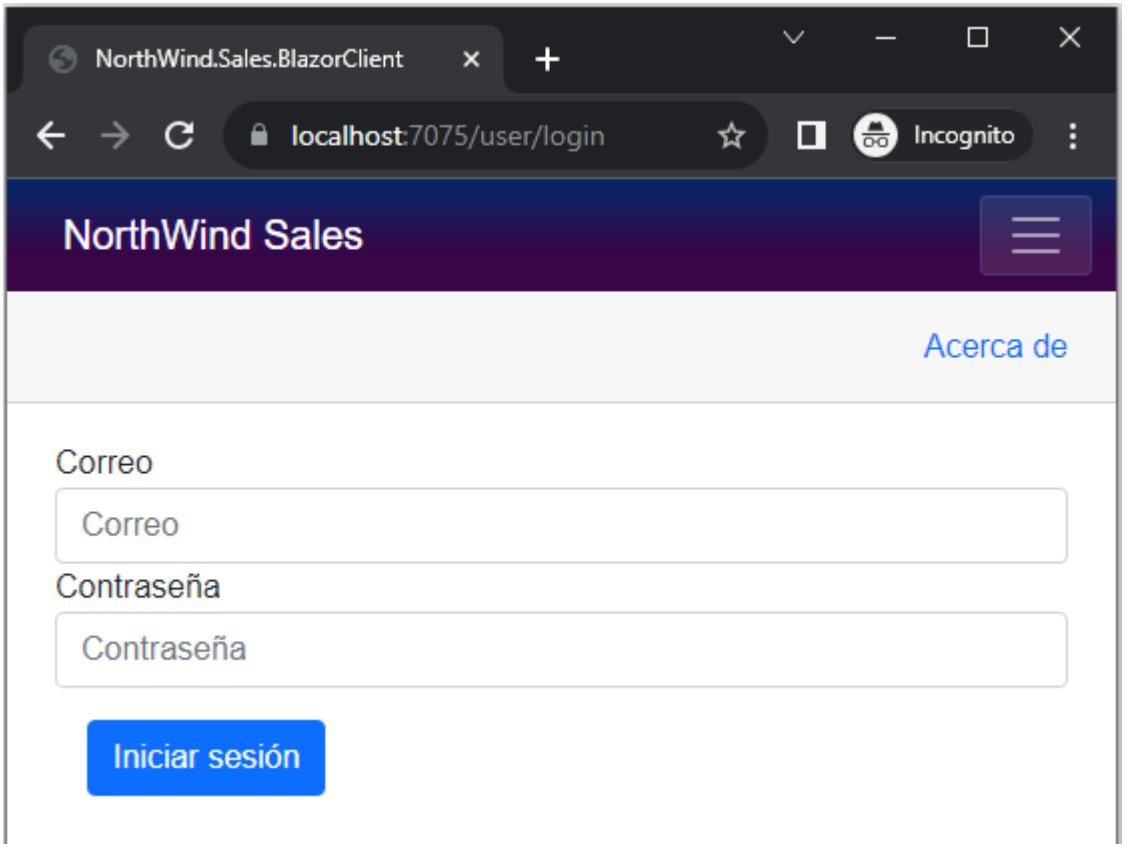
Reinstalar el paquete NorthWind.Membership.Frontend.RazorViews

1. Abre la solución *NorthWind*.
2. En la ventana ***Solution Explorer,*** expande el nodo ***Dependencies*** del proyecto *NorthWind.Sales.Frontend.IoC*.
3. Expande el elemento ***Packages***.
4. Selecciona la opción ***Open Folder in File Explorer*** del menú contextual del paquete *NorthWind.Membership.Frontend.RazorViews* para abrir el explorador de Windows. Se mostrarán los archivos en cache del paquete.
5. Elimina los archivos del directorio.
6. Cierra el explorador de Windows y regresa a Visual Studio.
7. Elimina y vuelve a agregar el paquete *NorthWind.Membership.Frontend.RazorViews*.

Probar la funcionalidad

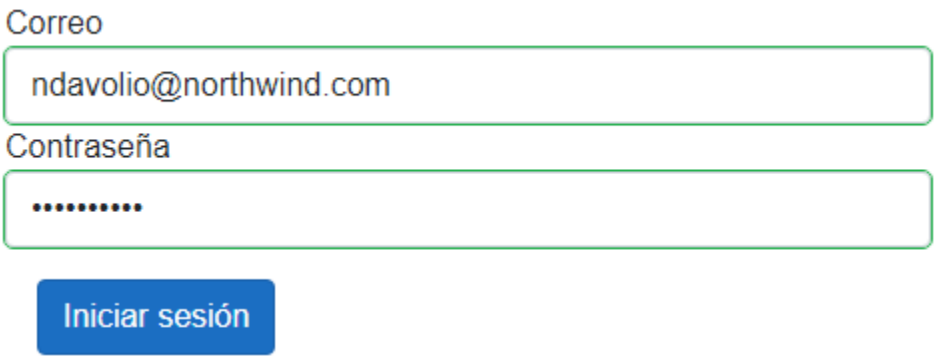
Podemos ahora verificar la funcionalidad de inicio de sesión de usuarios.

1. Ejecuta la aplicación.
2. Navega a la Uri ***/user/login*** de la aplicación *Blazor*. Podrás ver una página similar a la siguiente.

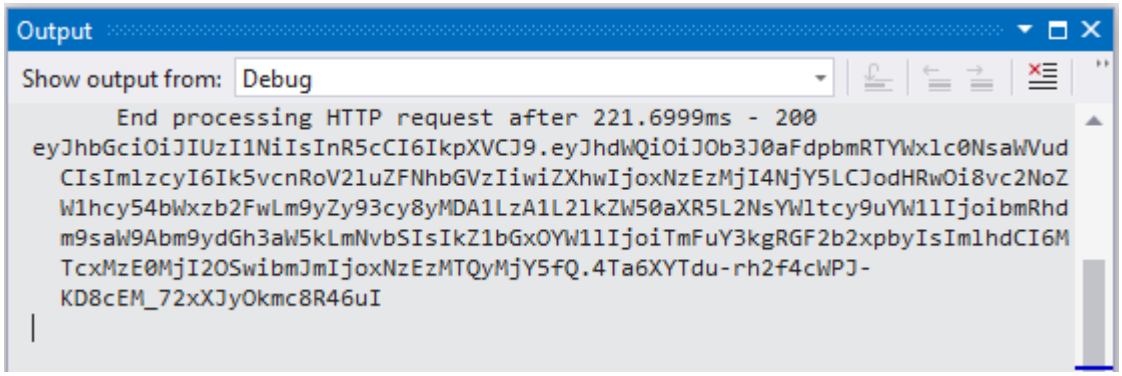


Probar con credenciales válidas

1. Proporciona las credenciales correctas de un usuario registrado y haz clic en el botón ***Iniciar sesión***.

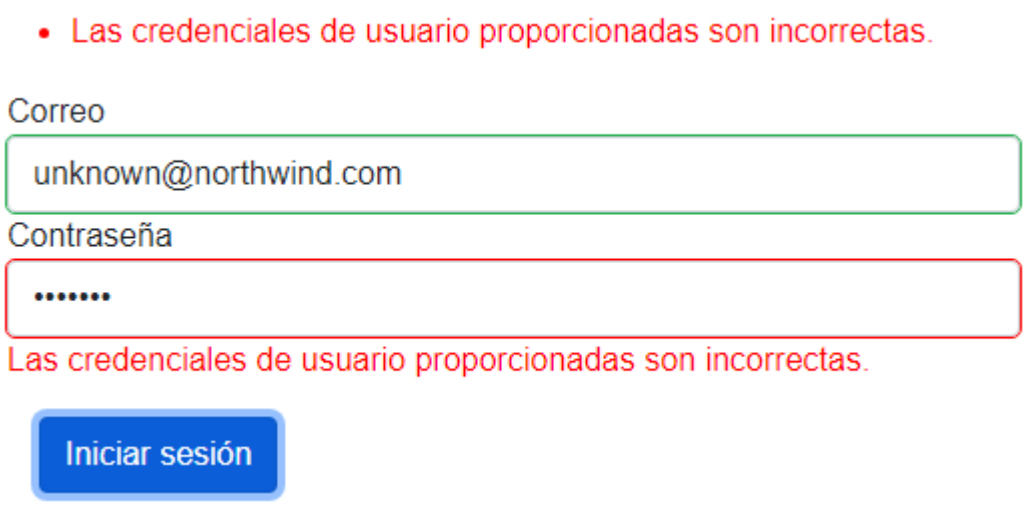


Después de haber proporcionado las credenciales correctas y hacer clic en el botón de inicio de sesión, serás dirigido a la página principal. En la ventana ***Output*** de Visual Studio, podrás ver el token de acceso obtenido similar al siguiente.



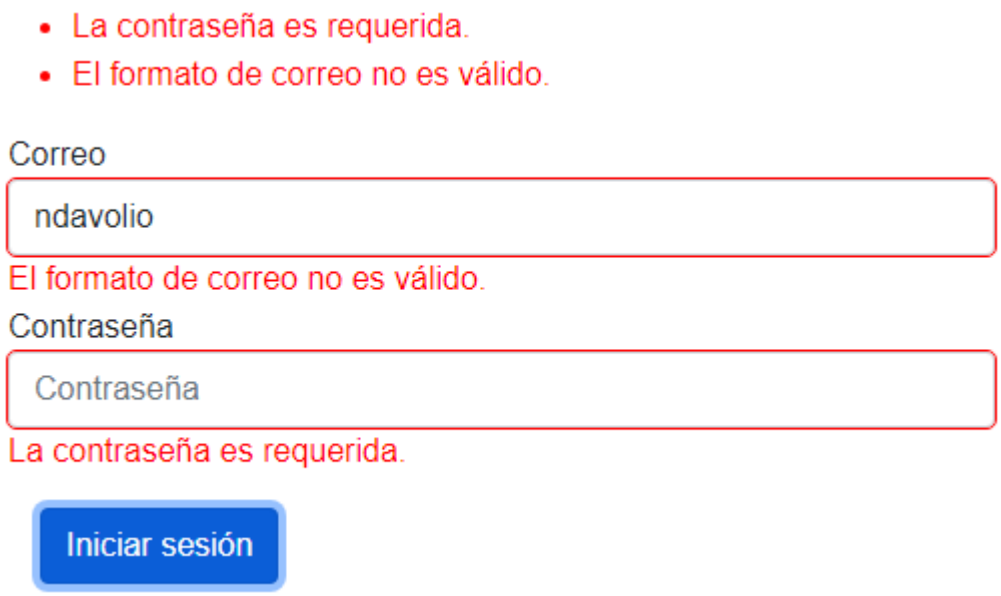
Probar con credenciales incorrectas

1. Intenta iniciar sesión proporcionando una cuenta de usuario no existente o con una clave de acceso incorrecta. Podrás ver una respuesta similar a la siguiente.



Probar con errores de validación

1. Intenta iniciar sesión con un formato de correo no válido y sin contraseña. Podrás ver una respuesta similar a la siguiente.



2. Regresa a Visual Studio y detén la ejecución.

Hemos implementado el caso de uso que permite al usuario iniciar sesión y obtener el token de acceso. Podemos continuar ahora con la configuración de los servicios de autenticación y autorización de *ASP.NET Core*.

Práctica 16:

Configurar los servicios de autenticación y autorización

Para que un usuario pueda acceder a recursos protegidos en la API Web, este deberá enviar el token de acceso durante la petición del recurso.

En esta práctica configuraremos los servicios de autenticación y autorización de *ASP.NET Core* para que el usuario pueda acceder a los recursos protegidos de la aplicación. El esquema de autenticación que utilizaremos será *JWT Bearer*.

Objetivos

Al finalizar esta práctica los estudiantes:

- Configurarán los servicios de autenticación y autorización de *ASP.NET Core* en la *API Web*.
- Configurarán el servicio *Swagger* para poder acceder a recursos protegidos de la API Web utilizando el token de acceso.
- Configurarán los servicios de autenticación y autorización de *ASP.NET Core* en la aplicación *Blazor WebAssembly.*

Configurar los servicios en el backend

Empecemos por configurar los servicios de autenticación y autorización en el código backend.

Agregar el proveedor de autenticación

Hemos implementado el caso de uso de inicio de sesión del usuario para obtener un token de acceso. Ahora, cuando el usuario quiera acceder a un recurso protegido en la API Web, deberá enviar el token en la petición. El token será analizado por el proveedor de autenticación que configuraremos a continuación.

Realiza los siguientes pasos en el proyecto *NorthWInd.Sales.WebApi.*

1. Agrega el paquete NuGet ***Microsoft.AspNetCore.Authentication.JwtBearer***. Este paquete contiene el middleware de *ASP.NET Core* que permite que una aplicación pueda recibir un token de seguridad conocido como *Bearer Token*.
2. Abre el archivo *Startup.cs*.
3. Agrega el siguiente código antes de la última línea de código del método *CreateWebApplication*.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        // Establecer la configuración del Token.
        builder.Configuration.GetSection(JwtOptions.SectionKey)
            .Bind(options.TokenValidationParameters);

        // Establecer la llave para validación de la firma.
        string SecurityKey = builder.Configuration
            .GetSection(JwtOptions.SectionKey)[nameof(JwtOptions.SecurityKey)];

        byte[] SecurityKeyBytes = Encoding.UTF8.GetBytes(SecurityKey);

        options.TokenValidationParameters.IssuerSigningKey =
            new SymmetricSecurityKey(SecurityKeyBytes);
    });
```

El código anterior registra los servicios requeridos por los servicios de autenticación. El argumento *JwtBearerDefaults.AuthenticationScheme* especifica el nombre del esquema que será utilizado de manera predeterminada cuando no se especifique un esquema durante una acción de autenticación y/o autorización.

El método ***AddJwtBearer*** habilita la autenticación *JWT-bearer* utilizando el esquema predeterminado *JwtBearerDefaults.AuthenticationScheme*.

4. Agrega el siguiente código después del código anterior para registrar los servicios de políticas de autorización.

```
builder.Services.AddAuthorization();
```

5. Agrega el siguiente código antes de la última línea de código del método *ConfigureWebApplication*.

```
app.UseAuthentication();
app.UseAuthorization();
```

El código anterior agrega los middlewares de autenticación y autorización respectivamente.

6. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Agregar valores de configuración

1. Abre el archivo *appsettings.json*.
2. Agrega los siguientes valores en la sección de configuración *JwtOptions*.

```
"JwtOptions": {
  "SecurityKey": "KeyMustBeAtLeast32CharactersLong",
  "ValidIssuer": "NorthWindSales",
  "ValidAudience": "NorthWindSalesClient",
  "ExpireInMinutes": 1440,
  "ValidateIssuer": true,
  "ValidateAudience": true,
  "ValidateLifetime": true,
  "ValidateIssuerSigningKey": true
}
```

El código anterior hablita las opciones del token que deben validarse.

Configurar el servicio Swagger

Configuremos ahora el servicio de *Swagger* para habilitar el soporte de autenticación. Esto nos permitirá realizar pruebas de consumo de recursos protegidos proporcionando el token de acceso.

1. Agrega un nuevo directorio llamado ***Extensions*** en la raíz del proyecto *NorthWind.Sales.WebApi*.
2. Agrega un nuevo archivo de clase llamado ***SwaggerExtensions.cs*** en el directorio *Extensions* y remplaza el contenido del archivo por el siguiente.

```
namespace Microsoft.Extensions.DependencyInjection;
public static class SwaggerExtensions
{
    public static IServiceCollection AddSwaggerGenBearer(
        this IServiceCollection services)
    {
        services.AddSwaggerGen(options =>
        {
            // Agregar definición de seguridad.
            options.AddSecurityDefinition("BearerToken",
                new OpenApiSecurityScheme
                {
                    // Nombre del encabezado HTTP que se agregará en la petición.
                    Name = "Authorization",
                    Description = "Proporciona el valor del token.",
                    // Ubicación del Token
                    In = ParameterLocation.Header,
                    // Tipo del esquema de seguridad a utilizar.
                    // Al especificar HTTP, no necesitamos proporcionar
                    // la palabra bearer antes del token, solo necesitamos
                    // proporcionar el Token.
                    Type = SecuritySchemeType.Http,
                    // Nombre del esquema de autenticación.
                    // Para HTTP puede ser Basic o Bearer (no es sensible
                    // a mayúsculas/minúsculas)
                    Scheme = "Bearer"
                });

            // Para agregar el encabezado de autorización en las
            // peticiones de endpoints.
            options.AddSecurityRequirement(
                new OpenApiSecurityRequirement
                {
                    {
                        new OpenApiSecurityScheme
                        {
                            Reference = new OpenApiReference
                            {
                                Type = ReferenceType.SecurityScheme,
                                // Id definido en AddSecurityDefinition.
                                Id = "BearerToken"
                            }
                        },
                        // Solo se requiere en oauth2 o OIDC para enviar
                        // nombres de scopes.
                        new string[] { }
                    }
                });
        });

        return services;
    }
}
```

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.
4. Abre el archivo *Startup.cs*.

5. Localiza la siguiente línea de código en el método *CreateWebApplication*.

```
builder.Services.AddSwaggerGen();
```

6. Remplaza el código anterior por el siguiente.

```
builder.Services.AddSwaggerGenBearer();
```

Modificar el controlador UseCreateOrderController

1. Abre el archivo ***CreateOrderController.cs*** del proyecto ***NorthWind.Sales.Backend.Controllers***.
2. Modifica el código del método *UseCreateOrderController* para agregar la política de autorización predeterminada. Esto requerirá que el usuario esté autenticado para poder acceder al endpoint.

```
    public static WebApplication UseCreateOrderController(
        this WebApplication app)
    {
        app.MapPost(Endpoints.CreateOrder, CreateOrder)
            .RequireAuthorization();

        return app;
    }
```

Actualizar el registro del servicio IUserService

Realiza los siguientes pasos en el proyecto *NorthWind.UserServices.*

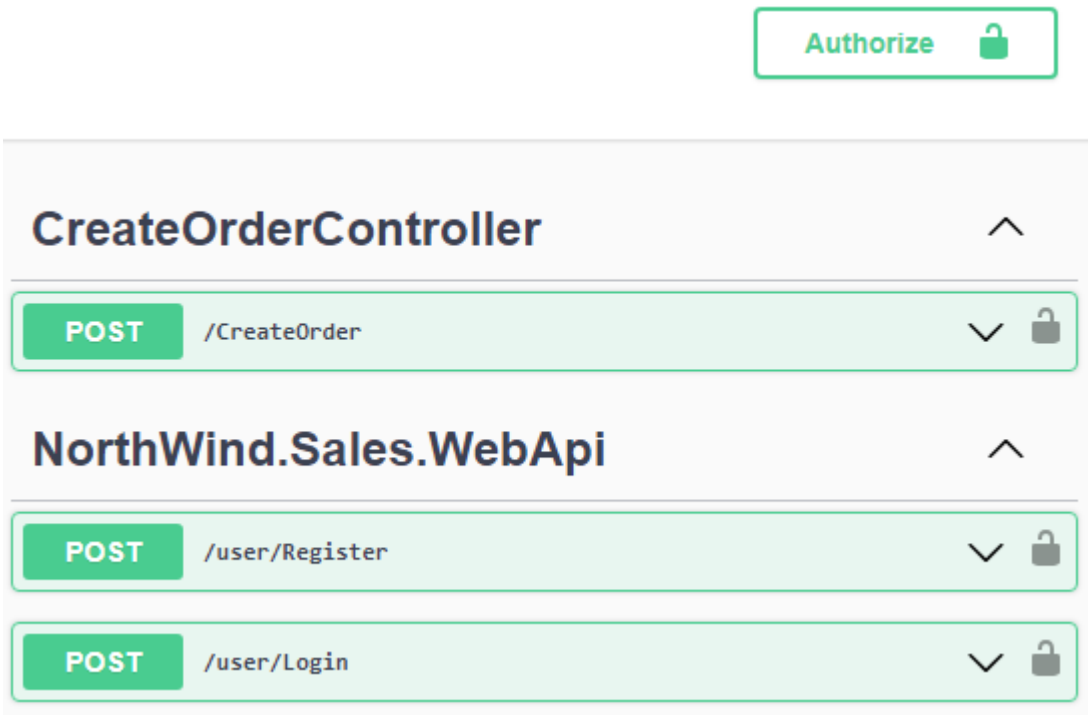
1. Abre el archivo *DependencyContainer.cs*.
2. Modifica el código del método *AddUserServices* para utilizar el tipo ***UserService*** en lugar del tipo ***UserServiceFake***.

```
services.AddSingleton<IUserService, UserService>();
```

Probar la funcionalidad

Podemos ahora probar la funcionalidad para verificar que el endpoint */CreateOrder* solo autorice el acceso a usuarios autenticados.

1. Ejecuta la aplicación. Podrás notar que se muestran los endpoints disponibles además del botón ***Authorize***.

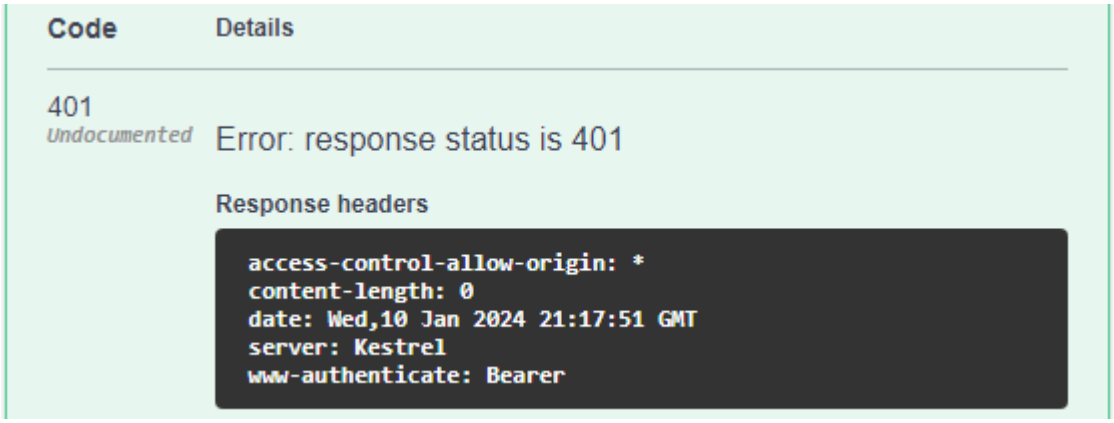


Probar el acceso sin autenticación

1. En la página *Index* de la aplicación Web API intenta crear una orden proporcionando los siguientes datos (aunque realmente podrías proporcionar cualquier información ya que el acceso no estará autorizado).

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        }
    ]
}
```

Podrás ver un resultado similar al siguiente mostrando el código de estado ***HTTP 401 Unauthorized***. Puedes notar además que la respuesta indica que se requiere un tipo de autenticación *Bearer*.

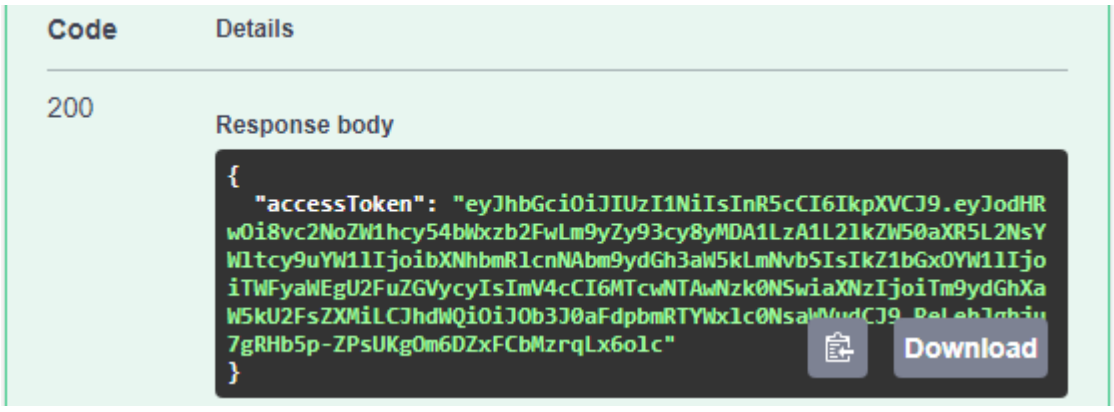


Probar el acceso con autenticación

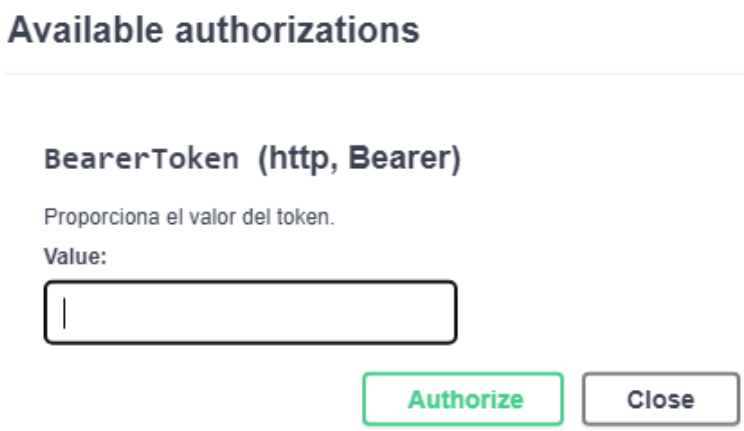
1. Intenta iniciar sesión utilizando los siguientes datos.

```
{
    "email": "msanders@northwind.com",
    "password": "Mm1234567$"
}
```

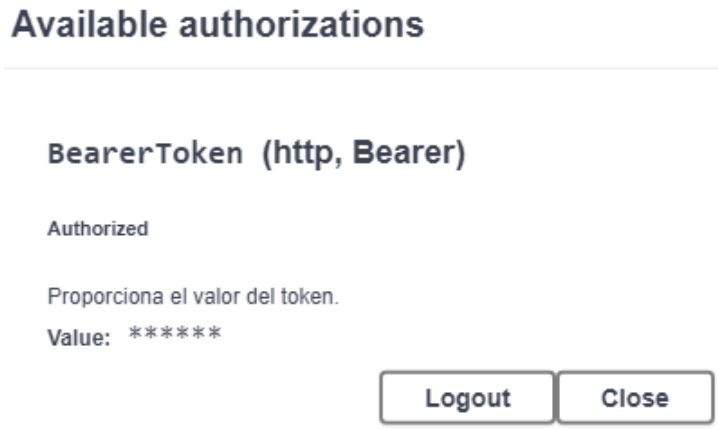
Podrás ver un resultado similar al siguiente mostrando el token devuelto.



2. Copia el valor del token.
3. Haz clic en el botón ***Authorize*** ubicado en la parte superior derecha de la página.
4. Se mostrará la ventana de autorizaciones disponibles.



5. En la ventana de autorizaciones disponibles, pega el valor del token en el cuadro de texto ***Value***.
6. Haz clic en el botón ***Authorize***. Se mostrará una ventana similar a la siguiente.

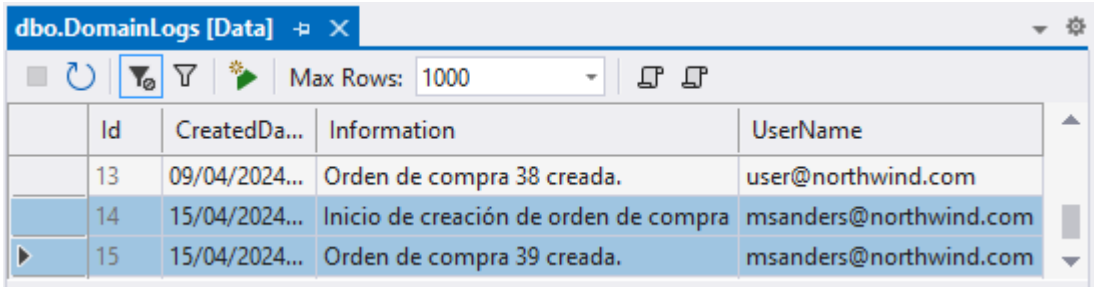


7. Haz clic en el botón ***Close***.
8. Intenta crear una orden proporcionando los siguientes datos.

```
{
    "customerId": "ALFKI",
    "shipAddress": "31 Oriente 810",
    "shipCity": "Puebla",
    "shipCountry": "México",
    "shipPostalCode": "72100",
    "orderDetails": [
        {
            "productId": 1,
            "unitPrice": 35,
            "quantity": 10
        }
    ]
}
```

Podrás ver un resultado similar al siguiente mostrando el número de orden creada.

La tabla ***DomainLogs*** de la base de datos deberá mostrar los nuevos registros creados que ahora incluyen el nombre del usuario que ha creado la orden.



9. Regresa a Visual Studio y detén la ejecución.

Implementar la funcionalidad frontend del framework de membresía

Hemos creado la interfaz de usuario de inicio de sesión para obtener un token de acceso. Ahora, cuando el usuario quiera acceder a un recurso protegido en la API Web, deberá enviar el token en la petición.

Blazor utiliza los mecanismos de autenticación de *ASP.NET Core* para establecer la identidad del usuario. Debido a que las aplicaciones *Blazor WebAssembly* se ejecutan en el cliente, la autorización solamente es utilizada para determinar las opciones de la interfaz de usuario que serán mostradas.

Para administrar la autenticación en *Blazor*, podemos utilizar un *Proveedor de estado de autenticación* predefinido o podemos utilizar un *Proveedor de estado de autenticación* personalizado.

AuthenticationStateProvider es el servicio sobre el cual los componentes ***AuthorizeView*** y ***CascadingAuthenticationState*** obtienen el estado de autenticación. El servicio *AuthenticationStateProvider* puede proporcionar los datos del *ClaimsPrincipal* del usuario actual.

Trabajemos en el proyecto *NorthWind.Membership.Frontend.RazorViews* para implementar la nueva funcionalidad.

Definir una interface para almacenamiento del Token

Definamos una interface que nos permita abstraer un servicio de almacenamiento del token de inicio de sesión. El token podrá ser almacenado por ejemplo en el objeto ***sessionStorage*** o el objeto ***localStorage*** del navegador web.

1. Agrega un nuevo directorio llamado ***Interfaces*** en la raíz del proyecto.
2. Agrega un nuevo archivo de tipo *Interface* llamado ***ITokenStorage.cs*** en el directorio *Interfaces* y remplaza el código de la interface por el siguiente.

```
public interface ITokenStorage
{
    Task<TokensDto> GetTokensAsync();
    Task StoreTokensAsync(TokensDto tokens);
    Task RemoveTokensAsync();
}
```

Implementar el servicio predeterminado de almacenamiento de token

1. Agrega un nuevo directorio llamado ***Services*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***TokenLocalStorage.cs*** en el directorio *Services* y remplaza el contenido de la clase por el siguiente.

```
internal class TokenLocalStorage(IJSRuntime jsRuntime) : ITokenStorage
{
    const string StorageKey = "tokens";
    const string StoreIdentifier = "localStorage.setItem";
    const string GetIdentifier = "localStorage.getItem";
    const string RemoveIdentifier = "localStorage.removeItem";

    public async Task StoreTokensAsync(TokensDto tokens)
    {
        string Value = JsonSerializer.Serialize(tokens);

        await jsRuntime.InvokeVoidAsync(StoreIdentifier,
            StorageKey, Value);
    }

    public async Task<TokensDto> GetTokensAsync()
    {
        string Value = await jsRuntime.InvokeAsync<string>(
            GetIdentifier, StorageKey);

        return Value == null ? null :
            JsonSerializer.Deserialize<TokensDto>(Value);
    }

    public async Task RemoveTokensAsync()
    {
        await jsRuntime.InvokeVoidAsync(RemoveIdentifier,
            StorageKey);
    }
}
```

Recibimos el servicio *IJSRuntime* a través del constructor para poder acceder al objeto ***localStorage*** del navegador web donde se encontrará almacenado el token de acceso cuando el usuario haya sido autenticado. El objeto *TokensDto* es serializado a un tipo *string* para guardar el valor *string* serializado.

3. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Crear un proveedor de estado de autenticación

Implementemos un proveedor de estado de autenticación que pueda ser capaz de administrar *Tokens Web JSON*.

1. Agrega el paquete NuGet ***Microsoft.AspNetCore.Components.Authorization*** al proyecto. Este paquete incluye el soporte para autenticación y autorización en aplicaciones *Blazor*.
2. Agrega el paquete NuGet ***Microsoft.IdentityModel.JsonWebTokens*** al proyecto. Este paquete incluye los tipos que proporcionan soporte para crear, serializar y validar *Tokens Web JSON*.
3. Agrega un nuevo directorio llamado ***AuthenticationStateProviders*** en la raíz del proyecto.

4. Agrega un nuevo archivo de clase llamado ***JWTAuthenticationStateProvider.cs*** en el directorio creado y remplaza el código de la clase por el siguiente.

```
public class JWTAuthenticationStateProvider(ITokenStorage storage) :
    AuthenticationStateProvider
{
    public override async Task<AuthenticationState>
        GetAuthenticationStateAsync()
    {
        // Un ClaimsIdentity vacío indica que no hay un usuario autenticado.
        ClaimsIdentity Identity = new ClaimsIdentity();

        // Recuperar el token.
        var Tokens = await storage.GetTokensAsync();

        if (Tokens != null)
        {
            // Se encontró el Token. Procesarlo.
            var Handler = new JsonWebTokenHandler();
            var Token = Handler.ReadJsonWebToken(Tokens.AccessToken);

            // Obtener la identidad del usuario.
            Identity = new ClaimsIdentity(
                        Token.Claims,
                        nameof(JWTAuthenticationStateProvider));
        }

        return new AuthenticationState(
            new ClaimsPrincipal(Identity));
    }

    // Almacenar el Token.
    public async ValueTask LoginAsync(TokensDto tokens)
    {
        await storage.StoreTokensAsync(tokens);
        NotifyAuthenticationStateChanged(GetAuthenticationStateAsync());
    }

    // Eliminar el Token.
    public async ValueTask LogoutAsync()
    {
        await storage.RemoveTokensAsync();
        NotifyAuthenticationStateChanged(GetAuthenticationStateAsync());
    }
}
```

Puedes notar que la clase deriva de *AuthenticationStateProvider* y estamos sobrescribiendo el método *GetAuthenticationStateAsync* encargado de devolver el estado de autenticación. Este es el único método que necesitamos implementar en un proveedor de estado de autenticación personalizado de *Blazor*.

El método ***LoginAsync*** almacena el Token de acceso y notifica al proveedor de estado de autenticación que hubo un cambio en el estado.

El método ***LogoutAsync*** deberá invocarse para finalizar la sesión del usuario. Para finalizar la sesión del usuario, simplemente eliminamos el token de acceso y notificamos al proveedor de estado de autenticación.

El método ***NotifyAuthenticationStateChanged*** es utilizado durante las operaciones de inicio y fin de sesión con el fin de notificar que hay un cambio en el estado actual de autenticación.

5. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Modificar el ViewModel de inicio de sesión

1. Abre el archivo ***UserLoginViewModel.cs***.
2. Modifica el constructor primario de la clase para requerir la nueva dependencia.

```
public class UserLoginViewModel(
      MembershipGateway gateway,
      IModelValidatorHub<UserLoginViewModel> validator,
      JWTAuthenticationStateProvider authenticationStateProvider)
```

3. Localiza la siguiente línea de código en el método *Login*.

```
Console.WriteLine(Tokens.AccessToken);
```

4. Remplaza el código anterior por el siguiente.

```
await authenticationStateProvider.LoginAsync(Tokens);
```

5. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Agregar el componente para finalizar sesión

1. Agrega los siguientes valores en el archivo *CommonMessages.resx* del directorio *Resources*.

Nombre	Valor
CreateAccountText	Crear cuenta
LoginText	Iniciar sesión
CloseSessionText	Cerrar sesión

2. Agrega un nuevo componente razor llamado ***LogoutPage.razor*** en el directorio *Pages*.
3. Agrega un nuevo archivo de clase llamado ***LogoutPage.razor.cs*** en el directorio *Pages* y remplaza el código de la clase por el siguiente.

```
public partial class LogoutPage
{
	[Inject]
	JWTAuthenticationStateProvider AuthenticationStateProvider { get; set; }

	[Inject]
	NavigationManager NavigationManager { get; set; }

	async ValueTask CloseSession()
	{
		await AuthenticationStateProvider.LogoutAsync();

		// Navegar a la raíz de la aplicación
		NavigationManager.NavigateTo("/");
	}
}
```

4. Modifica el contenido del archivo *LogoutPage.razor* para que sea similar al siguiente.

```
@page "/user/logout"

<button class="btn btn-primary"
		@onclick=@(async () => await CloseSession())>
	@CommonMessages.CloseSessionText
</button>
```

Agregar el componente para mostrar el estado de autenticación

1. Agrega un nuevo componente razor llamado ***LoginDisplayComponent.razor*** en el directorio *Components* y modifica el contenido del archivo por el siguiente.

```
<AuthorizeView>
	<Authorized>
		@GetUserName(context)
		<a href="user/logout">
			@CommonMessages.CloseSessionText
		</a>
	</Authorized>
	<NotAuthorized>
		<a href="user/register">
			@CommonMessages.CreateAccountText
		</a>
		<a href="user/login">
			@CommonMessages.LoginText
		</a>
	</NotAuthorized>
</AuthorizeView>

@code{
	string GetUserName(AuthenticationState context) =>
		context.User.Claims
```

```
            .Where(c => c.Type == "FullName")
            .Select(c => c.Value)
            .FirstOrDefault() ?? context.User.Identity.Name;
}
```

El código anterior muestra el nombre completo del usuario cuando este se encuentra autenticado junto con un enlace para finalizar la sesión. Si el nombre completo no puede ser obtenido, se muestra el correo del usuario almacenado en *context.User.Identity.Name*.

Cuando el usuario no se encuentre autenticado, se mostrará el enlace para crear una cuenta de usuario y un enlace para iniciar sesión.

2. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Authorization
```

Agregar un manejador de mensajes HTTP

Para poder consumir los recursos protegidos de la API Web será necesario incluir el token de acceso en el encabezado de la petición. Para facilitar el envío del token de acceso en cada petición, implementaremos un manejador de mensajes HTTP.

1. Agrega un nuevo directorio llamado ***HttpMessageHandlers*** en la raíz del proyecto.
2. Agrega un nuevo archivo de clase llamado ***MembershipBearerTokenHandler.cs*** en el directorio creado y remplaza el código de la clase por el siguiente.

```
internal class MembershipBearerTokenHandler(ITokenStorage storage) :
    DelegatingHandler
{
    protected async override Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request, CancellationToken cancellationToken)
    {
        // Intentamos obtener el token de acceso
        var Tokens = await storage.GetTokensAsync();

        // Si el token es encontrado, lo agregamos en el encabezado
        // de autorización.
        if (Tokens != null)
        {
            request.Headers.Authorization =
                new AuthenticationHeaderValue("Bearer", Tokens.AccessToken);
        }

        return await base.SendAsync(request, cancellationToken);
    }
}
```

En el código anterior, estamos heredando de ***DelegatingHandler*** que es un manejador de mensajes HTTP. Simplemente sobrescribimos el método ***SendAsync*** para que, en el caso de existir un token de acceso, este sea agregado al encabezado de autorización de la petición.

3. Importa el espacio de nombres requerido en el archivo *GlobalUsings.cs*.

Registrar los servicios

1. Abre el archivo *DependencyContainer.cs*.
2. Agrega el siguiente código en el método *AddMembershipServices* para agregar los servicios de autorización.

```
services.AddAuthorizationCore();
```

3. Agrega el siguiente código para registrar el servicio de almacenamiento predeterminado.

```
services.AddScoped<ITokenStorage, TokenLocalStorage>();
```

4. Agrega el siguiente código para registrar el proveedor de estado de autenticación personalizado.

```
services.AddScoped<JWTAuthenticationStateProvider>();
```

5. Agrega el siguiente código para indicar que se utilice el proveedor personalizado para el manejo del estado de autenticación.

```
services.AddScoped<AuthenticationStateProvider>(provider =>
      provider.GetService<JWTAuthenticationStateProvider>());
```

6. Agrega el siguiente código a la clase para definir el método que permita registrar el manejador de mensajes HTTP.

```
public static IHttpClientBuilder AddMembershipBearerTokenHandler(
      this IHttpClientBuilder builder)
{
      builder.Services.TryAddTransient<MembershipBearerTokenHandler>();
      builder.AddHttpMessageHandler<MembershipBearerTokenHandler>();

      return builder;
}
```

Este método permitirá agregar el manejador de mensajes a un objeto *HttpClient*.

7. Importa los espacios de nombres requeridos en el archivo *GlobalUsings.cs*.

Actualizar el paquete NuGet

1. Abre una ventana de línea de comandos.
2. Desplázate al directorio del proyecto *NorthWind.Membership.Frontend.RazorViews*.
3. Ejecuta el siguiente comando para crear el paquete NuGet remplazando `<path>` con la ruta del directorio donde se ubica el directorio *NorthWind.Packages* que creaste previamente.

```
dotnet pack -o <path>\NorthWind.Packages --include-source
```

Configurar los servicios en el frontend

Hagamos ahora algunos cambios en el proyecto *NorthWind.Sales.Frontend.Views*.

Modificar el componente MainLayout

1. Abre la solución *NorthWind*.
2. En el proyecto *NorthWind.Sales.Frontend.Views,* agrega una referencia del paquete *NorthWind.Membership.Frontend.RazorViews*.
3. Abre el archivo *MainLayout.razor* del directorio *Layout*.
4. Localiza el siguiente código.

```
<SectionContent SectionId="HeaderSection">
    <a href="https://learn.microsoft.com/aspnet/core/"
       target="_blank">
       @CommonMessages.AboutTitle
    </a>
</SectionContent>
```

5. Modifica el código anterior para mostrar el componente *LoginDisplay* en lugar del enlace actual. El código será similar al siguiente.

```
<SectionContent SectionId="HeaderSection">
    <LoginDisplayComponent/>
</SectionContent>
```

6. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using NorthWind.Membership.Frontend.RazorViews.Components
```

7. Localiza el siguiente código que muestra el enlace para crear una orden**.**

```
<NavMenuItem HRef="CreateOrder">
    <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
fill="currentColor" class="bi bi-card-checklist" viewBox="0 0 16 16">
        <path d="M14.5 3a.5.5 0 0 1 .5.5v9a.5.5 0 0 1-.5.5h-13a.5.5 0 0 1-.5-.5v-
9a.5.5 0 0 1 .5-.5zm-13-1A1.5 1.5 0 0 0 0 3.5v9A1.5 1.5 0 0 0 1.5 14h13a1.5 1.5 0
0 0 1.5-1.5v-9A1.5 1.5 0 0 0 14.5 2z" />
        <path d="M7 5.5a.5.5 0 0 1 .5-.5h5a.5.5 0 0 1 0 1h-5a.5.5 0 0 1-.5-.5m-
1.496-.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-.708 0l-.5-.5a.5.5 0 1 1 .708-
.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0M7 9.5a.5.5 0 0 1 .5-.5h5a.5.5 0 0 1 0
1h-5a.5.5 0 0 1-.5-.5m-1.496-.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-.708 0l-
.5-.5a.5.5 0 0 1 .708-.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0" />
    </svg>
```

```
    @CreateOrderMessages.CreateOrderMenuItemTitle
</NavMenuItem>
```

8. Coloca el código anterior dentro del elemento *<Authorized>* del componente *AuthorizeView* para que solo sea mostrado cuando el usuario actual esté autorizado.

```
<AuthorizeView>
    <Authorized>
        <NavMenuItem HRef="CreateOrder">
            <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
fill="currentColor" class="bi bi-card-checklist" viewBox="0 0 16 16">
                <path d="M14.5 3a.5.5 0 0 1 .5.5v9a.5.5 0 0 1-.5.5h-13a.5.5 0 0
1-.5-.5v-9a.5.5 0 0 1 .5-.5zm-13-1A1.5 1.5 0 0 0 0 3.5v9A1.5 1.5 0 0 0 1.5
14h13a1.5 1.5 0 0 0 1.5-1.5v-9A1.5 1.5 0 0 0 14.5 2z" />
                <path d="M7 5.5a.5.5 0 0 1 .5-.5h5a.5.5 0 0 1 0 1h-5a.5.5 0 0 1-
.5-.5m-1.496-.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-.708 0l-.5-.5a.5.5 0 1 1
.708-.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0M7 9.5a.5.5 0 0 1 .5-.5h5a.5.5 0
0 1 0 1h-5a.5.5 0 0 1-.5-.5m-1.496-.854a.5.5 0 0 1 0 .708l-1.5 1.5a.5.5 0 0 1-
.708 0l-.5-.5a.5.5 0 1 1 .708-.708l.146.147 1.146-1.147a.5.5 0 0 1 .708 0" />
            </svg>
            @CreateOrderMessages.CreateOrderMenuItemTitle
        </NavMenuItem>
    </Authorized>
</AuthorizeView>
```

9. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Authorization
```

Modificar la aplicación Blazor

Hagamos algunos cambios en el proyecto *NorthWind.Sales.BlazorClient*.

1. Abre el archivo *App.razor*.
2. Agrega el siguiente código antes de la etiqueta de inicio del elemento ***Router***.

```
<CascadingAuthenticationState>
```

3. Agrega el siguiente código después de la etiqueta final del elemento ***Router***.

```
</CascadingAuthenticationState>
```

El componente *Router* ha quedado envuelto en el componente *CascadingAuthenticationState*.

El componente *CascadingAuthenticationState* proporciona el parámetro en cascada *Task<AuthenticationState>* recibido del proveedor de estado de autenticación. Este parámetro permite a los componentes contenidos obtener el estado de autenticación.

4. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Components.Authorization
```

5. Localiza el siguiente código dentro del componente ***Found***.

```
<RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
```

6. Remplaza el código anterior por el siguiente.

```
<AuthorizeRouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />
```

El componente *AuthorizeRouteView* combina el comportamiento de los componentes *AuthorizeView* y *RouteView* para mostrar una página únicamente si el usuario está autorizado a verla.

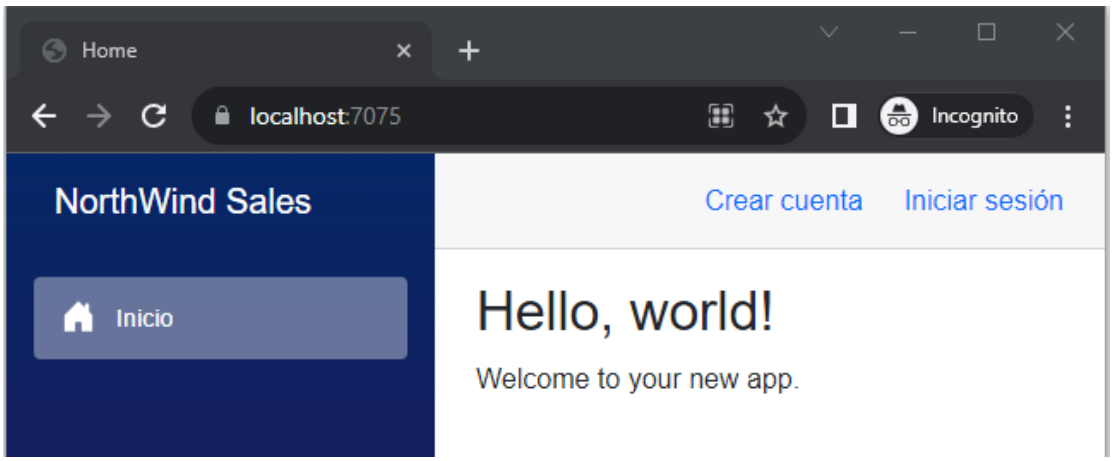
7. En el proyecto *NorthWind.Sales.Frontend.IoC,* elimina la referencia del paquete NuGet *NorthWind.Membership.Frontend.RazorViews*.

Probar la funcionalidad

Probemos ahora la funcionalidad con los cambios que hemos realizado.

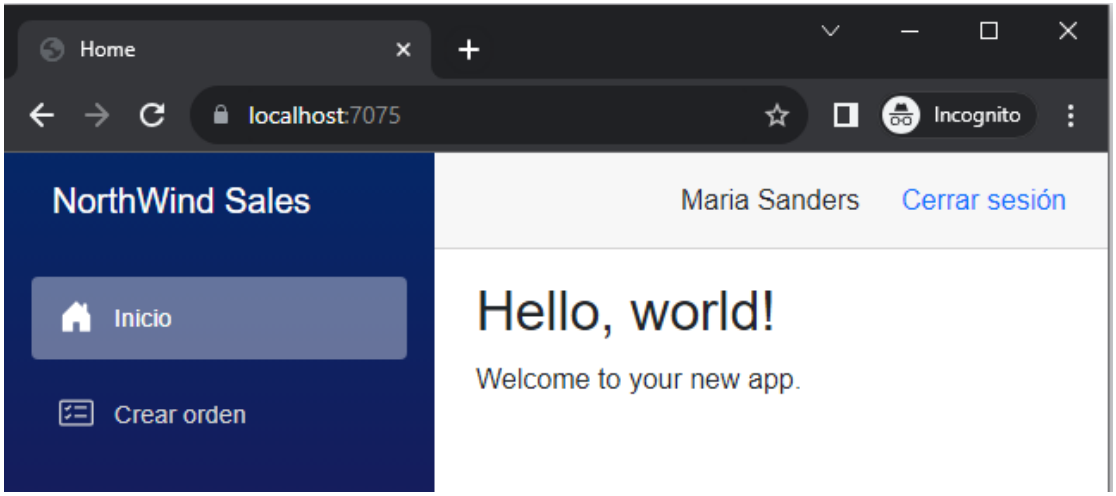
1. Ejecuta la aplicación.

   Verifica que ahora no se muestra el enlace para crear la orden de compra debido a que el usuario no se ha autenticado.



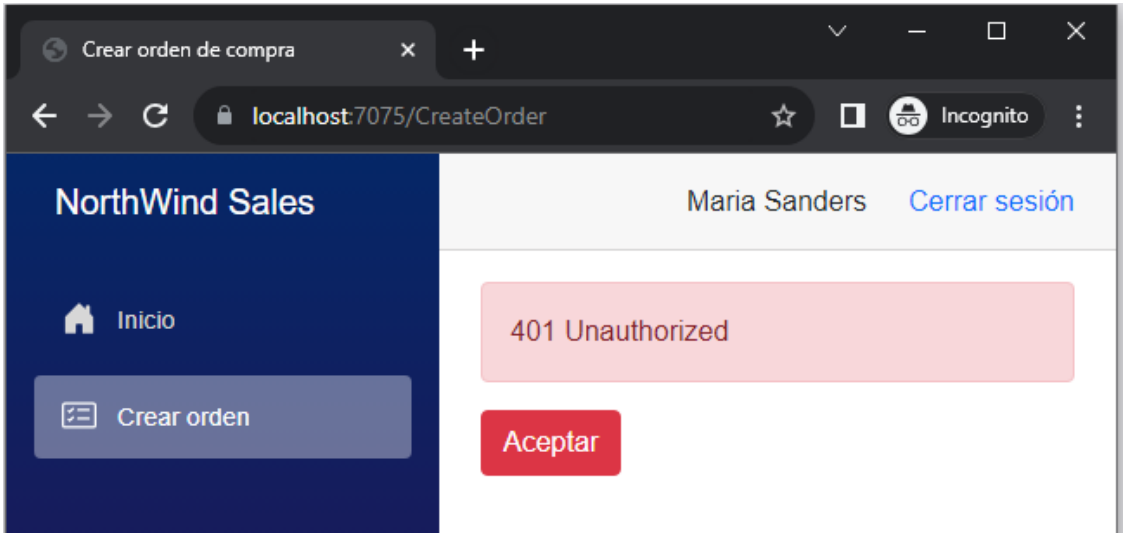
Puedes notar también que debido a que el usuario no se encuentra autenticado, se muestran los enlaces para crear una cuenta de usuario y para iniciar sesión.

2. Haz clic en el enlace ***Iniciar sesión***.
3. Proporciona las credenciales de un usuario registrado y haz clic en el botón ***Iniciar sesión***. Podrás notar que ahora se muestra en la página el nombre del usuario autenticado y el enlace para finalizar la sesión.



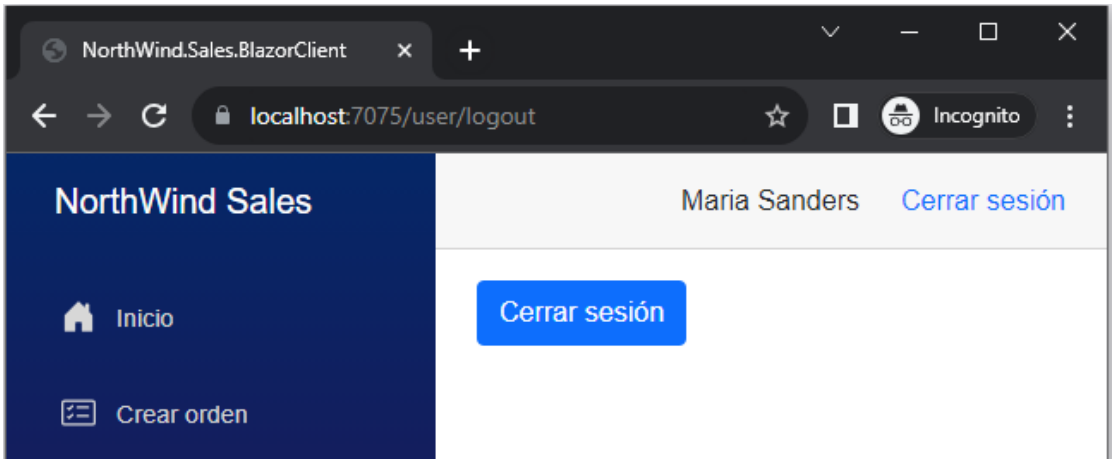
Puedes notar también que ahora ya se muestra el enlace para crear una orden.

4. Haz clic en el enlace ***Crear orden***.
5. Intenta crear una orden proporcionando los datos correctos. Podrás ver un resultado similar al siguiente.

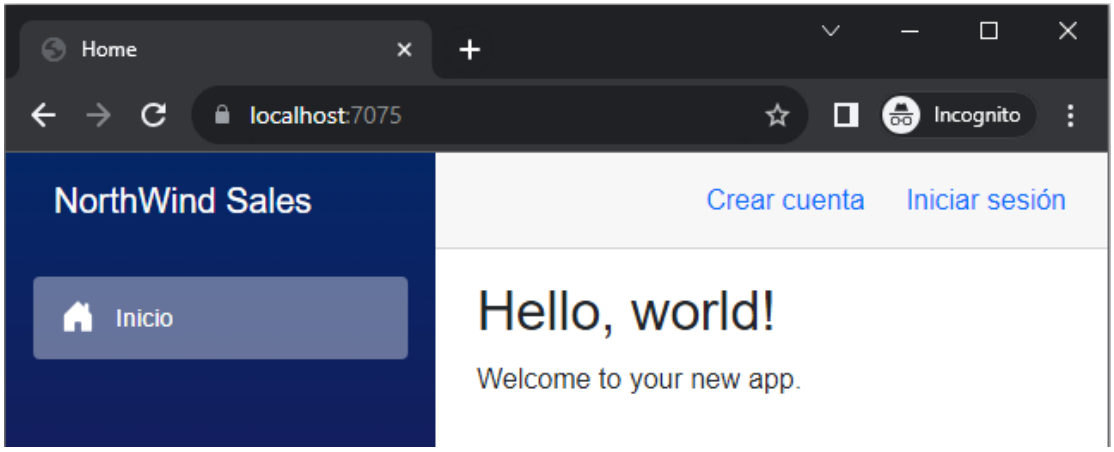


Aunque el usuario haya iniciado sesión, no estamos enviando el token de acceso como parte de la petición y por eso obtenemos el mensaje de error.

6. Haz clic en el enlace ***Cerrar sesión***. Se mostrará la página con el botón para cerrar la sesión.



7. Haz clic en el botón ***Cerrar sesión***. Se mostrará la página de inicio junto con los enlaces para crear cuenta e iniciar sesión. Además, el enlace para crear una orden ya no será mostrado.



8. Regresa a Visual Studio y detén la ejecución.

Consumir recursos protegidos

Para poder consumir los recursos protegidos de la API Web será necesario incluir el token de acceso en el encabezado de la petición.

Actualizar el proyecto NorthWind.Sales.Frontend.WebApiGateways.

1. Abre el archivo *DependencyContainer.cs*.
2. Modifica el código del método ***AddWebApiGateways*** para permitir que se pueda agregar el manejador de mensajes HTTP que hemos creado. El código será similar al siguiente.

```
public static IServiceCollection AddWebApiGateways(
    this IServiceCollection services,
    Action<HttpClient> configureHttpClient,
    Action<IHttpClientBuilder> configureHttpClientBuilder)
{
    services.AddExceptionDelegatingHandler();
    var Builder = services
        .AddHttpClient<ICreateOrderGateway,
            CreateOrderGateway>(configureHttpClient)
        .AddHttpMessageHandler<ExceptionDelegatingHandler>();

    configureHttpClientBuilder(Builder);

    return services;
}
```

Actualizar el proyecto NorthWind.Sales.Frontend.IoC

1. Abre el archivo *DependencyContainer.cs*.
2. Modifica el código del método ***AddNorthWindSalesServices*** para incluir la nueva dependencia. El código será similar al siguiente.

```
public static IServiceCollection AddNorthWindSalesServices(
    this IServiceCollection services,
    Action<HttpClient> configureHttpClient,
    Action<HttpClient> configureMembershipHttpClient,
    Action<IHttpClientBuilder> configureHttpClientBuilder)
{
    services.AddWebApiGateways(configureHttpClient, configureHttpClientBuilder)
        .AddViewsServices()
        .AddValidationService()
        .AddValidators()
        .AddMembershipServices(configureMembershipHttpClient);

    return services;
}
```

Actualizar el proyecto NorthWind.Sales.BlazorClient

1. Abre el archivo *Program.cs*.
2. Localiza el siguiente código.

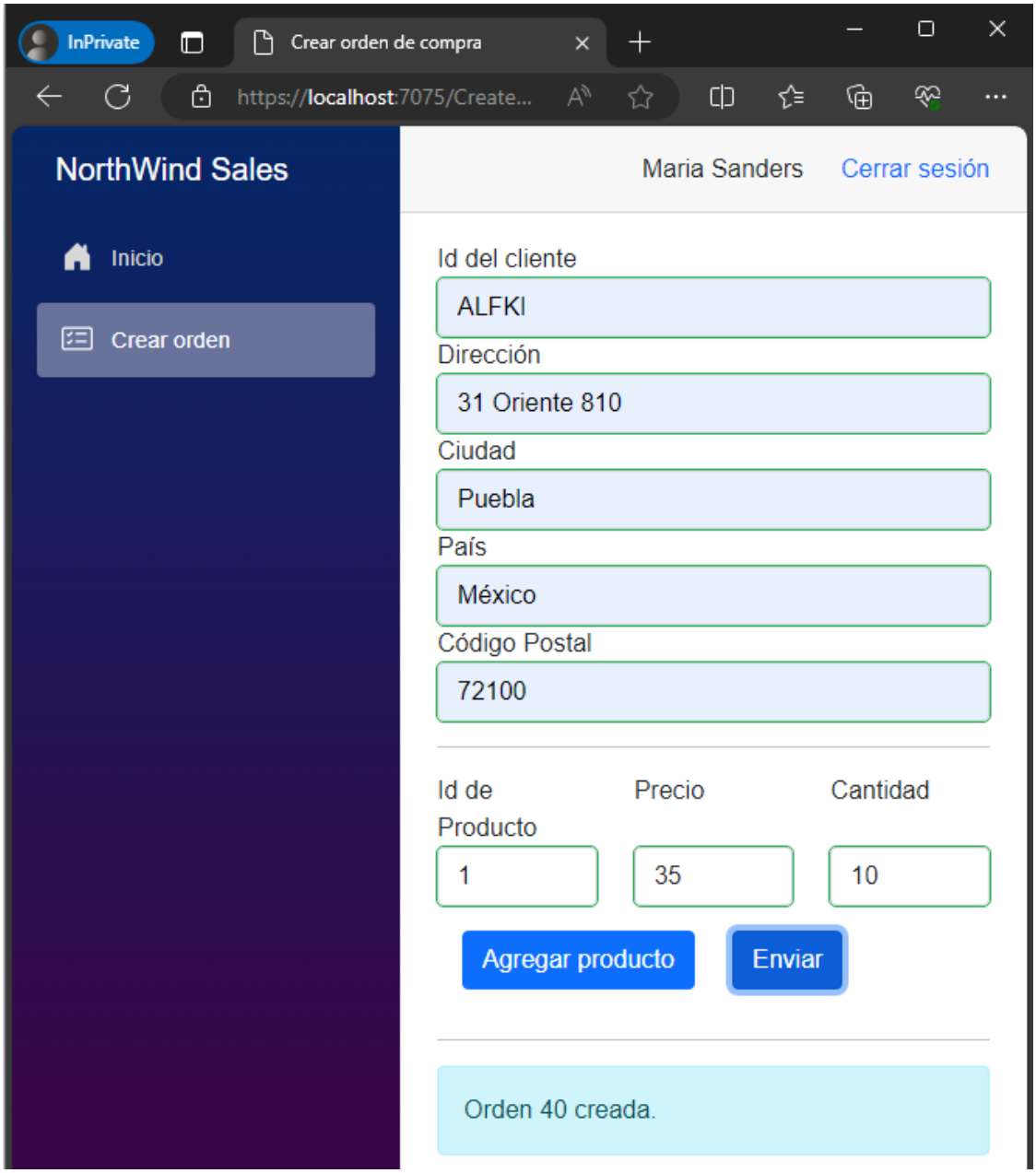
```
builder.Services.AddNorthWindSalesServices(client =>
{
    client.BaseAddress =
    new Uri(builder.Configuration["WebApiAddress"]);
},
membershipHttpClient =>
{
    membershipHttpClient.BaseAddress =
    new Uri(builder.Configuration["WebApiAddress"]);
});
```

3. Modifica el código anterior para agregar la nueva dependencia. El código será similar al siguiente.

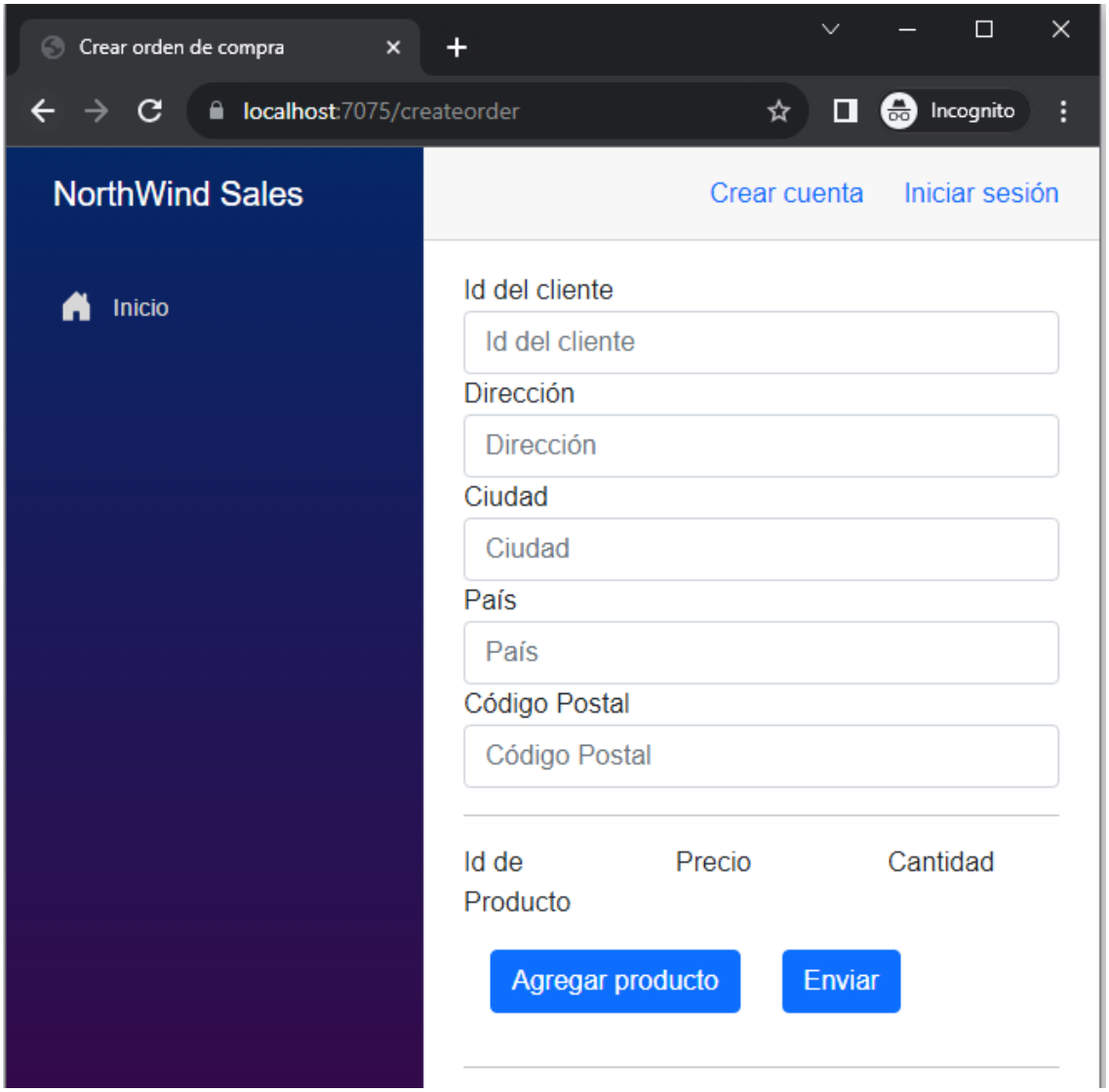
```
builder.Services.AddNorthWindSalesServices(client =>
{
    client.BaseAddress =
    new Uri(builder.Configuration["WebApiAddress"]);
},
membershipHttpClient =>
{
    membershipHttpClient.BaseAddress =
    new Uri(builder.Configuration["WebApiAddress"]);
},
httpClientBuilder =>
    httpClientBuilder.AddMembershipBearerTokenHandler());
```

Probar la funcionalidad

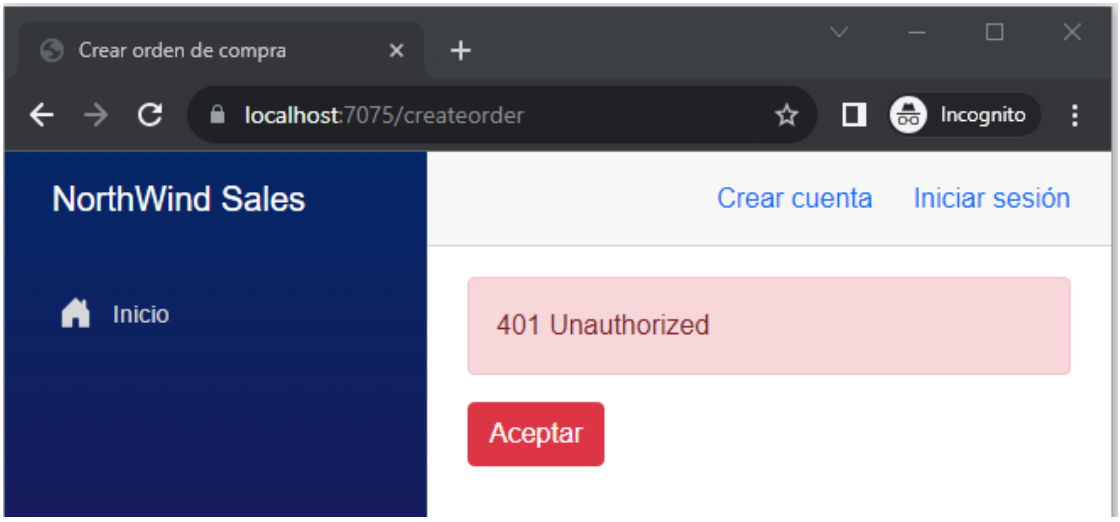
1. Ejecuta la aplicación.
2. Inicia sesión con las credenciales de un usuario registrado.
3. Intenta crear una orden de compra proporcionando datos válidos. Podrás notar que ahora ya es posible crear la orden debido a que el usuario ya se ha autenticado y ya se ha enviado el token de acceso como parte de la petición para ser autorizado.



4. Cierra la sesión del usuario. El enlace para crear una orden ya no estará visible, sin embargo, el usuario puede acceder a la página de creación de orden.
5. Navega al Uri ***/createorder***. Puedes notar que se muestra la página, aunque el usuario no haya sido autenticado.



6. Intenta crear una orden. Podrás ver un mensaje indicando que no se ha autorizado la petición.



Aunque el usuario puede acceder a la página que no está autorizado, el endpoint en la API web no lo autoriza ya que no recibe el token de acceso.

7. Regresa a Visual Studio y detén la ejecución.

Evitar el acceso a la página de creación de orden

Hagamos unos cambios para evitar que un usuario no autorizado pueda acceder a la página de creación de órdenes.

1. Abre el archivo *CreateOrder.razor* del proyecto *NorthWind.Sales.Frontend.Views*.
2. Agrega el siguiente código debajo de la línea de código que contine la directiva *@page*.

```
@attribute [Authorize]
```

El atributo *Authorize* permite indicar que para acceder a la página se requiere una autorización.

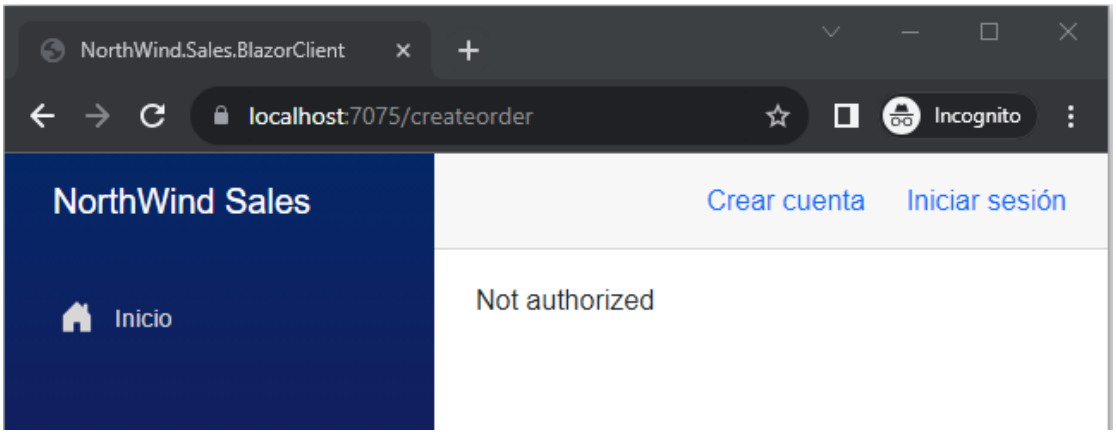
En lugar de colocar la directiva *@attribute* en el archivo *.razor*, podríamos colocar el atributo *[Authorize]* a la clase en el archivo *.razor.cs*.

3. Agrega el siguiente código en el archivo *_Imports.razor* para importar el espacio de nombres requerido.

```
@using Microsoft.AspNetCore.Authorization
```

Probar la funcionalidad

1. Ejecuta la aplicación.
2. Navega al Uri ***/createorder***. Puedes notar que ahora se muestra un mensaje indicando que el usuario no está autorizado.



3. Regresa a Visual Studio y detén la ejecución.

Hemos implementado la autenticación y autorización para la aplicación *Blazor WebAssembly* y para la *API Web* con *Clean Architecture*.